

Programação Orientada a Objetos em C++

Um curso construído sobre um sistema que compila

O QUE ESTE LIVRO AFERE

sistema-base	Deriva v2.7, roguelike de terminal
padrão-alvo	C++17, e ele é teto
portão	4 de 4 · 188 testes verdes · 0 avisos
estrutura	26 capítulos · 3 anexos
trechos de código	156 extraídos por âncora, nenhum digitado

Prefácio	1
1 Da programação procedural à orientação a objetos	4
1.1 Gerenciamento de Complexidade em Software	4
1.2 Limitações do Modelo Procedural	5
1.3 Os Quatro Pilares da POO	6
Encapsulamento	6
Herança	6
Polimorfismo	6
Abstração	6
1.4 Exemplo Comparativo: a grade da estação em C e em C++	6
A grade em estilo C	7
A grade em C++	7
1.5 Tabela Comparativa: C Procedural vs. C++ OO	7
Exercícios Propostos	8
O código, extraído do Deriva	8
2 Infraestrutura do programador C++	11
2.1 O Modelo de Compilação de C++	11
2.2 CMake - O Sistema de Build da Disciplina	12
Estrutura mínima de projeto	12
2.3 FetchContent - dependência externa sem submódulo	14
2.4 O portão make verifica	14
2.5 GDB - depuração básica, e o ponto de parada em destrutor	15
2.6 Sanitizers - confirmação, não oráculo	16
O que o ASan diz, e o que ele não diz	16
2.7 Transição de Python/C para C++	17
Exercícios Propostos	18
O código, extraído do Deriva	18
3 Fundamentos de C++17 para POO	22
3.1 Entrada e saída: de printf/scanf a cout/cin	22
3.2 Strings: de char[] a std::string	24
3.3 std::string_view e a armadilha de tempo de vida	25
3.4 Ponteiro e referência	28
3.5 C++ moderno essencial	29

auto - dedução de tipo	29
nullptr - ponteiro nulo com tipo próprio	30
Inicialização uniforme com {}	31
3.6 Ligações estruturadas	32
3.7 [[nodiscard]] e [[maybe_unused]]	33
[[nodiscard]] - descartar este resultado é erro	33
[[maybe_unused]] - não usar isto é intencional	34
Exercícios Propostos	35
O código, extraído do Deriva	35
4 Git, LLM como copiloto e a rubrica de revisão	39
4.1 Por que Versionar Código?	39
4.2 Conceitos Fundamentais	39
4.3 Fluxo de Trabalho Individual	40
4.4 Branches e Merge	41
4.5 GitHub - Colaboração e Entrega	42
4.6 .gitignore para C++ com CMake	43
4.7 O que um LLM faz, e o que ele não faz	43
4.8 A estrutura de um prompt que serve	44
4.9 Prompts por contexto de desenvolvimento OO	45
4.10 Análise crítica de código gerado	46
4.11 Limites, riscos e responsabilidades	47
4.12 O ciclo OO assistido, e o que conferir em cada etapa	47
4.13 A rubrica de revisão de código OO gerado por IA	48
Exercícios Propostos	49
O código, extraído do Deriva	50
5 Classificação de linguagens e sistemas de tipos	52
5.1 Baseadas em Objetos vs. Orientadas a Objetos	52
5.2 Sistemas de Tipos: os Eixos Fundamentais	52
Eixo 1 - Tipagem Estática vs. Dinâmica	53
Eixo 2 - Tipagem Forte vs. Fraca	54
Eixo 3 - Single Dispatch vs. Multiple Dispatch	54
5.3 Mecanismos de Reutilização: Herança, Composição e Traits	54
5.4 Posicionando as Linguagens	55
Exercícios Propostos	55
O código, extraído do Deriva	56

6	UML leve	58
6.1	Por que UML? A Modelagem Antes do Código	58
6.2	Diagrama de Classes - Notação Essencial	58
6.3	Relacionamentos entre Classes	59
	As relações que o Deriva de fato tem	59
6.4	Diagrama de Sequência - Interação entre Objetos	62
6.5	Fluxo de Trabalho com UML Leve	63
	Exercícios Propostos	64
	O código, extraído do Deriva	64
7	Classes e objetos; o contador de instâncias vivas	66
7.1	Declaração e Definição de Classes	66
7.2	Modificadores de Acesso	67
7.3	O Ponteiro this	67
7.4	Const-Correctness	68
7.5	Membros Estáticos e o Contador de Instâncias Vivas	69
	A parte de C++17: inline static	69
	O contador vivos, e por que ele é o exemplo canônico da disciplina	69
	O que o contador não acusa	70
7.6	O Leiaute do Objeto em Memória	70
	Exercícios Propostos	71
	O código, extraído do Deriva	71
8	Ciclo de vida e RAII	76
8.1	O que é o Ciclo de Vida de um Objeto?	76
8.2	Tipos de Construtores	76
	Construtor Padrão	76
	Construtor Parametrizado	76
	Construtor de Cópia	77
8.3	Listas de Inicialização - Por que Usar Sempre	77
8.4	Destruítores e a Ordem de Destruição	78
8.5	RAII - Resource Acquisition Is Initialization	78
8.6	Instrumentação de Ciclo de Vida	80
8.7	terminal_bruto: RAII com Consequência Física	81
	Exercícios Propostos	82
	O código, extraído do Deriva	82

9 Operações especiais: a regra do zero e do três	86
9.1 O que o Compilador Gera Automaticamente	86
9.2 A Regra do Zero	87
9.3 A Regra dos Três	88
9.4 Caça ao Bug 1: cópia rasa em grade	89
A ordem de observação	89
O conserto, e por que o conserto certo é o mais curto	89
Exercícios Propostos	90
O código, extraído do Deriva	90
 10 Herança simples	 96
10.1 Herança como Modelagem de Domínio	96
10.2 Sintaxe e Tipos de Herança em C++	97
10.3 A Hierarquia do Deriva - v1.0	98
10.4 Construtores e Destrutores na Hierarquia	100
10.5 O Subobjeto Base no Leiaute do Objeto	101
Exercícios Propostos	101
O código, extraído do Deriva	102
 11 Funções virtuais e classes abstratas	 107
11.1 A Necessidade de Funções Virtuais	107
11.2 override e final	108
11.3 Funções Virtuais Puras e Classes Abstratas	109
11.4 Mecanismo Interno: vtable e vptr	110
11.5 O Destrutor Virtual, e o Vazamento que a Ausência Dele Produz	111
O que cada instrumento vê	111
O aviso fica ligado, e é isto que a caça ao bug 2 caça	112
A forma da correção	113
11.6 Caça ao Bug 2: o destrutor que não é virtual	113
A ordem de observação	113
A frase única, e por que ela vem antes	114
Exercícios Propostos	114
O código, extraído do Deriva	115
 12 Ponteiros inteligentes I - posse exclusiva	 120
12.1 Por que Ponteiros Inteligentes?	120
12.2 O Contraexemplo: Posse Crua	121

O contraexemplo que o estudante vai encontrar sozinho	121
12.3 unique_ptr - Posse Exclusiva	122
O par que precisa dos dois: unique_ptr<entidade> e o destrutor virtual	123
12.4 O Que unique_ptr Custa, e Onde Ele Não Serve	124
Exercícios Propostos	124
O código, extraído do Deriva	125
13 Ponteiros inteligentes II - posse compartilhada	128
13.1 shared_ptr - Posse Compartilhada	128
13.2 weak_ptr - Quebrando Ciclos	129
13.3 O Ciclo, Medido	130
13.4 Regra de Escolha	131
Exercícios Propostos	131
O código, extraído do Deriva	132
14 Semântica de movimento e a regra dos cinco	135
14.1 O Problema: Cópias Desnecessárias	135
14.2 lvalues e rvalues	136
14.3 std::move e Referências a rvalue	137
14.4 O Que Sobra na Origem, e o Que o Padrão Promete	138
14.5 A Regra dos Cinco	139
14.6 noexcept, e Por Que o vector Depende Dele	140
14.7 Encaminhamento Perfeito e std::forward - Panorama	141
14.8 Semântica de Valor e Semântica de Referência	142
Exercícios Propostos	142
O código, extraído do Deriva	143
15 Sobrecarga de operadores	148
15.1 Por que Sobrecarregar Operadores?	148
15.2 Regras de Ouro	148
15.3 A Construção Composto-para-Simétrico, no Deriva	149
15.4 O Par const e Não-const de operator[]	150
15.5 operator<< e a Saída que o Replay Compara	151
Exercícios Propostos	152
O código, extraído do Deriva	152
16 Testes com Catch2 e replay determinístico	155

16.1 O Teste como Especificação Executável	155
16.2 Configurando Catch2 com CMake	156
16.3 Primeiro Arquivo de Teste	157
Sobre integração contínua, e por que ela não está neste livro	157
16.4 Testando Exceção e Casos de Fronteira	158
16.5 O Replay Determinístico	158
Por que este capítulo vem antes dos Caps. 24 e 25	159
16.6 O Que o Teste Não Faz	159
Exercícios Propostos	160
O código, extraído do Deriva	161
17 Herança múltipla e o diamante	166
17.1 Herança Múltipla: Capacidades e Riscos	166
17.2 O Problema do Diamante	167
17.3 Resolvendo com Herança Virtual	168
17.4 Contraste com Java/C#, e o Que o Deriva Escolhe	169
17.5 Neste Encontro: a Caça ao Bug 2	170
Exercícios Propostos	170
O código, extraído do Deriva	171
18 Polimorfismo dinâmico e RTTI	175
18.1 Polimorfismo Dinâmico em Ação	175
18.2 dynamic_cast - Downcasting Seguro	176
18.3 RTTI - typeid e type_info	177
18.4 Quando dynamic_cast é Sintoma, e Quando Ele é a Resposta	178
Exercícios Propostos	179
O código, extraído do Deriva	180
19 Templates, polimorfismo estático e CRTP	183
19.1 O que São Templates?	183
19.2 Templates de Função	184
19.3 grade<T>: o Template de Classe	185
19.4 Restringir o Tipo: static_assert e if constexpr	186
if constexpr, e por que ele substitui SFINAE	186
19.5 Polimorfismo Estático vs. Dinâmico	187
19.6 CRTP, e o Contador que Já Foi Escrito Três Vezes	187
O que o CRTP não conserta	189

19.7 Vinte Minutos de C++20: Concepts e Ranges	189
Exercícios Propostos	190
O código, extraído do Deriva	190
20 Tratamento de erros	194
20.1 Exceções em C++	194
20.2 As Garantias de Exceção	195
Onde o Deriva está, hoje, e o que falta	196
20.3 O Desenrolar da Pilha, e por que RAII Depende Dele	196
20.4 <code>std::optional</code> - Resultado Ausente sem Exceção	197
20.5 <code>std::variant</code> - Resultado ou Erro	197
20.6 <code>std::filesystem</code> no Carregamento de Mapa	198
20.7 Qual dos Três, e por Quê	199
Exercícios Propostos	199
O código, extraído do Deriva	200
21 STL panorâmica e lambdas	204
21.1 Contêineres Essenciais	204
21.2 Iteradores e o Intervalo Semiaberto	205
21.3 Lambdas: a Classe Anônima que o Compilador Escreve	206
Captura por valor e por referência	206
O que a lambda substitui	207
21.4 Algoritmos: Trocar o Laço pelo Nome do que Ele Faz	207
21.5 <code>std::clamp</code> e <code>std::size</code>	208
Exercícios Propostos	208
O código, extraído do Deriva	209
22 Concorrência em C++ - panorâmica	212
22.1 <code>std::thread</code> - Criação e Join	212
22.2 O que as Threads Compartilham	213
22.3 A Corrida de Dados sobre o Contador vivos	213
22.4 <code>std::mutex</code> , <code>std::lock_guard</code> e <code>std::scoped_lock</code>	214
22.5 O que a Concorrência Cobra do Projeto Orientado a Objetos	215
22.6 A Ponte com Programação Concorrente	216
Exercícios Propostos	216
O código, extraído do Deriva	217
23 Serialização	223

23.1 Estratégias de Serialização	223
23.2 O Formato da Partida Salva	224
23.3 Versionamento de Formato	225
23.4 O que Quebra a Compatibilidade Regressiva	226
23.5 A Invariante, o Contador, e o Teste de Ida e Volta	226
23.6 JSON, Formato Binário e Serialização Polimórfica	227
Exercícios Propostos	228
O código, extraído do Deriva	228
24 SOLID e invariância de comportamento	231
24.1 O mundo como God Class	231
24.2 S - Responsabilidade Única (SRP)	232
24.3 O - Aberto/Fechado (OCP)	233
24.4 L - Substituição de Liskov (LSP)	233
24.5 I - Segregação de Interfaces (ISP)	234
24.6 D - Inversão de Dependência (DIP)	235
24.7 Invariância de Comportamento, Verificada por Replay	236
24.8 Caça ao Bug 3: a Refatoração que Mudou a Saída	237
Exercícios Propostos	238
O código, extraído do Deriva	238
25 Padrões de projeto canônicos em C++ moderno	244
25.1 Strategy com Lambdas, e não com Herança	244
25.2 Command: a Entrada, e o Desfazer	246
25.3 State: as Telas	247
25.4 Observer: o Registro de Eventos	247
25.5 Factory: a Geração de Entidades	248
25.6 Composite: o Inventário	249
25.7 Decorator: a Apresentação	249
25.8 Singleton: a Forma Correta e os Quatro Problemas	250
25.9 Quando o Padrão Não se Paga	251
Exercícios Propostos	252
O código, extraído do Deriva	253
26 Qt e a separação domínio/apresentação	258
26.1 O que o Segundo Front-end Prova	259
26.2 Arquitetura do Qt	259

26.3 Signals e Slots como Observer Implementado pelo Framework	261
26.4 A Separação Domínio/Apresentação com Qt	261
26.5 O Esqueleto Publicado	262
Onde o critério da §26.1 é provado, e onde ele é conferido à mão . . .	263
26.6 O que Este Capítulo Fecha	263
Exercícios Propostos	264
O código, extraído do Deriva	264
Anexo A - Concepts e Ranges (C++20)	269
A.1 O Problema dos Templates sem Restrições (C++20)	270
A.2 Ranges - Programação Funcional sobre Sequências (C++20)	271
Exercícios Propostos	271
O código, extraído do Deriva	272
Anexo B - Referência rápida de C++17	273
B.1 As quinze construções	273
B.2 Como usar isto em prova	275
B.3 O que está fora, de propósito	275
Anexo C - O Deriva: as 20 versões	276
C.1 Por que a trilha existe	276
C.2 A tabela	277
C.3 A variante deliberadamente quebrada	278
C.4 As três caças ao bug	279
Glossário	280
Referências Bibliográficas	286
Referências Básicas	287
Referências Complementares	287
Documentação de Ferramenta e de Biblioteca	288
Referências em Português, no Acervo da BC/UFPB	288
Material da Disciplina	288
Nota sobre a seção de documentação	289

Prefácio

Esta apostila cobre integralmente o plano de ensino de Programação Orientada a Objetos em C++, no semestre 2026.2. Ela é o resultado da fusão e da expansão dos Volumes I e II utilizados em semestres anteriores, com material novo em todos os tópicos que o inventário de conteúdo apontou como ausentes ou rasos.

O livro tem **26 capítulos e 3 anexos**, e a correspondência com o curso é estrita: **Aula N = Capítulo N**. Onde o curso divide um tema em dois encontros, o livro tem dois capítulos; onde funde, o livro funde. A ordem é a do curso, e não a de uma exposição sistemática do C++, porque o que se pretende é que você encontre em casa exatamente o que viu projetado em aula. Concepts e Ranges, que eram um capítulo próprio, passaram ao Anexo A: deixaram de ser aula, e o conteúdo mudou de estatuto em vez de ser descartado.

A disciplina é organizada em torno de um projeto fio condutor chamado **Deriva**, um *roguelike* de terminal por turnos em que uma sonda de inspeção percorre uma estação orbital abandonada, através do console. O projeto evolui ao longo das três unidades: da posição e da célula da grade (vetor2, célula, grade, mapa) na Unidade I, passa pela hierarquia de entidades (entidade → sonda, drone, item) com posse por ponteiros inteligentes e testes na Unidade II, e chega a genericidade, refatoração sob SOLID e um segundo front-end em Qt na Unidade III. Cada capítulo mostra como as decisões de projeto discutidas afetam o Deriva, e cada exercício o faz crescer.

A escolha do Deriva tem uma razão que atravessa o livro inteiro: nele **RAII tem consequência física**. A classe `terminal_bruto` põe o terminal em modo bruto no construtor e o restaura no destrutor; sem o destrutor, o terminal fica sem eco e sem Enter *depois* que o programa sai, e o conserto é digitar `reset` às cegas. Vazamento de memória é abstrato, porque o sistema operacional devolve tudo quando o processo morre; vazamento do modo do terminal, não.

O **padrão-alvo é C++17**, e ele é o teto, não o piso: o laboratório compila com `g++ -std=c++17` e `CXX_EXTENSIONS OFF`, de forma que código que só compila com `-std=gnu++17` não conta como C++17. Onde algo de C++20 aparece, ele vem rotulado, e o Anexo A é o lugar em que se apresenta como reconhecimento de API, sem ser base de exemplo nem de entrega.

As máquinas do laboratório não têm sanitizer nem Valgrind, e neste livro isso é conteúdo, e não limitação. No lugar do detector automático estão três técnicas que você mesmo constrói, e que aparecem antes de serem exigidas: o **contador de instâncias vivas** (Cap. 7), a **instrumentação de ciclo de vida** (Cap. 8) e o **gdb com ponto de parada em destrutor** (Caps. 2, 8 e 11). Os sanitizers aparecem como ferramenta de confirmação no Cap. 2, e não como critério de aceitação.

Todo trecho de código deste livro é extraído de exemplos/deriva/, que compila sem um aviso sob `-Wall -Wextra -Wpedantic` e passa o portão `make verifica`; nenhum foi digitado no texto. Os números que a prosa afirma - tamanhos de estrutura, contagens de teste, bytes presos por um ciclo de `shared_ptr` - são medi-

0 avisos

avisos do compilador

aferido por

make verifica

Makefile

dos nesse código, e não estimados. Se o código mudar de forma a desmentir um número, o build falha antes de o livro sair errado.

Convenções adotadas:

- **Código:** `snake_case` para identificadores, identificadores e comentários em português, C++17 como padrão base, com o que for de C++20 explicitamente rotulado.
- **Caixas:** ▲ ATENÇÃO (situação em que o compilador não avisa), √ DICA (o idioma que a disciplina cobra), ◇ LLM (uso crítico de assistente), ► DERIVA (a ligação com o sistema-base).
- **UML:** diagramas de classes e de sequência em Mermaid, no Cap. 6.
- **Exercícios:** cada capítulo termina com exercícios graduados, do conceitual ao prático, e a maior parte deles é uma peça a mais no Deriva.

O material pressupõe familiaridade com programação em C (structs, ponteiros, funções) e noções básicas de Python. Não pressupõe experiência anterior com C++ nem com orientação a objetos.

Prof. Carlos Eduardo Coelho Freire Batista

João Pessoa, Setembro de 2026

Fundamentos

Da programação procedural ao objeto: infraestrutura, fundamentos de C++17, Git e revisão de código gerado por IA, tipos, UML, classes, ciclo de vida e as operações especiais.

Da programação procedural à orientação a objetos

O QUE ESTE CAPÍTULO ENTREGA

- ▶ Reconhecer as limitações do modelo procedural em sistemas de software de grande escala.
- ▶ Compreender os quatro pilares da POO: encapsulamento, herança, polimorfismo e abstração.
- ▶ Contrastar implementações equivalentes em C procedural e C++ orientado a objetos.
- ▶ Identificar quando a orientação a objetos representa ganho real de expressividade.

§1.1 Gerenciamento de Complexidade em Software

À medida que sistemas crescem, o principal desafio do programador deixa de ser “fazer funcionar” e passa a ser “entender, modificar e testar sem quebrar”. Que se leia código muito mais do que se escreve é observação corrente na literatura de engenharia de software, e a proporção exata não importa para o argumento: basta notar que toda linha escrita será lida muitas vezes, por outra pessoa ou por você mesmo seis meses depois, e que é a leitura, não a digitação, que a orientação a objetos existe para baratear.

Os paradigmas de programação são estilos fundamentalmente distintos de organizar um programa. Os mais relevantes para esta disciplina são:

- **Procedural/estruturado** (C, Pascal): o programa é uma sequência de funções que operam sobre dados globais ou passados por parâmetro.
- **Orientado a objetos** (C++, Java, Python, C#): dados e operações são agrupados em objetos que se comunicam por mensagens.
- **Funcional** (Haskell, Erlang, partes de Python e C++): computação como aplicação de funções puras sem efeitos colaterais.
- **Genérico** (templates em C++, generics em Java): algoritmos parametrizados por tipo, avaliados em tempo de compilação.

C++ é multiparadigma: suporta POO, programação genérica (templates), programação procedural e aspectos funcionais. Isso o torna especialmente poderoso - e especialmente exigente do programador.

| ▶ DERIVA |

O sistema que atravessa esta disciplina é o **Deriva**: um *roguelike* de terminal por turnos em que uma sonda de inspeção percorre uma estação orbital abandonada, através do console.

Nesta unidade ele é pouco mais que uma grade de células: vetor2 para posição, célula para o que há em cada posição, grade para o conjunto, mapa para o setor carregado de arquivo. Nada disso exige orientação a objetos - e é exatamente por isso que o Deriva serve ao argumento deste capítulo: o mesmo problema pode ser escrito nos dois paradigmas e comparado.

§1.2 Limitações do Modelo Procedural

Para entender por que a POO foi criada, é preciso sentir as dores reais do modelo procedural. Considere a grade de células da estação escrita em C: uma struct com o que há numa célula, um bloco de memória com todas elas, e um punhado de funções soltas que operam sobre esse bloco. Cada célula guarda energia, massa, o glifo com que é desenhada e a sigla do compartimento; a grade guarda largura, altura e o ponteiro para as células.

O arranjo funciona, e é assim que a maior parte do software de sistema do mundo está escrita. Os três problemas abaixo, porém, não são acidentes de implementação: são propriedades do modelo, e os três aparecem no código extraído no fim deste capítulo, tal como a grade em estilo C do Deriva os apresenta.

Falta de encapsulamento. A struct `grade_c` expõe largura, altura e o ponteiro para as células, e o contrato inteiro fica na cabeça de quem chama: são cinco regras, listadas no comentário de cabeçalho do primeiro trecho extraído, e nenhuma delas está escrita de forma que o compilador possa cobrá-la. Escrever em largura depois de a grade estar criada é uma dessas regras, e violá-la faz a mesma célula passar a ser outra, sem que nada reclame.

Dados e funções desconectados. `criar`, `destruir`, `em` e `escrever` são funções livres que recebem o endereço da struct, e nada nas assinaturas delas as prende à grade a que pertencem. Quem chamar `em` sem ter chamado `criar` desreferencia um ponteiro que nunca foi preenchido; quem copiar a struct leva o ponteiro, e não os dados, e é o que o segundo trecho extraído mostra medido, com as duas grades escrevendo no mesmo buffer.

Gerenciamento manual e frágil. A liberação é responsabilidade de quem escreve a função, em toda saída possível, inclusive nas que ainda não existem; e o terceiro trecho extraído mostra o que custa esquecer: dez kilobytes vazam e o teste passa, porque o vazamento não tem sintoma enquanto o processo é curto.

| ▲ ATENÇÃO |

Em C, nada impede que outro módulo acesse e modifique diretamente os campos de uma struct. Projetos grandes com centenas de structs e milhares de funções se tornam rapidamente impossíveis de manter - qualquer mudança pode quebrar código em qualquer lugar.

§1.3 Os Quatro Pilares da POO

A POO resolve os problemas acima por meio de quatro mecanismos fundamentais:

— Encapsulamento

Dados e operações são agrupados em uma unidade chamada classe. O acesso aos dados é controlado: o que é `public` pode ser acessado de fora; o que é `private` só pode ser acessado pelos próprios métodos da classe. Isso garante que os dados estejam sempre em estado válido. Na grade do Deriva, `largura_`, `altura_` e o vetor de células são privados, e quem quer saber a largura pergunta através de `largura()`, que é `const` e não deixa ninguém escrever nela.

— Herança

Novas classes são construídas a partir de classes existentes, herdando seus atributos e métodos. A classe derivada pode especializar o comportamento sem repetir código. Modela a relação “é um” (is-a): um drone *é uma* entidade, e o que vale para toda entidade da estação vale para ele.

— Polimorfismo

Objetos de tipos diferentes respondem à mesma mensagem de maneiras distintas. Um método virtual `desenhar()` declarado em entidade se comporta de um jeito na sonda, de outro no drone e de outro no item largado no chão - e o código que percorre o setor desenhando tudo não precisa saber qual tipo concreto tem em mãos. A hierarquia completa do Deriva chega na v1.0 (Cap. 10); o que existe desde já, em `include/deriva/leiaute.hpp`, é o par mínimo que permite medir quanto esse mecanismo custa em bytes.

— Abstração

O programador expõe apenas o que é essencial, ocultando detalhes de implementação. A classe `mapa` oferece `carregar()` e `despejar()` sem revelar se a grade por baixo é um `std::vector<celula>` ou um buffer alocado à mão - e essa liberdade é o que permite, no Cap. 9, trocar uma coisa pela outra e observar a diferença sem tocar em quem chama.

— DIAGRAMA UML —

Diagrama de classes: relação entre os 4 pilares - classe com encapsulamento, hierarquia mostrando herança, interface mostrando polimorfismo.

§1.4 Exemplo Comparativo: a grade da estação em C e em C++

— A grade em estilo C

A `struct grade_c` do primeiro trecho extraído no fim deste capítulo é o lado procedural do par, e o que interessa nela é o comentário de cabeçalho: cinco regras que quem chama precisa cumprir, e nenhuma delas escrita de forma que o compilador possa cobrá-la. Chamar `criar` antes de tudo, chamar `destruir` uma vez só, não copiar a `struct`, não escrever nas dimensões depois de criada, conferir o limite antes de indexar. As cinco são verdadeiras, as cinco são invisíveis, e é essa assimetria que a orientação a objetos existe para desfazer.

Os dois trechos seguintes, os dois marcados como quebrados de propósito, mostram duas dessas regras sendo violadas sem que nada reclame. No primeiro, `grade_c b = a` copia o ponteiro e não as células: escrever em `b` muda `a`, e `a.celulas == b.celulas` prova por que. No segundo, um bloco sai de escopo sem destruir, dez kilobytes vazam, e o teste passa - porque num processo curto o vazamento não tem sintoma nenhum.

— A grade em C++

Do outro lado, a grade de `include/deriva/grade.hpp`, que reúne dado e operação e passa a poder garantir coisas sobre si mesma. A métrica desta aula está aferida nos dois lados, e é o quarto trecho extraído que a afirma: **sete maneiras de errar em C, uma em C++17**. As seis que somem não somem por disciplina de quem programa, e sim porque o compilador passou a impedi-las.

O quinto trecho mostra uma dessas seis de perto. A dimensão, que em C é um campo público que qualquer módulo reescreve, virou invariante: `largura()` devolve por valor, não existe função que a altere, e o próprio teste afirma isso com um `static_assert` sobre `std::is_assignable_v`. A garantia deixou de ser lembrança e passou a ser compilação, que é a diferença que o capítulo inteiro defende.

— DICA —

Observe, nos trechos extraídos, o que a versão em C++ não precisa escrever: (1) os dados são privados, e ninguém escreve numa dimensão inválida de fora; (2) o construtor estabelece a invariante, e a validação acontece na lista de inicialização, antes de o corpo rodar; (3) não há ponteiro cru, não há inicialização à mão, e não há liberação - o `std::vector` cuida das três coisas.

§1.5 Tabela Comparativa: C Procedural vs. C++ OO

Aspecto	C Procedural	C++ Orientado a Objetos
Organização	<code>struct celula</code> + funções soltas	<code>class grade</code> reúne dados e operações
Inicialização	<code>grade_iniciar(&g, 80, 24)</code> - manual	Construtor automático: <code>grade g{80, 24}</code>
Proteção	Acesso direto a qualquer campo	<code>largura_</code> , <code>altura_</code> , <code>celulas_</code> <code>private</code>
Validação	Manual em cada função	Na lista de inicialização do construtor
Liberação	<code>free()</code> em cada saída de função	Destrutor, ou nenhum: o vetor cuida disso
Reutilização	Duplicação de código	Herança: <code>class drone : public entidade</code>
Strings	<code>char[]</code> + <code>strcpy/strcmp</code>	<code>std::string</code> e <code>std::string_view</code>

Coleções

Arrays de tamanho fixo

`std::vector<celula>` - dimensionado em execução

A linha da liberação é a que mais interessa por ora, e ela volta no Cap. 9: na versão em C, quem escreve a função é responsável por lembrar de liberar em toda saída possível, inclusive nas que ainda não existem; na versão em C++, se o único membro que gerencia recurso é um `std::vector`, não há o que lembrar, porque não há o que escrever.

Exercícios Propostos

1. Identifique três limitações reais da grade em C do §1.4, lendo-a no trecho extraído no fim deste capítulo. Para cada limitação, descreva um cenário concreto em que ela causaria um bug difícil de encontrar - e diga, em cada caso, se algum aviso do compilador o pegaria.
2. Reescreva a versão em C acrescentando uma função `celula_definir_energia()` que valide o valor antes de gravá-lo. Por que isso não resolve o problema de encapsulamento de forma satisfatória? O que ainda pode ser feito com a `struct` por quem não usar a sua função?
3. Escreva, em papel e sem compilar, a declaração da classe `grade` que você poria no lugar da `struct` em C: quais membros ficam privados, quais métodos são `const`, e qual método você não escreveria por não haver invariante a proteger. Guarde a folha: no Cap. 7 você compara a sua declaração com a de `include/deriva/grade.hpp`.
4. Pesquise e descreva com suas palavras como o Python implementa encapsulamento (convenção de sublinhado) em comparação com C++ (modificadores de acesso). Qual abordagem oferece garantias mais fortes, e em que momento cada uma detecta a violação?

O código, extraído do Deriva

Todo trecho abaixo vem de `exemplos/deriva/`, que compila com `-std=c++17 -Wall -Wextra -Wpedantic` sem um aviso e passa `make verifica`. Nenhum foi digitado neste texto.

```
comparativo/grade_procedural.hpp:10 | a grade em estilo C
10 /// A grade da estação em estilo C: dado exposto, funções livres, e o
11 /// contrato inteiro na cabeça de quem chama.
12 ///
13 /// Isto não é caricatura. É a forma que o C permite e que o C++ herdou, e
14 /// funciona - com uma condição: que todo mundo lembre das regras. As regras
15 /// são cinco, e nenhuma delas está escrita no código.
16 ///
17 /// 1. chame 'criar' antes de qualquer outra coisa;
18 /// 2. chame 'destruir' exatamente uma vez, e nunca depois de copiar;
19 /// 3. não copie a struct - ela leva o ponteiro, e não os dados;
20 /// 4. não escreva em 'largura' nem 'altura' depois de criada;
21 /// 5. confira o limite antes de indexar, porque ninguém confere por você.
22 struct grade_c {
23     int largura;
24     int altura;
25     char* celulas;
26 };
```

Cinco regras que só existem na cabeça de quem chama, e nenhuma delas está escrita no código. A métrica da Aula 01 não é elegância: é quantas maneiras de errar o desenho permite - sete aqui, uma na versão OO.

```
testes/test_comparativo.cpp:21 | quebrado de propósito - a cópia que não copia
21 TEST_CASE("em C, a copia da struct compartilha o buffer, e ninguem avisa") {
22     grade_c a = criar(4, 3);
23     escrever(&a, 0, 0, '@');
24
25     grade_c b = a; // copia a struct: leva o PONTEIRO
26     escrever(&b, 0, 0, '#');
27
28     REQUIRE(em(&a, 0, 0) == '#'); // a "original" mudou
29     REQUIRE(a.celulas == b.celulas); // porque e o mesmo buffer
30
31     destruir(&a);
32     // destruir(&b) aqui seria a segunda liberacao. Nao a chamamos - e o fato de
33     // termos de LEMBRAR disso e o defeito.
34 }
```

Copiar a struct leva o ponteiro, não os dados, e ninguém avisa. A versão C++ faz a cópia profunda pela regra do zero, sem escrever uma linha - e é essa diferença, e não a sintaxe, que o capítulo defende.

```
└─|▲ testes/test_comparativo.cpp:63 |───────────────── quebrado de propósito · o vazamento que passa no teste ─────────┐
63 TEST_CASE("em C, esquecer destruir nao produz sintoma nenhum hoje") {
64     {
65         grade_c g = criar(100, 100);
66         escrever(&g, 0, 0, '@');
67         // sem `destruir(&g)`: 10 KB vazam, e o programa termina bem
68     }
└────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────┘
```

Dez kilobytes vazam e o teste passa, porque o vazamento não tem sintoma. É o argumento inteiro do contador de instâncias da Aula 07.

```
└─| testes/test_comparativo.cpp:13 |───────────────── sete contra uma ─────────┐
13 // A metrica da Aula 01 nao e elegancia: e quantas maneiras de errar o desenho
14 // permite. Sete contra uma, e as seis que somem nao somem por disciplina do
15 // programador - somem porque o compilador passou a impedi-las.
16 TEST_CASE("a versao 00 fecha seis das sete maneiras de errar") {
17     REQUIRE(maneiras_de_errar_em_c() == 7);
18     REQUIRE(maneiras_de_errar_em_cpp() == 1);
19 }
└────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────┘
```

As seis maneiras de errar que somem não somem por disciplina de quem escreve: somem porque o compilador passou a impedi-las.

```
└─| testes/test_comparativo.cpp:55 |───────────────── o campo público virou invariante ─────────┐
55 TEST_CASE("em C++, a dimensao e invariante, e o tipo diz isso") {
56     const grade g(4, 3);
57     REQUIRE(g.largura() == 4);
58     // `g.largura() = 2;` nao compila: o retorno e por valor, e nao ha setter.
59     static_assert(!std::is_assignable_v<decltype(g.largura()), int>);
60     SUCCEED("a invariante e garantida em compilacao, nao por lembranca");
└────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────┘
```

Em C, largura é pública e mexer nela corrompe a indexação. Aqui a garantia é de compilação, e não de lembrança.

Infraestrutura do programador C++

O QUE ESTE CAPÍTULO ENTREGA

- ▶ Compilar programas C++ com GCC/Clang usando os flags corretos para a disciplina.
- ▶ Criar projetos organizados com CMake e trazer dependência externa com FetchContent.
- ▶ Executar o portão `make` e saber o que cada uma das suas quatro condições afirma.
- ▶ Depurar programas com GDB, inclusive com ponto de parada em destrutor.
- ▶ Reconhecer o lugar dos sanitizers: ferramenta de confirmação, e não critério de aceitação.

§2.1 O Modelo de Compilação de C++

C++ é compilado direto para código de máquina, e não interpretado como Python nem traduzido para o bytecode de uma máquina virtual como Java. Entre o texto que você escreve e o `./build/deriva` que roda existem três etapas: o pré-processamento, que expande `#include` e `#define` e entrega ao compilador uma unidade de tradução única; a compilação, que transforma cada unidade num objeto `.o`; e a ligação, ou linking, que junta `main.o`, `grade.o`, `mapa.o` e o resto num executável.

Saber em qual das três etapas um erro aconteceu economiza mais tempo de depuração que qualquer outra coisa deste capítulo. Símbolo não declarado é erro de compilação; símbolo declarado e não definido é erro de ligação, e a mensagem não se parece nada com a primeira.

A linha de compilação da disciplina não é digitada à mão em cada exercício: ela é o que o CMake gera a partir de `exemplos/deriva/CMakeLists.txt`, e o conjunto de avisos que ela carrega está extraído no fim deste capítulo, em `CMakeLists.txt` · o portão de warning. Duas coisas ali valem ser conferidas antes de seguir.

A primeira é que o conjunto é maior que `-Wall -Wextra -Wpedantic`, e a tabela abaixo traz os que mais aparecem no Deriva. Os quatro últimos existem por experiência acumulada: `-Wconversion` e `-Wsign-conversion` foram os que apontaram, em `terminal_bruto`, um `int` negativo sendo atribuído a um campo `unsigned` e funcionando por acidente do complemento de dois.

A segunda é que não há `-fsanitize` na linha de desenvolvimento. Os sanitizers são ferramenta do §2.6, ficam atrás da opção `DERIVA_SANITIZERS`, desligada por padrão, e as máquinas do laboratório não os têm; pôr um deles no conjunto padrão faria a compilação de todo exercício depender de algo que só existe na máquina de quem escreveu o material.

Flag	Significado
-std=c++17	Usa o padrão C++17 (obrigatório na disciplina)
-Wall	Ativa a maioria dos warnings úteis
-Wextra	Ativa warnings adicionais além de -Wall
-Wpedantic	Exige conformidade estrita com o padrão
-g	Inclui informações de depuração (necessário para GDB)
-O0	Sem otimização - melhor para depuração
-O2	Otimização padrão para release
-Wshadow	Variável local que esconde outra de escopo externo
-Wnon-virtual-dtor	Classe polimórfica com destrutor não virtual (Cap. 11)
-Wold-style-cast	Cast à moda de C, onde cabia static_cast
-Wconversion	Conversão numérica que pode perder valor
-Wsign-conversion	Conversão que troca o sinal - a que quebrou o terminal_bruto
-fsanitize=address	AddressSanitizer: ferramenta do §2.6, não flag padrão
-fsanitize=undefined	UndefinedBehaviorSanitizer: idem

▲ ATENÇÃO

NUNCA entregue código com warnings não resolvidos. -Wall -Wextra -Wpedantic são seus melhores aliados. Um warning silenciado é uma bug esperando acontecer.

§2.2 CMake - O Sistema de Build da Disciplina

O CMake não compila nada: ele gera o que compila. A partir da descrição em CMakeLists.txt, ele produz os arquivos de build da sua máquina - os Makefiles, o projeto do Ninja, o do Visual Studio -, já com o padrão da linguagem fixado e com o conjunto de avisos do §2.1 embutido em cada linha de compilação. É por isso que o Deriva se opera em dois comandos, e não em uma linha de g++ por arquivo, e é por isso que a mesma descrição serve à sua máquina e à do laboratório.

— Estrutura mínima de projeto

A árvore mínima separa o cabeçalho da implementação, e include/ é o diretório que entra no caminho de inclusão do alvo, de forma que o cabeçalho seja escrito como "deriva/grade.hpp" a partir de qualquer unidade de tradução:

```
1 meu_projeto/  
2 |— CMakeLists.txt  
3 |— src/  
4 |   |— main.cpp  
5 |   |— grade.cpp  
6 |— include/  
7 |   |— deriva/  
8 |       |— grade.hpp
```

O cabeçalho real do projeto está extraído no fim deste capítulo, em `CMakeLists.txt` · o topo, com as quatro opções, e é ele que serve de gabarito para o seu: `cmake_minimum_required(VERSION 3.16)`, o padrão fixado nas três linhas de `CMAKE_CXX_STANDARD`, `STANDARD_REQUIRED` e `EXTENSIONS OFF`, o tipo de build padrão para Debug quando ninguém pede outro, e as quatro opções do projeto. Repare em duas escolhas ali, porque as duas contrariam o que se copia da internet.

A versão mínima do CMake é **3.16**, e não a mais recente. Exigir uma versão que o laboratório não tem transforma a primeira linha do arquivo num impedimento; 3.16 é o que há, e é o que basta para `FetchContent` e para as propriedades de alvo que o Deriva usa.

E os sanitizers **não** entram como opção de Debug. Eles ficam atrás de `DERIVA_SANITIZERS`, desligada por padrão, pelo motivo do §2.6: as máquinas do laboratório não os têm, e uma linha que os ligue em toda compilação de depuração faz a sua entrega passar a depender de algo que só existe na máquina de quem escreveu o material.

Os quatro comandos com que se opera o projeto, depois de o arquivo existir:

```
1 cmake -S . -B build -DCMAKE_BUILD_TYPE=Debug    # configura  
2 cmake --build build --parallel                  # compila  
3 ./build/deriva --render                          # executa  
4 make verifica                                   # o portão
```

De `CMAKE_CXX_EXTENSIONS OFF` depende uma afirmação que o resto do livro faz sem qualificar: que o alvo é C++17. Sem essa linha, o CMake compila com `-std=gnu++17`, e código que usa extensão GNU passa a compilar na sua máquina sem compilar no compilador de outro. A linha custa nada e fecha a porta.

§2.3 FetchContent - dependência externa sem submódulo

O Deriva usa duas bibliotecas de terceiro: FTXUI, para a camada de terminal, e Catch2, para os testes. Nenhuma das duas é instalada no laboratório, e pedir que cada estudante as instale à mão seria transformar a primeira aula em suporte técnico. A saída é o módulo FetchContent do CMake, que baixa e configura a dependência durante a etapa de configuração do seu próprio projeto.

Duas decisões de projeto acompanham cada FetchContent_Declare, e as duas estão no código extraído no fim deste capítulo.

A primeira é que **a tag não flutua**. GIT_TAG v5.0.0 é uma tag imutável, e não um nome de branch: a FTXUI v5.0.0 declara cxx_std_17, e as versões seguintes podem elevar o padrão exigido e derrubar o alvo da disciplina de uma semana para a outra. O mesmo vale para o Catch2, fixo em v3.5.2. Dependência sem versão fixa é o que faz um projeto que compilava em abril parar de compilar em maio sem que ninguém tenha tocado numa linha.

A segunda é a palavra SYSTEM. Ela diz ao CMake que os cabeçalhos daquela dependência são de sistema, e o compilador deixa de emitir aviso sobre eles. Isso importa porque o portão desta disciplina é **zero aviso**, e o portão precisa incidir sobre o seu código, e só. Sem SYSTEM, uma versão nova da FTXUI que introduzisse um aviso interno reprovaria a sua entrega por algo que você não escreveu; e, na direção oposta, um projeto cheio de avisos de terceiro é um projeto em que o seu aviso não aparece. É por isso que, no Deriva, target_compile_options com a lista longa de -W... está em deriva_nucleo, e não numa opção global do projeto.

0 avisos

avisos do compilador

aferido por

make verifica

Makefile

§2.4 O portão make verifica

A entrega de todo exercício e de todo laboratório desta disciplina passa por um único comando, e ele afirma quatro coisas independentes:

Condição	O que roda	O que ela afirma
(1/4) warning	cmake --build	Zero aviso no código do estudante, com o conjunto completo de -W
(2/4) ctest	ctest --test-dir build	188 testes verdes na v2.7
(3/4) replay	deriva --replay roteiro.txt --semente 7	O despejo de estado é idêntico byte a byte ao de referência
(4/4) contadores	deriva --replay ... --contadores	vivos=0 no fim: nenhum destrutor deixou de rodar

As quatro são objetivas, e nenhuma delas depende de sanitizer nem de Valgrind. Isso é deliberado: as máquinas do laboratório não têm nenhum dos dois, e um portão que só passa na máquina de quem escreveu o material não é portão.

A terceira condição é a que costuma surpreender. **Replay** é rodar o Deriva com uma semente de sorteio fixa e um roteiro gravado de comandos, e comparar o despejo de estado resultante, byte a byte, com um arquivo de referência. Enquanto o comportamento observável não muda, o despejo não muda; assim, a saída do programa passa a funcionar como oráculo de refatoração. É esse mecanismo que torna objetiva a afirmação, que só chega no Cap. 24, de que refatoração correta é a que não altera o comportamento observável.

A quarta é a que fecha o laço com o Cap. 7. `vivos` é um contador de instâncias vivas escrito à mão em cada classe de domínio, incrementado no construtor e decrementado no destrutor. Sistema que passa nos testes mas termina com objeto vivo não passa no portão, porque objeto vivo no fim de `main` significa destrutor que não rodou.

✓ DICA

Rode `make` verifica antes de cada commit, e não só antes de entregar. As quatro condições levam segundos, e a que falha diz qual das quatro é. Descobrir que o replay divergiu vinte commits depois custa a tarde inteira de bissecção.

§2.5 GDB - depuração básica, e o ponto de parada em destrutor

GDB (GNU Debugger) permite executar o programa passo a passo, inspecionar variáveis e encontrar onde e por que o programa trava ou produz resultados errados. Compile sempre com `-g` para usar o GDB.

Comando GDB	Ação
<code>gdb ./programa</code>	Inicia o GDB com o programa
<code>run</code>	Executa o programa
<code>break main</code>	Define breakpoint na função <code>main</code>
<code>break grade.cpp:42</code>	Define breakpoint na linha 42 do arquivo
<code>next (n)</code>	Avança uma linha (sem entrar em funções)
<code>step (s)</code>	Avança uma linha (entra em funções)
<code>print variavel</code>	Exibe o valor de uma variável
<code>backtrace (bt)</code>	Mostra a pilha de chamadas (onde estamos)
<code>quit (q)</code>	Sai do GDB

O uso do GDB que esta disciplina cobra, porém, não é nenhum desses. É um só, e ele responde a uma pergunta que nenhuma outra ferramenta disponível no laboratório responde: **este destrutor rodou, e quem o chamou?**

Comando GDB	Ação
<code>break deriva::mapa::~~mapa</code>	Para na entrada do destrutor de <code>mapa</code> . O nome é qualificado pelo namespace, e o <code>~</code> faz parte dele
<code>info locals</code>	Mostra o estado do objeto no momento da destruição
<code>bt</code>	Diz de onde a destruição foi chamada: fim de escopo, <code>delete</code> , ou desenrolar de pilha por exceção
<code>continue (c)</code>	Segue até o próximo destrutor, e conta quantos são
<code>make gdb-dtor</code>	Roda a sequência acima em lote, sem interação

A sequência inteira da tabela acima roda em lote, sem interação, pelo alvo `gdb-dtor` do Makefile, extraído no fim deste capítulo: é o único lugar do material em que ela aparece verbatim, e vale compará-la com a tabela linha por linha.

A razão de o ponto de parada em destrutor ganhar seção própria é que ele é a terceira das três técnicas de verificação de posse que a disciplina usa no lugar do sanitizer que não tem. As outras duas são o contador de instâncias vivas (Cap. 7) e a

instrumentação de ciclo de vida (Cap. 8). O contador diz *que* faltou um destrutor; a instrumentação diz *em que ordem* os destrutores rodaram; o gdb diz *qual* destrutor rodou e de onde foi chamado. No Cap. 11, quando a ausência de destrutor virtual fizer o destrutor da derivada nunca ser chamado, é este ponto de parada que mostra a diferença: você põe a parada no destrutor da derivada e ele simplesmente não é alcançado.

§2.6 Sanitizers - confirmação, não oráculo

Os sanitizers são instrumentações que o compilador acrescenta ao programa para detectar classes de bug em tempo de execução. O AddressSanitizer (ASan) detecta uso de memória após liberação, leitura e escrita fora dos limites, liberação dupla e vazamento; o UndefinedBehaviorSanitizer (UBSan) detecta overflow de inteiro com sinal, desreferência de ponteiro nulo e deslocamento bit a bit inválido.

Eles são excelentes, e não são o portão desta disciplina. As máquinas do laboratório não os têm, e o material foi reescrito com essa restrição no centro em vez de na nota de rodapé: no Deriva, os sanitizers ficam atrás da opção `DERIVA_SANITIZERS`, desligada por padrão, e se rodam por `make sanitizers`, que é alvo separado de `make verifica`.

A ordem de uso importa, e é ela que este livro cobra. Diante de um vazamento, você primeiro **reproduz e explica** a falha com o contador que construiu, e só depois vê a ferramenta apontar a alocação que você já havia identificado. O caminho inverso ensina a operar a ferramenta, não a raciocinar sobre posse - e na prova, que é em papel, não há ferramenta.

— O que o ASan diz, e o que ele não diz

Os dois defeitos que os sanitizers pegam melhor - acesso fora de limite e estouro de inteiro com sinal - **não existem em exemplos/deriva/**, e a ausência é consequência do portão do §2.1, e não descuido: com `-Wconversion` e `-Wsign-conversion` ligados e zero aviso como critério, a conversão que produziria o estouro não passa da compilação, e todo acesso à grade é precedido de `dentro()` ou da pré-condição escrita no cabeçalho. Os dois exemplos abaixo são, por isso, mínimos e de fora do projeto: eles existem para que você reconheça o relatório da ferramenta, e não para representar código do Deriva.

O primeiro é de escrita fora do bloco alocado, e é o ASan que o acusa:

```
1 void escrever_fora_do_limite() {
2     int* p = new int[5];
3     p[10] = 42;          // escrita fora do bloco: o ASan aborta com relatório
4     delete[] p;
5 }
```

O segundo é de estouro de inteiro com sinal, que é comportamento indefinido, e é o UBSan que o relata:

```

1 #include <climits>
2
3 int somar_um_ao_maximo() {
4     int a = INT_MAX;
5     return a + 1;    // estouro com sinal: é aqui que o UBSan interrompe
6 }

```

O limite dos sanitizers aparece na primeira variante quebrada do Deriva, e vale conhecê-lo desde já. A v0.2-quebrada omite o destrutor de `terminal_bruto`, e com isso vaza o modo do terminal - um recurso do sistema operacional que sobrevive ao processo. O ASan não conhece termos e não diz uma palavra. O contador de instâncias vivas acusa um objeto sem destrutor, e é a única pista automática que existe para esse caso.

✓ DICA

Se a sua máquina tem sanitizer, use: `make sanitizers` no Deriva, ou `-fsanitize=address, undefined -g` na linha de `g++`. O custo de desempenho (cerca de 2x) é irrelevante em ambiente de aprendizado. Só não confunda “o ASan não reclamou” com “o programa está correto”: ele não vê recurso que não seja memória, e a metade dos defeitos deste livro é dessa espécie.

§2.7 Transição de Python/C para C++

Conceito	Python	C	C++
Variável	<code>x = 42</code> (dinâmico)	<code>int x = 42;</code>	<code>int x = 42;</code> ou <code>auto x = 42;</code>
String	<code>str</code> - imutável	<code>char[]</code> - manual	<code>std::string</code> - automático
Lista/Array	<code>list</code>	array estático	<code>std::vector<T></code>
Dicionário	<code>dict</code>	- (implementação manual)	<code>std::map<K,V></code> ou <code>std::unordered_map<K,V></code>
Função	<code>def f(x):</code>	tipo <code>f(tipo x)</code>	tipo <code>f(tipo x)</code> ou <code>auto f(auto x)</code>
Classe	<code>class C:</code>	<code>struct</code> (sem métodos)	<code>class C { ... };</code>
Importar	<code>import módulo</code>	<code>#include <header.h></code>	<code>#include <header.hpp></code>
Executar	<code>python prog.py</code>	<code>gcc ... && ./prog</code>	<code>g++ ... && ./prog</code> (ou <code>cmake</code>)

Exercícios Propostos

1. Crie um projeto CMake mínimo com um `main.cpp` que imprime “Deriva v0.0 ativo”. Use `CMAKE_CXX_STANDARD 17`, `CMAKE_CXX_STANDARD_REQUIRED ON` e `CMAKE_CXX_EXTENSIONS OFF`, e o conjunto de avisos da tabela do §2.1. Compile, execute, e confirme que a compilação não emite uma linha de aviso. Este é o portão do **LAB-01**.
2. Acrescente ao projeto do exercício 1 o Catch2 por FetchContent, com tag fixa e SYSTEM, e um único teste de fumaça. Depois **remova** a palavra SYSTEM e recompile. O que muda na saída do compilador? Explique por que o portão de zero aviso da disciplina depende dessa palavra.
3. Use o GDB para responder, no Deriva já compilado, quantas vezes o destrutor de mapa é chamado numa execução de `./build/deriva --render`, e de onde cada chamada vem. Ponha o ponto de parada com `break deriva::mapa::~~mapa`, use `bt` a cada parada, e compare o número que você obteve com o que `--contadores` imprime.
4. Escreva um `CMakeLists.txt` para um projeto com dois arquivos de origem, um diretório de includes, uma dependência trazida por FetchContent como SYSTEM e uma opção desligada por padrão que ligue os sanitizers. Justifique, em duas linhas no próprio arquivo, por que a opção é desligada por padrão.

O código, extraído do Deriva

Todo trecho abaixo vem de `exemplos/deriva/`, que compila com `-std=c++17 -Wall -Wextra -Wpedantic` sem um aviso e passa `make verifica`. Nenhum foi digitado neste texto.


```

CMakeLists.txt:93 |----- FTXUI com tag fixa -----
93 FetchContent_Declare(ftxui
94   GIT_REPOSITORY https://github.com/ArthurSonzogni/FTXUI.git
95   GIT_TAG v5.0.0
96   GIT_SHALLOW TRUE
97   SYSTEM
98 )

```

A tag não flutua. A v5.0.0 declara `cxx_std_17`; as v6/v7 podem elevar o padrão exigido e derrubar o alvo da disciplina.

```

Makefile:71 |----- gdb com ponto de parada em destrutor -----
71 # Aula 08 e 11: provar que o destrutor roda - e onde ele não roda.
72 gdb-dtor: compila
73 @gdb -batch -ex 'break deriva::mapa::~~mapa' -ex run -ex 'info locals' -ex bt \
74     -ex 'continue' --args ./$(BUILD)/deriva --render 2>&1 | head -30

```

A terceira das três técnicas que substituem o sanitizer ausente. É ela que mostra, na Aula 11, o destrutor da derivada nunca sendo alcançado quando se deleta por ponteiro da base.

```

▲ sanitizers/defeitos_de_memoria.cpp:57 |----- quebrado de propósito · sem 'dentro()' ao lado -----
57 // DEFEITO 1 · acesso fora de limite.
58 //
59 // Sem 'dentro()' ao lado, esta função aceita qualquer posição. 0
60 // 'operator[]' do vector não confere - 'at()' conferiria, e é a escolha que
61 // a versão boa deixa explícita.
62 [[nodiscard]] celular& em(int x, int y) {
63   return celulas_[static_cast<std::size_t>(y) * static_cast<std::size_t>(largura_) +
64                   static_cast<std::size_t>(x)];
65 }

```

O `operator[]` do vector não confere limite - `at()` conferiria, e é a escolha que a versão boa deixa explícita para quem chama. O ASan aponta a linha da escrita E a linha da alocação, que é a informação que faltava.

```

└─▲ sanitizers/defeitos_de_memoria.cpp:99 ─┬───────────────── quebrado de propósito · o estouro que não avisa
99 // As dimensões vêm de FORA - de linha de comando, como vêm de um arquivo
100 // de mapa no Deriva de verdade. E é essa a diferença que importa.
101 //
102 // Escrito com literais - `const int p = 50000 * 50000;` - o g++ dobra a
103 // conta em tempo de compilação, vê o estouro e avisa: `-Woverflow`,
104 // "integer overflow in expression of type 'int' results in
105 // '-1794967296'". O defeito não embarca.
106 //
107 // Com os valores vindo de fora, o compilador não tem o que dobrar, o aviso
108 // desaparece, e o estouro passa a acontecer em execução. É assim que ele
109 // chega em produção: não com número escrito no código, mas com número
110 // lido de um arquivo que alguém editou.
111 const int largura = std::atoi("50000");
112 const int altura = std::atoi("50000");
113 const int produto = largura * altura;          // <-- o estouro, sem aviso
114 std::printf("  largura x altura ... %d <-- deveria ser 2500000000\n", produto);
115 std::printf("  como size_t ..... %zu\n", static_cast<std::size_t>(produto));
116 std::puts("  a conta acontece em int, e nenhum static_cast depois a conserta.");
117 std::puts("  a versao boa converte CADA fator ANTES de multiplicar.");

```

Com literais, o g++ dobra a conta, vê o estouro e avisa por -Woverflow. Com os valores vindo de fora ele não tem o que dobrar, o aviso desaparece, e o estouro passa a acontecer em execução. O compilador pega o que consegue ver, e é por isso que o defeito real nunca chega com número escrito no código.

```

└─CMakeLists.txt:2 ─┬───────────────── o topo, com as quatro opções
2 cmake_minimum_required(VERSION 3.16)
3 project(deriva LANGUAGES CXX VERSION 0.3.0)
4
5 # C++17 é o teto e o foco da disciplina. `CXX_EXTENSIONS OFF` desliga as
6 # extensões GNU: código que só compila com -std=gnu++17 não é C++17.
7 set(CMAKE_CXX_STANDARD 17)
8 set(CMAKE_CXX_STANDARD_REQUIRED ON)
9 set(CMAKE_CXX_EXTENSIONS OFF)
10
11 if(NOT CMAKE_BUILD_TYPE AND NOT CMAKE_CONFIGURATION_TYPES)
12   set(CMAKE_BUILD_TYPE Debug CACHE STRING "" FORCE)
13 endif()
14
15 option(DERIVA_SANITIZERS "Compilar com ASan e UBSan" OFF)
16 option(DERIVA_TESTES     "Compilar os testes (baixa o Catch2)" ON)
17 option(DERIVA_COM_FTXUI  "Camada de terminal com FTXUI v5.0.0" OFF)
18 option(DERIVA_COM_QT     "Segundo front-end em Qt6 (Aula 26)" OFF)

```

CXX_EXTENSIONS OFF é o que recusa -std=gnu++17: código que só compila com extensão não é C++17. E DERIVA_SANITIZERS nasce desligada, porque o laboratório não os tem.

Fundamentos de C++17 para POO

O QUE ESTE CAPÍTULO ENTREGA

- ▶ Dominar a entrada e saída com fluxos (`std::cin` e `std::cout`) e as suas diferenças em relação a `printf/scanf`.
- ▶ Utilizar `std::string` no lugar de arranjos de `char`.
- ▶ Usar `std::string_view` onde ela cabe, e reconhecer a armadilha de tempo de vida.
- ▶ Compreender ponteiro e referência, e saber o que a escolha entre os dois declara na assinatura.
- ▶ Aplicar `auto`, `nullptr` e inicialização uniforme sobre os tipos da v0.1 do Deriva.
- ▶ Ler e escrever ligações estruturadas, e anotar código com `[[nodiscard]]` e `[[maybe_unused]]`.

O Capítulo 2 deixou um esqueleto que compila: a v0.0 do Deriva, com CMake, as duas dependências trazidas por FetchContent, um teste de fumaça e o portão `make verifica`. Este capítulo não acrescenta versão à trilha; ele instala o vocabulário de C++17 com que as versões seguintes são escritas, e cada construção aparece aqui no lugar exato em que o Deriva a usa depois - `vetor2` e `celula` na v0.1, `grade` na v0.2, `mapa` e o carregamento na v0.3.

§3.1 Entrada e saída: de `printf/scanf` a `cout/cin`

A transição de C para C++ é imediatamente visível no sistema de entrada e saída. Em C, `printf()` e `scanf()` recebem especificadores de formato (`%d`, `%f`, `%s`) que precisam corresponder aos argumentos, e a correspondência não faz parte do tipo da função: `printf` é variádica, e o que ela recebe depois da cadeia de formato ela não sabe. Em C++, `operator<<` e `operator>>` são sobrecarregados por tipo, de forma que o tipo do argumento é o que escolhe a função chamada.


```
1 // C: o especificador e o argumento são conferidos por fora do sistema de tipos
2 int n = 42;
3 printf("%f\n", n);          /* comportamento indefinido */
4
5 // C++: o tipo do argumento escolhe a sobrecarga de operator<<
6 std::cout << n << '\n';    // sempre correto
```

Uma qualificação, porque o livro anterior afirmava mais do que é verdade: o g++ **confere** essa primeira linha, através de -Wformat, que vem dentro de -Wall, e o conjunto de avisos do Capítulo 2 a reprova. O que ele não pode conferir é a chamada em que a cadeia de formato não é um literal visível - printf(fmt, n), com fmt vindo de uma variável, volta a ser indefinida em silêncio. A sobrecarga de operator<<, por não depender de o compilador ler uma cadeia de caracteres, não tem essa fronteira.

No Deriva toda a saída passa por std::cout, e desde a v2.6 através da interface i_apresentacao (Cap. 24). Duas escolhas dessa saída valem ser vistas agora, porque as duas se repetem em todas as vinte versões.

A primeira é que não há uma ocorrência de std::endl no núcleo, nos cabeçalhos ou na suíte de testes. std::endl escreve '\n' e descarrega o fluxo; o '\n' sozinho escreve o mesmo byte e deixa a descarga para quando ela for necessária. Num despejo feito fileira por fileira, std::endl seria uma descarga por linha, e nenhuma delas mudaria um byte do que sai.

A segunda é que os despejos não escrevem em std::cout: mapa::despejar() e mundo::despejar() devolvem std::string, e é main que a imprime numa inserção só. A razão é a condição 3 do portão, o replay byte a byte: um teste pode afirmar sobre a cadeia devolvida, e não pode afirmar sobre o que já foi para o terminal.

Na entrada, a assimetria entre >> e std::getline é a origem de um defeito clássico. >> para no primeiro caractere branco e **deixa** o resto da linha, o '\n' incluído, no fluxo; std::getline lê até o '\n' e o consome.

```
1 #include <iostream>
2 #include <limits>
3 #include <string>
4
5 int main() {
6     std::string setor;
7     int largura = 0;
8
9     std::cout << "setor: ";
10    std::getline(std::cin, setor); // a linha inteira, espaços incluídos
11
12    std::cout << "largura: ";
13    std::cin >> largura;           // para no primeiro branco, e deixa o '\n'
14    std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
15
16    std::cout << setor << ' ' << largura << '\n';
17    return 0;
18 }
```

Sem a linha do ignore, um segundo `std::getline` logo depois do `>>` devolveria cadeia vazia: ele encontra o `'\n'` residual no primeiro caractere e para ali.

▲ ATENÇÃO

Misturar `std::cin >>` com `std::getline` sem tratar o `'\n'` residual é o defeito de leitura mais comum da disciplina, e ele não produz erro nenhum: o `getline` devolve cadeia vazia e o programa segue. Use `std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n')` entre os dois.

O Deriva usa as duas formas, e em cada caso pelo motivo certo. `ler_roteiro`, em `src/main.cpp`, lê o roteiro de replay com `std::getline(arq, linha)`, porque um comando ocupa uma linha inteira e `arq >> linha` pararia no primeiro branco. Já o leitor de partida salva da v2.5 (Cap. 23) **mistura** os dois de propósito: ele lê a chave com `is >> chave` e, quando a chave é desconhecida, chama `std::getline(is, resto)` justamente para engolir o resto daquela linha e seguir para a próxima. O `'\n'` residual, que no parágrafo acima é armadilha, ali é o mecanismo.

§3.2 Strings: de `char[]` a `std::string`

Arranjo de `char` em C exige que o tamanho seja carregado à mão, ao lado de `strcpy`, `strcat` e do terminador, e é a origem histórica dos estouros de buffer. `std::string` administra a sua própria memória, cresce quando precisa, tem `operator+` e `operator==` e devolve o tamanho em tempo constante.

```
1 #include <iostream>
2 #include <string>
3
4 int main() {
5     const std::string setor = "eclusa";
6     const std::string lado = "norte";
7
8     const std::string nome = setor + "-" + lado;    // concatenação com +
9     std::cout << nome << '\n';                    // eclusa-norte
10    std::cout << nome.size() << '\n';               // 12
11    std::cout << nome.substr(7, 5) << '\n';         // norte
12
13    if (nome.find("norte") != std::string::npos)
14        std::cout << "setor do lado norte\n";
15
16    return 0;
17 }
```

`find` devolve `std::string::npos` quando não acha, e não `-1`: `npos` é o maior valor de `std::size_t`, tipo sem sinal, de forma que comparar o retorno com `-1` funciona por acidente da conversão e `-Wsign-compare` reclama. A comparação correta é com `std::string::npos`.

No Deriva, `std::string` é o tipo de tudo que um objeto **guarda** em texto: o nome_ de mapa (v0.3), o nome_ de item (v1.0) e o traço de ciclo de vida que instrumento acumula (v0.2). São todos casos em que os bytes precisam sobreviver ao fim da chamada que os trouxe, e possuir é a única forma de garantir isso.

`std::string` possui os seus bytes, e isso tem preço: toda cópia aloca. A função que só quer *ler* uma sequência de caracteres não precisa possuí-los, e é para esse caso que o C++17 traz `std::string_view`.

§3.3 `std::string_view` e a armadilha de tempo de vida

Uma `std::string_view` é uma vista sobre caracteres que **alguém mais** possui. Por dentro são dois valores, um ponteiro para o primeiro caractere e um tamanho, e nada mais - testes/test_string_view.cpp fixa isso num `static_assert` de `sizeof(std::string_view) == 16`, ao lado de outro que afirma que o tipo é trivialmente copiável. Construí-la a partir de uma `std::string` ou de um literal não aloca e não copia, e passá-la por valor custa o mesmo que passar dois inteiros de 64 bits.

O ganho aparece no parâmetro de função. Uma função que declara `const std::string&` obriga quem chama a ter uma `std::string`: passar um literal ou um pedaço de outra cadeia força a construção de uma temporária, com alocação. A mesma função declarada com `std::string_view` aceita literal, `std::string`, pedaço de `std::string` e buffer de caracteres, sem alocar em nenhum dos casos.

O carregamento de mapa do Deriva usa as duas coisas na mesma assinatura, e a assimetria é o conteúdo desta seção:

```
1 // include/deriva/mapa.hpp
2 [[nodiscard]] static std::optional<mapa> de_texto(std::string_view texto,
3                                                    std::string nome);
```

texto é uma **vista**: a função o percorre para achar as fileiras e nunca o guarda. Depois que ela retorna, ninguém dentro do mapa aponta para lá. nome é uma **std::string por valor**: ele é *armazenado* em nome_, e o objeto precisa possuir os bytes para sobreviver ao fim da chamada.

A regra prática é essa mesma: **string_view para o que você lê durante a chamada; tipo que possui para o que você guarda**. O trecho extraído no fim deste capítulo mostra o corpo da função, e nele as fileiras são std::string_view apontando para dentro de texto, o que só é seguro porque nada disso atravessa o return.

A hierarquia de entidades da v1.0 (Cap. 10) mostra o outro lado do mesmo contrato, com dois donos diferentes por trás do mesmo tipo de retorno:

```
1 // em `sonda`: o literal tem duração estática, e a vista nunca pendura
2 [[nodiscard]] std::string_view nome() const override { return "sonda"; }
3
4 // em `item`: `nome_` é um `std::string` membro, e a vista vale
5 // enquanto aquele item viver
6 [[nodiscard]] std::string_view nome() const override { return nome_; }
```

As duas sobrescritas satisfazem a mesma declaração da base, e o tempo de vida que cada uma promete é diferente. Nenhuma linha da assinatura diz qual é qual, e é dessa lacuna que a armadilha inteira nasce.

Como a vista não possui nada, ela não tem opinião sobre o tempo de vida do que aponta: quando o dono morre, a vista continua existindo, apontando para memória que já foi devolvida, e ler dali é comportamento indefinido. Quatro formas de cair nisso, em ordem crescente de sutileza:

1. **A temporária que morre no fim da instrução.** `std::string_view v = montar_nome();` onde `montar_nome()` devolve `std::string` por valor: a temporária é destruída no ponto e vírgula, e `v` já nasce pendurada.
2. **O membro que sobrevive ao argumento.** Uma classe que guarda `std::string_view nome_;` recebida no construtor passa a depender de o chamador manter a cadeia viva por todo o tempo de vida do objeto - dependência que não aparece na assinatura e que ninguém lembra seis meses depois. É exatamente o que `mapa` **não** faz: `nome_` é `std::string`, e testes/`test_string_view.cpp` prova a diferença destruindo o texto de origem e lendo o mapa depois.
3. **A vista devolvida por quem não possui.** Se `de_texto` devolvesse `string_view` para dentro do seu parâmetro `texto`, a vista sobreviveria ao dono sempre que o chamador tivesse passado uma temporária.
4. **A vista para dentro de um objeto que muda.** `mapa::nome()` devolve `std::string_view` para `nome_`, e isso é correto porque o mapa possui `nome_` e não o altera. A vista, porém, vale enquanto aquele mapa viver: guardá-la, destruir o mapa e usá-la depois é o caso 2 disfarçado.

▲ ATENÇÃO

Nenhum dos quatro casos acima emite aviso com o conjunto do Capítulo 2. `string_view` pendurada é, junto com a cópia rasa do Cap. 9, o defeito desta unidade que o compilador não pega - e por isso é o assunto do **LAB-02**, cujo portão exige que você reproduza a armadilha de tempo de vida e a explique por escrito antes de consertá-la.

`std::string_view` não termina em `'\0'`. Passá-la para uma função de C que espera `const char*` é erro, e o compilador não deixa: não existe conversão implícita de `string_view` para `const char*`, justamente porque ela não pode prometer o terminador. O caso extraído no fim do capítulo mede o estrago quando alguém contorna isso com `.data()`: sobre a vista de seis caracteres tirada de "eclusa-norte", `std::strlen(vista.data())` devolve doze, que é o tamanho do dono. Quando é preciso chamar API de C, construa uma `std::string` a partir da vista e use `c_str()` - e aí a alocação, que você tinha evitado, acontece no único lugar em que era realmente necessária.

§3.4 Ponteiro e referência

Ponteiro e referência são os dois mecanismos de C++ para acessar um objeto indiretamente, e a escolha entre eles é uma declaração de contrato: a referência promete que existe um objeto, o ponteiro admite que talvez não exista.

Característica	Ponteiro (T*)	Referência (T&)
Pode ser nulo?	Sim, <code>nullptr</code>	Não, designa sempre um objeto válido
Pode ser reassinalado?	Sim	Não, liga-se uma vez e não muda de alvo
Sintaxe de acesso	<code>p->campo</code> ou <code>(*p).campo</code>	<code>r.campo</code> , igual ao objeto
Aritmética?	Sim, <code>p++</code> , <code>p + n</code>	Não
Uso típico	resultado que pode ser ausência, posse por ponteiro inteligente	passagem de parâmetro, retorno de acesso a membro

A assinatura de `grade::em` é o exemplo canônico da coluna da direita:

```

1 // include/deriva/grade.hpp
2 [[nodiscard]] bool dentro(vetor2 p) const noexcept;
3
4 /// Sem verificação de limite: quem chama já perguntou `dentro()`.
5 [[nodiscard]] const celula& em(vetor2 p) const;
6 [[nodiscard]] celula& em(vetor2 p);

```

Três decisões numa declaração de três linhas. `vetor2` entra **por valor**, porque são oito bytes (`static_assert(sizeof(vetor2) == 8)`) e uma referência a ele custaria os mesmos oito, com uma indireção a mais. O retorno é **referência**, e não ponteiro, porque a célula existe: a pergunta sobre existência é `dentro()`, e ela é feita antes. E as duas sobrecargas existem porque uma `grade` constante tem de deixar ler a célula sem deixar escrever nela - uma sobrecarga só, devolvendo `celula&`, permitiria escrever através de uma `grade const`.

Onde a ausência é uma resposta possível, o Deriva volta ao ponteiro cru. `mundo::primeira_com(char glifo)` (v1.2, Cap. 12) procura a primeira entidade com determinado glifo e devolve `entidade*`, que é `nullptr` quando não há nenhuma. O tipo carrega a informação de que quem chama precisa testar, e o `nullptr` é o único jeito de dizer “não achei” sem inventar uma entidade vazia.

As três formas de passar um objeto para uma função, e o que cada uma promete:

```
1 #include <iostream>
2 #include <string>
3
4 // por valor: cópia; o que a função escreve não sai dela
5 void por_valor(std::string s) { s += "-copia"; }
6
7 // por referência: escreve no objeto de quem chamou
8 void por_referencia(std::string& s) { s += "-mudado"; }
9
10 // por referência const: lê sem copiar, e o tipo promete não escrever
11 void por_referencia_const(const std::string& s) { std::cout << s << '\n'; }
12
13 int main() {
14     std::string nome = "eclusa";
15
16     por_valor(nome);           // nome continua "eclusa"
17     por_referencia(nome);      // nome passa a "eclusa-mudado"
18     por_referencia_const(nome); // imprime, sem copiar
19
20     return 0;
21 }
```

✓ DICA

Regra de passagem de parâmetro em C++17, na ordem em que se decide: (1) tipo pequeno e copiável (int, char, vetor2, std::string_view) por valor; (2) objeto que a função vai modificar, por T&; (3) objeto só de leitura, por const T&, ou por std::string_view quando for cadeia de caracteres; (4) parâmetro que pode ser ausente, por T*; (5) objeto que a função vai **guardar**, por valor, e a função o move para dentro do membro - que é o caso do nome de mapa::de_texto.

§3.5 C++ moderno essencial

— auto - dedução de tipo

auto instrui o compilador a deduzir o tipo a partir da expressão de inicialização. O tipo continua sendo conferido em tempo de compilação; o que desaparece é a obrigação de escrevê-lo.

```
1 auto turno = 0; // int
2 auto energia = 100.0; // double
3 auto setor = std::string{"eclusa"}; // std::string
```

Onde ele paga por si é no tipo que ninguém quer digitar - o iterador, e o resultado de um algoritmo da STL:

```
1 #include <vector>
2
3 int massa_total(const std::vector<int>& massas) {
4     auto total = 0; // int, deduzido do 0
5
6     // sem auto: std::vector<int>::const_iterator it = massas.begin();
7     for (auto it = massas.begin(); it != massas.end(); ++it) total += *it;
8
9     for (auto m : massas) total += m; // cópia de cada elemento
10    for (const auto& m : massas) total += m; // sem cópia, e sem escrita
11
12    return total;
13 }
```

A forma `for (const auto& x : v)` é a que o Deriva usa em toda travessia de contêiner, e o motivo é a linha do meio: `for (auto m : v)` copia cada elemento. Sobre `std::vector<int>` a cópia é um inteiro e não se sente; sobre `std::vector<std::unique_ptr<entidade>>` ela nem compila, porque `unique_ptr` não se copia, e é o compilador quem denuncia o descuido. Sobre um contêiner de mapa, que compõe uma grade com um `std::vector<celula>` dentro, a cópia é silenciosa, alocada e por volta do laço.

Em `mundo::primeira_com`, `const auto it = std::find_if(...)` guarda um iterador cujo tipo escrito por extenso ocuparia uma linha inteira e não diria mais do que `auto` diz.

— nullptr - ponteiro nulo com tipo próprio

`nullptr` é um literal de tipo `std::nullptr_t`. `NULL` é uma macro que expande para uma constante inteira, e em presença de sobrecarga isso muda qual função é chamada.


```
1 #include <iostream>
2
3 void f(int n) { std::cout << "inteiro " << n << '\n'; }
4 void f(int* p) { std::cout << "ponteiro " << (p == nullptr) << '\n'; }
5
6 int main() {
7     // f(NULL);      // o compilador recusa: a chamada é ambígua
8     f(nullptr);      // sem ambiguidade: std::nullptr_t só casa com f(int*)
9     return 0;
10 }
```

Descomente a linha do NULL e o g++ responde error: call of overloaded 'f(NULL)' is ambiguous, listando as duas candidatas logo abaixo. nullptr tem tipo próprio, que não converte para inteiro, e a ambiguidade deixa de existir.

No Deriva o nullptr aparece nos dois lugares em que ausência é resposta: no return nullptr de mundo::primeira_com, quando nenhuma entidade tem o glifo pedido, e no return nullptr de mundo::retirar_de, quando não há ninguém na posição - ali o nullptr constrói um std::unique_ptr<entidade> vazio, e quem chamou recebe um dono de nada.

— Inicialização uniforme com {}

O C++11 trouxe a sintaxe de chaves para inicializar qualquer objeto pela mesma forma, e o efeito que interessa aqui é o segundo: a inicialização com chaves **proíbe** conversão que estreita o valor, em tempo de compilação.

```
1 #include <string>
2 #include <vector>
3
4 struct ponto { int x = 0; int y = 0; };
5
6 int main() {
7     const int a{42}; // ok
8     // const int b{3.7}; // ERRO: estreitamento de double para int
9     const std::vector<int> v{1, 2, 3, 4, 5}; // lista de inicializadores
10    const std::string s{"eclusa"};
11    const ponto p{10, 20}; // agregado, membro por membro
12
13    return a + v.front() + static_cast<int>(s.size()) + p.x + p.y;
14 }
```

► DERIVA

vetor2 é um agregado de dois int com inicializador de membro em cada um, e por isso aceita as três formas: `vetor2{} dá {0, 0}`, `vetor2 p{10, 20}` preenche na ordem de declaração, e `vetor2 d{s.ate(g.largura()), s.ate(g.altura())}` é como `src/main.cpp` semeia item na grade. A proibição de estreitamento é o que impede alguém de escrever `vetor2{10.5, 3}` e receber uma posição truncada em silêncio. Em célula, o mesmo mecanismo aparece nos inicializadores de membro - `int energia = 0, char glifo = '.'` -, que é o que faz uma célula recém-construída ser piso vazio e não lixo de pilha.

§3.6 Ligações estruturadas

Uma **ligação estruturada** dá nome a cada pedaço de um objeto agregado, numa única declaração. Onde antes se escrevia `it->first` e `it->second`, ou se guardava um `std::pair` e se acessava `.first` mais adiante, o C++17 permite declarar os dois nomes de uma vez:

```
1 const std::pair<std::string_view, deriva::vetor2> par{"norte", {0, -1}};
2
3 const auto [nome, delta] = par;    // dois nomes numa só declaração
```

A forma serve a três espécies de objeto: `std::pair` e `std::tuple`; um `struct` cujos membros de dados são todos públicos, tais como `vetor2`, que se decompõe em `auto [x, y] = pos;`; e um arranjo de tamanho conhecido. Não serve a classe com membro privado, e é bom que não sirva: decompor mapa nos seus quatro campos por trás dos métodos de acesso seria furar o encapsulamento pela sintaxe.

O ganho maior não é a economia de digitação, é o nome. Um laço que percorre uma tabela de comandos e trabalha com `par.first` e `par.second` obriga o leitor a subir até a declaração para descobrir o que cada metade significa; o mesmo laço escrito com `[nome, delta]` diz. O trecho extraído no fim deste capítulo mostra esse laço, sobre a tabela que traduz o comando digitado no roteiro de replay em deslocamento na grade.

Duas advertências, e as duas aparecem no trecho:

A primeira é sobre a **forma de captura**. `auto [a, b]` copia o elemento; `auto& [a, b]` liga por referência e permite escrever; `const auto& [a, b]` liga por referência e proíbe escrever. Num laço sobre um contêiner de objetos grandes, escrever `auto` em vez de `const auto&` significa copiar o contêiner inteiro, um elemento por volta, sem que nada no código pareça uma cópia. É a mesma armadilha do `for (auto x : v)` do §3.5, agora com dois nomes em vez de um.

A segunda é que **os nomes não são variáveis independentes**. Eles são nomes para os subobjetos do mesmo objeto oculto que a declaração criou; assim, não se pode declarar um deles `const` e o outro não, nem capturar um só numa `lambda`, nem aplicar-lhes atributo individualmente. Onde é preciso tratar as metades de forma diferente, ainda cabe a variável nomeada à moda antiga.

§3.7 `[[nodiscard]]` e `[[maybe_unused]]`

Os dois são **atributos**: anotações que não mudam o que o programa faz, e mudam o que o compilador diz sobre ele. Num projeto cujo portão é zero aviso, isso equivale a mudar o que compila.

0 avisos

avisos do compilador

aferido por

make verifica

Makefile

— `[[nodiscard]]` - descartar este resultado é erro

`[[nodiscard]]` antes do tipo de retorno de uma função faz o compilador emitir aviso quando alguém a chama e joga o resultado fora. O critério para usá-la é direto: **se o único efeito da função é o valor que ela devolve, descartar esse valor não pode ser intencional**.

O `Deriva` a usa em toda função dessa espécie, e vale ler a lista pelo que ela tem em comum. `vetor2::operator==` e `operator!=`: chamar `a == b` sem usar a resposta é sempre erro de digitação. `grade::largura()`, `grade::altura()`, `grade::dentro()`, `grade::em()`: métodos de acesso que não alteram nada e cuja chamada isolada não tem sentido. `deriva::total_vivos()`, a soma dos contadores que a condição 4 do portão lê: idem.

O caso em que o atributo passa de higiene a mecanismo é `mundo::retirar_de(vetor2)`, da v1.2 (Cap. 12). Ela remove a entidade que estiver na posição e devolve `std::unique_ptr<entidade>`, o que significa que **a posse muda de mãos na expressão de retorno**: o mundo deixou de ser dono, e quem chamou passou a ser. Descartar esse retorno destrói o objeto no fim da instrução, silenciosamente, porque o destrutor do `unique_ptr` temporário roda e leva a entidade com ele. A entidade desaparece do mapa sem que ninguém tenha pedido, e nenhum aviso apareceria - o `delete` mora dentro de um cabeçalho do sistema. Com o atributo, e com o portão de zero aviso, essa linha não entra no repositório.

`mapa::carregar()` é o mesmo argumento pelo outro lado: ela devolve `std::optional<mapa>`, e descartar o retorno significa ter aberto o arquivo, lido a grade, construído o mapa e atirado tudo fora - e, pior, significa ter perdido a única informação sobre o carregamento ter falhado, que está no `optional` estar vazio.

O atributo também pode ser posto num tipo, e não numa função: `struct [[nodiscard]] resultado { ... };` faz com que **qualquer** função que devolva resultado herde a exigência. É o que se usa quando o próprio valor carrega uma obrigação, tais como um código de erro que precisa ser examinado.

— `[[maybe_unused]]` - não usar isto é intencional

`[[maybe_unused]]` é o movimento oposto: ele diz ao compilador que uma entidade pode ficar sem uso, e desliga o aviso `-Wunused` para ela. Sem o atributo, a saída é o velho `(void)x;`, que funciona e mente sobre a intenção - ele parece um uso, quando o que se quer dizer é que não há uso.

Os três lugares em que ele cabe:

1. **Parâmetro de função que a implementação não usa.** Acontece toda vez que se implementa uma interface: a assinatura vem da classe base e esta sobrescrita específica não precisa de um dos argumentos. No Deriva é o caso de entidade: `agir(mundo&)`, cuja base recebe o mundo e não o usa - ali o nome do parâmetro foi apagado, que é a outra saída legítima. `[[maybe_unused]]` é a saída que mantém o nome, e com ele a documentação do que aquele argumento é.
2. **Variável usada só em assert, ou só em compilação de depuração.** Com `NDEBUG` definido, o `assert` desaparece e a variável fica sem uso; o aviso aparece exatamente na configuração de release, que é a que ninguém compila enquanto desenvolve.
3. **Objeto que existe apenas para o seu construtor e o seu destrutor rodarem.** É o caso de todo guarda RAII cujo nome nunca é mencionado depois - um `std::lock_guard`, ou o `terminal_bruto` do Cap. 8 - e é o caso dos objetos de instrumentação do Cap. 8, que existem só para deixar rastro no traço de ciclo de vida.

O terceiro está extraído no fim deste capítulo: `testes/test_ciclo_de_vida.cpp` declara objetos `marca_de_vida` cujos nomes nunca são mencionados depois, porque o que interessa neles é o que o construtor e o destrutor anotam no traço. Antes eram `(void)a;`, que parece um uso; agora são `[[maybe_unused]] const marca_de_vida a("a")`, que diz o que se quer dizer.

Exercícios Propostos

1. Escreva um programa que leia, do console, o nome de um setor e as suas duas dimensões, usando `std::getline` para o nome e `std::cin >>` para os números. Trate o `'\n'` residual e imprima as três informações. Confirme, compilando com o conjunto de avisos do Cap. 2, que não há uma linha de aviso.
2. Escreva a função `std::vector<std::string_view> fileiras(std::string_view texto)`, que parte o texto em linhas sem alocar uma única `std::string`. Depois responda por escrito: em que condição o vetor devolvido fica com todas as vistas penduradas, e por que nenhum aviso do compilador aponta isso?
3. Tome os quatro casos de tempo de vida do §3.3 e escreva, para cada um, o menor programa que o exibe. Rode cada um; anote quais produzem saída errada, quais parecem funcionar e quais abortam. A lição está na segunda coluna: o caso que “parece funcionar” é o que chega à produção.
4. Reescreva o laço `for (auto it = tabela.begin(); it != tabela.end(); ++it)` com ligação estruturada e `const auto&`. Depois troque `const auto&` por `auto` e meça a diferença com uma tabela de dez mil elementos cujo valor seja uma `std::string` longa. Explique de onde vem a diferença.
5. Escreva uma função `std::unique_ptr<int> criar(int v)` e chame-a descartando o retorno. Compile com o conjunto do Cap. 2, marque a função com `[[nodiscard]]` e compile de novo. Compare as duas saídas do compilador e explique, no vocabulário do §3.7, o que o atributo passou a afirmar sobre a posse.
6. Explique com um exemplo concreto por que `NULL` pode ser ambíguo em C++ na presença de sobrecarga, e como `nullptr` resolve o problema. Compile o exemplo e transcreva a mensagem do compilador.

O código, extraído do Deriva

Todo trecho abaixo vem de exemplos/deriva/, que compila com `-std=c++17 -Wall -Wextra -Wpedantic` sem um aviso e passa `make verifica`. Nenhum foi digitado neste texto.

```
src/mape.cpp:66 |----- string_view e tempo de vida -----
66 std::optional<mapa> mapa::de_texto(std::string_view texto, std::string nome) {
67     // `string_view` não possui os bytes: `texto` tem de continuar vivo durante
68     // toda esta função. É por isso que as fileiras abaixo são `string_view`
69     // para dentro de `texto`, e nada disso é guardado depois (Aula 03).
70     std::vector<std::string_view> fileiras;
71     std::size_t inicio = 0;
72     while (inicio <= texto.size()) {
73         const std::size_t fim = texto.find('\n', inicio);
74         std::string_view linha = texto.substr(
75             inicio, fim == std::string_view::npos ? std::string_view::npos : fim - inicio);
76         if (!linha.empty()) fileiras.push_back(linha);
77         if (fim == std::string_view::npos) break;
78         inicio = fim + 1;
79     }
80
81     if (fileiras.empty()) return std::nullopt;
```

`string_view` não possui os bytes. As fileiras apontam para dentro de `texto`, e nada disso é guardado depois - se fosse, seriam referências penduradas no instante em que a função retornasse.

```
src/main.cpp:78 |----- ligação estruturada -----
78 // ligação estruturada (C++17) sobre a tabela de comandos
79 static const std::pair<std::string_view, deriva::vetor2> tabela[] = {
80     {"norte", {0, -1}}, {"sul", {0, 1}}, {"leste", {1, 0}},
81     {"oeste", {-1, 0}}, {"esperar", {0, 0}};
82 for (const auto& [nome, delta] : tabela) {
83     if (linha == nome) {
84         passos.push_back({linha, delta});
85         break;
86     }
```

`for (const auto& [nome, delta] : tabela)` - por referência, e `const` porque só se lê. Copiar por valor aqui seria copiar a tabela inteira a cada volta.

```
testes/test_string_view.cpp:42 |----- a vista não termina em nul-----
42 TEST_CASE("caso 3: a vista NAO termina em nul") {
43     const std::string dono = "eclusa-norte";
44     const std::string_view prefixo = std::string_view(dono).substr(0, 6);
45
46     REQUIRE(prefixo == "eclusa");
47     REQUIRE(prefixo.size() == 6);
48     // O byte seguinte ao fim da vista é '-', não '\0'. Passar `prefixo.data()`
49     // a uma função que espera `const char*` leria doze caracteres, não seis.
50     REQUIRE(prefixo.data()[prefixo.size()] == '-');
51     REQUIRE(std::strlen(prefixo.data()) == dono.size()); // e não 6
52
53     INFO("para uma API de C, o caminho é std::string(prefixo), que copia de "
54          "propósito - e aí o custo está declarado");
55 }
```

É o caso que mais morde: `strlen(vista.data())` devolve o tamanho da string de origem, não o da vista. Para uma API de C o caminho é `std::string(vista)`, que copia de propósito - e aí o custo está declarado.

```
testes/test_ciclo_de_vida.cpp:22 |----- [[maybe_unused]]-----
22 // `[[maybe_unused]]` diz ao compilador o que `(void)x` dizia por gesto:
23 // o objeto existe pelo efeito do construtor e do destrutor, não pelo
24 // valor. O atributo é C++17 e é o lugar mais natural dele em todo o
25 // Deriva (Aula 03).
26 [[maybe_unused]] const marca_de_vida a("a");
27 [[maybe_unused]] const marca_de_vida b("b");
28 {
29     [[maybe_unused]] const marca_de_vida c("c");
30 }
```

O atributo diz ao compilador o que `(void)x` dizia por gesto: o objeto existe pelo efeito do construtor e do destrutor, não pelo valor.

```

| include/deriva/vetor2.hpp:15 |----- v0.1 |
15 struct vetor2 {
16     int x = 0;
17     int y = 0;
18
19     [[nodiscard]] constexpr bool operator==(const vetor2& o) const noexcept {
20         return x == o.x && y == o.y;
21     }
22     [[nodiscard]] constexpr bool operator!=(const vetor2& o) const noexcept {
23         return !(*this == o);
24     }
25
26     // v1.5 · Aula 15. Os compostos são MEMBROS porque modificam o objeto da
27     // esquerda; os binários são funções LIVRES, logo abaixo, porque tratam os
28     // dois lados igual. Escrever '+' em termos de '+=' é a forma que não
29     // duplica a regra.
30     constexpr vetor2& operator+=(const vetor2& o) noexcept {
31         x += o.x;
32         y += o.y;
33         return *this;
34     }
35     constexpr vetor2& operator-=(const vetor2& o) noexcept {
36         x -= o.x;
37         y -= o.y;
38         return *this;
39     }
40     constexpr vetor2& operator*=(int k) noexcept {
41         x *= k;
42         y *= k;
43         return *this;
44     }
45
46     /// Distância de Manhattan, que é a métrica da grade: a sonda não anda em
47     /// diagonal, então a distância euclidiana mentiria sobre o número de turnos.
48     [[nodiscard]] constexpr int manhattan(const vetor2& o) const noexcept {
49         const int dx = x > o.x ? x - o.x : o.x - x;
50         const int dy = y > o.y ? y - o.y : o.y - y;
51         return dx + dy;
52     }
53 };

```

[[nodiscard]] numa comparação: chamar a == b e jogar o resultado fora é sem-pre erro, e agora o compilador diz isso.

Git, LLM como copiloto e a rubrica de revisão

O QUE ESTE CAPÍTULO ENTREGA

- ▶ Usar Git para versionar código individualmente: init, add, commit, log, diff.
- ▶ Trabalhar com branches para isolar funcionalidades e experimentos.
- ▶ Colaborar via GitHub: fork, clone, pull request, code review.
- ▶ Interpretar e resolver conflitos de merge.
- ▶ Compreender o fluxo de trabalho da disciplina (repositório por atividade, PR para entrega).
- ▶ Escrever prompts eficazes para modelagem OO, implementação e refatoração.
- ▶ Avaliar criticamente código gerado por LLM sob a ótica de correção e de projeto.
- ▶ Aplicar, item por item, a rubrica de revisão de código OO gerado por IA.

§4.1 Por que Versionar Código?

Desenvolvimento de software sem controle de versão é como escrever um documento importante sem nunca salvar um rascunho. Com Git: você pode desfazer qualquer mudança, comparar versões, trabalhar em paralelo em funcionalidades diferentes sem conflito, e colaborar com outros sem sobrescrever o trabalho alheio.

No contexto desta disciplina, Git e GitHub são a infraestrutura pedagógica: exercícios são entregues como commits em repositórios, pull requests são revisados como parte do processo formativo. O Deriva evolui em vinte versões nomeadas, da v0.0 à v2.7, e cada uma delas é um estado do repositório que você pode voltar a visitar; o histórico é o registro das decisões de projeto do semestre, e o DECISA0.md de cada entrega é onde a justificativa fica escrita.

§4.2 Conceitos Fundamentais

Conceito	O que é
Repositório (repo)	Diretório com todo o histórico do projeto, guardado em <code>.git/</code>
Commit	Estado do projeto inteiro num dado momento, com mensagem que o descreve
Branch	Linha de desenvolvimento independente, para trabalhar em paralelo
HEAD	Ponteiro para o commit atual, em geral o topo da branch corrente
Remote	Repositório remoto; <code>origin</code> é o nome convencional do seu
Clone	Cópia local de um repositório remoto, com o histórico inteiro

Conceito	O que é
Pull Request (PR)	Pedido para integrar uma branch em outra, e o lugar do code review
Merge	Integração de duas branches
Conflict	Duas branches mudaram a mesma linha, e a resolução é manual

Dois desses termos costumam ser confundidos com o que não são, e a confusão sai caro no meio do semestre. **Commit não é backup**: ele registra o estado do projeto e a razão da mudança, e um commit sem mensagem útil é um estado sem razão, o que reduz o histórico a uma pilha de arquivos datados. E **branch não é pasta**: mudar de branch não copia nada, troca o que a árvore de trabalho mostra, e é por isso que trocar de branch com mudança não commitada é a primeira forma de perder trabalho que todo mundo descobre por conta própria.

§4.3 Fluxo de Trabalho Individual

O ciclo que você repete a cada exercício tem seis passos, e nenhum deles precisa de interface gráfica:

```
1 # 1. criar o repositório local
2 git init deriva
3 cd deriva
4
5 # 2. configurar a identidade, uma vez por máquina
6 git config --global user.name "Seu Nome"
7 git config --global user.email "voce@ufpb.br"
8
9 # 3. pôr o que mudou na área de preparação
10 git add CMakeLists.txt include/deriva/vetor2.hpp
11 git add .          # tudo que é rastreável e não está no .gitignore
12
13 # 4. commitar, com mensagem que diz o que mudou
14 git commit -m "feat: vetor2 com operadores constexpr e nodiscard"
15
16 # 5. ler o histórico
17 git log --oneline
18
19 # 6. ver o que mudou, antes e depois de preparar
20 git diff
21 git diff --staged
```

✓ DICA

Escreva a mensagem no imperativo, dizendo o que o commit faz: “acrescenta validação na lista de inicialização” em vez de “validação adicionada” ou “mudanças”. Os prefixos que esta disciplina usa são feat: para funcionalidade nova, fix: para correção, test: para teste, refactor: para refatoração sem mudança de comportamento, e docs: para documentação.

O Deriva evolui em vinte versões nomeadas, e há uma disciplina de commit que vale a pena a partir da primeira: **um commit por decisão de projeto, e não um commit por dia de trabalho**. Quando a caça ao bug do Cap. 9 pedir que você mostre em que momento a cópia rasa entrou no código, a resposta sai de `git log` em segundos se cada commit fizer uma coisa, e sai de uma tarde de bissecção se um commit fizer sete.

§4.4 Branches e Merge

Branch serve para isolar o que ainda pode não dar certo, e o uso que esta disciplina recomenda é direto: **cada exercício que quebra o Deriva de propósito nasce numa branch**. As variantes que o material já traz, tais como a v0.2-quebrada do Cap. 8 e a v0.3-quebrada do Cap. 9, moram em `exemplos/deriva/variantes/` como diretórios, de forma a poderem ser lidas lado a lado com a versão boa; quando for você a quebrar a sua, a branch é o que deixa a diferença legível através de um `git diff` e reversível através de um `git switch`.

```

1 # criar e entrar na branch
2 git switch -c feat/contador-de-instancias
3
4 # a forma antiga, que você vai encontrar em tutorial:
5 git checkout -b feat/contador-de-instancias
6
7 # ver em que branch se está, e quais existem
8 git branch
9
10 # trabalhar, e commitar ali
11 git add . && git commit -m "feat: contador vivos e criados em mapa"
12
13 # voltar à principal e integrar
14 git switch main
15 git merge feat/contador-de-instancias
16
17 # apagar a branch já integrada
18 git branch -d feat/contador-de-instancias

```

Quando as duas branches mudaram a mesma linha, o merge para e deixa no arquivo os marcadores <<<<<<, ===== e >>>>>>, com a sua versão de um lado e a de quem veio do outro. Resolver é editar o arquivo até ele ficar como você quer, apagar os três marcadores, e commitar o resultado. O erro que se comete uma vez só é commitar com os marcadores dentro do código; e o portão de zero aviso do Cap. 2 pega esse erro na hora, porque <<<<<< não é C++ válido.

0 avisos

avisos do compilador

aferido por

make verifica

Makefile

§4.5 GitHub - Colaboração e Entrega

Comando	O que faz
git clone <url>	Traz o repositório remoto inteiro
git push origin main	Envia os commits locais ao remoto
git pull origin main	Traz e integra os commits do remoto
git remote -v	Mostra a que remoto o repositório aponta

A entrega de atividade desta disciplina é um pull request, e o fluxo é sempre o mesmo: fork do repositório da disciplina pela interface web; clone do seu fork; branch nomeada pela atividade; desenvolvimento, commits e push; e o PR aberto do seu fork para o repositório de origem.

O PR é o lugar do code review, e é onde a rubrica do §4.13 é aplicada por escrito. Duas coisas fazem parte da entrega e não do código: o `make verifica` verde, que é o portão do Cap. 2, e o `DECISA0.md`, que é onde você registra por que resolveu do jeito que resolveu. O portão sem a justificativa vale metade; a justificativa sem o portão vale zero.

§4.6 .gitignore para C++ com CMake

O que **não** entra no repositório é decisão de projeto, e num projeto C++ com CMake ela é sempre a mesma: nada que o build produz, e nada que o build baixa.

```
1 # diretório de build, e as variações que as IDEs criam
2 build/
3 cmake-build-*/
4
5 # o que o FetchContent baixa: FTXUI e Catch2 vêm por tag fixa
6 _deps/
7
8 # objetos, bibliotecas e executáveis
9 *.o
10 *.a
11 *.so
12 *.out
13
14 # IDE e sistema operacional
15 .vscode/
16 .idea/
17 .DS_Store
```

As duas primeiras linhas resolvem quase tudo, porque FetchContent clona a FTXUI e o Catch2 para dentro do próprio diretório de build; a linha de _deps/ está ali para o caso de alguém configurar o projeto com outro diretório binário, que é o arranjo em que o código de terceiro escapa do .gitignore sem ninguém perceber. O que sustenta a reprodutibilidade não é a cópia dos arquivos baixados: é a tag imutável declarada no CMakeLists.txt, e é ela que garante que quem clonar o seu projeto compile a mesma coisa.

§4.7 O que um LLM faz, e o que ele não faz

Um Large Language Model é treinado em grande volume de texto, código-fonte de repositórios públicos incluído, e aprende a completar texto de forma coerente com o contexto recebido. A consequência que interessa a este capítulo é negativa e precisa ser dita sem rodeio: **o modelo não executa o código que escreve**. Ele não sabe se aquilo compila, não sabe se o teste passa, e não sabe se a assinatura que chamou existe na versão da biblioteca que você tem. Ele produz texto que se parece com código correto, e a verificação é sua.

Onde eles ajudam	Onde eles falham
Esboçar declaração de classe a partir de especificação em texto	Raciocinar sobre a complexidade exata de um algoritmo
Explicar conceito, idioma e mensagem de erro do compilador	Garantir ausência de defeito em lógica de posse
Sugerir nome de método e forma de interface	Manter coerência com código que não está no contexto
Gerar teste para o caso típico	Achar o caso de fronteira que ninguém pensou
Refatorar seguindo padrão conhecido	Raciocinar sobre concorrência e comportamento indefinido
Traduzir idioma de uma linguagem para outra	Saber qual biblioteca está disponível na sua máquina

Onde eles ajudam**Onde eles falham**

A coluna da direita é a razão de este capítulo existir na segunda semana, e não no fim do semestre: as competências que ela lista são exatamente as que a disciplina forma, e quem terceirizar a leitura de código para o modelo fica sem meio de avaliar o que o modelo devolveu.

§4.8 A estrutura de um prompt que serve

A resposta acompanha a qualidade da pergunta, e prompt vago produz resposta genérica. Um prompt de programação que serve tem quatro partes, e a quarta é a que quase nunca aparece:

- **Contexto.** Qual é o projeto, a linguagem, o padrão e a convenção: C++17, snake_case, identificadores e comentários em português.
- **Restrições.** O que não pode ser usado: sem new nem delete crus, sem função de C++20, sem dependência que não esteja no CMakeLists.txt.
- **Tarefa.** O que exatamente deve ser gerado, com a interface pública nomeada membro a membro.
- **Portão.** Como o resultado será verificado: compila com o conjunto de avisos do Cap. 2 sem uma linha de aviso, e passa nestes testes.

O quarto item muda o que o modelo devolve, e vale experimentar a diferença. Prompt sem portão declarado produz código que parece pronto; prompt com portão declarado produz código que o modelo já tenta defender, e às vezes traz o teste junto, o que é ótimo e é armadilha - o §4.10 é sobre isso.

Os dois pedidos abaixo são sobre a mesma classe. O fraco:

```
1 Escreva uma classe para representar uma grade de células em C++
```

E o forte, que nomeia projeto, convenção, requisito e portão:

```
1 Preciso da classe `grade` em C++17 para o projeto Deriva.  
2  
3 Convenções: snake_case, identificadores e comentários em português.  
4  
5 Requisitos:  
6 - guarda as células em std::vector<celula>, e `celula` já existe;  
7 - construtor recebe largura e altura, e valida que as duas são positivas  
8   na lista de inicialização, não no corpo;  
9 - métodos: largura(), altura(), dentro(vetor2), e o par de sobrecargas  
10  const e não-const de em(vetor2);  
11 - todo método que não altera o objeto é const, e todo método que só  
12  devolve valor é [[nodiscard]];  
13 - nenhuma das seis operações especiais é declarada: regra do zero.  
14  
15 Portão: compila com -std=c++17 -Wall -Wextra -Wpedantic -Wconversion  
16 -Wsign-conversion sem uma linha de aviso.
```

§4.9 Prompts por contexto de desenvolvimento OO

Três contextos aparecem toda semana nesta disciplina, e cada um pede uma forma diferente de pedido.

Modelagem. O que se pede é o diagrama antes do código, porque é ali que a decisão entre herança e composição custa um traço em vez de uma tarde: *“Dado este domínio, identifique as classes, os atributos e as operações; aplique responsabilidade única; devolva um diagrama de classes em Mermaid e, só depois dele, o esboço das declarações em C++17.”* O Cap. 6 é sobre como ler o que voltar.

Refatoração. O que se pede é o nome do princípio violado antes do conserto, porque conserto sem diagnóstico é indistinguível de reescrita: *“Identifique as violações de SOLID no código abaixo; para cada uma, nomeie o princípio, explique a violação em uma frase, e mostre o trecho corrigido mantendo a interface pública existente.”* O Cap. 24 é onde isso é cobrado com o replay do Cap. 2 como oráculo.

Geração de teste. O que se pede é a fronteira, e não o caso típico, porque o caso típico é o que o modelo escreve sozinho e é o que não prova nada: *“Gere testes Catch2 para esta classe cobrindo construção com argumento inválido, índice de fronteira, e o estado do objeto depois de cada operação que o modifica.”* Compare o que voltar com o item R7 da rubrica.

§4.10 Análise crítica de código gerado

O código gerado que este capítulo usa está em `exemplos/deriva/revisao_ia/`, compila sem uma linha de aviso, e passa no teste que veio junto com ele. Ele tem três defeitos plantados, cada um correspondendo a um item da rubrica do §4.13, e nenhum deles é erro de digitação: os três são decisões plausíveis, e é isso que os torna caros.

Os dois trechos extraídos no fim deste capítulo são a base da hierarquia gerada e o teste que o próprio modelo escreveu para ela. Leia os dois antes de continuar, e procure os defeitos sozinho.

O primeiro é de **posse**, item R1: o vetor de leituras é público, de forma que a validação que o método `registrar` faz é contornável de fora, e a média passa a mentir sem que nada no código pareça errado. O segundo é de **hierarquia**, item R5: a base tem método virtual e destrutor não virtual, e deletar por ponteiro da base não roda o destrutor da derivada; posto dentro de um `unique_ptr`, isso não produz aviso nenhum. O terceiro é de **const-correctness**, item R3: o método de acesso devolve cópia da string a cada chamada e não é `const`, de modo que chamá-lo num objeto constante não compila, e o custo passa despercebido dentro de um laço.

O quarto problema é o teste que veio junto, e ele é o mais instrutivo dos quatro: o item R7. Ele passa, e afirma que a média de 20 e 22 é 21, o que é verdade e é irrelevante: nenhuma das três decisões acima muda esse resultado. Teste que passa não é prova de que o código está certo; ele é prova de que o comportamento medido está certo, e a pergunta que importa é se o comportamento medido é o que a classe promete.

— ◇ LLM —

A ordem de trabalho com código gerado, nesta disciplina:

1. Tente resolver o problema sozinho primeiro, ainda que sem terminar. É o que lhe dá critério para julgar o que voltar.
2. Leia cada linha do que voltou, e marque as que você não sabe explicar.
3. Compile com o conjunto de avisos do Cap. 2, e rode. Código gerado que não passou pelo compilador é rascunho.
4. Aplique os sete itens da rubrica, um por um, por escrito.
5. Escreva um teste que **falhari**a na versão anterior à sua correção.
6. Registre no `DECISA0.md` o que você aceitou, o que recusou, e por quê.

§4.11 Limites, riscos e responsabilidades

- **API inventada.** O modelo chama função que não existe, ou que existe com outra assinatura, ou que existe só a partir do C++20. É o defeito mais barato de encontrar e o mais fácil de aceitar sem ver, porque a chamada inventada parece exatamente com uma chamada real. O item R7 da rubrica cobra a conferência de cada função de biblioteca contra o C++17.
- **Licenciamento.** O código devolvido pode derivar de código sob licença restritiva do conjunto de treinamento, e a política de uso do serviço que você usa é o que diz o que fazer com isso. Em trabalho de disciplina, declare o uso; em trabalho que vá a público, leia a política antes.
- **Confidencialidade.** Nada de credencial, dado pessoal nem código de terceiro sob acordo vai para prompt de serviço externo. Isso não é conselho de etiqueta; é o mesmo raciocínio de posse do Cap. 12, aplicado a informação.
- **Responsabilidade.** Você responde por todo código que entrega, tenha ou não escrito cada linha. “O modelo gerou” descreve a origem e não transfere a autoria.
- **Dependência.** Usar o modelo para tudo sem entender o que voltou produz uma competência que se perde no dia em que a ferramenta não está disponível, e a prova desta disciplina é em papel exatamente por isso.

§4.12 O ciclo OO assistido, e o que conferir em cada etapa

As seis etapas abaixo são o ciclo de desenvolvimento OO com assistente, na ordem em que ele acontece, e cada uma traz o que se confere ao receber a resposta. A segunda metade de cada etapa é a que separa usar a ferramenta de ser usado por ela.

1. **Modelagem.** Peça o diagrama de classes em Mermaid, com as relações nomeadas. Confira se cada relação de herança passa o teste duplo do Cap. 5, e se nenhuma composição virou herança por conveniência de escrita.
2. **Esqueleto.** Peça as declarações em C++17, com destrutor virtual na base e posse declarada no tipo. Confira o item R5 antes de qualquer outra coisa, porque é o defeito que o compilador não pega e o teste não vê.
3. **Implementação.** Peça uma classe por vez, e não o sistema. Confira o item R6: o construtor estabelece a invariante, e nenhum método público retorna com ela quebrada.
4. **Teste.** Peça o caso de fronteira e o caso de erro, nomeados. Confira o item R7, e desconfie de todo teste que afirme detalhe interno em vez de comportamento.
5. **Refatoração.** Peça a mudança com a interface pública fixada. Confira com o replay do Cap. 2: se o despejo mudou byte a byte, aquilo não foi refatoração.
6. **Documentação.** Peça a documentação das pré-condições, pós-condições e invariantes, e não a paráfrase do corpo da função. Comentário que repete o código é ruído que envelhece.

Uma observação minha, sobre este material e não sobre modelos em geral: os três defeitos plantados em `revisao_ia/gerado.hpp` não foram inventados para o livro. São os três que eu vi voltar com mais frequência ao pedir hierarquia com posse em C++, e é por isso que eles são os três, e não outros. Posse por contêiner público, destrutor não virtual na base, e método de acesso não-const que copia. O que os

une é que os três compilam sem aviso e passam num teste plausível, o que é a definição de defeito caro.

§4.13 A rubrica de revisão de código OO gerado por IA

A rubrica é a consolidação, num instrumento só, do que este capítulo dispersou: os problemas do §4.10, os riscos do §4.11 e as conferências do §4.12. São sete itens, cada um com a pergunta que se faz ao código e o instrumento com que se responde, e ela se aplica a código gerado por modelo, a código de colega em code review, e ao seu próprio código de três semanas atrás.

Ela **não** mora neste capítulo. A definição única é `conteudo/mapa.py`, e dela saem três coisas: a tabela abaixo, a página `rubrica.html` do site da disciplina, que é a forma publicada e utilizável fora do livro, e os comentários dos defeitos plantados em `exemplos/deriva/revisao_ia/`, que apontam para o item que cada um viola. O portão `make verifica` recusa o build se os três divergirem na numeração, e a razão de existir esse portão é que eles já divergiram: os mesmos sete itens estiveram numerados de três maneiras diferentes, em três lugares, ao mesmo tempo.

item	a pergunta	como se verifica	capítulos
R1 · Posse	Quem possui cada recurso, e em que linha ele é liberado? Há um <code>delete[]</code> para cada <code>new[]</code> - ou, melhor, nenhum dos dois?	O contador vivos fecha em zero no fim de <code>main</code>	7, 12, 13
R2 · Operações especiais	Se há destrutor, há também cópia e atribuição, ou as duas estão = <code>delete</code> ? A regra do zero não resolveria melhor?	Traço de ciclo de vida: a cópia que ninguém pediu aparece nele, e o contador acusa a que sobrou	8, 9, 14
R3 · const-correctness	Todo método que não altera o objeto é <code>const</code> ? Todo parâmetro só de leitura é <code>const&</code> ou <code>string_view</code> ? Nenhum acesso devolve referência não- <code>const</code> a membro privado?	O compilador: chame o método num objeto <code>const</code> e veja se recusa	3, 7
R4 · Limites e ausência	Todo índice é verificado, ou a pré-condição está escrita e quem chama a cumpre? Ausência de resultado é <code>std::optional</code> , e não valor mágico?	Teste que passa índice de fronteira; <code>optional</code> no lugar de -1	3, 20
R5 · Hierarquia	Base com método virtual tem destrutor virtual? Toda sobrescrita está marcada <code>override</code> ?	<code>gdb</code> com ponto de parada no destrutor da derivada: sem <code>virtual</code> , ele não é alcançado	2, 11
R6 · Invariante e estado	O construtor estabelece a invariante, e nenhum método público a deixa quebrada no retorno? Nada lê o objeto de origem depois de um <code>std::move</code> ?	Um caso que viola a invariante por fora e um que lê a origem movida: os dois têm de ser impossíveis ou recusados	7, 14

item	a pergunta	como se verifica	capítulos
R7 · Prova e justificativa	O teste falha se o comportamento mudar - e não se a implementação mudar? Cada função de biblioteca chamada existe, com essa assinatura, em C++17? Para cada decisão aceita há uma frase que a sustenta, escrita por quem entrega?	DECISA0.md na entrega, e o <code>make</code> verifica verde antes dela	16, 24

A rubrica se aplica desde esta segunda semana, com uma ressalva. Os itens R5 e R6 só se tornam plenamente aplicáveis quando houver hierarquia e movimento no Deriva, o que acontece nos Caps. 11 e 14; até lá, a resposta a eles é “não se aplica”, **escrita**, e não deixada em branco, porque item de rubrica em branco é item que ninguém leu. O item R7 é o único sem instrumento automático nenhum, e é o que mais separa quem entendeu de quem colou: o portão sem a justificativa vale metade, e a justificativa sem o portão vale zero.

Exercícios Propostos

1. Crie um repositório Git local para o Deriva v0.0. Faça pelo menos três commits com mensagens descritivas representando etapas de desenvolvimento, tais como “feat: CMakeLists com padrao C++17 e extensoes desligadas”, “feat: Catch2 por FetchContent como SYSTEM”, “test: teste de fumaca”.
2. Simule um conflito de merge: crie duas branches que modificam a mesma linha de um arquivo, tente fazer merge e resolva o conflito manualmente. Descreva o processo com a transcrição dos comandos usados.
3. Publique seu repositório do Deriva no GitHub. Configure um `.gitignore` adequado para C++/CMake e verifique, na interface do GitHub, que nenhum arquivo de build e nenhuma dependência baixada pelo FetchContent foi enviada.
4. Pesquise o conceito de rebase e explique a diferença entre rebase e merge. Em que situação cada um é mais adequado? Por que rebase pode ser perigoso em branches compartilhadas?
5. Escreva um prompt para gerar a classe grade do Deriva, seguindo a estrutura do §4.8 e declarando explicitamente o portão. Execute-o em um LLM, aplique os sete itens da rubrica ao resultado e escreva o parecer, item por item. Este é o portão do **LAB-03**.
6. O código a seguir foi gerado por um LLM. Aplique a rubrica e diga qual item cada defeito viola:

```
class Pilha { int* arr; int topo; public: Pilha() { arr = new int[100]; topo=0; } void empilhar(int x) { arr[topo++]=x; } int desempilhar() { return arr[--topo]; } };
```
7. Crie um prompt que instrua um LLM a refatorar o código do exercício anterior aplicando `std::vector`, validação nos métodos e `const-correctness`. Depois aplique a rubrica à **refatoração**: o LLM introduziu defeito novo ao consertar o antigo?
8. Escreva um post-mortem do seu Deriva: o que foi mais difícil de implementar, quais decisões de projeto (herança ou composição, virtual ou template, quem tem a posse) você mudaria com o que aprendeu depois, e em que momentos exatos o

LLM ajudou e em que momentos atrapalhou. Cite pelo menos um caso em que a rubrica pegou algo que você teria aceitado.

O código, extraído do Deriva

Todo trecho abaixo vem de exemplos/deriva/, que compila com `-std=c++17 -Wall -Wextra -Wpedantic` sem um aviso e passa `make verifica`. Nenhum foi digitado neste texto.

```

29 /// A base da hierarquia gerada.
30 struct sensor_base {
31     explicit sensor_base(std::string nome) : nome_(std::move(nome)) {}
32
33     // DEFEITO 2 · item R5 da rubrica (Hierarquia): destrutor NÃO virtual numa base com
34     // método virtual. Deletar por `sensor_base*` não roda o destrutor da
35     // derivada. Com o `delete` dentro de um `unique_ptr`, o compilador não
36     // emite aviso algum.
37     ~sensor_base() = default;    // R5
38
39     virtual double media() const = 0;
40
41     // DEFEITO 3 · item R3 da rubrica (const-correctness): devolve cópia da string a cada chamada,
42     // e não é `const` nem `[[nodiscard]]`. Chamar num objeto `const` não
43     // compila, e o custo passa despercebido em laço.
44     std::string nome() { return nome_; }    // R3
45
46 protected:
47     std::string nome_;
48 };

```

Compila sem um aviso e passa no teste que o próprio modelo escreveu. Tem três defeitos plantados, um por item da rubrica, e nenhum é erro de digitação: os três são decisões plausíveis, e é isso que os torna caros.

```
┌───| testes/test_revisao_ia.cpp:12 |─── o teste que o modelo escreveu ──┐
12 // 0 teste que o proprio modelo escreveu para o codigo dele. Passa. E nao prova
13 // nada do que importa - e essa e a licao do item R7 da rubrica.
14 TEST_CASE("o teste que veio com o codigo gerado passa") {
15     sensor_termico t("termico-01");
16     t.registrar(20.0);
17     t.registrar(22.0);
18     REQUIRE(t.media() == 21.0);
19     REQUIRE(t.nome() == "termico-01");
20 }
```

Ele passa, e não prova nada do que importa. É o item R7 da rubrica: o teste tem de falhar se o comportamento mudar, e não se a implementação mudar.

```
┌───| testes/test_revisao_ia.cpp:31 |─── R1, medido nos dois lados ──┐
31 TEST_CASE("R1: o vetor publico anula a invariante da classe") {
32     sensor_termico t("x");
33     t.registrar(20.0);
34     t.leituras_.push_back(-999.0); // ninguem impede, e a media mente
35     REQUIRE(t.media() < 0.0);
36
37     SECTION("na versao revisada, registrar RECUSA e o vetor e privado") {
38         revisado::sensor_termico r("x");
39         REQUIRE(r.registrar(20.0));
40         REQUIRE_FALSE(r.registrar(-999.0));
41         REQUIRE(r.quantas() == 1);
42         REQUIRE(r.media() == 20.0);
43     }
```

Com o vetor público, registrar protege uma invariante que qualquer um contorna de fora - e a média passa a mentir. A versão revisada recusa o valor e mantém o vetor privado.

Classificação de linguagens e sistemas de tipos

O QUE ESTE CAPÍTULO ENTREGA

- ▶ Distinguir linguagens “baseadas em objetos” de linguagens “orientadas a objetos”.
- ▶ Compreender os eixos de classificação: tipagem estática vs. dinâmica, forte vs. fraca, herança vs. composição vs. traits.
- ▶ Posicionar C++, Python, Java, JavaScript e Rust nessa taxonomia.
- ▶ Entender o sistema de tipos de C++ e suas implicações práticas.

§5.1 Baseadas em Objetos vs. Orientadas a Objetos

Nem toda linguagem que usa a palavra “objeto” é verdadeiramente orientada a objetos. A distinção clássica de Wegner (1987) estabelece:

Categoria	Características	Exemplos
Baseadas em objetos	Tem objetos e encapsulamento, mas sem herança nem polimorfismo	Ada 83, primeiras versões do Visual Basic
Orientadas a objetos	Objetos + herança + polimorfismo (despacho dinâmico)	C++, Java, Python, Ruby, Smalltalk
Multiparadigma	OO + funcional + genérico + procedural	C++, Scala, Kotlin, Rust (sem herança clássica)

C++ é a linguagem multiparadigma por excelência: você pode escrever C puro, POO com herança e polimorfismo, código genérico com templates, código funcional com lambdas e algoritmos da STL - tudo no mesmo projeto. O Deriva usa três desses estilos ao mesmo tempo, e de propósito: `vetor2` e `celula` são agregados sem um método virtual, porque não há invariante a proteger; `grade` e `mapa` são classes com invariante e encapsulamento; a hierarquia entidade → sonda/drone/item do Cap. 10 é polimórfica; e o contador de instâncias vivas do Cap. 7 vira template por CRTP no Cap. 19. A pergunta que este capítulo prepara não é qual estilo é melhor, é qual deles o problema em mãos está pedindo.

§5.2 Sistemas de Tipos: os Eixos Fundamentais

— Eixo 1 - Tipagem Estática vs. Dinâmica

Em C++, Java e Rust o tipo é propriedade da expressão, e o compilador o conhece antes de o programa existir: a conferência acontece na compilação, e o que não passa nela não chega a gerar binário. Em Python, JavaScript e Ruby o tipo é propriedade do valor que a variável guarda no momento, e a conferência acontece na execução, quando a linha em falta é finalmente alcançada. O primeiro caso está em código no Deriva: `classificar<T>()`, em `tipos/despacho.hpp`, responde “inteiro”, “ponteiro” ou “classe” através de `if constexpr` sobre os traits de `<type_traits>`, e responde sem executar nada.

O contraste está na mesma atribuição escrita nas duas linguagens. Em C++ ela não chega a produzir binário: o g++ recusa a unidade de tradução, e a mensagem nomeia a conversão que não existe.

```
1 tipos.cpp: In function 'int main()':
2 tipos.cpp:2:13: error: invalid conversion from 'const char*' to 'int' [-fpermissive]
3     2 |         int n = "ola";
4       |               ^~~~~
5       |               |
6       |               const char*
```

Em Python a mesma atribuição é aceita, e `python3 -m py_compile` passa sem uma palavra: o nome `n` passa a referir uma `str`, e nada é conferido ali. O defeito existe, porém aparece só quando a linha seguinte é executada, com `TypeError: can only concatenate str (not "int") to str`.

```
1 n = "ola"
2 print(n + 1)
```

O que a disciplina cobra dessa diferença é o momento em que o defeito aparece, e em C++ ele aparece antes de o programa existir - o mesmo mecanismo de que o portão de avisos do Cap. 2 se serve para transformar conversão duvidosa em erro de build.

— Eixo 2 - Tipagem Forte vs. Fraca

Tipagem forte significa que conversões implícitas entre tipos incompatíveis são proibidas ou altamente restritas. C++ é predominantemente forte (narrowing com {} é erro), mas herda algumas coerções de C (int para float implicitamente). Python é forte dinâmico: “1” + 1 é erro. JavaScript é fraco dinâmico: “1” + 1 é “11” (coerção silenciosa).

As coerções herdadas do C são justamente o motivo de o Deriva compilar com `-Wconversion` e `-Wsign-conversion`, apresentados no Cap. 2: a linguagem permite a conversão, e o aviso é o que faz o programador decidir se ela era intencional. O caso concreto está em `terminal_bruto`, onde um `int` negativo atribuído a um campo `unsigned` funcionava por acidente do complemento de dois.

— Eixo 3 - Single Dispatch vs. Multiple Dispatch

C++ tem despacho **simples**: `virtual` resolve por um tipo só, o do objeto sobre o qual a chamada acontece, e os tipos dos argumentos não entram na conta. Java e Python fazem o mesmo. Em Common Lisp, Julia e Clojure o método escolhido depende dos tipos de todos os argumentos ao mesmo tempo, e é isso que se chama despacho **múltiplo**.

A diferença aparece quando a resposta depende de dois objetos, e é o caso da colisão: sonda contra parede, sonda contra sonda, parede contra parede. `tipos/despacho.cpp` faz esse despacho à mão, na função `resolver(const colisao&, const colisao&)`, que pergunta o tipo de cada operando com `dynamic_cast` justamente porque `virtual` responde por um. O trecho está extraído no fim deste capítulo, com a conta do que ele custa quando os tipos aumentam. O contorno que C++17 oferece para o mesmo problema é `std::visit` sobre `std::variant`, que cobra em compilação o tratamento de todas as alternativas, e o Cap. 20 mostra onde isso ajuda e onde atrapalha.

§5.3 Mecanismos de Reutilização: Herança, Composição e Traits

Mecanismo	Como funciona	Exemplos	C++ equivalente
Herança de classes	Subclasse herda implementação e interface da superclasse	C++, Java, Python	<code>class B : public A</code>
Composição	Objeto contém referência a outro; delega comportamento	Todas as linguagens	Membro do tipo A em B
Interfaces / Protocolos	Contrato de métodos sem implementação	Java, C#, Swift, Go	Classe abstrata pura em C++
Traits / Mixins	Pacotes de comportamento reutilizáveis sem hierarquia fixa	Rust (traits), Scala, Ruby (modules)	CRTP, herança múltipla de interfaces

C++ suporta todos esses mecanismos de alguma forma. A escolha entre herança e composição é uma das decisões de projeto mais importantes, e é o assunto dos Caps. 10 a 14. A forma da pergunta, porém, já cabe aqui, e ela tem um enunciado concreto no Deriva: um mapa *é* uma grade, ou um mapa *tem* uma grade? A resposta que o Cap. 9 defende é a segunda, e o critério é duplo. O mapa tem nome e ponto

de entrada, que grade nenhuma tem; e não faz sentido passar um mapa para uma função que espera uma grade. Onde qualquer uma das duas coisas falha, herança pública é modelagem errada, ainda que compile.

§5.4 Posicionando as Linguagens

Linguagem	Tipagem	Herança	Despacho dinâmico	Genérico
C++	Estática forte	Simples e múltipla	virtual (vtable)	Templates (Turing-completos)
Java	Estática forte	Simples (classes) + múltipla (interfaces)	Toda chamada de método	Generics (type erasure)
Python	Dinâmica forte	Múltipla (MRO)	Toda chamada (duck typing)	Implícito (duck typing)
JavaScript	Dinâmica fraca	Prototípica	Toda chamada	Implícito
Rust	Estática forte	Sem herança de dados	Traits (vtable ou static)	Generics + traits (monomorphization)
Go	Estática forte	Sem herança	Interfaces implícitas	Generics (Go 1.18+)

A coluna do despacho dinâmico é a que custa bytes, e o número aparece no Cap. 11: em C++ o programador escolhe, classe por classe, se paga o `vptr` de 8 bytes por objeto; em Java, toda chamada de método é virtual por padrão. `include/deriva/leiaute.hpp` mede as duas situações no mesmo arquivo, e os `static_assert` de lá são o que impede este livro de afirmar um número que o compilador não produza.

8 bytes

custo do `vptr`

aferido por

`./build/deriva --leiaute`

`testes/test_leiaute.cpp`

— ◇ LLM —

Prompt útil para este capítulo:

“Compare as implementações do padrão Observer em C++ com virtual functions, em Python com duck typing e em Rust com traits. Mostre código concreto para cada uma e explique os trade-offs de desempenho e segurança de tipos.”

Aplique a rubrica do Cap. 4 à resposta: o LLM provavelmente acertará a estrutura geral, mas pode cometer erros sutis no código Rust (lifetimes) ou no código C++ (ponteiro cru com posse, destrutor não virtual na base).

Exercícios Propostos

1. Pesquise o conceito de duck typing em Python. Escreva um exemplo em Python e um equivalente em C++ usando templates que demonstrem a mesma ideia. Explique as diferenças de quando os erros são detectados.
2. Java não permite herança múltipla de classes, mas permite implementar múltiplas interfaces. Qual problema isso resolve em relação ao problema do diamante? Como C++ lida com o mesmo problema?
3. Rust não tem herança de dados, mas tem traits. Pesquise como o padrão Iterator é implementado em Rust usando traits e compare com o uso de classes abstratas em C++.

4. Explique por que JavaScript é considerado de tipagem fraca: dê três exemplos de coerção implícita silenciosa que podem causar bugs. Como TypeScript resolve esse problema?

5. Para cada uma das quatro estruturas do Deriva que já foram nomeadas neste livro - vetor2, célula, grade e mapa - decida se ela pede herança, composição ou nenhuma das duas, e escreva a frase que sustenta a decisão. Guarde as quatro frases: o Cap. 9 as compara com as decisões que o código de fato tomou.

O código, extraído do Deriva

Todo trecho abaixo vem de exemplos/deriva/, que compila com `-std=c++17 -Wall -Wextra -Wpedantic` sem um aviso e passa `make verifica`. Nenhum foi digitado neste texto.

```
tipos/despacho.cpp:11 |----- C++ tem despacho simples-----
11 std::string resolver(const colisao& a, const colisao& b) {
12     // Duas perguntas de tipo, uma para cada operando, porque `virtual` resolve
13     // por um só. Com N tipos, esta função cresce com N² - e é exatamente esse
14     // crescimento que faz o Visitor existir.
15     const bool a_sonda = dynamic_cast<const sonda_c*>(&a) != nullptr;
16     const bool b_sonda = dynamic_cast<const sonda_c*>(&b) != nullptr;
17
18     if (a_sonda && b_sonda) return "sonda x sonda: as duas param";
19     if (a_sonda && !b_sonda) return "sonda x parede: a sonda para";
20     if (!a_sonda && b_sonda) return "parede x sonda: a sonda para";
21     return "parede x parede: nada acontece";
22 }
```

Duas perguntas de tipo, uma por operando, porque `virtual` resolve por um só. Com N tipos esta função cresce com N^2 , e é esse crescimento que faz o Visitor existir. Se houvesse despacho múltiplo, a função não existiria.

```
tipos/despacho.hpp:33 |----- forte por padrão, fraco por convite-----
33 struct fahrenheit {
34     // Sem `explicit`, `fahrenheit f = 30.0;` compilaria, e 30 graus Celsius
35     // viraria 30 Fahrenheit em silêncio. Com ele, o compilador exige a
36     // conversão escrita - e é aí que o sistema de tipos fica forte.
37     explicit fahrenheit(double v) : valor(v) {}
38     double valor;
39 };
```

Sem `explicit`, trinta graus Celsius viraria trinta Fahrenheit em silêncio. Com ele, o compilador exige a conversão escrita - e é aí que o sistema de tipos fica forte.

```
tipos/despacho.hpp:66 |----- trait em C++ é template, não construção-----  
66 struct tem_glifo : std::false_type {};
```

Em Rust ou Scala o trait é construção da linguagem; aqui é um template que responde sobre um tipo. O teste o aplica a sonda e a mapa, e a resposta vem sem executar nada.

UML leve

aula 06

O QUE ESTE CAPÍTULO ENTREGA

- ▶ Ler e produzir diagramas de classes UML básicos.
- ▶ Representar herança, composição, agregação e dependência em UML.
- ▶ Ler e produzir diagramas de sequência UML simples.
- ▶ Usar UML como rascunho de projeto antes de escrever código.

§6.1 Por que UML? A Modelagem Antes do Código

UML (Unified Modeling Language) é uma notação gráfica padronizada para modelar sistemas de software. Nesta disciplina usamos UML “leve”: apenas os elementos essenciais dos diagramas de classes e de sequência, sem burocracia.

O valor de fazer um diagrama de classes antes de codificar é semelhante ao de fazer um esboço arquitetônico antes de construir: é muito mais barato corrigir um retângulo no papel do que refatorar 500 linhas de código. Ferramentas tais como Mermaid (usada no site da disciplina), draw.io e PlantUML permitem criar diagramas rapidamente.

Há uma segunda razão, específica deste material, e ela vale mais que a primeira: o diagrama é onde a decisão entre herança e composição fica visível antes de custar caro. Trocar um losango por uma seta no papel é um traço; trocar `class mapa : public grade` por `class mapa { grade grade_; }` depois de vinte arquivos dependerem da primeira forma é uma tarde.

§6.2 Diagrama de Classes - Notação Essencial

A classe é um retângulo de três compartimentos: o nome no topo, os atributos no meio, as operações no fundo. A visibilidade vem como prefixo de cada linha - + para public, - para private, # para protected -, e o tipo de retorno de cada operação vem à direita, depois dos parênteses. É notação de leitura, e não de geração: nada aqui produz código, e o que a notação tem de fazer é caber no quadro da sala.

O diagrama abaixo é a grade de `include/deriva/grade.hpp` na notação `classDiagram`, e o que vale conferir nele é a ausência: nenhuma das cinco operações especiais aparece, porque grade segue a regra do zero, e o Cap. 9 é sobre por que isso é decisão e não omissão. As seis operações públicas são todas `[[nodiscard]]`, inclusive a sobrecarga não-const de `em()`, e todas `const` exceto o construtor e essa mesma sobrecarga, que existe justamente para deixar escrever na célula.

```

1 classDiagram
2   class grade {
3     -int largura_
4     -int altura_
5     -vector~celula~ celulas_
6     +grade(int largura, int altura)
7     +largura() int
8     +altura() int
9     +dentro(vetor2 p) bool
10    +em(vetor2 p) celula&
11    +bytes_das_celulas() size_t
12  }

```

Desenhe o seu antes de abrir o cabeçalho, e depois compare. Onde os dois divergi-rem, um dos dois está errado, e descobrir qual é o exercício 2 deste capítulo.

§6.3 Relacionamentos entre Classes

Relacionamento	Símbolo UML	Significado	Exemplo no Deriva
Herança (generalização)	Seta fechada vazia —▷	“é um” - subclasse herda da superclasse	class drone : public entidade
Composição forte	Losango preenchido ◆—	“tem um” - parte não existe sem o todo	class mapa { grade grade_;};
Composição fraca (agregação)	Losango vazio ◇—	“tem um” - parte pode existir sem o todo	Coleção de ponteiros para entidade (Cap. 12)
Dependência (uso)	Seta tracejada -->	“usa” - métrica de acoplamento	grade::em(vetor2) recebe vetor2 por parâmetro
Associação	Linha —	Relação estrutural genérica	marca_de_vida escreve no traço do instrumento

A distinção entre composição forte e agregação decide quem chama o destrutor, e é por isso que ela não é detalhe de notação. Em `class mapa { grade grade_; }`, a `grade` é um membro por valor: ela nasce quando o mapa nasce, morre quando o mapa morre, e o mapa não escreve uma linha de código para que isso aconteça. A alternativa - guardar um ponteiro para uma `grade` que vive em outro lugar - transfere a responsabilidade para quem nem aparece no diagrama, e é a origem da metade dos defeitos dos Caps. 9 e 12.

— As relações que o Deriva de fato tem

Vale conferir o diagrama contra o código, e nesta versão do Deriva a lista inteira cabe em cinco linhas:

- mapa **compõe** uma `grade`, uma `vetor2` (o ponto de entrada) e uma `marca_de_vida`, todas por valor, todas 1 para 1.
- `grade` **compõe** as `celula` num `std::vector`, 1 para largura × altura. Numa `grade` de 80 por 24 são 1920 células, e o Cap. 7 mostra por que 12 bytes por célula em vez de 16 importa nessa conta.

- `grade` **depende de** `vetor2`: ela o recebe como parâmetro em `dentro()` e em `em()`, e não guarda nenhum.
- `mapa` **depende de** `std::filesystem::path`, que é o parâmetro de `carregar()`, e de `std::optional<mapa>`, que é o seu retorno.
- `marca_de_vida` **depende de** `instrumento`, onde ela anota o nascimento e a morte de quem a carrega.

Nenhuma herança. A primeira relação de generalização do Deriva chega na v1.0 (Cap. 10), quando entidade ganhar sonda, drone e item - e é significativo que os nove primeiros capítulos deste livro se passem sem uma.

O diagrama da v0.3 tem as cinco relações da lista acima, e nenhuma sexta:

```

1 classDiagram
2   class mapa {
3     -string nome_
4     -grade grade_
5     -vetor2 entrada_
6     -marca_de_vida marca_
7     +carregar(path) optional~mapa~
8     +de_texto(string_view, string) optional~mapa~
9     +despejar() string
10  }
11  class grade {
12    -int largura_
13    -int altura_
14    -vector~celula~ celulas_
15    +dentro(vetor2) bool
16    +em(vetor2) celula&
17  }
18  class celula {
19    +int energia
20    +int massa
21    +char glifo
22    +char sigla
23  }
24  class vetor2 {
25    +int x
26    +int y
27    +manhattan(vetor2) int
28  }
29  class marca_de_vida {
30    -string nome_
31  }
32  class instrumento {
33    +anotar(string_view, string_view) void
34    +traco() vector~string~
35  }
36  mapa "1" *-- "1" grade
37  mapa "1" *-- "1" vetor2 : entrada
38  mapa "1" *-- "1" marca_de_vida
39  grade "1" *-- "larguraxaltura" celula
40  grade ..> vetor2 : parâmetro
41  marca_de_vida ..> instrumento

```

Uma diferença de versão entre o desenho e a verificação, e vale dizê-la em vez de esconder: o diagrama acima é o da v0.3, que é o Deriva que existe na sexta aula, e os dois trechos extraídos no fim deste capítulo vêm de testes/test_um1.cpp, que confere a hierarquia **inteira**, até a v1.7. Modelar antes de codificar significa desenhar o que ainda não existe, e o teste que confere o desenho só pode existir depois dele. O que este capítulo pede é que o desenho da v0.3 esteja certo sobre a v0.3, e que você volte aqui quando a hierarquia chegar - é para esse retorno que o teste está no fim.

DERIVA

O diagrama da v0.3 tem quatro classes e nenhuma herança, e as duas coisas são decisões. A generalização chega na v1.0, e o teste no fim deste capítulo já a confere - porque é ele que vai acusar o desenho quando ele envelhecer.

mapa compõe grade porque um mapa *tem* uma grade: ele tem nome e ponto de entrada, que grade nenhuma tem, e não faz sentido passar um mapa onde se espera uma grade. É a mesma pergunta do Cap. 5, agora com resposta no código.

Desenhe o diagrama da versão que vai escrever ANTES de escrevê-la, e confira-o contra o cabeçalho depois. Onde os dois divergirem, um dos dois está errado - e descobrir qual é o exercício.

§6.4 Diagrama de Sequência - Interação entre Objetos

O diagrama de sequência mostra como objetos interagem ao longo do tempo para realizar um caso de uso. É lido de cima para baixo, com o tempo fluindo para baixo e as mensagens trocadas horizontalmente entre as “lifelines” (linhas de vida).

No Deriva há uma coincidência útil: o traço de ciclo de vida do Cap. 8 é um diagrama de sequência em texto. Cada linha do traço é uma mensagem - construção, cópia, destruição - na ordem exata em que aconteceu, e é essa ordem que os testes afirmam. Assim, quando você desenhar o diagrama de sequência de um caso de uso do Deriva, existe um jeito de conferir se ele está certo: rodar o programa com a instrumentação ligada e comparar.

O caso de uso que vale desenhar é o carregamento de um setor, e ele tem dois desfechos, porque a ausência de resultado é o assunto: `mapa::carregar` devolve `std::optional<mapa>`, e o `nullopt` é resposta, e não falha.


```

1 sequenceDiagram
2   participant m as main()
3   participant mp as mapa (estático)
4   participant g as grade
5   m->>mp: carregar(caminho)
6   mp->>mp: exists() e is_regular_file()
7   alt arquivo ausente ou irregular
8     mp-->>m: nullopt
9   else arquivo lido
10    mp->>mp: de_texto(texto, nome)
11    mp->>mp: parte em fileiras (string_view, sem alocar)
12    alt fileira de largura divergente
13      mp-->>m: nullopt
14    else fileiras consistentes
15      mp->>g: grade(largura, altura)
16      g-->>mp: grade construída
17      mp->>g: em(p) para cada célula
18      mp->>mp: acha a entrada '@'
19      mp-->>m: optional~mapa~ com valor
20    end
21  end

```

Duas coisas conferem esse desenho contra o código. A primeira são os trechos de `src/mapa.cpp` extraídos no Cap. 3 e no Cap. 20, que são as duas funções desenhadas. A segunda é o traço: rodar `./build/deriva --replay roteiro.txt --traco` imprime uma linha por construção e por destruição, na ordem em que aconteceram, e cada linha do traço é uma mensagem do diagrama. Diagrama de sequência que discorda do traço está errado, e é a única espécie de UML deste livro que se pode reprovar automaticamente.

§6.5 Fluxo de Trabalho com UML Leve

1. Leia o enunciado e sublinhe os substantivos, que são candidatos a classe, e os verbos, que são candidatos a operação. No Deriva, “a sonda gasta energia para agir” dá sonda e energia_ de um lado e agir() do outro.
2. Esboce o diagrama de classes no papel ou em mermaid: atributos, operações e relacionamentos, e só os que você consegue defender.
3. Verifique os relacionamentos: é realmente “é um” (herança) ou “tem um” (composição)? Aplique o teste duplo do Cap. 5 - a parte tem estado que o todo não tem, e faz sentido passar o todo onde se espera a parte?
4. Para fluxos complexos, esboce um diagrama de sequência para o caso de uso principal, e inclua o caminho em que nada dá certo.
5. Só então comece a escrever código - o diagrama é o contrato.
6. Ao terminar, confira o diagrama contra o cabeçalho que você escreveu, e atualize-o quando a implementação revelar decisão de projeto não prevista. Diagrama que não acompanha o código deixa de ser documentação e passa a ser desinformação.

DICA

UML não é burocracia, é comunicação. Um diagrama de classes de 5 minutos no papel pode evitar 2 horas de refatoração. Mas não exagere: UML “leve” significa o suficiente para clarificar o design, não um manual técnico completo.

Exercícios Propostos

1. Desenhe em Mermaid o diagrama de classes para um sistema de biblioteca: Livro (título, ISBN, ano), Autor (nome, bio), Biblioteca (coleção de livros), Empréstimo (livro, usuário, data). Indique os relacionamentos corretos, e diga em cada caso por que não é herança.
2. Desenhe o diagrama de classes da v0.3 do Deriva a partir dos cabeçalhos em `exemplos/deriva/include/deriva/`, sem olhar a lista do §6.3. Depois compare com ela: você acertou as cinco relações e não inventou nenhuma sexta?
3. Escreva um diagrama de sequência em Mermaid para o empréstimo de livro do exercício 1: usuário solicita → biblioteca verifica disponibilidade → registra empréstimo → retorna confirmação. Inclua o desfecho em que o livro não está disponível.
4. Analise os seguintes pares e identifique se a relação é herança, composição ou nenhuma: (a) Avião e Motor; (b) Animal e Mamífero; (c) Conta Bancária e Transação; (d) Forma Geométrica e Círculo; (e) Carro e Proprietário; (f) mapa e grade; (g) grade e vetor2.
5. O Deriva da v1.0 (Cap. 10) terá a hierarquia entidade → sonda, drone, item. Desenhe agora o diagrama que você espera, com a seta de herança, o destrutor virtual na base e o método `desenhar()` puro. Guarde-o: no Cap. 10 você compara o seu desenho com o que a hierarquia de fato ficou, e o que interessa é a divergência.

O código, extraído do Deriva

Todo trecho abaixo vem de `exemplos/deriva/`, que compila com `-std=c++17 -Wall -Wextra -Wpedantic` sem um aviso e passa `make verifica`. Nenhum foi digitado neste texto.

```
┌─── testes/test_uuml.cpp:37 ──── o diagrama, conferido ────┐
37 TEST_CASE("relacao 3: composicao, e nao heranca") {
38     // `mapa` TEM uma grade. O desenho usa losango, e nao triangulo.
39     static_assert(!std::is_base_of_v<grade, mapa>);
40     // `mundo` TEM um mapa, e possui as entidades.
41     static_assert(!std::is_base_of_v<mapa, mundo>);
42     SUCCEED("as duas sao composicao, e o desenho tem de mostrar losango");
43 }
```

mapa TEM uma grade, e o desenho usa losango e não triângulo. Diagrama que ninguém confere envelhece calado, e a versão desenhada passa a descrever um sistema que já não existe.

```
┌─── testes/test_uuml.cpp:72 ──── o que o desenho não pode afirmar ────┐
72 // O que o desenho NAO deve afirmar, e este teste protege contra a tentacao.
73 TEST_CASE("sonda NAO e final, e o desenho nao pode dizer que e") {
74     static_assert(!std::is_final_v<sonda>,
75                  "era final na v1.0, e a v1.7 retirou a promessa");
76     static_assert(std::is_final_v<drone>, "esta continua sendo folha");
77     static_assert(std::is_final_v<item>);
78     static_assert(std::is_final_v<sonda_reparadora>);
79     SUCCEED("tres folhas e uma base intermediaria, e o desenho tem de refletir isso");
80 }
```

sonda era final na v1.0 e deixou de ser na v1.7. Este teste protege contra a tentação de o diagrama continuar dizendo que é - foi um diagrama envelhecido que motivou escrever este arquivo.

Classes e objetos; o contador de instâncias vivas

O QUE ESTE CAPÍTULO ENTREGA

- ▶ Declarar e definir classes em C++ com encapsulamento correto.
- ▶ Usar modificadores de acesso (`public`, `protected`, `private`) apropriadamente.
- ▶ Compreender o papel do ponteiro `this`.
- ▶ Aplicar `const-correctness`: métodos `const`, parâmetros `const`, referências `const`.
- ▶ Usar membros estáticos (`static`) para dados e comportamento compartilhados por todas as instâncias.
- ▶ Escrever o contador de instâncias vivas, e reconhecer o que ele acusa e o que ele não acusa.
- ▶ Determinar o tamanho de um objeto a partir da ordem de declaração dos seus membros.

§7.1 Declaração e Definição de Classes

Em C++, a convenção é separar a declaração (interface) em um arquivo de cabeçalho (`.hpp`) e a implementação (definição dos métodos) em um arquivo de implementação (`.cpp`). Isso reduz o tempo de recompilação e separa a interface do cliente da implementação.

A separação, porém, não é obrigatória, e o Deriva a usa de forma desigual de propósito. `grade` e `mapa` têm cabeçalho e implementação, porque os seus métodos têm corpo grande e validação. `vetor2` e `celula` vivem inteiros no cabeçalho, sem um `.cpp`: são agregados de dois e de quatro campos, com operadores `constexpr` de uma linha, e criar um arquivo de implementação para eles acrescentaria um arquivo e nenhuma informação.

O par completo está extraído no fim deste capítulo, e vale lê-lo nas duas metades. A interface de `grade` declara os métodos e não os define, e é o trecho que o Cap. 9 traz; a implementação está aqui, em `src/grade.cpp` · a lista que constrói uma vez só, com a definição qualificada por `grade::`, fora da classe, e a lista de inicialização que dimensiona `celulas_` uma vez só. O caso oposto, do tipo que não tem `.cpp` nenhum, é a `celula` extraída no mesmo lugar, e a `vetor2` que o Cap. 3 mostra: agregados cujos operadores são `constexpr` de uma linha e vivem inteiros no cabeçalho.

DERIVA

A primeira “classe” do Deriva (v0.1) é `vetor2`, e ela é um `struct` com dois `int` públicos.

Isso não é descuido, é a decisão: **não há invariante a proteger**. Qualquer par de inteiros é um `vetor2` válido, inclusive um fora da grade, porque `vetor2` também serve de deslocamento. Pôr os campos em `private` e escrever `x()` e `set_x()` acrescentaria cerimônia e nenhuma garantia.

`grade` é o contrário: largura e altura têm de ser positivas, e o vetor de células tem de ter exatamente largura $\times$ altura elementos. Aí o `private` está protegendo algo, e o construtor é quem estabelece a invariante. O critério é esse, e não a regra de que “atributo é sempre privado”.

§7.2 Modificadores de Acesso

Modificador	Quem pode acessar	Uso típico
<code>public</code>	Qualquer código	Interface da classe: construtores, getters, setters, operações
<code>private</code>	Apenas a própria classe	Dados internos, métodos auxiliares de implementação
<code>protected</code>	A própria classe e classes derivadas	Dados que subclasses precisam acessar diretamente

DICA

Regra geral: comece tudo como `private`. Torne `public` apenas o que é necessário para o contrato da classe. Use `protected` somente quando projetar herança - e mesmo assim, considere getters/setters `protected` em vez de dados diretamente `protected`.

Há uma exceção que este capítulo já usou, e vale nomeá-la para que a regra não seja aplicada sem pensar: quando a classe não tem invariante nenhuma, o `private` não protege nada, e o tipo é melhor como agregado de campos públicos. `vetor2` e `celula` são assim, e é por isso que ligações estruturadas funcionam sobre eles, como o Cap. 3 mostrou. Encapsular é meio, e a invariante é o fim.

§7.3 O Ponteiro `this`

Dentro de qualquer método não-estático existe um ponteiro para o objeto sobre o qual o método foi chamado, e ele se chama `this`. Está lá mesmo quando ninguém o escreve: entidade::pos() devolve pos_, e o que ela de fato devolve é `this->pos_`. Explicitá-lo se paga quando o objeto precisa ser mencionado inteiro, e não um membro dele.

Os dois usos que o Deriva tem são os do operador de atribuição, e estão em mapa: `:operator=`. O primeiro é `if (this != &o)`, a guarda de autoatribuição: sem ela, `m = m` liberaria os recursos do objeto antes de copiá-los de si mesmo. O segundo é `return *this`, que devolve uma referência ao próprio objeto e é o que permite escrever `a = b = c`, encadeando à direita como manda a linguagem.

Os dois estão extraídos no fim deste capítulo, em `src/ mapa.cpp` · os dois usos de `this` que o `Deriva` tem, junto com o comentário que explica por que o contador de instâncias vivas não se move numa atribuição: nenhum objeto nasceu, e nenhum morreu.

O `Deriva` **não** tem interface fluente, e é a decisão que fecha esta seção. Encadear `set_x().set_y()` exige que todo modificador devolva `*this`, e inventar essa cadeia no sistema para demonstrar `this` seria acrescentar cerimônia ao código a serviço do livro, que é a inversão que este material evita. Onde você encontrar interface fluente em biblioteca de terceiro - e vai encontrar, em montadores de consulta e de configuração - o mecanismo é este, e a partir daqui você o reconhece.

§7.4 Const-Correctness

const-correctness significa usar `const` em todos os lugares onde um objeto não deve ser modificado. É uma das práticas mais importantes de C++ moderno: ela torna as intenções explícitas, permite otimizações do compilador e detecta bugs.

No `Deriva`, `const` e `[[nodiscard]]` andam juntos, e a razão é que os dois marcam a mesma coisa por ângulos diferentes. Um método `const` que devolve valor e não altera nada tem exatamente uma finalidade, que é o valor devolvido; assim, descartar esse valor não pode ser intencional, e o `[[nodiscard]]` do Cap. 3 transforma o descarte em aviso. Toda função de acesso de `grade`, de `mapa` e de `vetor2` carrega os dois.

O par de sobrecargas de `grade::em()` merece atenção. Há uma versão `const` que devolve `const celula&` e uma versão não-`const` que devolve `celula&`, e as duas existem porque o compilador escolhe pela constância do objeto: numa `grade` `const`, só a primeira é viável, e ela não deixa ninguém escrever na célula. É assim que uma classe fica utilizável em contexto de leitura sem abrir mão da escrita em contexto de escrita.

O par está extraído no fim deste capítulo, em `include/ deriva/ grade.hpp` · o par de sobrecargas, com `[[nodiscard]]` nas duas e o comentário que registra a pré-condição: a verificação de limite é de quem chama, que já perguntou `dentro()`.

Do lado de quem consome a classe, o mesmo `const` aparece no parâmetro. A função livre `operator<<(std::ostream&, const mapa&)`, que o Cap. 15 extrai, recebe o `mapa` por referência constante e por isso só alcança método `const` - `despejar()`. Tirar aquele `const` não mudaria uma linha do corpo, e faria o programa deixar de compilar em todo lugar que passe um `mapa` constante.

▲ ATENÇÃO

Se um método logicamente não modifica o objeto, declare-o como `const`. Sem o `const`, você não pode chamar esse método em objetos `const` - o que limita a utilidade da classe em interfaces de leitura.

§7.5 Membros Estáticos e o Contador de Instâncias Vivas

Membros `static` pertencem à classe, e não a instâncias individuais. Uma variável `static` é compartilhada por todos os objetos da classe; um método `static` pode ser chamado sem que exista objeto algum.

A consequência que mais confunde, e que o interativo desta aula mostra, é que o **membro estático não está dentro do objeto**. Acrescentar um `static int` a uma classe não muda o `sizeof` dela: a variável vive uma única vez, em memória estática, e todos os objetos falam da mesma. É por isso que ela serve de contador.

— A parte de C++17: `inline static`

Antes do C++17, declarar um dado estático mutável exigia dois lugares. A declaração ia na classe, dentro do cabeçalho, e a definição ia em exatamente um `.cpp` - `int classe::vivos = 0;` - porque a declaração não reserva memória e a regra de uma definição só exige que a reserva aconteça uma vez em todo o programa. Esquecer a segunda linha produz o erro de ligação clássico, `undefined reference to`, que não aparece na compilação e só aparece no fim.

O C++17 acabou com a dança através de **variável inline**: `inline static int vivos = 0;` declara e define na mesma linha, dentro do cabeçalho, e o ligador aceita a repetição em todas as unidades de tradução que incluírem aquele arquivo. Não há `.cpp` a manter, e não há erro de ligação a esquecer.

Um detalhe que fecha o assunto: `static constexpr` já se comportava assim desde o C++17 para constantes, e o `inline static` generaliza o mesmo para estado mutável. O contador precisa mudar, logo `constexpr` não serve; `inline static` serve.

— O contador vivos, e por que ele é o exemplo canônico da disciplina

O laboratório desta disciplina não tem sanitizer nem Valgrind. Sem detector automático, a posse precisa ser verificável à mão, e o contador de instâncias vivas é a primeira das três técnicas que substituem a ferramenta ausente. A forma é curta o bastante para caber numa frase: **incrementa no construtor, decrementa no destrutor; se não fecha em zero no fim de main, um destrutor não rodou**.

Duas variáveis, e não uma. `vivos` sobe e desce; `criados` só sobe. A diferença entre elas diz quantos objetos morreram, e é o que permite responder à pergunta que aparece no Cap. 9: quantos objetos foram construídos para produzir um mapa? Se `criados` é três e o programa pediu um, houve duas cópias que ninguém pediu.

O incremento tem de estar em **todos** os construtores, e essa é a armadilha. O construtor de cópia de mapa incrementa o contador, e o comentário no código explica o motivo: sem esse incremento, o destrutor da cópia decrementaria algo que ninguém incrementou, e `vivos` fecharia negativo. Contador que fecha em número negativo não é falso alarme, é o mesmo defeito visto pelo outro lado.

O contador é o que sustenta a quarta condição do portão `make verifica`, apresentada no Cap. 2: `vivos=0` no fim da execução do roteiro. Sistema que passa nos testes mas termina com objeto vivo não passa.

— O que o contador não acusa

Ele conta objetos, e não recursos. Reconhecer esse limite é parte da lição, e a caça ao bug do Cap. 9 depende disso: na variante com cópia rasa, duas grades apontam para o mesmo buffer, os dois objetos são construídos e destruídos corretamente, o contador **fecha em zero** e o programa libera a mesma memória duas vezes. O contador não vê, porque nada de errado aconteceu com objetos.

O caso complementar está na variante do Cap. 8, em que `terminal_bruto` não tem destrutor: aí o contador é a única pista automática que existe, porque o ASan não conhece termos e não diria uma palavra sobre o modo do terminal vazado.

Duas outras limitações, para não sobrar dúvida. O contador não é seguro entre threads: o Cap. 22 mostra exatamente esta variável perdendo um incremento, e é o interativo de corrida de dados. E ele é escrito **à mão** em cada classe que o quer, o que é repetição deliberada: é ela que motiva, no Cap. 19, generalizá-lo em `contador_de_instancias<T>` por CRTP. Template que chega antes de o estudante sentir a repetição é solução para um problema que ele não teve.

As duas variáveis declaradas estão em `include/deriva/contador.hpp` · o detector de vazamento, e o contador em uso está em `src/mapa.cpp` · o contador em uso, os dois extraídos no fim deste capítulo: `++contador_mapa::vivos` e `++contador_mapa::criados` no construtor, e um destrutor de uma linha que só decrementa o primeiro. O construtor de cópia, que é onde o incremento deixa de ser óbvio e passa a ser obrigatório, é o assunto do §9.3, e é lá que ele está extraído.

— ATENÇÃO —

O idioma **pré-C++17** que você vai encontrar em código antigo são duas linhas em dois arquivos: `static int count_;` dentro da classe, no cabeçalho, e `int classe::count_ = 0;` em exatamente um `.cpp`. Reconheça-o pelo erro que ele produz quando a segunda linha falta - `undefined reference to`, na ligação, e não na compilação, que é onde ninguém está olhando. No Deriva, e nas suas entregas, o contador é `inline static int vivos = 0;` dentro do cabeçalho, sem definição em `.cpp` nenhum. Se você escrever as duas coisas, o ligador reclama de novo, e a mensagem é a menos informativa das duas.

§7.6 O Leiaute do Objeto em Memória

Um objeto em C++ é a soma dos seus membros, na ordem em que foram declarados, com o alinhamento que o compilador precisa respeitar. A ordem é sua; o alinhamento não é. Da combinação das duas sai o `sizeof`, e ele pode mudar sem que uma linha de significado mude.

A célula do Deriva é o caso mínimo e completo. São quatro campos - dois `int` de 4 bytes, dois `char` de 1 byte - e portanto 10 bytes de dado. Agrupada por tamanho, com os dois `int` primeiro, ela ocupa **12 bytes**: os dois inteiros nos offsets 0 e 4, os dois caracteres em 8 e 9, e 2 bytes de padding no fim para que o próximo elemento de um array comece alinhado a 4. Declarada na ordem em que se pensa nela, com o glifo primeiro porque é o que se vê, ela ocupa **16 bytes**: cada `char` seguido de um `int` obriga o compilador a inserir 3 bytes de padding para alinhar o inteiro.

Quatro bytes por célula parece nada. Numa grade de 80 por 24 são 1920 células, e a diferença é de 7,5 KB - 23 KB contra 30 KB, um terço a mais de memória, sem uma linha de código diferente e sem um campo a mais.

```
12 bytes
sizeof(celula)
aferido por
./build/deriva --leiaute
include/deriva/leiaute.hpp
```


O que faz esses números confiáveis não é o texto, é o compilador. `include/deriva/celula.hpp` termina com quatro `static_assert` que afirmam `sizeof(celula) == 12`, `sizeof(celula_ingenua) == 16`, `alignof(celula) == 4` e `offsetof(celula, glifo) == 8`. Se o leiaute mudar, o Deriva **não compila**, e este livro não passa a exibir número errado em silêncio. O trecho está extraído no fim deste capítulo, e vale ler os três blocos na ordem: a versão boa, a ingênua, e as quatro linhas que travam as duas.

A pergunta que fica para o Cap. 11: e quando a classe tem método virtual? Aí entra um ponteiro para a vtable como primeiro campo do objeto, e são mais 8 bytes por objeto, e não por classe. `include/deriva/leiaute.hpp` já mede isso, e o número é o mesmo que o interativo do site exibe.

Exercícios Propostos

1. Implemente `include/deriva/vetor2.hpp` do zero, sem olhar o arquivo: um agregado com `x` e `y`, os operadores de igualdade e desigualdade `constexpr`, `noexcept` e `[[nodiscard]]`, e um `static_assert` afirmando `sizeof(vetor2) == 8`. Compile com o conjunto de avisos do Cap. 2 e confirme que não há uma linha de aviso.
2. Implemente `grade` com cabeçalho e implementação separados: construtor que recebe largura e altura, `largura()`, `altura()`, `dentro(vetor2)` e o par de sobrecargas de `em(vetor2)`. Marque como `const` todos os métodos que não alteram o objeto, e como `[[nodiscard]]` todos os que só devolvem valor. Depois tente, de propósito, chamar a versão não-const de `em()` numa `grade const`, e leia a mensagem do compilador até entendê-la.
3. Acrescente o contador de instâncias vivas a `grade`, com `inline static int vivos` e `inline static int criados`. Escreva um `main` que: cria três grades, imprime o contador; destrói uma delas usando `escopo`, imprime; **copia** uma delas, imprime; e ao final confirma que `vivos` voltou a zero. Explique cada número impresso antes de rodar o programa, e só depois rode. Este é o portão do **LAB-04**.
4. Faça o contador fechar em número **negativo**, de propósito: declare o construtor de cópia e esqueça o incremento nele. Rode o exercício 3 outra vez. Por que o número é negativo, e o que isso diz sobre onde o incremento tem de estar?
5. Declare uma `struct` com um `char`, um `double` e um `int`, nessa ordem, e escreva o `static_assert` com o tamanho que você prevê. Compile. Se errou, reordene os campos para obter o menor tamanho possível e escreva o `static_assert` do novo valor. Explique, em duas linhas, de onde vem cada byte de padding.
6. Explique com um exemplo concreto por que o compilador recusa chamar um método não-const em um objeto const. O que acontece internamente com o tipo do ponteiro `this`? Como isso ajuda a detectar bugs?

O código, extraído do Deriva

Todo trecho abaixo vem de `exemplos/deriva/`, que compila com `-std=c++17 -Wall -Wextra -Wpedantic` sem um aviso e passa `make verifica`. Nenhum foi digitado neste texto.

```

| include/deriva/celula.hpp:9 |----- 12 bytes -----
9  /// Uma célula da grade da estação.
10 ///
11 /// A ordem dos membros aqui NÃO é estética: agrupada por tamanho, a célula
12 /// ocupa 12 bytes. Declarada na ordem "natural" - o glifo primeiro, porque é
13 /// o que se vê - , ocupa 16. Numa grade de 80×24 são 1920 células, logo 23 KB
14 /// contra 30 KB, e é o mesmo código.
15 ///
16 /// `celula_ingenua` existe só para ser medida: é o contraexemplo do
17 /// interativo "inspetor de objeto" (Aula 07), e os static_assert abaixo são o
18 /// que impede o material de mentir sobre esses números.
19 struct celula {
20     int energia = 0;    // 4 B @ 0
21     int massa = 0;      // 4 B @ 4
22     char glifo = '.';    // 1 B @ 8
23     char sigla = ' ';    // 1 B @ 9
24                             //      + 2 B de padding no fim
25 };

```

Agrupada por tamanho. Os offsets no comentário não são estimativa: os static_assert no fim do arquivo os afirmam, e o build falha se mudarem.

```

| include/deriva/celula.hpp:27 |----- 16 bytes -----
27 /// A MESMA célula, na ordem em que se pensa nela. Não use.
28 struct celula_ingenua {
29     char glifo = '.';    // 1 B @ 0    + 3 B de padding
30     int energia = 0;     // 4 B @ 4
31     char sigla = ' ';    // 1 B @ 8    + 3 B de padding
32     int massa = 0;       // 4 B @ 12
33 };

```

A MESMA célula, na ordem em que se pensa nela. Quatro bytes a mais por célula, 7,5 KB numa grade de 80×24, e nada em troca.

```

| include/deriva/celula.hpp:35 |----- os números, aferidos -----
35 static_assert(sizeof(celula) == 12, "agrupada por tamanho");
36 static_assert(sizeof(celula_ingenua) == 16, "a ordem ingênua custa 4 bytes por célula");
37 static_assert(alignof(celula) == 4, "o maior membro manda no alinhamento");
38 static_-
assert(offsetof(celula, glifo) == 8, "os dois int primeiro, sem buraco entre eles");

```

É esta linha que impede o material de mentir. Se o leiaute mudar, o Deriva não compila - e o interativo da Aula 07 não passa a exibir número errado em silêncio.

```

| include/deriva/contador.hpp:7 |----- o detector de vazamento -----|
7 /// O detector de vazamento da disciplina, e ele não depende de sanitizer.
8 ///
9 /// `vivos` é `inline static` - variável inline é C++17, e é o que dispensa a
10 /// definição num .cpp. Incrementa no construtor, decrementa no destrutor: se
11 /// não fecha em zero no fim de `main`, um destrutor não rodou.
12 ///
13 /// Nesta versão o contador é escrito À MÃO em cada classe que o quer. Essa
14 /// repetição é deliberada: é ela que motiva o `contador_de_instancias<T>` por
15 /// CRTP na Aula 19. Template que chega antes de o estudante sentir a
16 /// repetição é solução para um problema que ele não teve.
17 ///
18 /// Não é seguro entre threads. A Aula 22 mostra exatamente esta variável
19 /// perdendo um incremento - e é o interativo de corrida de dados.
20 struct contador_mapa {
21     inline static int vivos = 0;
22     inline static int criados = 0;
23 };

```

`inline static` é variável inline, C++17: dispensa a definição num .cpp. A repetição em cada classe é deliberada - é ela que motiva o `contador_de_instancias<T>` da Aula 19.

```

| src/grade.cpp:32 |----- a lista que constrói uma vez só -----|
32 // `celulas_` é construído UMA vez, com o tamanho certo. Atribuir no corpo o
33 // construiria vazio antes e o redimensionaria depois.
34 grade::grade(int largura, int altura)
35     : largura_(exigir_positivo(largura, "largura")),
36       altura_(exigir_positivo(altura, "altura")),
37       celulas_(static_cast<std::size_t>(largura_) * static_cast<std::size_t>(altura_ -
)) {}

```

`celulas_` é construído com o tamanho certo de uma vez. Atribuir no corpo o construtoria vazio antes e o redimensionaria depois.

```

src/mapa.cpp:31 | os dois usos de 'this' que o Deriva tem
31 mapa& mapa::operator=(const mapa& o) {
32     if (this != &o) {
33         nome_ = o.nome_;
34         grade_ = o.grade_;
35         entrada_ = o.entrada_;
36         marca_ = o.marca_;
37     }
38     return *this; // o contador não muda: nenhum objeto nasceu nem morreu

```

`this != &o` para recusar a autoatribuição, e `return *this` para encadear. São os dois únicos lugares em que `this` aparece explícito em todo o projeto - fora deles, ele é implícito e escrevê-lo é ruído.

```

include/deriva/grade.hpp:33 | o par de sobrecargas
33 /// Sem verificação de limite: quem chama já perguntou 'dentro()'.
34 /// O 'operator[]' de 'mapa' chega na v1.5 (Aula 15).
35 [[nodiscard]] const celula& em(vetor2 p) const;
36 [[nodiscard]] celula& em(vetor2 p);

```

`[[nodiscard]]` nas duas, e `const` só na primeira: é a versão não-`const` que existe para deixar escrever, e é por isso que ela não pode ser `const`.

```

testes/test_grade.cpp:11 | o objeto const, e o que ele alcança
11 TEST_CASE("grade conhece seus limites") {
12     const grade g(20, 10);
13     REQUIRE(g.largura() == 20);
14     REQUIRE(g.altura() == 10);
15     REQUIRE(g.dentro({0, 0}));
16     REQUIRE(g.dentro({19, 9}));
17     REQUIRE_FALSE(g.dentro({20, 9}));
18     REQUIRE_FALSE(g.dentro({-1, 0}));
19 }

```

`const grade g(20, 10)` só deixa chamar método `const`, e todas as chamadas aqui o são. Trocar uma delas pela sobrecarga não-`const` de `em()` faz este teste deixar de compilar - e é essa recusa, e não a palavra no cabeçalho, que é a garantia.

```
src/mape.cpp:10 | o contador em uso
10 mape::mape(std::string nome, int largura, int altura)
11     : nome_(std::move(nome)),
12       grade_(largura, altura),
13       marca_("mape:" + nome_) {
14     ++contador_mape::vivos;
15     ++contador_mape::criados;
16 }
17
18 mape::~mape() { --contador_mape::vivos; }
```

`++vivos` e `++criados` no construtor, `--vivos` no destrutor. A declaração `inline static` está no cabeçalho, e é ela que dispensa a definição num `.cpp`.

Ciclo de vida e RAI

O QUE ESTE CAPÍTULO ENTREGA

- ▶ Dominar os tipos de construtores: padrão, parametrizado, cópia, conversão.
- ▶ Compreender listas de inicialização e a ordem obrigatória de inicialização.
- ▶ Implementar destrutores corretamente e entender quando são chamados.
- ▶ Aplicar o idioma RAI para gerenciamento automático de recursos.
- ▶ Instrumentar construtores e destrutores, e ler o traço que eles produzem.
- ▶ Determinar a ordem exata de construção e destruição de um trecho sem executá-lo.

§8.1 O que é o Ciclo de Vida de um Objeto?

Todo objeto em C++ passa por três fases: construção (alocação de memória e inicialização dos membros), uso (chamadas de métodos, acesso a atributos) e destruição (liberação de recursos e desalocação de memória). O programador controla esse ciclo por meio de construtores e destrutores.

A habilidade que este capítulo existe para formar é mais estreita e mais difícil que isso: **determinar a ordem exata em que as três fases acontecem, para todos os objetos de um trecho, sem executar o trecho.** Quem só sabe responder compilando confia na execução observada, e não no texto lido; e a prova desta disciplina é em papel justamente por isso.

§8.2 Tipos de Construtores

Construtor Padrão

Chamado quando o objeto é criado sem argumentos. Se não declarado e a classe não tiver outros construtores, o compilador gera um automaticamente. Membros de tipo primitivo ficam não-inicializados, com valor indefinido, salvo se tiverem inicializador na própria declaração - como `int x = 0;` em `vetor2`, que é o que faz `vetor2 p;` nascer na origem em vez de nascer com lixo.

Construtor Parametrizado

É o construtor que estabelece a invariante da classe. Em `grade`, ele recebe largura e altura, valida as duas e dimensiona o vetor de células; a partir do momento em que ele retorna, não existe `grade` com dimensão não-positiva no programa.

— Construtor de Cópia

Chamado quando um objeto é inicializado a partir de outro do mesmo tipo: `mapa b{a}` ou `mapa b = a`. Se não declarado, o compilador gera uma cópia membro a membro, e ela funciona bem para classes cujos membros já saibam se copiar - o que é o caso de `grade`, cujo único membro que gerencia recurso é um `std::vector`.

Os três aparecem juntos no código extraído no fim deste capítulo, em `src/instrumento.cpp`: `marca_de_vida` tem construtor parametrizado que move o nome recebido, na lista de inicialização; destrutor de uma linha; e construtor de cópia que existe para marcar a cópia no traço. Vale ler o que o de cópia faz com o nome, porque é disso que o Cap. 9 depende - ele monta `o.nome_ + ""`, e é essa aspa que distingue, no traço, a cópia que ninguém pediu de uma construção legítima.

§8.3 Listas de Inicialização - Por que Usar Sempre

A lista de inicialização (`: membro_{expr}` antes do corpo do construtor) inicializa os membros antes que o corpo do construtor execute. Isso é mais eficiente, porque evita construção padrão seguida de atribuição, e é obrigatório para membros `const`, referências e classes sem construtor padrão.

Há um motivo mais forte, e ele custou uma depuração no Deriva. **Validação de argumento que protege a construção de um membro tem de estar na lista, e não no corpo.** A primeira versão do construtor de `grade` validava no corpo, e um teste pegou o erro: com `grade(5, -1)`, o `-1` virava `size_t` enorme na conta do tamanho, e o `std::vector` lançava `length_error` antes de o corpo rodar. A mensagem que o estudante veria era `cannot create std::vector larger than max_size()`, que não diz nada sobre a `grade` nem sobre a altura negativa que ele digitou.

A ordem em que a lista roda é a **ordem de declaração dos membros na classe**, e não a ordem em que você escreve a lista. Em `grade`, `largura_` e `altura_` vêm declarados antes de `celulas_`, e é por isso que a validação chega a tempo. Inverter a declaração dos membros quebraria a validação sem mudar uma linha do construtor, e é o tipo de defeito que só aparece no caso de erro.

O trecho extraído no fim deste capítulo mostra essa função de validação. Vale notar que ela está fora da classe, é livre, e devolve o valor validado, para poder ser chamada *dentro* da lista: `largura_(exigir_positivo(largura, "largura"))`.

O contraste entre inicializar na lista e atribuir no corpo está na mesma seção extraída, no construtor de `marca_de_vida`: `: nome_(std::move(nome))` constrói o membro uma vez, já com o valor recebido movido para dentro dele. Escrito no corpo, `nome_ = nome` construiria a `std::string` vazia primeiro e só depois lhe daria o valor, e o movimento se perderia numa cópia.

— ▲ ATENÇÃO —

A ordem de inicialização na lista NÃO é a ordem que você escreve - é a ordem de declaração dos membros na classe. Escreva a lista na mesma ordem dos membros para evitar bugs sutis (e warnings do compilador com `-Wall`).

§8.4 Destruítores e a Ordem de Destruição

O destrutor (`~NomeDaClasse()`) é chamado automaticamente quando o objeto sai de escopo, no caso de objeto automático, ou quando `delete` é chamado, no caso de objeto dinâmico. A ordem de destruição é sempre o inverso da ordem de construção, e isso vale em três níveis ao mesmo tempo: entre objetos do mesmo escopo, entre escopos aninhados, e entre os membros de um mesmo objeto.

A garantia mais importante, porém, é a que trata do caso em que nada dá certo: **quando uma exceção é lançada, os destrutores dos objetos já construídos no caminho são chamados, de dentro para fora, enquanto a pilha é desenrolada.** A exceção não pula os destrutores; ela os chama. É essa garantia, e nada mais, que faz RAII funcionar, e é por isso que ela tem um teste próprio no Deriva, extraído no fim deste capítulo.

A ordem simples, sem exceção nenhuma, é a primeira coisa a se saber prever, e o Deriva a afirma num teste em vez de a comentar: `testes/test_ciclo_de_vida.cpp` · a ordem, afirmada linha por linha monta dois objetos no escopo externo e um no interno, e compara o traço inteiro com a cadeia esperada, linha por linha. O escopo interno fecha primeiro; o externo desmonta na ordem inversa da construção. O outro trecho extraído no fim deste capítulo é o mesmo experimento com uma exceção no meio, e é ele que mostra a garantia do parágrafo acima em funcionamento.

§8.5 RAII - Resource Acquisition Is Initialization

RAII é o idioma mais importante de C++ moderno. A ideia: ao adquirir um recurso (arquivo, mutex, conexão de rede, memória dinâmica, modo do terminal), inicialize-o no construtor; libere-o no destrutor. Como o destrutor é chamado automaticamente ao sair do escopo, inclusive no desenrolar por exceção, o recurso é sempre liberado.


```

1 #include <cstdio>
2 #include <fstream>
3 #include <iostream>
4 #include <stdexcept>
5 #include <string>
6
7 // Sem RAII: a liberação é manual, e depende de o fluxo chegar até ela
8 void ler_arquivo_a_mao(const std::string& caminho) {
9     std::FILE* f = std::fopen(caminho.c_str(), "r");
10    // ... processar ...
11    // e se uma exceção for lançada aqui?
12    std::fclose(f);           // pode não executar
13 }
14
15 // Com RAII: o std::ifstream fecha no destrutor, desenrolar de pilha incluído
16 void ler_arquivo_com_raii(const std::string& caminho) {
17     std::ifstream arquivo{caminho}; // abre no construtor
18     if (!arquivo) throw std::runtime_error("arquivo não encontrado");
19     std::string linha;
20     while (std::getline(arquivo, linha))
21         std::cout << linha << "\n";
22 }                                     // arquivo.close() aqui, sempre

```

✓ DICA

Use sempre tipos RAII da STL: `std::ifstream/ofstream` para arquivos, `std::unique_ptr` para memória dinâmica, `std::lock_guard` para mutexes. Você raramente precisa escrever seu próprio RAII.

O “raramente” da caixa acima merece qualificação, porque este capítulo termina justamente com o caso em que você precisa. A STL cobre os recursos que a STL conhece; o modo do terminal não é um deles, e nem o descritor de arquivo do sistema, nem a trava de banco, nem a entrada em `/tmp`. Para esses, a forma é a mesma, e escrevê-la uma vez é o que faz entender por que a STL a usa.

O caso em que se precisa escrever o próprio RAII está no fim deste capítulo, extraído do Deriva: o construtor de `terminal_bruto` adquire o modo bruto do terminal e o destrutor o devolve, e o §8.7 é sobre por que esse recurso, e não um arquivo, é o exemplo que a disciplina escolheu.

§8.6 Instrumentação de Ciclo de Vida

A ordem de construção e destruição é determinada pelo texto do programa, e a seção anterior afirmou qual é. Afirmar não basta: sem sanitizar no laboratório, a segunda das três técnicas de verificação da disciplina é fazer os próprios construtores e destrutores **imprimirem a própria execução**, de modo que você compare o traço com o que previu.

A classe que faz isso no Deriva é `marca_de_vida`. Ela anota `+nome` ao nascer e `-nome` ao morrer, e é posta como membro de quem quiser ser rastreado - `mapa` carrega uma, chamada `mapa`: mais o nome do setor. É RAI aplicado ao próprio rastreamento: o registro do fim acontece porque o destrutor roda, e não porque alguém lembrou de chamar algo.

Três decisões de projeto valem ser lidas, porque nenhuma delas é óbvia.

O traço vai para um vetor, não para `std::cout`. Dois motivos: teste pode afirmar sobre um vetor, e a saída do programa não fica poluída, o que importa porque a saída do Deriva é comparada byte a byte pelo replay do Cap. 2. Quem quer ver ao vivo liga `instrumento::ecoa(true)`, e a linha vai também para `stderr`.

A cópia é anotada com sufixo próprio. O construtor de cópia de `marca_de_vida` monta o nome como `o.nome_ + "'"`, e é assim que você vê, no traço, que houve uma cópia que ninguém pediu. Sem essa marca, a cópia apareceria como um `+mapa: teste a mais`, indistinguível de uma construção legítima - e é exatamente a confusão que o Cap. 9 precisa desfazer.

O que se compara é o despejo inteiro, e não linha a linha. `instrumento::despejo()` devolve uma linha por evento, na ordem, e é essa cadeia de caracteres que o teste compara com a esperada. Comparar a cadeia inteira faz o teste falhar quando a *ordem* muda, e não só quando um evento falta.

O que a instrumentação acusa é aquilo que o contador do Cap. 7 não vê: o contador diz que faltou um destrutor; o traço diz em que **ordem** os destrutores rodaram, e qual deles rodou fora de hora. Os dois juntos, mais o `gdb` do Cap. 2, cobrem o terreno do sanitizar ausente.

DERIVA

O portão do **LAB-05** é que o traço impresso bata, linha por linha, com o esperado no roteiro. E o roteiro se escreve **antes** de rodar o programa.

A ordem importa porque é o regime de predição da disciplina: enquanto você não escreveu a previsão, o traço é só uma saída de programa; depois de escrita, ele é o veredito sobre a sua leitura do código. Previsão errada é resultado de aprendizagem, e não é avaliada.

Rode `./build/deriva --replay roteiro.txt --traco` para ver o traço da execução gravada.

§8.7 terminal_bruto: RAII com Consequência Física

O melhor exemplo de RAII que existe está no Deriva, e ele não é metáfora.

Um jogo de terminal por turnos precisa ler tecla sem esperar Enter e sem que a tecla apareça na tela. Isso se faz pondo o terminal em **modo bruto**, que é uma chamada de sistema que altera o estado do dispositivo. A classe `terminal_bruto` faz a chamada no construtor, guarda o estado anterior, e restaura o estado anterior no destrutor.

Se o destrutor não rodar, o terminal fica sem eco e sem Enter **depois** que o programa sai. O estado do terminal é um recurso do sistema operacional, e ele **sobre-vive ao processo**: ninguém devolve nada quando o programa morre, e o conserto é digitar reset seguido de Enter, às cegas.

É aqui que RAII deixa de ser doutrina. Vazar memória é abstrato: o sistema operacional recolhe tudo no fim do processo, e o estudante nunca sente. Vazar o modo do terminal significa que a próxima coisa que ele digitar não vai aparecer na tela. A variante `variantes/v0.2-quebrada/` omite o destrutor de propósito, e o roteiro de sala de aula é: rodar com `< /dev/null` primeiro, onde não há tty e portanto não há o que estragar, ler o contador de instâncias vivas, e só então rodar de verdade num terminal que se possa perder.

Nenhuma ferramenta acusa esse vazamento. O ASan não conhece termos. O contador de instâncias vivas acusa **um objeto** sem destrutor, e é a única pista automática disponível.

Duas outras decisões da classe fecham o capítulo e abrem o seguinte.

A guarda de isatty. O construtor pergunta se a entrada padrão é um terminal de verdade antes de mexer em qualquer coisa. Em teste, em pipe ou em integração contínua a saída não é tty: o objeto se constrói, conta como vivo, e não altera nada. Sem essa guarda, `ctest` deixaria o terminal de quem roda os testes em estado imprevisível, o que é um preço absurdo a pagar por rodar a suíte.

Não é copiável nem movível. Há exatamente um terminal, e posse de recurso único não se duplica: copiar um `terminal_bruto` produziria dois objetos cujos destrutores restaurariam o mesmo estado duas vezes. As duas operações são declaradas `= delete`, e isso é a **regra do três** na sua forma mais curta - assunto do Cap. 9, onde a mesma pergunta reaparece com resposta diferente.

▲ ATENÇÃO

A pergunta que fecha este capítulo, e que vale responder por escrito: **onde mais, no código que você escreve, existe recurso que sobrevive ao processo?** Arquivo aberto com bloqueio, socket, entrada em `/tmp`, linha travada em banco de dados, semáforo nomeado. Todos têm a mesma forma, e nenhum deles é recolhido pelo sistema operacional quando o seu programa termina de qualquer jeito.

Exercícios Propostos

1. Implemente o construtor de grade validando **na lista de inicialização**, com uma função livre que devolve o valor validado. Depois escreva a mesma classe validando **no corpo** e chame `grade(5, -1)` nas duas versões. Transcreva as duas mensagens de erro e explique por que a segunda não menciona a grade.
2. Acrescente uma `marca_de_vida` como membro da sua grade e escreva um `main` com dois objetos no escopo externo e um no interno. **Antes de compilar**, escreva no papel o traço que você espera, linha por linha. Depois rode, compare, e explique cada divergência. Este é o portão do **LAB-05**.
3. Repita o exercício 2 lançando uma exceção de dentro do escopo interno, sem capturá-la ali. Preveja o traço antes de rodar. Quantos destrutores rodam, em que ordem, e qual objeto ainda não havia sido construído quando a exceção passou?
4. Escreva `terminal_bruto`: construtor que põe o terminal em modo bruto guardando o estado anterior, destrutor que restaura, guarda de `isatty`, e as operações de cópia declaradas = `delete`. Rode. Depois **apague o destrutor** e rode outra vez com `< /dev/null`, lendo o contador de instâncias vivas. Só então, se quiser, rode num terminal que você possa perder - e conserte com `reset`.
5. Escreva uma classe `guarda_de_arquivo` que use RAII para garantir que um `FILE*` seja fechado quando o guarda sai de escopo: o construtor abre e lança exceção se falhar, o destrutor chama `fclose()`. Demonstre, com o traço do exercício 2, que o arquivo é fechado mesmo quando uma exceção é lançada no meio do processamento.
6. Explique com código o que acontece quando um objeto com um `std::vector` como membro é copiado. O vetor é copiado raso, compartilhando a memória, ou profundo, com memória nova? Como isso mudaria se o membro fosse um `celula*` cru? Guarde a resposta: ela é o assunto do Cap. 9 e da primeira caça ao bug.

O código, extraído do Deriva

Todo trecho abaixo vem de `exemplos/deriva/`, que compila com `-std=c++17 -Wall -Wextra -Wpedantic` sem um aviso e passa `make verifica`. Nenhum foi digitado neste texto.

```

src/grade.cpp:9 |----- validar NA lista de inicialização-----
 9 /// Valida NA lista de inicialização, não no corpo.
10 ///
11 /// A primeira versão deste construtor validava no corpo, e um teste pegou o
12 /// erro: com `grade(5, -1)`, o `-1` virava `size_t` enorme e o `std::vector`
13 /// lançava `length_error` **antes** de o corpo rodar. A mensagem que o
14 /// estudante veria era "cannot create std::vector larger than max_size()", que
15 /// não diz nada sobre a grade.
16 ///
17 /// A lista de inicialização roda inteira antes da primeira linha do corpo:
18 /// validação de argumento que protege a construção de um membro tem de estar
19 /// na lista, e a ordem é a de DECLARAÇÃO dos membros - `largura_` e `altura_`
20 /// vêm antes de `celulas_`, então a checagem acontece a tempo (Aula 08).
21 [[nodiscard]] int exigir_positivo(int valor, const char* qual) {
22     if (valor <= 0) {
23         throw std::invalid_argument(std::string("grade: ") + qual +
24                                     " precisa ser positiva, recebeu " +
25                                     std::to_string(valor));
26     }

```

A primeira versão validava no corpo, e um teste pegou: com `grade(5, -1)` o vetor lançava `length_error` antes de o corpo rodar. A lista de inicialização acontece inteira antes da primeira linha do corpo.

```

src/instrumento.cpp:40 |----- instrumentação de ciclo de vida-----
40 marca_de_vida::marca_de_vida(std::string nome) : nome_(std::move(nome)) {
41     instrumento::anotar("+", nome_);
42 }
43
44 marca_de_vida::~marca_de_vida() { instrumento::anotar("-", nome_); }
45
46 // A cópia é anotada com sufixo próprio: é assim que o estudante vê, no traço,
47 // que houve uma cópia que ele não pediu.
48 marca_de_vida::marca_de_vida(const marca_de_vida& o) : nome_(o.nome_ + "'") {
49     instrumento::anotar("+", nome_);
50 }
51
52 marca_de_vida& marca_de_vida::operator=(const marca_de_vida& o) {

```

Construtor e destrutor imprimindo a própria execução. Substitui o sanitizer que o laboratório não tem: o estudante LÊ o traço e o compara com o roteiro.

```

src/terminal_bruto.cpp:14 |----- o construtor adquire -----
14 terminal_bruto::terminal_bruto() {
15     ++contador_terminal::vivos;
16     ++contador_terminal::criados;
17
18     if (::isatty(STDIN_FILENO) == 0) return; // pipe, teste, CI: não há o que mexer
19     if (::tcgetattr(STDIN_FILENO, &salvo()) != 0) return;
20
21     termios bruto = salvo();
22
23     // ICANON e ECHO são macros de `int`. `~(ICANON | ECHO)` é um int NEGATIVO,
24     // e atribuí-lo a `c_lflag`, que é `unsigned`, muda o valor de -11 para
25     // 4294967285. O resultado funciona por acidente do complemento de dois, e
26     // -Wsign-conversion está certo em reclamar. Converter primeiro, complementar
27     // depois: aí a operação toda acontece em `tcflag_t`.
28     const tcflag_t cru = static_cast<tcflag_t>(ICANON) | static_cast<tcflag_t>(ECHO);
29     bruto.c_lflag &= ~cru;
30     bruto.c_cc[VMIN] = 1;
31     bruto.c_cc[VTIME] = 0;
32     if (::tcsetattr(STDIN_FILENO, TCSAFLUSH, &bruto) == 0) ativo_ = true;
33 }

```

isatty primeiro: em pipe, em teste ou em CI não há modo bruto a alterar. Sem essa guarda, ctest deixaria o terminal de quem roda os testes em estado imprevisível.

```

src/terminal_bruto.cpp:35 |----- o destrutor libera -----
35 // É esta linha que separa "o terminal do estudante volta ao normal" de "ele
36 // digita `reset` às cegas". A variante quebrada não a tem.
37 terminal_bruto::~~terminal_bruto() {
38     if (ativo_) ::tcsetattr(STDIN_FILENO, TCSAFLUSH, &salvo());
39     --contador_terminal::vivos;
40 }

```

Duas linhas. São elas que separam “o terminal volta ao normal” de “o estudante digita reset às cegas” - e o recurso aqui sobrevive ao processo, então ninguém vai desfazer isso por ele.

```

┌─| testes/test_ciclo_de_vida.cpp:41 |────────────────── o desenrolar da pilha ─────────┐
41  TEST_CASE("a excecao nao pula destrutor - ela o chama ao desenrolar a pilha") {
42      instrumento::limpar();
43      try {
44          [[maybe_unused]] const marca_de_vida externo("externo");
45          {
46              [[maybe_unused]] const marca_de_vida interno("interno");
47              throw std::runtime_error("falha de leitura do setor");
48          }

```

O teste afirma a ordem exata, linha por linha. A exceção não pula os destrutores: ela os chama, de dentro para fora. É essa garantia, e nada mais, que faz RAII funcionar.

```

┌─| testes/test_ciclo_de_vida.cpp:19 |────────────────── a ordem, afirmada linha por linha ─────────┐
19  TEST_CASE("a ordem de destruicao e a inversa da de construcao") {
20      instrumento::limpar();
21      {
22          // `[[maybe_unused]]` diz ao compilador o que `(void)x` dizia por gesto:
23          // o objeto existe pelo efeito do construtor e do destrutor, não pelo
24          // valor. O atributo é C++17 e é o lugar mais natural dele em todo o
25          // Deriva (Aula 03).
26          [[maybe_unused]] const marca_de_vida a("a");
27          [[maybe_unused]] const marca_de_vida b("b");
28          {
29              [[maybe_unused]] const marca_de_vida c("c");
30          }
31      }
32      REQUIRE(instrumento::despejo() ==
33          "+a\n"
34          "+b\n"
35          "+c\n"
36          "-c\n" // o escopo interno fecha primeiro
37          "-b\n" // e o externo desmonta na ordem inversa
38          "-a\n");

```

Dois objetos no escopo externo, um no interno, e a saída comparada texto a texto. Nenhuma linha de código pediu essa ordem.

Operações especiais: a regra do zero e do três

O QUE ESTE CAPÍTULO ENTREGA

- ▶ Compreender a regra do zero: quando o compilador gera tudo corretamente.
- ▶ Aplicar a regra dos três: destrutor, cópia e atribuição de cópia.
- ▶ Reconhecer, no texto do código, a cópia rasa e o que ela produz.
- ▶ Conduzir a caça ao bug 1 na ordem: reproduzir, explicar em uma frase, corrigir, provar.

§9.1 O que o Compilador Gera Automaticamente

Para qualquer classe, o compilador pode gerar automaticamente até seis operações especiais: construtor padrão, destrutor, construtor de cópia, operador de atribuição por cópia, construtor de movimento e operador de atribuição por movimento. Quando e o quê ele gera dependem do que você declarou.

Se você declarar...	O compilador NÃO gera...
Qualquer construtor	Construtor padrão
Destrutor	Construtor/atribuição de movimento
Construtor de cópia	Construtor/atribuição de movimento
Atribuição de cópia	Construtor/atribuição de movimento
Construtor de movimento	Construtor/atribuição de cópia
Atribuição de movimento	Construtor/atribuição de cópia

A segunda linha da tabela é a que se costuma explicar errado, e no Deriva ela está medida. `mapa` declara destrutor e cópia, porque precisa mexer no contador de instâncias vivas; com isso, o compilador **deixa de gerar** as operações de movimento, e é a v1.4, no Cap. 14, que passa a declará-las à mão.

O que se espera dessa correção é que ela poupe construções, e não é o que acontece. `testes/test_mapa.cpp` mede os dois estados do código e afirma que **de_texto custa duas construções com o construtor de movimento e duas sem ele**: a do `mapa` local e a do que vai para dentro do `std::optional`. Dois objetos nascem nos dois casos, e o contador, que conta objetos, não registra diferença nenhuma.

O que o movimento muda é o **custo** da segunda construção. Com ele, o ponteiro de heap da grade troca de dono; sem ele, as células são copiadas uma a uma, e a única forma de ver a diferença é comparar endereços, que é o que o teste faz. O trecho extraído no fim deste capítulo é o do contador, e a lição que ele carrega é sobre o instrumento antes de ser sobre o movimento: o contador do Cap. 7 não distingue cópia de movimento, e saber o que o instrumento não vê vale tanto quanto saber usá-lo.

```
duas construções em de_texto
aferido por
./build/testes "mapa"
testes/test_mapa.cpp
```


Guarde o número e guarde a surpresa. Os dois são o argumento da regra dos cinco, no Cap. 14, medidos antes de serem explicados.

§9.2 A Regra do Zero

Se a sua classe gerencia recursos apenas através de tipos RAII da biblioteca padrão - `std::vector`, `std::string`, `std::unique_ptr` e assemelhados - declare **zero** das seis operações especiais. O compilador gera versões corretas, que chamam as operações corretas de cada membro.

A regra do zero é a primeira escolha, e não um caso particular. Ela tem duas vantagens, e a segunda é a que mais paga a longo prazo.

A primeira é que as operações geradas são melhores do que as que escreveríamos: o compilador copia cada membro com o construtor de cópia daquele membro, move cada membro com o de movimento, destrói na ordem inversa da construção, e não erra.

A segunda é que elas **não podem ficar desatualizadas**. Quando um membro novo aparece na classe, as operações geradas passam a tratá-lo, no mesmo commit, sem que ninguém precise lembrar. Cópia escrita à mão, em contraste, precisa ser atualizada a cada membro novo, e o defeito de esquecer não produz aviso nenhum: o campo simplesmente não é copiado, e a classe passa a ter dois estados que divergem em silêncio.

A grade do Deriva é a regra do zero em forma pura. Ela não declara destrutor, cópia, movimento nem atribuição; o único membro que gerencia recurso é o `std::vector<celula>`, e ele já sabe se copiar, se mover e se destruir. O trecho está extraído no fim deste capítulo, e o que vale ler nele é o que **não** está escrito.

E vale copiar o hábito que o cabeçalho de `grade.hpp` tem: listar em comentário as operações que não foram declaradas, para que quem lê saiba que a omissão é decisão, e não esquecimento.

— | DICA | —

Prefira sempre a regra do zero. Se você precisa declarar qualquer uma das seis operações especiais, provavelmente está gerenciando um recurso que deveria ser encapsulado em sua própria classe RAII - e então essa classe interna seguiria a regra dos cinco.

Vale registrar a exceção que o Deriva tem, para que a regra não pareça absoluta. mapa declara destrutor e cópia, e não por gerenciar recurso: é para mexer no contador de instâncias vivas. O comentário no código explica por que o incremento na cópia é obrigatório - sem ele, o destrutor da cópia decrementaria algo que ninguém incrementou, e vivos fecharia negativo. É um caso legítimo de sair da regra do zero, e o preço dele é o do §9.1: a partir do momento em que o destrutor foi declarado, as cinco operações passaram a ser responsabilidade de quem escreve a classe, e o Cap. 14 é onde essa conta se fecha.

§9.3 A Regra dos Três

Onde a classe gerencia o recurso à mão - `celula*` cru com posse, descritor de arquivo, socket, o modo do terminal de `terminal_bruto` -, as três operações andam juntas: destrutor, construtor de cópia e `operator=` de cópia. E o defeito não é a falta das três, que ninguém comete: é declarar uma e esquecer as outras duas. O compilador continua gerando a que faltou, e a que ele gera copia membro a membro, de forma que o ponteiro é duplicado e o recurso não. Ficam dois objetos apontando para o mesmo recurso, cada um com um destrutor pronto para liberá-lo.

A regra tem duas formas de ser cumprida, e a mais curta é a que se esquece.

A **forma longa** é escrever as três: destrutor que libera, construtor de cópia que faz cópia profunda, e operador de atribuição que aloca o novo antes de liberar o antigo, com guarda de autoatribuição. Funciona, e é o que mapa faz.

A **forma curta** é declarar as duas operações de cópia = delete, e é o que `terminal_bruto` faz no Cap. 8. Ela cabe quando o recurso é único por natureza: há exatamente um terminal, e posse de recurso único não se duplica. Aí a regra dos três se cumpre em duas linhas, e a classe deixa de ter uma categoria inteira de defeitos por não ter a operação.

As duas formas estão extraídas no fim deste capítulo, e as duas vêm de código que compila.

A longa está em `src/mapa.cpp` · a forma longa da regra do três: o construtor de cópia de mapa, que copia cada membro na lista de inicialização e incrementa o contador de instâncias vivas. O incremento é a parte que não se pode esquecer, e o comentário do código diz por quê - sem ele, o destrutor da cópia decrementaria algo que ninguém incrementou, e vivos fecharia negativo, que é o mesmo defeito visto pelo lado em que ninguém desconfia. A terceira operação da forma longa, o `operator=`, está extraída no §7.3, com a guarda de autoatribuição `if (this != &o)` e o `return *this;` a ordem que ela obedece é sempre a mesma, e vale memorizá-la: **aloca o novo, copia para dentro dele, libera o antigo, assume o novo**. Liberar antes de alocar é o defeito clássico, porque a alocação pode falhar e o objeto fica sem nenhum dos dois estados.

A curta está em `include/deriva/terminal_bruto.hpp` · a forma curta: duas linhas de = delete e o comentário de cabeçalho que explica por que elas são a resposta certa ali, e não apenas a mais econômica.

▲ ATENÇÃO

A forma longa com ponteiro cru aparece neste livro num lugar só, e de propósito: a grade da v0.3-quebrada, extraída no fim deste capítulo, que é a caça ao bug do §9.4. Em código seu, use `std::vector` e a regra do zero. Se você se pega escrevendo a regra dos três, pergunte-se antes: existe um tipo RAII da biblioteca padrão que já faz isso?

§9.4 Caça ao Bug 1: cópia rasa em grade

A primeira das três caças ao bug do semestre é esta, na semana 5, no mesmo encontro da Prova 1. Cada dupla recebe uma versão do Deriva plausível e errada, e o protocolo tem quatro passos, na ordem, e a ordem é cobrada: **reproduzir a falha; explicá-la em uma frase, por escrito, antes de tocar no código; corrigir; provar a correção**. Os sete itens da rubrica do Cap. 4 se aplicam por escrito.

O defeito está em `variantes/v0.3-quebrada/`. A grade de lá guarda `celula*` cru em vez de `std::vector<celula>`, declara `~grade()` que faz `delete[]`, e **não** declara construtor de cópia nem atribuição. A cópia que o compilador gera copia o ponteiro, e não o buffer: duas grades, um buffer, dois `delete[]`. O trecho está extraído no fim deste capítulo, marcado como quebrado de propósito.

— A ordem de observação

Cinco olhares sobre o mesmo defeito, e o que se aprende é tanto o que cada instrumento vê quanto o que ele deixa de ver.

1. **Sem ferramenta nenhuma.** Rodar o programa mostra que escrever numa célula de `b` mudou a célula correspondente de `a`, e que os dois buffers têm o mesmo endereço. Isto basta para o diagnóstico, e é o único caminho disponível no laboratório. Comece por aqui.
2. **Com o contador de instâncias vivas.** Acrescente vivos a grade como no Cap. 7. Ele **fecha em zero**, e é isso que engana: dois objetos nasceram e dois morreram, corretamente. O contador conta objetos, e não recursos, e descobrir esse limite é parte da lição.
3. **Com o alocador.** Nesta máquina a própria `glibc` aborta, com `free(): double free detected in tcache 2`. Não é para se confiar: o `tcache` pega *este* arranjo de alocações, e um `delete[]` duplo separado por outras alocações passa. Detecção pelo alocador é sorte de leiaute.
4. **Com o ASan,** se a sua máquina tiver: `attempting double-free`, com as duas pilhas de liberação nomeadas. É a confirmação com garantia, e é a diferença entre “abortou” e “abortou e disse onde”.
5. **Com o compilador.** `-Wall -Wextra -Wpedantic` não emite uma palavra. Nenhum aviso existe para isto, porque a linguagem está fazendo exatamente o que foi pedida: cópia membro a membro é o que ela deve gerar.

— O conserto, e por que o conserto certo é o mais curto

Escrever o construtor de cópia e a atribuição funciona, e é o que a regra dos três manda. É a resposta correta, e não é a melhor.

A melhor é **apagar o destrutor**: troque `celula*` por `std::vector<celula>` e a classe volta à regra do zero, com as cinco operações corretas de graça - inclusive as duas de movimento, que só serão apresentadas no Cap. 14 e que a versão escrita à mão não teria. É o que a `v0.3` boa faz, e é o código extraído no §9.2.

A pergunta que fecha a discussão: *quantas linhas a mais a versão com `vector` tem?* Nenhuma.

▲ ATENÇÃO

Declarar destrutor e esquecer a cópia é a violação mais barata da regra do três, e a que mais sobrevive à revisão de código. Ela não produz aviso, não produz falha em teste que só verifique valores, e o contador de instâncias vivas fecha em zero. O item 2 da rubrica do Cap. 4 existe por causa dela: **se há destrutor, há também cópia e atribuição, ou as duas estão = delete.**

Exercícios Propostos

1. Implemente grade pela regra do zero, com `std::vector<celula>` internamente. Verifique, com testes, que cópia, atribuição e destruição funcionam sem que você escreva uma linha para isso: escreva numa célula da cópia e confirme que a original não mudou.
2. Tome a sua grade do exercício 1 e a quebre de propósito, como a v0.3-quebrada: troque o vetor por `celula*`, acrescente o destrutor com `delete[]`, e **não** escreva a cópia. Percorra os cinco olhares do §9.4 nesta ordem, e escreva para cada um o que ele acusou e o que ele deixou passar.
3. Escreva a frase única que explica o defeito do exercício 2, **antes** de consertá-lo. A frase precisa dizer o que acontece, e não o que fazer. Depois conserte pelas duas vias - a forma longa da regra dos três, e o retorno à regra do zero - e compare o número de linhas de cada conserto.
4. `terminal_bruto` cumpre a regra dos três em duas linhas de `= delete`. Explique por que a forma longa seria errada nesse caso, e não apenas desnecessária. O que faria o segundo destrutor?
5. Rode `testes/test_mapa.cpp` e confirme quantas construções de `_texto` custa. Depois responda, sem consultar o Cap. 14: qual das seis operações especiais, se existisse em `mapa`, mudaria o **custo** de uma dessas construções sem mudar o **número** delas, e por que o contador de instâncias vivas é incapaz de mostrar essa diferença?

O código, extraído do Deriva

Todo trecho abaixo vem de `exemplos/deriva/`, que compila com `-std=c++17 -Wall -Wextra -Wpedantic` sem um aviso e passa `make verifica`. Nenhum foi digitado neste texto.

```

| include/deriva/grade.hpp:13 |----- regra do zero -----
13 /// A grade de células da estação.
14 ///
15 /// Não declara destrutor, cópia, movimento nem atribuição: é a **regra do
16 /// zero** (Aula 09). O único membro que gerencia recurso é o
17 /// `std::vector`, e ele já sabe se copiar, se mover e se destruir
18 /// corretamente. As cinco operações que o compilador gera são melhores do que
19 /// as que escreveríamos aqui - e não podem ficar desatualizadas quando um
20 /// membro novo aparecer.
21 ///
22 /// A variante `variantes/v0.3-quebrada/` faz o oposto: guarda `celula*` cru,
23 /// declara destrutor e esquece a cópia. É a **caça ao bug 1** da semana 5, e
24 /// o contador de instâncias vivas é o que a acusa.
25 class grade {
26 public:
27     grade(int largura, int altura);
28
29     [[nodiscard]] int largura() const noexcept { return largura_; }
30     [[nodiscard]] int altura() const noexcept { return altura_; }
31     [[nodiscard]] bool dentro(vetor2 p) const noexcept;
32
33     /// Sem verificação de limite: quem chama já perguntou `dentro()`.
34     /// O `operator[]` de `mapa` chega na v1.5 (Aula 15).
35     [[nodiscard]] const celula& em(vetor2 p) const;
36     [[nodiscard]] celula& em(vetor2 p);
37
38     /// Quantos bytes as células ocupam de fato. Serve à Aula 07: com
39     /// `celula_ingenua` este número cresce um terço sem nada mudar de
40     /// significado.
41     [[nodiscard]] std::size_t bytes_das_celulas() const noexcept;
42
43 private:
44     int largura_;
45     int altura_;
46     std::vector<celula> celulas_;
47 };
|-----

```

Nenhuma das cinco operações especiais é declarada. O único membro que gerencia recurso é o `std::vector`, e as operações que o compilador gera são melhores que as que escreveríamos - e não ficam desatualizadas quando um membro novo aparecer.

```

|▲ variantes/v0.3-quebrada/grade_quebrada.cpp:26|———— quebrado de propósito · cópia rasa · CAÇA AO BUG 1
26 /// A grade da v0.3, com uma diferença: o buffer é um ponteiro cru, e a posse
27 /// dele é do destrutor.
28 class grade {
29 public:
30     grade(int largura, int altura)
31         : largura_(largura),
32           altura_(altura),
33           dados_(new celula[static_cast<std::size_t>(largura) *
34                           static_cast<std::size_t>(altura)]) {}
35
36     // Destrutor declarado. A partir daqui, a cópia gerada pelo compilador
37     // passou a ser um erro - e o compilador não avisa, porque gerar cópia
38     // membro a membro é exatamente o que a linguagem manda fazer.
39     ~grade() { delete[] dados_; }
40
41     // FALTA: grade(const grade&)
42     // FALTA: grade& operator=(const grade&)
43     // (é a regra do três, e ela está pela metade)
44
45     [[nodiscard]] celula& em(int x, int y) {
46         return dados_[static_cast<std::size_t>(y) * static_cast<std::size_t>(largura_) +
47                       static_cast<std::size_t>(x)];
48     }
49     [[nodiscard]] const celula* buffer() const noexcept { return dados_; }
50
51 private:
52     int largura_;
53     int altura_;
54     celula* dados_;
55 };

```

Destrutor declarado, cópia esquecida: a violação mais barata da regra do três, e a que mais sobrevive à revisão. -Wall -Wextra -Wpedantic não emite uma palavra sobre isto.

```

┌───| testes/test_mapa.cpp:60 |────────────────────────────────── o contador como portão ───────────────────────────────────┐
60 TEST_CASE("o contador de instancias vivas fecha em zero") {
61     deriva::zerar_contadores();
62     REQUIRE(deriva::contador_mapa::vivos == 0);
63     {
64         const auto m = mapa::de_texto(kSetor, "teste");
65         REQUIRE(deriva::contador_mapa::vivos == 1);
66         {
67             const mapa copia = *m;    // cópia: nasce um objeto, e o contador sabe
68             REQUIRE(deriva::contador_mapa::vivos == 2);
69             REQUIRE(copia.despejar() == m->despejar());
70         }
    }
}
└────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────┘

```

Três objetos nascem para um carregar, e o contador soma os três. Repare no que ele NÃO distingue: depois de a Aula 14 acrescentar o construtor de movimento, o número continua três, porque continuam sendo três nascimentos - o que mudou foi o custo de cada um. O contador conta objetos, não alocações, e saber o que o instrumento não vê vale tanto quanto saber usá-lo.

```

┌───| src/mapa.cpp:20 |────────────────────────────────── a forma longa da regra do três ───────────────────────────────────┐
20 // Destrutor declarado → a cópia precisa ser declarada também: é a regra do
21 // três (Aula 09). Aqui ela é rasa de propósito? Não: `grade_` é um vector, e
22 // copiá-lo copia as células. O que a cópia manual acrescenta é só o
23 // incremento do contador - sem ele, `vivos` fecharia negativo, porque o
24 // destrutor da cópia decrementaria algo que ninguém incrementou.
25 mapa::mapa(const mapa& o)
26     : nome_(o.nome_), grade_(o.grade_), entrada_(o.entrada_), marca_(o.marca_) {
27     ++contador_mapa::vivos;
28     ++contador_mapa::criados;
29 }
└────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────┘

```

O incremento na cópia é obrigatório: sem ele o destrutor da cópia decrementaria algo que ninguém incrementou, e vivos fecharia negativo - o contador passaria a mentir na direção que ninguém desconfia.

```

┌─| include/deriva/terminal_bruto.hpp:9 |────────────────────────── a forma curta ───────────────────┐
9  /// Põe o terminal em modo bruto no construtor e o restaura no destrutor.
10 ///
11 /// É a melhor demonstração de RAII que existe, e não é metáfora: se o
12 /// destrutor não rodar, o terminal do estudante fica sem eco e sem Enter
13 /// **depois** que o programa sai. Ele descobre RAII digitando `reset` às
14 /// cegas.
15 ///
16 /// A variante `variantes/v0.2-quebrada/` omite o destrutor de propósito.
17 ///
18 /// Não é copiável nem movível: há exatamente um terminal, e posse de recurso
19 /// único não se duplica. Declarar as duas apagadas é a **regra do três**
20 /// (Aula 09) na sua forma mais curta.
21 class terminal_bruto {
22 public:
23     terminal_bruto();
24     ~terminal_bruto();
25
26     terminal_bruto(const terminal_bruto&) = delete;
27     terminal_bruto& operator=(const terminal_bruto&) = delete;
28
29     /// Verdadeiro quando havia um terminal de verdade para alterar. Em teste,
30     /// em pipe ou em CI a saída não é tty: o objeto se constrói, conta como
31     /// vivo, e não mexe em nada. Sem isso, `ctest` deixaria o terminal de quem
32     /// roda os testes em estado imprevisível.
33     [[nodiscard]] bool ativo() const noexcept { return ativo_; }
34
35 private:
36     bool ativo_ = false;
37 };

```

Duas linhas de `= delete`, e a regra do três está cumprida. Há exatamente um terminal, e posse de recurso único não se duplica.

Hierarquias, posse e despacho

Herança antes de ponteiro inteligente: virtuais, posse exclusiva e compartilhada, movimento e a regra dos cinco, operadores, testes com replay, diamante e RTTI

Herança simples

O QUE ESTE CAPÍTULO ENTREGA

- ▶ Modelar hierarquias de classes com herança pública.
- ▶ Distinguir herança (is-a) de composição (has-a), e decidir entre as duas por pergunta de domínio.
- ▶ Compreender o encadeamento de construtores e destrutores na hierarquia.
- ▶ Usar `protected` corretamente.
- ▶ Localizar o subobjeto base no leiaute do objeto derivado, e dizer quanto ele custa.

§10.1 Herança como Modelagem de Domínio

Herança modela a relação “é um” (is-a). Antes de usar herança, pergunte: “essa relação realmente é is-a, ou é has-a?” Abusar de herança onde composição seria mais apropriado é um dos erros mais comuns em POO.

Esta pergunta já apareceu uma vez, no Cap. 9, e a resposta de lá foi composição. mapa tem nome, ponto de entrada e uma grade; ele não *é* uma grade, e passar um mapa onde se espera uma grade não faz sentido nenhum, porque o mapa não é substituível por ela nem ela por ele. O comentário de cabeçalho de `mapa.hpp` registra a decisão, e o registro é parte do trabalho: composição escolhida sem que ninguém saiba que houve escolha vira, três meses depois, uma herança que alguém acrescenta “para reaproveitar”.

Agora a resposta muda. Uma sonda de inspeção *é* uma entidade da estação; um drone *é* uma entidade da estação; um item no chão *é* uma entidade da estação. As três têm posição na grade, as três aparecem no render, e o código que desenha o setor precisa poder percorrer as três sem saber qual é qual. Isto é is-a, e é o caso em que a herança paga.

Use herança quando...	Use composição quando...
A relação é genuinamente is-a	A relação é has-a ou uses-a
A classe derivada pode ser usada no lugar da base (Liskov)	A funcionalidade é implementada em termos de outra sem expor interface
Você quer polimorfismo dinâmico (virtual)	Você quer flexibilidade: trocar implementação em runtime
A hierarquia é estável e bem compreendida	A hierarquia cresceria demais ou seria frágil

A segunda linha da tabela é a que o Cap. 24 vai cobrar pelo nome, com o princípio de substituição de Liskov, e vale antecipar o teste em uma frase: se existe uma função que aceita entidade& e que passa a se comportar errado quando recebe um drone, ou a função está errada ou o drone não é uma entidade. Não há terceira possibilidade, e é sempre a hierarquia que sai perdendo.

Herança também tem preço, e o preço é de acoplamento em tempo de compilação. A derivada carrega o subobjeto base dentro de si, vê tudo o que a base declarou protected, e depende de cada uma dessas decisões para sempre; mudar um campo protected da base recompila e pode quebrar toda derivada que existir, inclusive as que estiverem em outro repositório. Composição não tem esse alcance, porque o membro composto continua atrás da sua própria interface. A herança pública é, por isso, a mais forte das dependências que uma classe pode declarar sobre outra, e é o argumento para que ela seja rasa: no Deriva, dois níveis, e o terceiro só aparece na v1.7, quando o diamante do Cap. 17 exigir.

► DERIVA

A hierarquia da v1.0 tem três derivadas e nenhuma a mais, e cada uma existe por um requisito do jogo, e não por simetria:

- sonda é o que o estudante controla: recebe comando do teclado e se move.
- drone é o que se move sozinho, uma vez por turno, a partir da semente do sorteio.
- item não se move nunca: fica no chão até que alguém passe por cima.

O que as três compartilham é pouco - a posição e o glifo - e é exatamente por ser pouco que a base é pequena. Base grande é a forma mais comum de hierarquia errada: ela obriga toda derivada a carregar o campo que só uma delas usa.

§10.2 Sintaxe e Tipos de Herança em C++

```

1 class base {};
2
3 // Herança pública (é um): a interface da base continua pública na derivada
4 class derivada_publica : public base {};
5
6 // Herança protegida: a interface da base vira protected na derivada
7 class derivada_protegida : protected base {};
8
9 // Herança privada (implementada em termos de): a interface vira private
10 class derivada_privada : private base {};

```

Na prática, herança pública é a mais comum (modelagem is-a com polimorfismo). Herança privada é uma alternativa à composição quando precisamos de acesso à implementação interna da base (raro).

Duas armadilhas de sintaxe, e as duas custam tempo de laboratório. A primeira é que `class` sem qualificador herda **privado** por omissão, enquanto `struct` herda público; escrever `class drone : entidade` compila e produz uma hierarquia em que nenhum ponteiro `entidade*` aceita um `drone*`, com uma mensagem de erro que fala de “conversão inacessível” e não de herança. A segunda é que a base precisa estar **completa** no ponto da declaração: declaração adiantada de `class entidade`; basta para um ponteiro e não basta para herdar, porque o compilador precisa saber o tamanho do subobjeto base para montar o derivado.

No Deriva, toda herança é pública, e a razão é a do §10.1: ela existe para que o render percorra entidade sem saber de subtipo. Herança privada resolveria um problema que o sistema não tem.

§10.3 A Hierarquia do Deriva - v1.0

A v1.0 é a primeira versão em que o Deriva tem mais de um tipo de coisa dentro da estação, e o desenho dela responde a uma pergunta só: o que o laço de render precisa saber?

A resposta é: nada além da posição e do glifo. Percorrer o setor e desenhar significa, para cada entidade, pedir a posição, pedir o glifo, e escrever o caractere na coluna e na fileira correspondentes. Nem o render nem o laço de turno precisam saber se aquilo é uma sonda, um drone ou um item, e é essa ignorância que a base declara. Todo campo que ficar na base sem que o render ou o turno precisem dele é campo no lugar errado.

pos fica na base, porque as três derivadas têm posição e porque o render a lê. O rumo da patrulha fica no drone, e é o único campo dele. A carga, que o Cap. 11 usa para medir quanto o `vptr` custa quando a derivada acrescenta dado, não está no drone de verdade: ela vive em `leiaute::drone_com_carga`, um par de medição que não entra no jogo - e o cabeçalho de lá diz isso na primeira linha. O nome do item fica no item. A cada campo, a mesma pergunta.

Sobre `protected`: ele é o acesso do meio, e é o mais fácil de usar errado. Campo `protected` na base é parte da interface da base para as derivadas, com a mesma força de um campo público, exceto que a plateia é menor - e uma interface é tão difícil de mudar quanto o número de linhas que dependem dela. A regra prática

que este material segue é a do Cap. 7 estendida: campo `private` na base, com função de acesso `protected` quando a derivada precisar escrever nele. Assim a base conserva a possibilidade de trocar a representação, e a derivada continua a poder mexer no estado.

As duas metades desse desenho estão no trecho no fim deste capítulo, extraído do arquivo que compila. A entidade guarda `pos_` privada, recebe a posição no construtor `explicit`, e devolve-a por `pos()`; a sonda herda com `public`, e a lista de inicialização dela nomeia a base antes do campo próprio, `: entidade(pos)`, `energia_(energia)`, que é a ordem do §10.4 escrita em uma linha.

Duas diferenças em relação ao que este capítulo descreveu, e as duas são deliberadas. A primeira é que o arquivo já está na v1.1: `glifo()` e `nome()` são puras, o destrutor é virtual, e o comentário do cabeçalho diz em que versão cada linha entrou. Leia-o agora pela hierarquia, e volte a ele no Cap. 11 pelo virtual.

Vale desfazer uma confusão de trilha antes que ela apareça na leitura. A v1.0 traz final e não traz `override`, e as duas palavras não chegam juntas porque não dependem da mesma coisa. final **em classe** diz que ninguém herda dela, e é o que `drone` e `item` declaram desde a primeira versão: são folhas do desenho, e a palavra não pede virtual nenhum para valer. `override`, ao contrário, só existe quando há virtual na base para sobrescrever, então ele entra na v1.1, junto de `desenhar()` e `agir()` virtuais, e é por isso que o cabeçalho que você lê aqui já o traz nas quatro sobrescritas de cada derivada.

A segunda diferença responde à pergunta que o Cap. 11 vai fazer, e a resposta é uma decisão do sistema: a entidade do tronco **nasce com destrutor virtual**, na v1.0, e nunca o perde. A regressão que a caça ao bug 2 precisa não mora no tronco, mora em `exemplos/deriva/variantes/v1.1-quebrada/`, ao lado das outras variantes quebradas de propósito, com o `LEIA-ME.md` dizendo o que foi retirado e por quê. Entregar uma versão do tronco com defeito conhecido, e depois pedir ao estudante que a conserte, ensinaria que o repositório principal pode estar errado; a variante isolada ensina o contrário, que é o que se quer.

A segunda é sobre o `protected`, e ela vale como resposta à regra prática de dois parágrafos acima: no Deriva a posição sai por `pos()` e entra por `mover_para()`, as duas públicas, e função de acesso `protected` não aparece em nenhuma das quatro classes. A razão é que nenhuma derivada precisou escrever na posição por conta própria; quem move o drone é o `agir()` dele, que chama `mover_para` como qualquer cliente chamaria. A regra continua sendo a que se cobra quando a derivada precisar do estado da base para escrita, e este sistema não precisou.

▲ ATENÇÃO

SEMPRE declare o destrutor de classes base como virtual.

Sem virtual `~entidade()`, deletar uma sonda* através de um ponteiro `entidade*` chama apenas o destrutor de entidade - o destrutor de sonda NÃO é chamado, e o que ele liberaria fica vazado. O Cap. 11 mede esse vazamento com o contador `vivos` e localiza o destrutor que não roda com o `gdb`; é a caça ao bug 2. Aqui, guarde a regra e a razão dela.

§10.4 Construtores e Destrutores na Hierarquia

A ordem é fixa e não é escolha de ninguém: **a base se constrói primeiro, e se destrói por último**. Construir a derivada antes da base seria construir um objeto cujo subobjeto base ainda não existe, e o construtor da derivada pode chamar função da base na primeira linha do corpo; a linguagem garante que, quando essa linha roda, a base está pronta.

Três consequências, e a terceira é a que aparece na prova.

A primeira é que a base é inicializada na **lista de inicialização** da derivada, e antes de qualquer membro da derivada, mesmo que seja escrita depois deles. Se a base não tem construtor padrão e a lista não a nomeia, a compilação falha, e a mensagem é sobre construtor ausente e não sobre ordem.

A segunda é que o construtor da base não vê nada da derivada. Não há como o construtor de entidade consultar a carga do drone, porque no instante em que ele roda a parte drone do objeto ainda é memória não inicializada. É a mesma razão pela qual chamar função virtual em construtor não faz o que se espera, e o Cap. 11 fecha esse assunto.

A terceira é que a destruição é o espelho exato: o destrutor da derivada roda inteiro, e só então o da base. Isto é observável, e o Cap. 8 já deu o instrumento: `marca_de_vida` imprime a própria construção e a própria destruição.

É afirmado, e não descrito. Os dois últimos trechos no fim deste capítulo vêm de testes/`test_ciclo_de_vida.cpp`, e o primeiro deles trava a ordem inteira em seis linhas - `+base`, `+membro`, `+derivada`, e depois `-derivada`, `-membro`, `-base` -, com um `REQUIRE` por posição. O segundo isola a consequência que o Cap. 11 vai cobrar: o corpo do construtor da derivada roda por último, então nada do que ele faz está disponível ao construtor da base, e função virtual chamada de dentro da base chama a versão da base.

Repare em uma decisão do par instrumentado, porque ela é de projeto e não de teste: as duas classes marcadas são **locais ao arquivo de teste**, e não a entidade e o drone do jogo. Pôr `marca_de_vida` na base do jogo faria toda entidade anotar no traço e mudaria o despejo que o portão de replay compara byte a byte, o que custaria a terceira das quatro condições de `make verifica`. É o mesmo desenho de `include/deriva/leiaute.hpp`, que mantém fora do jogo as classes que existem para medir. O regime de predição desta disciplina consiste em escrever as seis linhas antes de rodar.

DIAGRAMA UML

Diagrama UML: hierarquia de entidades do Deriva v1.0 - entidade $\triangleleft$ — sonda, drone, item. Seta fechada de generalização das três derivadas para a base; na base, pos e o glifo; em cada derivada, só o campo que é dela (o rumo do drone, o nome do item). A sonda não tem campo novo, e é o caso que o §10.5 mede.

§10.5 O Subobjeto Base no Leiaute do Objeto

Um objeto derivado **contém** um objeto base. Isto não é figura de linguagem: o subobjeto base ocupa bytes contíguos dentro do derivado, normalmente no começo, e é por isso que um `drone*` se converte em `entidade*` sem custo nenhum - o endereço é o mesmo, e só o tipo estático muda.

O par mínimo que mede isso está em `include/deriva/leiaute.hpp`, e ele existe para que os números que o interativo desta aula exhibe sejam os que o compilador produz. `entidade_simples` tem só a posição, e ocupa **8 bytes**, que são os dois `int` do `vetor2`. `drone_simples` herda dela, não acrescenta campo nenhum, não tem método virtual, e ocupa **8 bytes** também. O `static_assert` no arquivo diz a razão em quatro palavras: herdar de classe não-polimórfica é grátis.

Grátis em espaço, e nada mais. O acoplamento do §10.1 continua lá inteiro, e é ele, não o `sizeof`, que decide se a herança valeu.

A pergunta que fica para o Cap. 11 é a que muda o número: e quando a base ganha um método virtual? Aí o compilador põe um ponteiro para a `vtable` como primeiro campo do objeto, e são mais 8 bytes por objeto, e não por classe. Os cinco `static_assert` de `leiaute.hpp` afirmam os dois lados dessa conta, e o trecho está extraído no fim do Cap. 11.

Os cinco trechos no fim deste capítulo são a hierarquia inteira em código que compila: a base com a posição privada e as duas virtuais puras, a sonda e o `final` que teve de sair dela quando a v1.7 acrescentou a `sonda_reparadora`, o `descrever()` que é a moldura não-virtual sobre virtuais, e o par de casos que afirma a ordem do §10.4.

8 bytes

`sizeof(entidade_simples)`

aferido por

`./build/deriva --leiaute`

`include/deriva/leiaute.hpp`

Exercícios Propostos

1. Escreva a entidade da v1.0 e as três derivadas - `sonda`, `drone`, `item` -, com a posição na base e nenhum campo a mais nela. Para cada campo que você for tentado a pôr na base, escreva antes a frase que o justifica em termos do laço de render ou do laço de turno; se a frase não sair, o campo é da derivada. A hierarquia que sair daqui é a que o Cap. 11 torna abstrata, e é o alvo sobre o qual o **LAB-06** roda no encontro seguinte.
2. Decida, para cada par abaixo, se a relação é *is-a* ou *has-a*, e escreva uma frase por decisão: `sonda` e `entidade`; `mapa` e `grade`; `drone` e `sonda`; `mundo` e `mapa`; `celula` e `glifo`. Duas delas são armadilhas.
3. Escreva um par de classes instrumentadas **local ao seu arquivo de teste** - uma base e uma derivada, cada uma com um `marca_de_vida` de nome próprio, e a derivada com um membro marcado também. Antes de compilar, escreva as seis linhas do traço que um objeto em escopo vai produzir, na ordem. Depois rode e compare com o que `testes/test_ciclo_de_vida.cpp` afirma. Se errou a ordem, explique qual das três consequências do §10.4 você não aplicou. E responda por que o par tem de ser local ao teste em vez de morar na entidade do jogo.
4. Tente, de propósito, escrever `class drone : entidade` sem o `public`. Compile, atribua um `drone*` a um `entidade*` e leia a mensagem do compilador inteira, até entendê-la. Depois explique, em duas linhas, por que a mensagem fala de conversão e não de herança.

5. Dê à entidade um construtor que recebe a posição inicial e **nenhum** construtor padrão. Escreva um drone cujo construtor esquece de nomear a base na lista de inicialização, e leia o erro. Depois conserte e confirme, com o traço do exercício 3, que a base é construída antes do primeiro campo do drone.
6. Meça o sizeof das suas quatro classes e escreva o static_assert de cada uma antes de rodar. A sonda, que não acrescenta campo, tem o mesmo tamanho da base? E se você acrescentar um bool a ela, quanto o objeto cresce, e por quê? Compare com os números de include/deriva/leiaute.hpp.

O código, extraído do Deriva

Todo trecho abaixo vem de exemplos/deriva/, que compila com `-std=c++17 -Wall -Wextra -Wpedantic` sem um aviso e passa `make verifica`. Nenhum foi digitado neste texto.


```

15  /// Qualquer coisa que ocupa uma célula e faz algo por turno.
16  ///
17  /// A hierarquia é a que o domínio pede, não taxonomia inventada para o
18  /// exercício: uma sonda, um drone e um item são coisas diferentes que ocupam
19  /// posição e desenharam um glifo. O que varia é o que cada uma faz no turno.
20  ///
21  /// **Abstrata** (v1.1): `desenhar` é puramente virtual, então `entidade` não
22  /// se instancia. Isso não é cerimônia - uma entidade sem glifo não tem
23  /// significado no domínio, e o compilador passa a dizer isso.
24  ///
25  /// **Destrutor virtual** (v1.1): sem ele, `delete` por `entidade*` não roda o
26  /// destrutor da derivada, e nenhum aviso aparece quando o `delete` mora dentro
27  /// de um `unique_ptr`. A variante `variantes/v1.1-quebrada/` mede os três
28  /// casos. É a caça ao bug 2.
29  ///
30  /// O contador de instâncias vivas mora em CADA classe concreta, escrito à mão,
31  /// e não na base. Duas razões: o contador da base contaria objetos e não tipos,
32  /// e a repetição é o que motiva o `contador_de_instancias<T>` da Aula 19.
33  class entidade {
34  public:
35      explicit entidade(vetor2 pos) noexcept : pos_(pos) {}
36
37      virtual ~entidade() = default;
38
39      // Base polimórfica não se copia por valor: copiar por `entidade&` fatiaria
40      // o objeto. A cópia correta é `clonar`, que a Aula 25 transforma em padrão.
41      entidade(const entidade&) = delete;
42      entidade& operator=(const entidade&) = delete;
43
44      [[nodiscard]] virtual char glifo() const = 0;
45      [[nodiscard]] virtual std::string_view nome() const = 0;
46
47      /// Um turno. A base não faz nada, e a derivada que não age não precisa
48      /// dizer nada - é o caso do `item`.
49      virtual void agir(mundo&) {}
50
51      [[nodiscard]] vetor2 pos() const noexcept { return pos_; }
52      void mover_para(vetor2 p) noexcept { pos_ = p; }
53
54      /// Não-virtual de propósito, e chamando o virtual: é o Template Method na
55      /// sua forma mais curta. A moldura do texto é da base; o glifo é da
56      /// derivada (Aula 11).
57      [[nodiscard]] std::string descrever() const;
58
59  private:
60      vetor2 pos_;
61  };

```

A hierarquia é a que o domínio pede: sonda, drone e item são coisas diferentes que ocupam posição e desenham um glifo. O contador de instâncias mora em cada classe concreta, não na base - o da base contaria objetos e não tipos.

```

┌─| include/deriva/entidade.hpp:63 |────────────────── o `final` que teve de sair ───────────────────┐
63 /// A sonda que o estudante opera. Tem energia, e gasta energia agindo.
64 ///
65 /// **Não é `final`, e já foi.** Na v1.0 esta classe era `final`, porque nada
66 /// derivava dela e a palavra documentava a intenção. A v1.7 introduziu
67 /// `sonda_reparadora`, e o compilador recusou: "cannot derive from final base".
68 /// A palavra saiu, e a lição fica: `final` é promessa, não decoração, e
69 /// retirá-la é admitir que a hierarquia mudou de forma. Quem depende de a
70 /// classe ser folha - devirtualização, por exemplo - perde a garantia nesse
71 /// instante (Aula 11 e Aula 17).
72 class sonda : public entidade {
73 public:
74     inline static int vivos = 0;
75     inline static int criados = 0;
76
77     explicit sonda(vetor2 pos, int energia = 100) noexcept
78         : entidade(pos), energia_(energia) {
79         ++vivos;
80         ++criados;
81     }
82     ~sonda() override { --vivos; }
83
84     [[nodiscard]] char glifo() const override { return '@'; }
85     [[nodiscard]] std::string_view nome() const override { return "sonda"; }
86     void agir(mundo& m) override;
87
88     [[nodiscard]] int energia() const noexcept { return energia_; }
89     void gastar(int quanto) noexcept;
90
91 private:
92     int energia_;

```

Na v1.0 esta classe era `final`. A v1.7 introduziu a `sonda_reparadora` e o compilador recusou: “cannot derive from final base”. A palavra saiu, e a lição fica - `final` é promessa, e retirá-la é admitir que a hierarquia mudou de forma.

```

┌──| src/entidade.cpp:9 |── Template Method na forma mais curta ──┐
    9 std::string entidade::descrever() const {
    10     // A moldura é da base, o glifo e o nome são da derivada. Nenhuma derivada
    11     // reescreve este texto, e nenhuma precisa.
    12     std::string s;
    13     s.push_back(glifo());
    14     s.append(" ").append(nome());
    15     s.append(" @ ").append(std::to_string(pos().x)).append(",")
    16     .append(std::to_string(pos().y));
    17     return s;
    18 }
└──────────────────────────────────────────────────────────────────┘

```

Não-virtual chamando virtual: a moldura do texto é da base, o glifo e o nome são da derivada, e nenhuma derivada reescreve o formato.

```

┌──| testes/test_ciclo_de_vida.cpp:116 |── a ordem, afirmada ──┐
    116 TEST_CASE("a ordem e base, membros, corpo da derivada - e o inverso ao morrer") {
    117     ordem.clear();
    118     { const derivada_marcada d; }
    119
    120     REQUIRE(ordem.size() == 6);
    121     REQUIRE(ordem[0] == "+base");           // a base primeiro, sempre
    122     REQUIRE(ordem[1] == "+membro");         // depois os membros, na ordem de declaracao
    123     REQUIRE(ordem[2] == "+derivada");       // e so entao o corpo do construtor
    124     REQUIRE(ordem[3] == "-derivada");       // e a destruicao inverte os tres
    125     REQUIRE(ordem[4] == "-membro");
    126     REQUIRE(ordem[5] == "-base");
    127 }
└──────────────────────────────────────────────────────────────────┘

```

Base, membros na ordem de declaração, corpo da derivada - e o inverso exato ao morrer. O par instrumentado é local ao teste: poluir o despejo que o replay compara custaria a condição 3 do portão.

```
┌───| testes/test_ciclo_de_vida.cpp:129 |─── a consequência da ordem ──┐
129 TEST_CASE("o corpo da derivada roda por ultimo, e isso tem consequencia") {
130     // A base nao pode contar com nada que o corpo da derivada faca: quando o
131     // construtor da base roda, a derivada ainda nao existe. E por isso que
132     // chamar metodo virtual dentro do construtor da base chama a versao DA
133     // BASE, e nao a sobrescrita - o objeto ainda nao e do tipo derivado.
134     ordem.clear();
135     { const derivada_marcada d; }
136     const auto pos_base = std::find(ordem.begin(), ordem.end(), "+base");
137     const auto pos_corpo = std::find(ordem.begin(), ordem.end(), "+derivada");
138     REQUIRE(pos_base < pos_corpo);
139 }
```

Quando o construtor da base roda, a derivada ainda não existe. É por isso que método virtual chamado dentro do construtor da base chama a versão DA BASE - o objeto ainda não é do tipo derivado.

Funções virtuais e classes abstratas

O QUE ESTE CAPÍTULO ENTREGA

- ▶ Usar virtual, override e final corretamente.
- ▶ Declarar classes abstratas com funções virtuais puras.
- ▶ Compreender o mecanismo interno das vtables, e dizer quanto ele custa por objeto.
- ▶ Reconhecer o vazamento que a ausência de destrutor virtual produz, e prová-lo com o contador vivos e com o gdb.
- ▶ Evitar chamadas a funções virtuais em construtores e destrutores.

§11.1 A Necessidade de Funções Virtuais

Sem virtual, a chamada a um método é resolvida em tempo de compilação pelo tipo estático do ponteiro/referência (ligação estática). Com virtual, a chamada é resolvida em tempo de execução pelo tipo dinâmico do objeto (ligação dinâmica). Isso é o que permite polimorfismo.

O Cap. 10 montou a hierarquia e deixou uma pergunta em aberto de propósito: o laço de render percorre entidade sem saber de subtipo, e cada derivada desenha um glifo diferente - como é que o glifo certo aparece? Com o que a v1.0 tem, não aparece. Uma função `desenhar()` não virtual na base é resolvida pelo tipo do que o laço tem na mão, que é entidade, e o resultado é que as três derivadas desenharam o mesmo caractere. O defeito não produz aviso, não produz falha de compilação, e na tela aparece como estação inteira preenchida com um glifo só.

A distinção que este capítulo existe para formar é entre **tipo estático** e **tipo dinâmico**. O tipo estático é o que está escrito no código, e é o que o compilador sabe; o tipo dinâmico é o do objeto que de fato está ali, e só a execução sabe. Toda vez que os dois diferem, a pergunta é qual dos dois manda, e a resposta é: manda o estático, salvo se a função for virtual. É o interativo *Despachante* desta aula que mostra os dois lados ao mesmo tempo, o que a execução real não faz.

O primeiro trecho de despacho no fim deste capítulo é essa distinção travada pelo portão. Uma sonda, um drone e um item entram num arranjo de `const entidade*`; o laço pede `glifo()` a cada um e concatena; o `REQUIRE` afirma "`@d!`". Os três ponteiros têm o mesmo tipo estático, e a saída tem três caracteres diferentes - o que decidiu foi o tipo dinâmico de cada objeto. Sem virtual, a mesma linha produziria o glifo da base três vezes, e o comentário do caso registra esse contrafactual ao lado da afirmação.

A palavra para o caso sem virtual é **ocultamento**, e não sobrescrita. Uma `glifo()` não-virtual redeclarada na derivada não substitui a da base, ela esconde o nome dentro do escopo da derivada: duas funções distintas passam a existir, e qual delas

roda depende de por onde se chama. É o defeito que o `override` do §11.2 elimina, e é a razão de o Deriva não ter uma única redeclaração sem essa palavra.

§11.2 `override` e `final`

O erro que o `override` pega é sempre o mesmo, e é de uma sutileza que passa em revisão. A base declara `virtual void desenhar() const;` e a derivada escreve `void desenhar();`, sem o `const`. As assinaturas diferem, então não há sobrescrita: há ocultamento, a base continua com a implementação dela, e o laço de render, que chama através de `const entidade&`, continua a chamar a versão da base. Nada avisa. Com `override` escrito, o compilador recusa a definição e diz que ela não sobrescreve nada, o que transforma um defeito de execução silencioso em erro de compilação com nome e linha.

Por isso a regra desta disciplina é dura: **toda** sobrescrita leva `override`, e nenhuma leva `virtual` repetido. Repetir `virtual` na derivada é legal e é redundante, porque a virtualidade se herda; escrever os dois numa mesma declaração é o sinal de que quem escreveu não sabia qual dos dois faz o trabalho.

`final` serve a duas coisas diferentes com a mesma palavra. Em função, ele diz que a cadeia de sobrescrita termina ali; em classe, que ninguém mais herda dela. As duas são declarações de intenção verificadas pelo compilador, e o Deriva usa a segunda no `drone` e no `item`, que são folhas do desenho e não têm subtipo previsto. Na sonda ela esteve e teve de sair: a v1.7 acrescentou a sonda_reparadora, e o compilador recusou a derivação com “cannot derive from final base”. O trecho extraído no Cap. 10 registra a retirada, com a razão no comentário, e a lição é a que a retirada ensina - `final` é promessa, e apagá-la é admitir que a hierarquia mudou de forma. Quem dependia de a classe ser folha, para devirtualização por exemplo, perde a garantia no mesmo instante.

O `drone` está extraído no fim deste capítulo, e ele é as duas regras numa declaração só: `class drone final : public entidade`, com `override` nas quatro sobrescritas, inclusive no destrutor, e `virtual` repetido em nenhuma. Leia-o ao lado da sonda do Cap. 10, que é a mesma forma sem o `final`, e a diferença entre os dois cabeçalhos é a história da v1.7 escrita em uma palavra.

As duas mensagens de compilador que esta seção descreve não têm arquivo no repositório, e não têm por um motivo declarado: código que não compila não entra no Deriva, e as duas só existem como falha de compilação. Quem quiser lê-las escreve as três linhas do exercício 3 e do exercício 4, que é onde elas pertencem - na máquina do estudante, com a mensagem inteira na tela.

— DICA —

Sempre use `override` em toda função virtual de uma classe derivada. O compilador detecta imediatamente se você errou a assinatura - um erro comum que sem `override` se manifesta como “método não está sendo chamado” em runtime.

§11.3 Funções Virtuais Puras e Classes Abstratas

Uma função virtual pura é declarada na base e não definida nela: `o = 0` no lugar do corpo é o que a torna pura, e `virtual char glifo() const = 0;` é a forma que entidade usa. Toda derivada concreta tem de responder a ela. Uma classe com ao menos uma pura é **abstrata**, e objeto dela não se cria.

entidade é abstrata a partir da v1.1, e a razão é de domínio antes de ser de técnica: não existe, na estação, uma coisa que seja *apenas* uma entidade. Toda coisa é uma sonda, um drone ou um item. Instanciar entidade seria criar um objeto que o jogo não sabe desenhar nem fazer agir, e a linguagem oferece a forma de tornar isso impossível em tempo de compilação em vez de documentado num comentário. Foi essa cláusula que o portão do LAB-06 pediu: entidade não instanciável.

Duas coisas que costumam surpreender.

A primeira é que classe abstrata **pode ter estado e pode ter método concreto**. Não é interface no sentido de Java: entidade guarda a posição, tem a função de acesso que a devolve, e nada disso deixa de valer por ela ter uma pura. O que ela não pode é existir sozinha.

A segunda é que função virtual pura **pode ter implementação**, e ela é chamada explicitamente, com o nome qualificado, pela derivada. É raro e é útil quando toda derivada precisa fazer a mesma coisa antes ou depois do trabalho dela.

O padrão que usa isso é o **Template Method**, e no Deriva ele está em `descrever()`, não no `turno`. `descrever()` é não-virtual, mora na base, e chama `glifo()` e `nome()`, que são virtuais puras: a moldura do texto - o glifo, o nome, a posição, nessa ordem, nesse formato - é uma decisão só, e nenhuma derivada a reescreve. Cada uma decide apenas o glifo e o nome. O trecho está extraído no Cap. 10, e o `TEST_CASE("descrever e Template Method")` o mede.

A sequência do **turno** não é Template Method, e a diferença vale ser dita porque a tentação é grande. Ela mora em `mundo::turno()`, que percorre as entidades e chama `agir()` em cada uma; `agir()` é virtual com corpo vazio na base, e cada derivada a sobrescreve inteira. Poderia ter sido o contrário - a base fixando “confira o limite, peça o passo, aplique o passo” e deixando só o passo virtual -, e a razão de não ser é concreta: `agir()` pode acrescentar entidade ao mundo, e é por isso que `mundo::turno()` percorre por índice e não por iterador, com o comentário de lá dizendo exatamente isso. Uma sequência fixa na base precisaria conhecer o mundo para consultar o limite, e a base passaria a depender do que a contém.

A regra que sai daí serve além deste caso: Template Method cabe onde a sequência é invariante e **pertence à base**. A do `turno` pertence ao mundo, e é lá que ela está.

A abstração em si não fica em prosa: o último trecho do capítulo são quatro `static_assert` sobre a base, e cada um trava uma decisão de projeto. `is_abstract_v<entidade>` afirma que ela tem pura; `!is_constructible_v<entidade, vetor2>` afirma que o construtor existe e não é utilizável de fora, que é a cláusula do portão do LAB-06; `has_virtual_destructor_v<entidade>` é a regra do §11.5 travada onde ela não pode ser esquecida; e `!is_copy_constructible_v<entidade>` é a recusa do fatiamento que o Cap. 14 §14.8 explica. Quatro asserções em tempo de compilação, e nenhuma delas custa um ciclo de execução.

§11.4 Mecanismo Interno: vtable e vptr

Para cada classe com ao menos uma função virtual, o compilador gera uma vtable: uma tabela de ponteiros para as implementações que aquela classe usa, uma entrada por função virtual. E põe em cada objeto da classe um campo oculto, o vptr, que aponta para a vtable da classe de que o objeto é. Nada mais é acrescentado ao objeto, e é esse campo que esta seção mede.

Quando você chama `ent.desenhar()` através de referência ou ponteiro, o compilador emite três passos: carrega o vptr do objeto, indexa a vtable pelo slot de `desenhar()`, e chama a função que encontrou lá. Esse é o overhead do despacho virtual, e ele é uma indireção a mais em relação à chamada direta.

O custo em espaço, ao contrário do custo em tempo, é fácil de medir, e o Deriva o mede. `include/deriva/leiaute.hpp` põe lado a lado a mesma estrutura com e sem método virtual, e cinco `static_assert` afirmam os números. `entidade_simples`, que tem só a posição, ocupa **8 bytes**. `entidade`, que é a mesma coisa com um destrutor virtual e uma função virtual pura, ocupa **16**: são os 8 da posição mais os 8 do vptr. E `drone_com_carga`, que herda de `entidade` e acrescenta um `int`, ocupa **24** - 8 do vptr, 8 da posição, 4 da carga e 4 de padding.

Três leituras desses números, e a terceira é a que decide projeto.

A primeira: o custo é **por objeto**, e não por classe. A vtable existe uma vez por classe, em memória estática; o vptr existe uma vez por objeto. Numa estação com centenas de entidades, são 8 bytes vezes centenas.

A segunda: o custo não desaparece quando a derivada cresce. `drone_com_carga` não reaproveita o espaço do vptr para guardar a carga; ele se soma, e o padding de 4 bytes no fim aparece por alinhamento, exatamente como no `celula_ingenua` do Cap. 7.

A terceira: 8 bytes por entidade é barato, e a alternativa é caríssima. Sem virtual, o laço de render precisaria descobrir o tipo de cada entidade por conta própria, através de um campo de etiqueta e um `switch` - o que gasta os mesmos bytes no campo, gasta uma comparação a mais por caso, e obriga a mexer no `switch` cada vez que uma derivada nova aparece. O despacho virtual é o `switch` que o compilador escreve e mantém.

O trecho está extraído no fim deste capítulo, e vale ler os três blocos como uma conta só.

```
8 bytes
sizeof(entidade_simples)

aferido por
./build/deriva --leiaute
include/deriva/leiaute.hpp
```

DIAGRAMA UML

Diagrama: um objeto `drone` em memória, com o vptr nos primeiros 8 bytes apontando para a vtable de `drone`, a posição nos 8 seguintes e a carga nos 4 finais mais 4 de padding; ao lado, a vtable de `drone` com os slots `~entidade` → `~drone`, `desenhar` → `drone::desenhar` e `passo` → `drone::passo`. Marcar que a vtable é uma por classe e o vptr um por objeto.

▲ ATENÇÃO

Nunca chame funções virtuais em construtores ou destrutores. Durante a construção de um drone, o `vp`tr ainda aponta para a `vtable` de entidade; a função virtual chamada será a da classe que está sendo construída no momento, e não a da derivada que você esperava. Se a função for pura, e portanto não tiver implementação na base, o programa chama uma função virtual pura e aborta. A razão é a mesma do §10.4: quando o construtor da base roda, a parte derivada do objeto ainda não existe, e chamar código dela seria usar memória não inicializada.

§11.5 O Destrutor Virtual, e o Vazamento que a Ausência Dele Produz

Este é o defeito que a Unidade II tem para ensinar, e ele merece seção própria por três razões: é o mais comum na prática, não produz aviso nenhum, e é o primeiro em que o instrumento construído no Cap. 7 acusa algo que nenhuma outra leitura acusa.

A situação é a de sempre. O mundo guarda entidades e as manipula através de ponteiro para a base, porque é isso que a hierarquia existe para permitir; na v1.2 esse ponteiro passa a ser um `unique_ptr<entidade>`, e o problema aqui é o mesmo com ponteiro cru ou com ponteiro inteligente, porque quem escolhe qual destrutor chamar não é o ponteiro, é a base. No instante em que a entidade sai do mundo, alguma coisa destrói o objeto através de um `entidade*`, e a linguagem precisa decidir qual destrutor rodar.

Se `~entidade()` for virtual, a decisão é pelo tipo dinâmico: roda `~drone()`, e depois `~entidade()`, na ordem espelhada do §10.4. Se não for, o padrão diz que o comportamento é **indefinido**, e o que acontece nesta plataforma, com este compilador, é o pior dos casos possíveis: o programa não aborta, não avisa, e chama apenas `~entidade()`. O destrutor da derivada nunca roda. Tudo o que ele fosse liberar fica onde estava, e o programa continua como se nada tivesse acontecido.

— O que cada instrumento vê

O contador de instâncias vivas é o que acusa, e ele acusa porque o decremento está no destrutor que não roda. Com o contador na derivada - `++vivos` no construtor de drone, `--vivos` no destrutor de drone -, o incremento acontece na criação e o decremento não acontece nunca; ao fim do roteiro, vivos vale exatamente o número de entidades que foram destruídas através da base. É a quarta condição do portão `make` verifica falhando, e é a diferença entre um defeito que se descobre por acidente e um que o `make` recusa.

Repare no detalhe que faz a diferença: o contador tem de estar **na derivada**. Se ele estiver só na base, o incremento e o decremento acontecem os dois, porque `~entidade()` roda; vivos fecha em zero e o defeito passa. É o mesmo limite do Cap. 7 visto por outro ângulo - o contador conta o que ele foi posto a contar, e nada mais.

O `gdb` é o que localiza. O Cap. 2 §2.5 montou a sequência e o alvo `make gdb-dtor` a roda em lote; aqui ela responde a uma pergunta precisa. Ponha o ponto de parada no destrutor da derivada, `break deriva::~drone::~~drone`, e rode. Com destrutor virtual na base, a parada é atingida e o `backtrace` mostra de onde a destruição veio.

Sem destrutor virtual, **a parada não é atingida uma única vez**, e é essa ausência que é a prova. Ponto de parada que nunca dispara é informação, e é a informação mais direta que existe sobre destrutor que não roda.

A instrumentação de ciclo de vida é o que torna a ausência legível sem depurador. Com `marca_de_vida` na base e na derivada, o traço de um objeto bem destruído tem quatro linhas; o do objeto destruído através da base tem três, e falta justamente a da derivada. Contar linhas do traço é o que o laboratório pode fazer sem ferramenta nenhuma.

E o compilador **diz**, e é aqui que este defeito se separa do da caça ao bug 1. Vale medir, porque a resposta tem três casos e o mais interessante é o do meio.

Com o conjunto mínimo, `-Wall -Wextra -Wpedantic`, e um `delete e;` escrito no seu código, o g++ acusa no ponto da liberação: *deleting object of abstract class type 'entidade' which has non-virtual destructor will cause undefined behavior*, pelo `-Wdelete-non-virtual-dtor`, que já vem dentro de `-Wall`.

Com o mesmo conjunto mínimo e **nenhum delete escrito** - o caso do Cap. 12, em que a liberação acontece dentro do destrutor de `std::unique_ptr`, o g++ não emite **uma palavra**. Medido: um programa com `std::vector<std::unique_ptr<entidade>>`, `make_unique` e `clear()`, com a base sem destrutor virtual, compila com zero aviso sob `-Wall -Wextra -Wpedantic`. A razão é que o `delete` que dispararia o aviso está num cabeçalho do sistema, e aviso em cabeçalho do sistema é suprimido. O ponteiro inteligente, que resolve o vazamento de memória, **esconde o aviso** do vazamento de comportamento.

E com `-Wnon-virtual-dtor`, que está no conjunto de avisos do Deriva e que o Cap. 2 já lista apontando para este capítulo, o g++ acusa na **declaração da classe**, antes de existir liberação nenhuma: são três avisos, um em entidade por ter função virtual e destrutor acessível não virtual, um em drone pela base, e um em drone por ela mesma. É por isso que aquele aviso está no conjunto, e é a diferença entre pegar o defeito onde ele foi escrito e pegá-lo onde ele explode.

0 avisos

avisos do compilador

aferido por

make verifica

Makefile

— O aviso fica ligado, e é isto que a caça ao bug 2 caça

Os três casos acima estão medidos em `variantes/v1.1-quebrada/`, e o LEIA-ME.md de lá os tabela: 1 aviso com `delete` textual sob o conjunto mínimo, 0 com o mesmo `delete` dentro de um `unique_ptr`, e 3 com `-Wnon-virtual-dtor`. Daí sai uma decisão de projeto do material, e ela vai escrita porque tem consequência no que o laboratório mede.

`-Wnon-virtual-dtor` **fica** no portão do núcleo. Não há alvo com `-Wno-non-virtual-dtor` para devolver o silêncio à variante, e o efeito colateral é que a `v1.1-quebrada` é a única das quatro variantes que falha na **condição 1 de 4** de `make verifica`, e não na 2: ela é acusada na compilação, antes de qualquer teste rodar. Isso não enfraquece a caça, muda o alvo dela. O defeito que o estudante persegue é o **caso do meio**, que é genuinamente silencioso sob `-Wall -Wextra -Wpedantic`: com a liberação dentro do `unique_ptr`, o único diagnóstico que existia desaparece, e nenhum instrumento além dos que o Cap. 7 e o Cap. 2 construíram acusa qualquer coisa. Desligar o aviso para fabricar silêncio seria simular no material a mesma prática que ele recusa no código do estudante; o silêncio de verdade já está ali, e é mais interessante que o simulado.

A ordem das cinco observações do §11.6 sobrevive intacta, e por isso: o compilador continua sendo o quinto olhar, porque no caso que se caça ele não diz nada até que

alguém peça o aviso pelo nome no `CMakeLists.txt`. É essa a lição, e ela é a mais transferível deste capítulo - o aviso existia, e bastava pedir por ele.

— A forma da correção

Uma linha: `virtual ~entidade() = default;` na base. Não há mais nada a fazer, e nenhuma derivada precisa mudar, porque a virtualidade da destruição se herda como a de qualquer outra função.

Duas notas de fecho. A primeira é que declarar o destrutor, ainda que como `= default`, **impede o compilador de gerar as operações de movimento**, pela segunda linha da tabela do Cap. 9. Numa base polimórfica isso costuma não custar nada, porque objeto polimórfico é manipulado por ponteiro e não é movido como valor; a regra continua sendo a do Cap. 14, e é lá que ela se fecha. A segunda é o par que o item 5 da rubrica do Cap. 4 cobra junto: base com método virtual tem destrutor virtual, e toda sobrescrita está marcada `override`. Os dois lados do par pegam defeitos diferentes, e é por isso que estão no mesmo item.

§11.6 Caça ao Bug 2: o destrutor que não é virtual

A segunda das três caças ao bug do semestre é esta, e ela usa o mesmo protocolo da primeira, do Cap. 9: cada dupla recebe uma versão do Deriva plausível e errada, e a ordem é cobrada - **reproduzir a falha; explicá-la em uma frase, por escrito, antes de tocar no código; corrigir; provar a correção**. Os sete itens da rubrica do Cap. 4 se aplicam por escrito, e aqui os itens 5 e 6, que até o Cap. 10 se respondiam com “não se aplica”, passam a ter resposta de verdade.

Uma nota de calendário, porque ela muda o que o exercício mede. O conteúdo do destrutor não virtual é deste capítulo, na semana 6, e a caça acontece no segundo encontro da **semana 9**, ao lado do Cap. 17. A distância de três encontros é deliberada: reconhecer o defeito duas semanas depois de o ter aprendido é a pergunta mais honesta das duas, e é a que se parece com o que acontece num projeto. O Cap. 17 §17.5 faz a chamada de volta e diz o que revisar antes; guarde este §11.6 para chegar lá com o roteiro na mão.

O defeito está em `variantes/v1.1-quebrada/`: a entidade tem funções virtuais e um destrutor **não** virtual, e o mundo destrói entidades através de `entidade*`. No jogo, a falha tem forma: a entidade continua desenhada no mapa depois de remoção, porque o que a removeria estava no destrutor que não rodou.

— A ordem de observação

Cinco olhares sobre o mesmo defeito, na ordem, e o que se aprende é tanto o que cada instrumento vê quanto o que ele deixa de ver.

1. **Sem ferramenta nenhuma.** Rodar o programa e olhar a tela: a entidade remoção continua lá. É o sintoma, não o diagnóstico, e é onde toda caça começa.
2. **Com a instrumentação de ciclo de vida.** O traço tem três linhas onde devia ter quatro, e a que falta é a do destrutor da derivada. Isto já basta para a frase única, e é o único caminho que não depende de nada instalado.
3. **Com o contador de instâncias vivas.** vivos não volta a zero, e o número que sobra diz quantas entidades foram destruídas pela base. Confirme antes que o

contador está **na derivada**: se estiver só na base, ele fecha em zero e o defeito passa.

4. **Com o gdb.** Ponto de parada em `deriva::drone::~~drone` e `run`. A parada não é atingida. É a prova mais curta que existe, e é a técnica que o Cap. 2 §2.5 preparou para este momento.
5. **Com o compilador**, e aqui em três passos, porque a resposta depende de com o que se compila. Com `-Wall -Wextra -Wpedantic` e um `delete` escrito no seu código, há aviso, e ele é `-Wdelete-non-virtual-dtor`. Com o mesmo conjunto e a liberação acontecendo dentro de um `unique_ptr`, **não há aviso nenhum** - o ponteiro inteligente esconde o aviso, e é o resultado que mais surpreende neste capítulo. Com o conjunto do Deriva, que traz `-Wnon-virtual-dtor`, há três avisos, na declaração das classes, antes de existir liberação. Anote os três resultados: a diferença entre eles é o conteúdo desta observação.

— A frase única, e por que ela vem antes

A frase tem de dizer o que acontece, e não o que fazer. “Falta virtual no destrutor” é o conserto, não a explicação. A explicação é: *destruir um objeto derivado através de um ponteiro para uma base cujo destrutor não é virtual não chama o destrutor da derivada, e o que ele liberaria fica alocado*. Escrever isso antes de mexer no código é o que separa a correção da tentativa; e é o que impede o conserto por sorte, que passa no teste e não se sustenta na explicação.

Exercícios Propostos

1. Torne entidade abstrata: `virtual void desenhar() const = 0;` e `virtual vetor2 passo() = 0;`. Implemente as três derivadas com `override`. Tente instanciar entidade e leia o erro. Depois percorra um vector de ponteiros para a base e confirme, pelo glifo de cada um, que o despacho é pelo tipo dinâmico.
2. Escreva o seu próprio `descrever()` como Template Method: não-virtual na base, chamando duas virtuais puras que cada derivada implementa. Depois tente o mesmo com o turno `-agir()` concreto na base, fixando a sequência, com só o passo virtual - e diga o que a base passa a precisar saber sobre o que a contém. Feche respondendo por escrito: se cada derivada tivesse o seu próprio laço de turno, o que aconteceria com o replay do Cap. 16?
3. Na sua hierarquia, esqueça o `const` numa sobrescrita, sem `override`. Compile - não há erro - e mostre, com uma chamada através de `const entidade&`, que a versão da base é a que roda. Depois acrescente `override` e leia a mensagem. Explique em duas linhas a diferença entre ocultar e sobrescrever.
4. Reproduza o vazamento do §11.5 na sua própria hierarquia: destrutor não virtual na base, contador vivos na derivada, um `main` que cria três drones, os guarda em `vector<entidade*>` e os destrói pelo laço. Percorra as cinco observações do §11.6 nesta ordem e escreva, para cada uma, o que ela acusou e o que deixou passar. Este é o portão do **LAB-06**, e o nome dele é o critério: o destrutor não virtual, acusado pelo contador vivos, provado sem ferramenta externa e localizado no gdb.
- 4b. Meça os três casos do compilador do §11.5, com o mesmo defeito e três compilações. Primeiro, com `delete` escrito e só `-Wall -Wextra -Wpedantic`; segundo, trocando o `delete` por um `vector<unique_ptr<entidade>>` que se limpa, com o mesmo conjunto de avisos; terceiro, acrescentando `-Wnon-virtual-dtor`. Regis-

tre a contagem e o texto de cada aviso. Depois responda em duas linhas: por que o segundo caso é o mais perigoso dos três?

5. Mova o contador do exercício 4 da derivada para a base e rode outra vez. vivos fecha em zero, e o defeito continua lá. Explique por quê, e diga o que isso acrescenta ao “o que o contador não acusa” do Cap. 7.

6. Ponha break no destrutor da derivada com o gdb, pelo alvo make gdb-dtor do Cap. 2, nas duas versões - com e sem destrutor virtual. Registre a diferença exata na saída. Depois responda: por que um ponto de parada que nunca dispara é uma prova, e do que exatamente ele é prova?

7. Demonstre a armadilha da função virtual em construtor: base cujo construtor chama uma função virtual, derivada que a sobrescreve. Mostre qual versão roda. Depois torne a função virtual **pura** e rode outra vez; explique a mensagem que aparece e ligue-a ao §10.4.

8. Meça o custo do vptr na sua hierarquia com sizeof e static_assert, antes de rodar: a base sem virtual, a base com virtual, e uma derivada que acrescenta um int. Compare com os três números de include/deriva/leiaute.hpp e explique cada byte de diferença, inclusive o padding.

O código, extraído do Deriva

Todo trecho abaixo vem de exemplos/deriva/, que compila com -std=c++17 -Wall -Wextra -Wpedantic sem um aviso e passa make verifica. Nenhum foi digitado neste texto.

```

┌─── testes/test_entidade.cpp:42 ──── a prova do destrutor virtual ────
42 // A prova do destrutor virtual: deletar por ponteiro da base tem de rodar o
43 // destrutor da derivada, e o contador de CADA classe concreta e o que acusa.
44 TEST_CASE("deletar por entidade* destroi a derivada") {
45     zerar_entidades();
46     {
47         std::unique_ptr<entidade> e = std::make_unique<sonda>(vetor2{1, 1});
48         REQUIRE(sonda::vivos == 1);
49     }

```

O contador de cada classe concreta é o que acusa. Sem virtual no destrutor da base, sonda::vivos ficaria em 1 no fim do escopo - e nenhum aviso apareceria, porque o delete mora dentro do unique_ptr.

```

└─▲ variantes/v1.1-quebrada/destrutor_quebrado.cpp:39 ── quebrado de propósito · destrutor não virtual · CAÇA AO BUG 2─┐
39 /// A base da hierarquia da v1.0. Tem função virtual, então é polimórfica -
40 /// e é para ser usada por ponteiro de base.
41 struct entidade {
42     explicit entidade(char glifo) : glifo_(glifo) {
43         ++contador::vivos;
44         ++contador::criados;
45     }
46
47     // FALTA a palavra `virtual` aqui. É a única diferença entre esta variante e
48     // a v1.1 boa.
49     ~entidade() {
50         ++contador::destrutores_de_base;
51         --contador::vivos;
52     }
53
54     virtual void desenhar() const { std::putchar(glifo_); }
55
56     protected:
57     char glifo_;
58 };

```

Medido em g++ 13.3: delete textual dá 1 aviso; o mesmo delete dentro de uni-que_ptr dá ZERO, porque passou a morar num cabeçalho do sistema; com -Wnon-virtual-dtor são 3, e nas declarações. C++ moderno, correto em tudo o mais, silenciou o único diagnóstico.

```

┌─| include/deriva/leiaute.hpp:16 |──────────────────────────────────────────┐ quanto custa o vptr ───────────────────────────────────────────┐
16 /// Sem nenhum método virtual: não há vtable, e portanto não há `vptr`.
17 struct entidade_simples {
18     vetor2 pos;
19 };
20 struct drone_simples : entidade_simples {};
21
22 /// Com um método virtual: o compilador põe um ponteiro para a vtable como
23 /// primeiro campo do objeto. São 8 bytes por OBJETO, não por classe.
24 struct entidade {
25     vetor2 pos;
26     virtual ~entidade() = default;
27     virtual void desenhar() const = 0;
28 };
29 struct drone : entidade {
30     void desenhar() const override {}
31 };
32
33 /// E quando a derivada acrescenta dado, o custo do vptr não desaparece - ele
34 /// se soma. É a v1.2, quando o drone ganha carga.
35 struct drone_com_carga : entidade {
36     int carga = 0;
37     void desenhar() const override {}
38 };
39
40 static_assert(sizeof(entidade_simples) == 8, "só a posição");
41 static_assert(sizeof(drone_simples) == 8, "herdar de classe não-polimórfica é grátis");
42 static_assert(sizeof(entidade) == 16, "8 do vptr + 8 da posição");
43 static_assert(sizeof(drone) == 16, "a derivada sem dado novo não cresce");
44 static_assert(sizeof(drone_com_carga) == 24, "8 vptr + 8 pos + 4 carga + 4 de padding");

```

8 bytes por OBJETO, não por classe. E o custo não desaparece quando a derivada acrescenta dado: ele se soma - 8 do vptr, 8 da posição, 4 da carga, 4 de padding.

```

┌─| testes/test_entidade.cpp:21 |────────────────────────── três objetos, um tipo de ponteiro ─────────────────┐
21 TEST_CASE("o despacho virtual escolhe pelo tipo do OBJETO") {
22     zerar_entidades();
23     const sonda s({1, 1});
24     const drone d({2, 2});
25     const item i({3, 3}, "celula-de-energia", 5);
26
27     const entidade* por_base[] = {&s, &d, &i};
28     std::string glifos;
29     for (const entidade* e : por_base) glifos.push_back(e->glifo());
30
31     REQUIRE(glifos == "@d!");    // e nao "eee": o ponteiro e entidade* nos tres
32 }
└────────────────────────────────────────────────────────────────────────────────────────────────────────────────┘

```

A saída é @d!, e não eee. Os três ponteiros são entidade*, e quem decidiu foi o objeto.

```

┌─| include/deriva/entidade.hpp:95 |────────────────────────── `final` na folha, `override` nas sobrescritas ─────────────────┐
95 /// Drone de patrulha. Anda sozinho, em linha, e inverte ao bater.
96 class drone final : public entidade {
97     public:
98         inline static int vivos = 0;
99         inline static int criados = 0;
100
101         explicit drone(vetor2 pos, vetor2 rumo = {1, 0}) noexcept
102             : entidade(pos), rumo_(rumo) {
103             ++vivos;
104             ++criados;
105         }
106         ~drone() override { --vivos; }
107
108         [[nodiscard]] char glifo() const override { return 'd'; }
109         [[nodiscard]] std::string_view nome() const override { return "drone"; }
110         void agir(mundo& m) override;
111
112         [[nodiscard]] vetor2 rumo() const noexcept { return rumo_; }
113
114     private:
115         vetor2 rumo_;
116 };
└────────────────────────────────────────────────────────────────────────────────────────────────────────────────┘

```

virtual não se repete na derivada: override já diz que sobrescreve, e diz melhor, porque o compilador o verifica.


```
┌─── testes/test_entidade.cpp:12 ──── a abstração, afirmada ────┐
12 TEST_CASE("entidade e abstrata, e o compilador diz isso") {
13     static_assert(std::is_abstract_v<entidade>);
14     static_assert(!std::is_constructible_v<entidade, vetor2>);
15     static_assert(std::has_virtual_destructor_v<entidade>);
16     static_assert(!std::is_copy_constructible_v<entidade>,
17                   "base polimorfica nao se copia por valor: fatiaria o objeto");
18     SUCCEED("verificado em tempo de compilacao");
19 }
```

`is_abstract_v`, `!is_constructible_v`, `has_virtual_destructor_v` e `!is_copy_constructible_v`. Quatro decisões de projeto que o compilador guarda.

Ponteiros inteligentes I - posse exclusiva

O QUE ESTE CAPÍTULO ENTREGA

- ▶ Explicar por que ponteiro inteligente existe, e o que ele resolve que ponteiro cru não resolve.
- ▶ Reconhecer, no texto do código, o ponteiro cru com posse, e nomear os defeitos que ele admite.
- ▶ Usar `std::unique_ptr` e `std::make_unique` para posse exclusiva.
- ▶ Transferir posse exclusiva, e dizer o que sobra na origem.
- ▶ Ligar a posse por ponteiro base ao destrutor virtual do Cap. 11.

§12.1 Por que Ponteiros Inteligentes?

Esta seção é a introdução comum aos dois capítulos de posse. O Cap. 13, que trata da posse compartilhada, refere-se a ela e não a repete.

O problema é de tipo, e não de disciplina. Um entidade* que apareceu numa função pode ser três coisas incompatíveis: a posse do objeto, que obriga quem recebeu a destruí-lo; uma vista sobre objeto de outro, que obriga a **não** destruí-lo; ou nada, um ponteiro nulo. O tipo é o mesmo nos três casos, então o compilador não tem como distinguir, e a única coisa que distingue é um comentário, ou o hábito da equipe, ou o nome do parâmetro. Nada disso é verificável, e é por isso que o defeito sobrevive à revisão: não há o que ler para saber se está errado.

Ponteiro inteligente resolve isso pondo a resposta no tipo. `std::unique_ptr<entidade>` diz “eu possuo, e sou o único”; `std::shared_ptr<entidade>` diz “eu possuo, e há outros”; `std::weak_ptr<entidade>` diz “eu observo e não possuo”; e entidade* ou entidade&, num parâmetro, passa a dizer “eu não tenho nada com a posse disto”. A partir do momento em que os quatro coexistem no mesmo código, a leitura de uma assinatura basta para saber quem libera o quê, e o compilador passa a recusar as combinações erradas.

Smart Pointer	Propriedade	Contagem de refs	Custo
<code>unique_ptr<T></code>	Exclusiva - um único dono	Não	Zero overhead vs. raw pointer
<code>shared_ptr<T></code>	Compartilhada - múltiplos donos	Sim (atômico)	Alocação extra + overhead atômico
<code>weak_ptr<T></code>	Nenhuma - observador	Não conta como strong ref	Similar ao <code>shared_ptr</code>

A terceira coluna é o que o Cap. 13 vai medir, e a quarta é o que decide a escolha na maior parte dos casos. Guarde a linha do `shared_ptr`: alocação extra e contagem atômica não são grátis, e escolher posse compartilhada por não querer pensar em posse é o erro que a Unidade II tem para corrigir.

O mecanismo, por baixo, é o do Cap. 8, e não é nenhum outro. Ponteiro inteligente é um objeto com destrutor, e o destrutor libera; o que muda em relação a `terminal_bruto` é só o recurso, que ali era o modo do terminal e aqui é um bloco de memória. RAII não ganhou capítulo novo - ele ganhou um caso a mais.

§12.2 O Contraexemplo: Posse Crua

O ponteiro cru não é proibido, e vale dizer com precisão o que é: **ponteiro cru com posse** é o que este material recusa. `entidade*` como parâmetro de função que só olha, ou como retorno de uma busca que pode não achar nada, é uso legítimo e continua legítimo até o fim do livro.

Posse crua admite cinco defeitos, e nenhum deles produz aviso.

O **vazamento** é o mais barato: um caminho de saída da função esquece o `delete`, e como raramente há um só caminho de saída, esquecer um é o normal. O `return` antecipado dentro de um `if` de validação é onde ele quase sempre está.

A **liberação dupla** é a da caça ao bug 1, no Cap. 9, vista agora do lado da posse: dois ponteiros com posse sobre o mesmo objeto, dois destrutores, um objeto.

O **uso depois da liberação** é o pior de diagnosticar, porque a memória liberada continua legível por um tempo e o programa continua a funcionar, com valores que já não significam nada, até parar de funcionar em outro lugar.

A **posse em comentário** é o defeito estrutural, e é o que produz os três acima. Quando a assinatura não diz quem possui, a informação passa a viver na cabeça de quem escreveu, e sai do projeto quando a pessoa sai.

A **exceção no meio** é a que quase ninguém considera. `entidade* e = new drone(...); validar(e); mundo.inserir(e);` vaza se `validar` lançar, e não há `delete` nenhum a esquecer - o `delete` que faltou é o que ninguém escreveu porque ninguém pensou naquele caminho. É o argumento que fecha o assunto: com posse crua, cada caminho de exceção é um vazamento a mais a auditar; com posse em ponteiro inteligente, não há caminho a auditar, porque o destrutor roda no desenrolar da pilha, como o Cap. 8 provou com o teste que afirma a ordem linha por linha.

— O contraexemplo que o estudante vai encontrar sozinho

Vale nomear onde o contraexemplo está, porque o estudante vai encontrá-lo, e provavelmente antes de encontrar este livro. O *Complete Roguelike Tutorial, using C++ and libtcod*, publicado no RogueBasin, ensina o jogo inteiro e funciona. O C++ dele é de antes de 2011, e é o que interessa aqui: as entidades vivem numa lista de ponteiros crus, os `delete` são escritos à mão, o destrutor da base varre a lista, e nenhuma assinatura diz quem possui o quê. Abrir o tutorial e ler o recorte em que as entidades são criadas e destruídas confirma cada uma dessas quatro frases, e é por isso que elas são o que este livro afirma sobre ele - descrição do código, e não juízo sobre o material.

Não há recusa a fazer, há exercício. Aquele código é o tipo de material que um modelo de linguagem reproduz quando se pede “um roguelike em C++”, pela razão que o Cap. 4 §4.8 expôs: o código mais provável é o mais publicado. Os sete itens da rubrica do Cap. 4 se aplicam a ele item por item, com resultado previsível nos itens 1, 2 e 5, e é o exercício 7 deste capítulo. Uma advertência de método: aquilo

é wiki, e muda; registre no seu relatório a data em que abriu a página e o recorte que avaliou, porque revisão sem o texto revisado anexado não se confere depois.

► DERIVA

A v1.2 do Deriva é a versão em que o mundo aparece, e ele guarda `std::vector<std::unique_ptr<entidade>>`.

Duas decisões que o vetor de `unique_ptr` toma de uma vez: a posse de cada entidade é do mundo e de mais ninguém, e o mundo não é copiável por acidente - `unique_ptr` não se copia, então o vetor não se copia, e a tentativa é erro de compilação em vez de duplicação silenciosa das entidades.

A consequência disso é o critério mais curto que a unidade tem, e é o que a entrega desta aula cobra: **zero delete no código do estudante**, e vivos fechando em zero. As duas condições juntas, porque cada uma sem a outra é fácil.

§12.3 `unique_ptr` - Posse Exclusiva

`std::unique_ptr<T>` é a posse exclusiva de um objeto alocado dinamicamente. Ele não se copia, se transfere; e quando ele morre, o objeto morre.

Quatro coisas a fixar sobre o uso.

Criar com `std::make_unique`. Escrever `std::unique_ptr<drone>(new drone(...))` funciona e é pior por dois motivos: o tipo aparece duas vezes, e há uma janela em que o objeto já existe e o ponteiro inteligente ainda não - se algo lançar naquele intervalo, vaza. `make_unique` fecha a janela e escreve o tipo uma vez.

Não copia. O construtor de cópia e a atribuição de cópia são `= delete`, e é a forma curta da regra dos três que o Cap. 9 §9.3 apresentou, aplicada a um recurso que por definição tem um dono só. Tentar copiar é erro de compilação, com mensagem sobre função apagada, e é essa mensagem que ensina.

Transfere com `std::move`. `auto outro = std::move(dono);` passa a posse para outro e deixa dono nulo. Aqui `std::move` faz uma coisa só, e é a única que este capítulo precisa: ele marca a origem como esvaziável, de forma que a operação escolhida seja a de transferência em vez da de cópia. O que ele é de fato, o que ele não é, e o que sobra na origem em geral, é o Cap. 14; para `unique_ptr` a resposta é precisa e vale antecipar, porque ela é garantida pelo padrão e não é “em geral”: **a origem fica nula**. É a única garantia desse tipo que este livro dá, e ela existe porque a operação de movimento do `unique_ptr` a promete explicitamente.

A ordem em que os dois assuntos chegam é deliberada, e vale dizê-la em vez de deixar o estudante estranhá-la. `std::move` aparece aqui, duas aulas antes de o movimento ser explicado, porque a posse exclusiva não se transfere de outra forma, e porque a hierarquia subiu na frente dos ponteiros inteligentes pela razão que o par 11/12 acabou de mostrar. Até o Cap. 14, trate `std::move` como o que ele é neste capítulo: a marca que escolhe a sobrecarga de transferência. A regra dos cinco, o estado da origem e o que `noexcept` decide fecham no Cap. 14, já na unidade seguinte de conteúdo, e nada aqui depende de eles estarem fechados antes.

Devolve por valor. Função de fábrica que cria entidade devolve `std::unique_ptr<entidade>`, e é o tipo de retorno que diz que quem chamou passou a ser o dono. É o Factory Method do Cap. 25 na sua forma mínima, e ele já funciona aqui.

— O par que precisa dos dois: `unique_ptr<entidade>` e o destrutor virtual

Este é o ponto em que os Caps. 11 e 12 se amarram, e é a razão de a herança ter subido cinco posições no plano v2.

`std::unique_ptr<entidade>` guarda um `entidade*` e, no destrutor, faz o equivalente a `delete` sobre esse ponteiro. Se o objeto de fato é um drone, quem decide qual destrutor roda é a base, exatamente como no §11.5, e a conclusão é a mesma: **sem virtual `~entidade()`, o `unique_ptr` não conserta nada**. Ele libera a memória, roda o destrutor da base, e o da derivada nunca é chamado. O contador vivos na derivada acusa, e o ponto de parada do gdb não é atingido.

E há um agravante que o Cap. 11 §11.5 mediu, e que é próprio deste capítulo: **o `unique_ptr` também esconde o aviso**. Com um `delete` escrito no seu código, `-Wall` acusa a liberação através de base sem destrutor virtual; com a liberação acontecendo dentro do destrutor do `unique_ptr`, que vive num cabeçalho do sistema, o aviso é suprimido, e o conjunto mínimo passa a compilar em silêncio. Trocar o ponteiro cru pelo inteligente, sem tocar na base, melhora a posse e **piora** o diagnóstico. É o que faz `-Wnon-virtual-dtor` estar no conjunto de avisos do Deriva desde o Cap. 2: ele acusa na declaração da classe, e não no ponto da liberação, e por isso é o único que sobrevive ao ponteiro inteligente.

É por isso que a ordem dos dois capítulos importa e que este material recusa a inversão. Ponteiro inteligente é sempre apresentado como “resolve o vazamento”, e resolve o vazamento de *memória*; o vazamento de *comportamento* que um destrutor ausente produz ele não vê, e o estudante que aprender a posse antes da hierarquia sai com a impressão de que `unique_ptr` é suficiente. Não é, e o item 5 da rubrica do Cap. 4 existe por isso.

Os dois trechos no fim deste capítulo são as duas metades disso em código que compila. `include/deriva/mundo.hpp` declara a posse no tipo, com `std::vector<std::unique_ptr<entidade>>` no membro privado e `acrescentar(std::unique_ptr<entidade>)` recebendo o ponteiro **por valor**, que é a assinatura que diz “entregue, e não fique com uma cópia”; as duas linhas de `cópia = delete` estão ali, e é delas que sai a mensagem sobre função apagada quando alguém tenta copiar um mundo - `testes/test_mundo.cpp` trava a recusa num `static_assert` e escreve a razão na própria mensagem de falha.

`src/mundo.cpp` mostra o outro lado. O `retirar_de` devolve `std::unique_ptr<entidade>` e é `[[nodiscard]]`, porque ignorar o retorno destrói o objeto na hora, e é o Factory Method do quarto item acima na forma mínima: o tipo de retorno é que diz que quem chamou passou a ser o dono. Compare com o `primeira_com` do mesmo cabeçalho, que devolve `entidade*` cru: ali é observação, e é o uso legítimo de ponteiro cru que o §12.2 preservou.

— ▲ ATENÇÃO —

`unique_ptr` não protege de posse dupla se você mesmo a criar. Construir dois `unique_ptr` a partir do **mesmo** ponteiro cru - `entidade* p = new drone(); std::unique_ptr<entidade> a(p); std::unique_ptr<entidade> b(p);` - compila e libera duas vezes. É a mesma liberação dupla do Cap. 9, e ela entrou pela porta que `make_unique` fecha: enquanto não existir um ponteiro cru nomeado, não há de onde criar o segundo dono.

§12.4 O Que `unique_ptr` Custa, e Onde Ele Não Serve

O custo em espaço é nenhum além do ponteiro, e está medido. `include/deriva/medida_posse.hpp` traz cinco `static_assert` sobre tamanho de ponteiro inteligente, e o trecho está extraído no fim deste capítulo: com o deletor padrão, `sizeof(std::unique_ptr<int>)` é igual a `sizeof(int*)`, porque `std::default_delete<T>` não tem estado e o padrão permite que ele desapareça do objeto. O custo em tempo é o do `delete` que você escreveria à mão, no mesmo lugar em que você o escreveria. É o único dos três ponteiros inteligentes de que se pode dizer isso, e é a razão de ele ser a escolha padrão - os outros dois medem $2 * \text{sizeof}(\text{int}^*)$, porque carregam também o ponteiro para o bloco de controle do Cap. 13.

Deletor personalizado é a extensão que o torna útil fora da memória. `std::unique_ptr<FILE, decltype(&std::fclose)>` fecha o arquivo no destrutor, e o mesmo desenho serve para qualquer recurso que tenha uma função de liberação. O tamanho passa a depender do deletor, e o cabeçalho separa os dois casos que se confundem. Deletor **vazio** - uma classe sem campo, com só o `operator()` - não custa nada: a otimização de base vazia o absorve, e o `unique_ptr` continua do tamanho de um ponteiro. Deletor **com estado** cobra o estado dele: com um `const char*` guardado dentro, o objeto passa a `sizeof(int*) + sizeof(const char*)`, e o dobro é visível no `static_assert`. A distinção decide projeto, porque um deletor que só chama uma função livre pode ser escrito das duas formas e só uma delas é grátis.

O `terminal_bruto` do Cap. 8 não usa esse caminho, e a escolha é deliberada: o recurso dele não é um ponteiro, é um estado global do terminal, e classe `RAII` escrita à mão diz isso melhor.

Onde ele não serve é onde a posse é genuinamente compartilhada, e essa palavra precisa de cuidado. Compartilhado não é “muita gente usa”: é “muita gente possui, e não se sabe qual delas morre por último”. Se dá para dizer quem é o dono, o dono tem `unique_ptr` e os outros têm referência ou ponteiro cru sem posse. O Cap. 13 trata do caso em que não dá.

Exercícios Propostos

1. Troque, na sua hierarquia do Cap. 11, o `std::vector<entidade*>` por `std::vector<std::unique_ptr<entidade>>`. Escreva `inserir(std::unique_ptr<entidade>)` recebendo por valor, e uma fábrica que devolva `std::unique_ptr<entidade>`. Ao final, confirme as duas condições juntas: **nenhum delete no seu código**, e vivos fechando em zero.
2. Tente copiar o seu mundo do exercício 1. Leia a mensagem do compilador inteira e diga qual das seis operações especiais está apagada, e por que a ausência dela se propaga do `unique_ptr` para o `vector` e do `vector` para o mundo.
3. Reproduza os cinco defeitos do §12.2, um por um, numa versão da sua hierarquia com posse crua: o vazamento no `return` antecipado, a liberação dupla, o uso depois da liberação, a posse que só existe em comentário, e o vazamento pelo caminho de exceção. Para cada um, diga qual instrumento o acusa - contador, traço de ciclo de vida, `gdb`, ou nenhum.
4. Escreva `entidade* p = new drone{...};` e construa **dois** `unique_ptr` a partir de `p`. Compile - não há erro - e rode. Depois explique, em duas linhas, por que `std::make_unique` torna este erro inescrivível, e não apenas desaconselhado.

5. Tire o virtual do destrutor de entidade e rode o exercício 1 outra vez, sem mudar mais nada. O `unique_ptr` continua liberando a memória, e vivos não fecha em zero. Explique o que exatamente vazou, e por que o ponteiro inteligente não tem como ver isso. Depois compile duas vezes, com `-Wall -Wextra -Wpedantic` e com o conjunto do Deriva, e registre a diferença na contagem de avisos.
6. Por que `make_unique` é preferível a `unique_ptr<T>(new T(...))`? Escreva a chamada `f(std::unique_ptr<drone>(new drone{}), g())` e explique, com a ordem de avaliação dos argumentos, qual é o vazamento possível ali. Depois reescreva com `make_unique` e diga se ele desaparece.
7. Abra o *Complete Roguelike Tutorial, using C++ and libtcod* do RogueBasin, tome a parte em que as entidades são guardadas e destruídas, e aplique os sete itens da rubrica do Cap. 4 por escrito. Depois reescreva **apenas** aquele recorte com posse em `unique_ptr`, e conte quantas linhas a mais ou a menos a sua versão tem.
8. Escreva um `std::unique_ptr` com deletor personalizado para um recurso que não seja memória - um `FILE*`, ou um descritor de arquivo. Meça o `sizeof` do seu ponteiro em três formas - deletor padrão, deletor vazio e deletor com um campo - e escreva o `static_assert` de cada uma **antes** de compilar. Depois compare com os cinco de `include/deriva/medida_posse.hpp` e explique qual dos três não cresceu, e por quê.

O código, extraído do Deriva

Todo trecho abaixo vem de `exemplos/deriva/`, que compila com `-std=c++17 -Wall -Wextra -Wpedantic` sem um aviso e passa `make verifica`. Nenhum foi digitado neste texto.

```

| include/deriva/mundo.hpp:16 |----- a posse declarada no tipo-----
16 /// O estado do domínio: um setor e as entidades que estão nele.
17 ///
18 /// A posse é **exclusiva e declarada no tipo**:
19 /// `std::vector<std::unique_ptr<entidade>>`. O `mundo` é o dono, e o tipo diz
20 /// isso sem precisar de comentário. Não há um `delete` em todo o Deriva.
21 ///
22 /// O par `unique_ptr<entidade>` + destrutor virtual é o que amarra as Aulas 11
23 /// e 12: sem o destrutor virtual, este vetor destruiria só a parte base de
24 /// cada objeto, e nenhum aviso apareceria - porque o `delete` mora dentro do
25 /// `unique_ptr`, num cabeçalho do sistema. É o achado da variante
26 /// `v1.1-quebrada`.
27 ///
28 /// `mundo` é movível e não copiável, e isso não é escolha de estilo: copiar
29 /// exigiria clonar polimorficamente cada entidade, o que é decisão de projeto
30 /// da Aula 25 (Factory), não operação implícita.
31 class mundo {
32 public:
33     explicit mundo(mapa m);
34
35     mundo(const mundo&) = delete;
36     mundo& operator=(const mundo&) = delete;
37     mundo(mundo&&) noexcept = default;
38     mundo& operator=(mundo&&) noexcept = default;
39     ~mundo() = default;
40
41     [[nodiscard]] const mapa& setor() const noexcept { return setor_; }
42     [[nodiscard]] mapa& setor() noexcept { return setor_; }
43
44     /// Toma posse. O `&&` na assinatura é o contrato: quem chama entrega o
45     /// ponteiro e não fica com uma cópia dele.
46     entidade& acrescentar(std::unique_ptr<entidade> e);
47
48     /// Célula dentro do setor e sem parede. Não considera entidades: duas podem
49     /// ocupar a mesma célula, e é a v2.6 que decide se isso é permitido.
50     [[nodiscard]] bool livre(vetor2 p) const;
51
52     /// Um turno para cada entidade, na ordem em que entraram. A ordem é parte
53     /// do comportamento observável, e o replay depende dela.
54     void turno();
55
56     [[nodiscard]] std::size_t quantas() const noexcept { return entidades_.size(); }
57     [[nodiscard]] const entidade& em(std::size_t i) const { return *entidades_[i]; }
58     [[nodiscard]] entidade& em(std::size_t i) { return *entidades_[i]; }
59
60     /// A primeira entidade cujo glifo casa, ou `nullptr`. Devolve ponteiro cru
61     /// de propósito: é observação, não posse, e o tipo tem de dizer a diferença
62     /// (Aula 12).
63     [[nodiscard]] entidade* primeira_com(char glifo) const;
64
65     /// Remove quem estiver na posição. Devolve o ponteiro: quem chamou passa a
66     /// ser o dono, e se ignorar o retorno o objeto morre - daí o
67     /// `[[nodiscard]]`.
68     [[nodiscard]] std::unique_ptr<entidade> retirar_de(vetor2 p);
69
70     /// Render determinístico: o setor com as entidades por cima, e a listagem.

```



```

71  /// Mesma entrada, mesma saída, byte a byte.
72  [[nodiscard]] std::string despejar() const;
73
74  private:
75  mapa setor_;
76  std::vector<std::unique_ptr<entidade>> entidades_;
77  };

```

`vector<unique_ptr<entidade>>` diz quem é o dono sem precisar de comentário, e não há um `delete` em todo o Deriva. O par com o destrutor virtual é o que amarra as Aulas 11 e 12.

```

src/mundo.cpp:35 |----- devolver posse é devolver o ponteiro-----
35  std::unique_ptr<entidade> mundo::retirar_de(vetor2 p) {
36      const auto it = std::find_if(entidades_.begin(), entidades_.end(),
37                                  [p](const std::unique_ptr<entidade>& e) {
38                                      return e->pos() == p;
39                                  });
40      if (it == entidades_.end()) return nullptr;
41      std::unique_ptr<entidade> saindo = std::move(*it);
42      entidades_.erase(it);
43      return saindo;
44  }

```

O retorno é `[[nodiscard]]` porque ignorá-lo destrói o objeto na hora. Compare com `primeira_com`, que devolve ponteiro cru: ali é observação, e o tipo tem de dizer a diferença.

```

include/deriva/medida_posse.hpp:118 |----- quanto custa cada ponteiro-----
118  static_assert(sizeof(std::unique_ptr<int>) == sizeof(int*),
119                  "com o deletor padrao, um ponteiro e nada mais");
120  static_assert(sizeof(std::unique_ptr<int, deletor_sem_estado>) == sizeof(int*),
121                  "deletor vazio: a otimizacao de base vazia o absorve");
122  static_assert(sizeof(std::unique_ptr<int, deletor_com_estado>) ==
123                  sizeof(int*) + sizeof(const char*),
124                  "deletor com estado cobra o estado dele, e o dobro e visivel");

```

`unique_ptr` com deletor padrão é um ponteiro e nada mais; deletor vazio é absorvido pela otimização de base vazia; deletor com estado cobra o estado. `shared_ptr` é o dobro, porque leva o bloco de controle.

Ponteiros inteligentes II - posse compartilhada

O QUE ESTE CAPÍTULO ENTREGA

- ▶ Usar `std::shared_ptr` quando a posse é genuinamente compartilhada.
- ▶ Explicar a contagem de referências, e o que a faz subir e descer.
- ▶ Reconhecer o ciclo de referências, e dizer quantos bytes ele prende.
- ▶ Usar `std::weak_ptr` para observar sem possuir, e desfazer o ciclo.
- ▶ Escolher, para cada requisito, entre `unique_ptr`, `shared_ptr`, `weak_ptr`, referência e ponteiro cru.

O argumento de por que ponteiro inteligente existe está no Cap. 12 §12.1, e não se repete aqui. O que este capítulo acrescenta é o caso que o `unique_ptr` não cobre, e o custo de cobri-lo.

§13.1 `shared_ptr` - Posse Compartilhada

`std::shared_ptr<T>` é a posse compartilhada: vários ponteiros possuem o mesmo objeto, e o objeto morre quando o último deles morre. A pergunta que autoriza usá-lo é a do Cap. 12 §12.4, e vale repeti-la, porque ela é o portão: *não se sabe qual dos donos morre por último*. Se dá para saber, há um dono, e ele tem `unique_ptr`.

O mecanismo é uma contagem, e ela vive num **bloco de controle** separado do objeto. O bloco guarda duas contagens - a de posse, que é a que decide quando destruir o objeto, e a de observação, que é a que decide quando destruir o próprio bloco - e um deletor. Copiar um `shared_ptr` incrementa a contagem de posse; destruí-lo decrementa; quando ela chega a zero, o objeto é destruído. É tudo, e é o que faz tanto a comodidade quanto os dois problemas do resto do capítulo.

Duas coisas que o mecanismo custa, e que a tabela do Cap. 12 §12.1 anunciou.

A primeira é **uma alocação a mais**. O bloco de controle é memória, e ele precisa vir de algum lugar. `std::make_shared` resolve isso alocando o objeto e o bloco juntos, numa única alocação, e é o motivo principal para preferi-lo a `std::shared_ptr<T>(new T(...))` - além do de sempre, que é fechar a janela em que o objeto existe sem dono. A alocação conjunta tem um efeito colateral que quase ninguém espera, e que o §13.3 mede: o objeto e o bloco morrem juntos, então um `weak_ptr` pendente mantém vivo o **espaço** do objeto depois de o objeto ter sido destruído.

A segunda é que a contagem é **atômica**. Ela tem de ser, porque dois threads podem copiar o mesmo `shared_ptr` ao mesmo tempo, e o Cap. 22 mostra o que acontece com contador não atômico nessa situação - é o interativo de corrida de dados, com

o contador vivos do Cap. 7 perdendo um incremento. Incremento atômico custa mais que incremento comum, e o custo se paga em todo lugar, inclusive nos programas de um thread só, que são todos os desta disciplina.

O último trecho no fim deste capítulo é a contagem afirmada em vez de descrita, e é o requisito desta seção verificado pelo portão. `use_count()` é lido em três pontos do mesmo escopo: **1** depois de a eclusa nascer, **3** depois de os dois corredores se ligarem a ela, e **1** outra vez depois de os dois corredores morrerem. O contador vivos confirma o que a contagem prometeu - a eclusa sobreviveu aos dois donos, e nenhum deles a destruiu sozinho. É esse o requisito que autoriza `shared_ptr`, e ele está travado num `REQUIRE` e não num comentário.

▲ ATENÇÃO

Nunca construa dois `shared_ptr` independentes a partir do mesmo ponteiro cru: cada um abre o seu bloco de controle, e o objeto passa a ter dois donos que não sabem um do outro. Use `std::make_shared`, e `shared_from_this` quando o `shared_ptr` tiver de sair de dentro do próprio objeto.

Este é o mesmo defeito do aviso do Cap. 12 §12.3, e no `shared_ptr` ele é mais traiçoeiro, porque a contagem dá a impressão de que alguém está cuidando. Dois blocos de controle sobre um objeto contam até zero cada um por sua conta, e o segundo a chegar libera o que já foi liberado. `std::enable_shared_from_this` existe para o caso em que um método precisa devolver um `shared_ptr` para o próprio objeto: sem ele, `std::shared_ptr<T>(this)` é exatamente o erro acima, escrito de um jeito que parece razoável.

§13.2 `weak_ptr` - Quebrando Ciclos

O ciclo não é um caso exótico, e é aqui que ele aparece no Deriva. O grafo de conexões da estação é um grafo, e não uma árvore: a eclusa liga ao corredor e o corredor liga de volta à eclusa, porque a sonda passa nos dois sentidos. Modelado com `shared_ptr` nas duas pontas, cada nó possui o outro, e nenhuma das duas contagens chega a zero quando os `shared_ptr` locais morrem. Os dois nós ficam alocados, sem ninguém que os alcance, até o fim do processo.

`weak_ptr` é a ponta que observa. Ele não conta na posse, então não sustenta o objeto; em troca, ele não pode ser desreferenciado direto, porque o objeto pode já não existir. O acesso é sempre pelo mesmo movimento de duas etapas: `lock()` devolve um `shared_ptr`, que é nulo se o objeto morreu e válido se não morreu, e é esse `shared_ptr` temporário que sustenta o objeto durante o uso. Perguntar `expired()` e depois usar o ponteiro é o erro clássico, porque entre a pergunta e o uso o objeto pode morrer; `lock()` faz as duas coisas de uma vez, e é por isso que ele é a única forma correta.

A pergunta que decide onde pôr o `weak_ptr` é de domínio, e não de técnica: **qual das duas pontas faz sentido sobreviver sozinha?** Numa árvore, o pai possui o filho e o filho observa o pai, porque filho sem pai não tem significado e pai sem filho tem. No grafo da estação a resposta é menos óbvia, e é por isso que ele é o exemplo: nenhum dos dois compartimentos é o dono natural do outro. A saída é a de sempre nesse caso - o mundo possui todos os compartimentos, com `unique_`

`ptr` ou com `shared_ptr`, e as ligações entre eles são todas de observação. Ciclo que não se resolve escolhendo uma ponta é ciclo que está pedindo um dono de fora.

Os trechos no fim deste capítulo mostram as duas pontas em código que compila. `src/estacao.cpp` traz o `ligar` inteiro em duas linhas, e a decisão está nelas: `adiante_.push_back(b)` possui, e `volta_ = a` observa, com o comentário de cada linha dizendo qual das duas contagens sobe. `include/deriva/medida_posse.hpp` traz as duas formas no mesmo `if`: com a volta em `weak_ptr`, a contagem chega a zero em cascata; com a volta em `shared_ptr`, o ciclo fecha, e o que fica preso é o que o §13.3 mede em bytes.

O `lock()` aparece sob o nome que o domínio lhe deu: `anterior()` devolve `volta_.lock()`, e o teste extraído afirma que ele devolve `nullptr` depois de o nó anterior morrer. É o movimento de duas etapas do parágrafo acima com a primeira etapa escondida atrás de uma função de acesso, que é como ele costuma aparecer em código de verdade - o `shared_ptr` temporário existe pelo tempo da expressão que o usa, e quem chama tem de conferir se ele é nulo antes de desreferenciá-lo.

§13.3 O Ciclo, Medido

O contador de instâncias vivas acusa o ciclo, e acusa bem: os destrutores não rodam, então vivos não volta a zero, e o portão `make_verifica` falha na quarta condição. Isto responde à pergunta “vazou?” e não responde à seguinte, que é a que o estudante faz: **quanto?**

A resposta não dá para travar em `static_assert`, e a razão é honesta: o tamanho do bloco de controle é escolha da implementação, e o padrão não o especifica. Então ele é **medido**. `include/deriva/medida_posse.hpp` traz um alocador que conta exatamente o que o `shared_ptr` pede, e `testes/test_posse.cpp` monta o ciclo mínimo - dois nós, “eclusa” e “corredor”, cada um com o nome, um `shared_ptr` e um `weak_ptr` -, deixa os dois ponteiros locais morrerem, e devolve os bytes que ficaram presos.

O número, nesta `libstdc++`, é **160 bytes**, e ele se decompõe inteiro: são $2 \times (64 \text{ do nó} + 16 \text{ do bloco de controle})$. Os 64 do nó são 32 da `std::string`, 16 do `shared_ptr` e 16 do `weak_ptr`; os 16 do bloco de controle são as duas contagens. Trocar uma das pontas por `weak_ptr` leva a medida a **zero**, em cascata: o segundo nó chega a zero, o destrutor dele solta o `shared_ptr` que ele guardava, e o primeiro chega a zero em seguida.

O material anterior dizia 96 bytes, herdados de uma estimativa do documento de design. Não havia código que sustentasse aquele número, e não havia como descobrir que ele estava errado sem medir. O portão `build/verifica_numeros.py` existe por causa deste caso e de dois outros iguais, e a regra dele é a mais curta do material: **para mudar um número, mude o código**.

Uma armadilha que custou depuração e virou comentário no cabeçalho, porque ela morde qualquer um que tente repetir a medida: `std::allocate_shared` **não** usa o alocador que você passa. Ele o rebinda para o tipo interno do bloco de controle, então um contador `inline static` declarado dentro do template do alocador conta na instancição errada - conta em `contando<no>` enquanto as alocações reais acontecem em `contando<_Sp_counted_ptr_inplace<...>>`. A primeira versão da medida deu zero byte vazado por isso, e zero byte vazado num ciclo é o tipo de resultado que parece confirmar a intuição de quem mediu. Os contadores vivem fora do template, e o trecho no fim deste capítulo mostra por quê.

DERIVA

A escolha de posse, versão por versão, e cada uma por um requisito:

- mundo possui as entidades com `std::vector<std::unique_ptr<entidade>>` (v1.2): há um dono, e é ele.
- O grafo de conexões da estação usa `shared_ptr` para os nós e `weak_ptr` para as voltas (v1.3): o grafo tem ciclo por natureza, e a volta é observação.
- O que uma função só consulta recebe `const entidade&`, e nunca ponteiro inteligente: passar `shared_ptr` por valor a quem só lê incrementa e decrementa a contagem atômica por nada.

§13.4 Regra de Escolha

- `unique_ptr`: padrão para posse exclusiva de objetos polimórficos em coleções de entidades, árvores, grafos sem ciclos.
- `shared_ptr`: quando múltiplos donos precisam sobreviver independentemente - e não se sabe qual deles morre por último.
- `weak_ptr`: para referências para trás em árvores (o filho observa o pai sem possuí-lo), observadores, caches sem propriedade.
- Referência (`T&`): quando não há transferência de propriedade - parâmetros de funções, laços.
- Raw pointer (`T*`): apenas em interfaces de baixo nível, código legado ou quando o tempo de vida é garantido por outro mecanismo.

A regra tem uma ordem, e a ordem é a metade que costuma se perder: comece pela referência, desça para `unique_ptr` quando houver posse, e só chegue a `shared_ptr` quando a pergunta do §13.1 não tiver resposta. `shared_ptr` como escolha padrão é o erro mais caro dos cinco, porque ele **funciona** - o programa não vaza, os testes passam, e o que sobra é uma contagem atômica em todo lugar e uma arquitetura em que ninguém sabe mais quem possui o quê. Posse compartilhada por todos é a posse em comentário do Cap. 12 §12.2 com um invólucro melhor.

O item 1 da rubrica do Cap. 4 é a forma de cobrar isso na revisão: *quem possui cada recurso, e em que linha ele é liberado?* Se a resposta for “todos, e não sei”, o ponteiro está errado, ainda que o programa esteja certo.

Exercícios Propostos

1. Monte o grafo de conexões de três compartimentos da estação com `shared_ptr` nas duas pontas de cada ligação, com o contador vivos em compartimento. Rode e mostre que vivos não fecha em zero. Depois troque as voltas por `weak_ptr` e mostre que fecha. Este exercício e o 7 são as duas metades do portão do **LAB-07**: provocar e desfazer o ciclo, aqui, e escolher a posse por requisito, lá.
2. Instrumente o exercício 1 com `use_count()` impresso em cinco pontos: depois de criar o primeiro nó, depois de ligar o segundo, depois de fechar o ciclo, depois de os ponteiros locais saírem de escopo, e no fim de `main`. Escreva os cinco números antes de rodar, e explique cada divergência.
3. Rode `./build/testes \"*copiar*\"` no Deriva e leia o número que ele imprime. Depois deduza os 160 bytes a partir dos tamanhos: quanto é a `std::`

string, quanto é o `shared_ptr`, quanto é o `weak_ptr`, e quanto sobra para o bloco de controle. Confira o `sizeof(no)` que o teste também imprime.

4. Escreva a sua própria medida do ciclo, com um alocador que conte, sem olhar `include/deriva/medida_posse.hpp`. Se o seu resultado for zero, você caiu na armadilha do rebind; descubra onde, e só então leia o cabeçalho.

5. Implemente um cache de mapas com `shared_ptr`: `cache::obter(nome)` devolve `std::shared_ptr<mapa>`, e vários setores podem compartilhar o mesmo mapa carregado. Verifique que o mapa só é liberado quando o último setor que o referencia é destruído. Depois responda: o cache deve guardar `shared_ptr` ou `weak_ptr` para o que ele serve, e o que muda em cada caso?

6. Tome um método que precise devolver um `shared_ptr` para o próprio objeto. Escreva-o primeiro como `std::shared_ptr<compartimento>(this)`, rode, e mostre a liberação dupla. Depois conserte com `std::enable_shared_from_this` e explique o que ele guarda a mais no objeto.

7. Percorra a regra do §13.4 sobre o Deriva que você escreveu até aqui e justifique, por escrito, a escolha de posse de **cada** membro que é ponteiro. Onde você escolheu `shared_ptr`, responda à pergunta do §13.1; se não conseguir respondê-la, troque por `unique_ptr` e veja o que quebra.

8. Por que `make_shared` é preferível a `shared_ptr<T>(new T(...))`? Trate os dois motivos separadamente - a janela de exceção e a alocação única - e depois diga o caso em que a alocação única é uma **desvantagem**, usando o que o §13.1 disse sobre `weak_ptr` pendente.

O código, extraído do Deriva

Todo trecho abaixo vem de `exemplos/deriva/`, que compila com `-std=c++17 -Wall -Wextra -Wpedantic` sem um aviso e passa `make verifica`. Nenhum foi digitado neste texto.

```

| include/deriva/medida_posse.hpp:72 |----- o ciclo, medido -----
72 /// Monta o ciclo, deixa os dois `shared_ptr` locais morrerem, e devolve
73 /// quantos bytes ficaram presos. Com `usar_weak`, uma das pontas passa a
74 /// observar em vez de possuir, e o resultado tem de ser zero.
75 template <class Aloc>
76 [[nodiscard]] std::size_t bytes_presos(bool usar_weak) {
77     contagem::zerar();
78     {
79         auto a = std::allocate_shared<no>(Aloc{}, no{"eclusa", nullptr, {}});
80         auto b = std::allocate_shared<no>(Aloc{}, no{"corredor", nullptr, {}});
81         a->vizinho = b;
82         if (usar_weak) {
83             b->volta = a;      // observa: a contagem de `a` não sobe
84         } else {
85             b->vizinho = a;    // possui: fecha o ciclo, e ninguém chega a zero
86         }
87     }
88 }

```

O vazamento do ciclo não dá para travar em `static_assert`: o tamanho do bloco de controle é escolha da implementação. Então ele é medido, por um alocador que conta exatamente o que o `shared_ptr` pede.

```

| include/deriva/medida_posse.hpp:24 |----- a armadilha do rebind -----
24 /// Os contadores vivem FORA do template, e há um motivo que custou uma
25 /// depuração: `std::allocate_shared` não usa o alocador que você passa. Ele o
26 /// rebinda para o tipo interno do bloco de controle, e um `inline static`
27 /// dentro de `contando<T>` passaria a contar em `contando<no>` enquanto as
28 /// alocações reais aconteceriam em `contando<Sp_counted_ptr_inplace<...>>`.
29 /// A primeira versão deste cabeçalho media zero byte vazado por isso.
30 struct contagem {
31     inline static std::size_t pedidos = 0;
32     inline static std::size_t devolvidos = 0;
33     inline static std::size_t bytes_pedidos = 0;
34     inline static std::size_t bytes_devolvidos = 0;
35
36     static void zerar() noexcept {
37         pedidos = devolvidos = bytes_pedidos = bytes_devolvidos = 0;
38     }
39     [[nodiscard]] static std::size_t vazados() noexcept {
40         return bytes_pedidos - bytes_devolvidos;
41     }
42 };

```

`std::allocate_shared` não usa o alocador que você passa: ele o rebinda para o tipo interno do bloco de controle. Contador `inline static` dentro do template conta na instanciação errada, e a primeira versão desta medida deu zero byte vazado por isso.

```

src/estacao.cpp:14 |----- a assimetria que impede o ciclo-----
14 void no_estacao::ligar(const std::shared_ptr<no_estacao>& a,
15                       const std::shared_ptr<no_estacao>& b) {
16     a->adiante_.push_back(b); // posse: a contagem de b sobe
17     b->volta_ = a;           // observação: a contagem de a NÃO sobe
18 }

```

Duas linhas, e é nelas que está a lição: a ligação para frente possui e a de volta observa. Trocar `volta_` por `shared_ptr` fecha o ciclo e prende 160 bytes por par de nós.

```

testes/test_estacao.cpp:45 |----- weak_ptr não pendura-----
45 TEST_CASE("weak_ptr obriga a perguntar, e por isso nao pendura") {
46     zerar_estacao();
47     std::shared_ptr<no_estacao> b;
48     {
49         auto a = std::make_shared<no_estacao>("a");
50         b = std::make_shared<no_estacao>("b");
51         no_estacao::ligar(a, b);
52         REQUIRE(b->anterior() != nullptr);
53     }

```

`lock()` devolve `nullptr` quando o objeto morreu. É a pergunta obrigatória que torna o ponteiro pendurado impossível - e é o que `shared_ptr` na volta impediria de acontecer, porque o objeto nunca morreria.

```

testes/test_estacao.cpp:10 |----- o requisito que shared_ptr atende-----
10 TEST_CASE("posse compartilhada: dois donos, e nenhum destroi sozinho") {
11     zerar_estacao();
12     {
13         auto eclusa = std::make_shared<no_estacao>("eclusa");
14         REQUIRE(eclusa.use_count() == 1);
15         {
16             auto corredor_a = std::make_shared<no_estacao>("corredor-a");
17             auto corredor_b = std::make_shared<no_estacao>("corredor-b");
18             no_estacao::ligar(corredor_a, eclusa);
19             no_estacao::ligar(corredor_b, eclusa);
20             REQUIRE(eclusa.use_count() == 3); // este escopo mais os dois corredores
21             REQUIRE(no_estacao::vivos == 3);
22         }

```

A contagem vai a 1, sobe a 3 e volta a 1 no mesmo escopo. É o requisito que `unique_ptr` não atende: a eclusa pertence aos dois corredores, e nenhum dos dois pode destruí-la sozinho.

Semântica de movimento e a regra dos cinco

O QUE ESTE CAPÍTULO ENTREGA

- ▶ Compreender rvalues e lvalues e a distinção entre eles.
- ▶ Usar `std::move` para transferir recursos eficientemente.
- ▶ Dizer o que o padrão promete, e o que ele não promete, sobre o objeto de origem depois do movimento.
- ▶ Implementar construtor e operador de atribuição de movimento, e marcá-los `noexcept`.
- ▶ Aplicar a regra dos cinco, e reconhecer quando ela é a resposta errada.
- ▶ Identificar quando o compilador aplica movimento e quando ele elide a cópia.
- ▶ Reconhecer uma referência universal e o papel de `std::forward`.

§14.1 O Problema: Cópias Desnecessárias

Antes de C++11, devolver um objeto grande de uma função significava copiar toda a memória dele. A semântica de movimento resolve isso: em vez de copiar, transferir os recursos do objeto de origem.

O Deriva já mediu o problema, e a medida corrigiu o argumento com que este capítulo costumava abrir. Vale contar a correção, porque ela ensina mais que o argumento antigo.

O Cap. 9 §9.1 mostrou que `mapa` declara destrutor e cópia - para mexer no contador de instâncias vivas - e que, por causa da segunda linha da tabela do Cap. 9, o compilador **deixou de gerar** as operações de movimento. A expectativa era a óbvia: dar a `mapa` as duas operações que faltavam faria a contagem de construções cair. `testes/test_mapa.cpp` mediu, nos dois estados do código, e a contagem **não caiu**. `de_texto` custa **duas** construções com o construtor de movimento declarado, e duas sem ele. Nasceram dois objetos nos dois casos, o local da função e o que vai para dentro do `std::optional`, e o contador conta objetos.

O que o movimento mudou é o **custo** da segunda construção, e ele não aparece em contagem nenhuma. Com o construtor de movimento, o ponteiro de heap da grade troca de dono; sem ele, as 1920 células de uma grade de 80 por 24 são copiadas uma a uma, a 12 bytes cada. A única forma de ver a diferença é por **identidade de endereço**: o teste extraído no fim deste capítulo guarda o endereço da primeira célula antes de mover, e afirma que o objeto movido devolve o **mesmo** endereço e o objeto copiado devolve outro.

Daí sai a lição de instrumento desta aula, e ela é do mesmo tipo da do Cap. 11 §11.5: **o contador de instâncias vivas não distingue cópia de movimento**. Ele foi posto a contar nascimentos, e conta nascimentos; o que custou cada nascimento é uma pergunta que ele não responde. Saber o que o instrumento não vê vale tanto

quanto saber usá-lo, e é a segunda vez na unidade que a frase se aplica ao mesmo contador.

Vale separar duas coisas que se misturam com facilidade em toda discussão de retorno de função, porque o C++17 mudou a fronteira entre elas.

Elisão de cópia é o compilador **não construir** o segundo objeto. Desde o C++17, quando o que se devolve é um prvalue - `return mapa{nome, 1, a};` -, a elisão é obrigatória, e não há cópia nem movimento: o objeto é construído diretamente no lugar de destino, e nenhuma das duas operações especiais precisa nem existir. Quando o que se devolve é uma variável local nomeada - `return m;` -, o caso é o NRVO, que continua sendo **permitido e não obrigatório**; se o compilador não o aplicar, ele usa o construtor de movimento, e se este não existir, a cópia.

Movimento é o compilador construir o segundo objeto de forma barata, transferindo o recurso do primeiro. É o que sobra quando a elisão não se aplica, e é o que este capítulo escreve.

A confusão entre as duas produz uma conclusão errada e comum: a de que declarar as operações de movimento acelera o retorno de função. Em geral não acelera, porque a elisão já resolveu; o que elas aceleram é tudo o mais - a passagem por valor, o `push_back` num vetor que realoca, o `std::swap`, a inserção num contêiner.

§14.2 lvalues e rvalues

A distinção que a linguagem faz não é entre tipos, e sim entre **expressões**, e é por isso que ela escapa a quem a procura na declaração da variável. Uma expressão é lvalue quando designa um objeto com endereço que o programa pode tomar, e é o caso de toda variável nomeada e de tudo o que uma função devolve por referência, tais como a `celula&` de `grade::em`. É rvalue quando designa um valor sem endereço acessível ao programador - o literal, o resultado de uma expressão aritmética, e o objeto que uma função devolve por valor, como o `std::optional<mapa>` que sai de `mapa::carregar`.

A regra prática que resolve 95% dos casos é mais curta que a definição: **se tem nome, é lvalue**. `m` é lvalue; `mapa::de_texto(texto, "setor")` é rvalue; `std::move(m)` é rvalue, ainda que `m` tenha nome, porque `std::move` é uma conversão que produz uma expressão sem nome.

A consequência que importa é de sobrecarga. `void inserir(mapa m)`, `void inserir(const mapa& m)` e `void inserir(mapa&& m)` são três funções diferentes, e qual delas o compilador escolhe depende da categoria do argumento, não do tipo. É esse mecanismo, e nada mais, que faz o movimento acontecer: o construtor de movimento é apenas a sobrecarga que recebe `mapa&&`, e ela é escolhida quando o argumento é rvalue.

```

1 #include <cstddef>
2 #include <string>
3 #include <utility>
4
5 std::size_t categorias_de_valor() {
6     int a = 42;                // a é lvalue; 42 é rvalue
7     int b = a + 1;            // (a + 1) é rvalue; b é lvalue
8
9     std::string s1 = "ola";    // "ola" é rvalue
10    std::string s2 = s1;        // s1 é lvalue: CÓPIA
11    std::string s3 = std::move(s1); // std::move produz rvalue: MOVIMENTO
12    // s1 fica em estado válido e não especificado, e nada abaixo o lê
13
14    return static_cast<std::size_t>(b) + s2.size() + s3.size();
15 }

```

Há uma armadilha de nome que custa tempo: **uma referência a rvalue, quando tem nome, é um lvalue**. Dentro de `mapa(mapa&& o)`, `o` tem nome, então `o` é lvalue, e escrever `grade_ = o.grade_` copia a `grade` em vez de movê-la. O construtor de movimento precisa de `std::move` no próprio corpo, membro por membro, e esquecer isso produz um construtor de movimento que é uma cópia com outro nome - que compila, passa nos testes de valor, e não economiza nada.

§14.3 std::move e Referências a rvalue

`std::move` não move nada, e o nome atrapalha mais do que ajuda: ele é um cast, que recebe um lvalue e devolve uma referência a rvalue - `T&&` -, de forma que a resolução de sobrecarga passe a escolher o construtor ou o operador de movimento no lugar dos de cópia. Quem move é a sobrecarga escolhida, e não a conversão.

A frase acima é a mais importante do capítulo e vale explicá-la pelo avesso: se você apagar todos os construtores de movimento do seu programa e mantiver todos os `std::move`, o programa continua a compilar e passa a copiar em todo lugar, sem um aviso. `std::move` é um pedido, e quem atende é a sobrecarga; onde não há sobrecarga de movimento, a de cópia atende, porque `const T&` aceita rvalue.

O idioma que este material recomenda para função que guarda o que recebe é **parâmetro por valor mais std::move para dentro**, e a razão é que ele escreve uma função só onde as duas sobrecargas escreveriam duas. Quem chama com lvalue paga uma cópia e um movimento; quem chama com rvalue paga dois movimentos. É um movimento a mais que a versão com duas sobrecargas, e é barato quando o movimento é barato, que é o caso de todo tipo que gerencia recurso por ponteiro.

O trecho extraído no fim deste capítulo é o idioma inteiro em duas linhas. `mundo::acrescentar` recebe `std::unique_ptr<entidade>` por valor e faz `push_back(std::move(e))`; num tipo que só se move, quem chama decide se move um ponteiro que já tinha ou constrói um no lugar, e a função move para dentro do vetor sem cópia nenhuma. Repare no contraste nas chamadas: em `testes/test_mundo.cpp` e em `testes/test_entidade.cpp`, todo `acrescentar` recebe o resultado de `std::make_unique`, que já é rvalue e por isso **não** leva `std::move`. Escrever `std::move` ali seria ruído, e é o segundo caso do §14.4 - `std::move` onde não há nada a transferir.

0 avisos

avisos do compilador

aferido por

make verifica

Makefile

§14.4 O Que Sobra na Origem, e o Que o Padrão Promete

Aqui o material antigo ensinava folclore, e o folclore é este: *depois de* `std::move(s)`, *s fica vazio*. Não é o que o padrão diz, e a diferença tem consequência prática.

O que o padrão promete para os tipos da biblioteca é **estado válido mas não-especificado**. Válido significa que o objeto continua a obedecer às invariantes da classe: você pode destruí-lo, pode atribuir-lhe um valor novo, pode chamar qualquer operação que não tenha pré-condição sobre o valor. Não-especificado significa que **qual** valor ele tem não está escrito em lugar nenhum, e não é a mesma coisa que “é vazio”.

A regra que decorre disso é dura e é simples: **nada, em nenhum ponto do seu código, lê o objeto de origem depois do movimento**. Não `assert(origem.empty())`, não `if (origem.size() == 0)`, não impressão de diagnóstico. As duas operações permitidas são destruir e reatribuir. Escrito assim, o código está correto em toda implementação, e a pergunta “o que sobrou?” deixa de existir.

E ela deixa de existir por um motivo que só se enxerga medindo. Considere `std::string`, que é o caso mais comum, e a otimização de string curta que ela usa: cadeias de até 15 caracteres, nesta `libstdc++`, moram dentro do próprio objeto, sem alocação no heap. Numa string curta **não há ponteiro de heap a roubar**, então a operação de movimento copia os bytes, exatamente como a de cópia faria. Isto é verificável, e a forma de verificar é olhar o endereço: movendo uma string longa, `data()` do destino é o mesmo endereço que a origem tinha, porque o ponteiro foi transferido; movendo uma string curta, `data()` do destino é outro endereço, dentro do objeto de destino, porque os bytes foram copiados.

A conclusão prática é sobre custo, e não sobre estado: **`std::move` sobre uma string curta não economiza nada**. Ele não é errado, e não é otimização; é a mesma cópia com outro nome. Quem escreve `std::move` em toda passagem de string na esperança de acelerar o programa está escrevendo ruído em metade dos casos.

E o estado da origem? Nesta `libstdc++`, com `g++ 13.3.0`, a origem é esvaziada nos quatro casos - construção e atribuição, curta e longa. É por isso que o folclore sobrevive: ele é **observavelmente verdadeiro** neste compilador, o teste errado `REQUIRE(origem.empty())` passa, e nada acusa que o programa está apoiado numa coisa que o padrão não garante. Defeito que o teste confirma é o mais difícil de tirar de um projeto.

Os dois parágrafos acima não são leitura de manual, são medida, e ela tem arquivo: `testes/test_move_string.cpp` roda dentro do portão e afirma as duas coisas separadamente. O primeiro caso percorre as quatro combinações - curta e longa, por construção e por atribuição - e afirma `origem.empty()` em todas as quatro, que é o que **este** alvo faz. O segundo caso mede a diferença que de fato existe entre curta e longa, por endereço: na curta, `data()` do destino é outro endereço e o da origem continua apontando para si mesma, porque os 8 bytes foram copiados; na longa, `data()` do destino é o endereço que a origem tinha, porque nenhum byte de conteúdo se moveu. O comentário de cabeçalho do arquivo registra que ele substituiu duas afirmações erradas, uma de cada lado - a de que a origem sempre fica vazia, e a de que a otimização de string curta deixaria a origem intacta.

A entrega desta aula cobra exatamente esta disciplina, e o portão do **LAB-08** a nomeia: *cópia versus movimento em grade, e o objeto de origem depois*. Nenhum teste da entrega lê o objeto de origem depois do movimento, e a instrumentação de ciclo de vida é o que se lê no lugar.

▲ ATENÇÃO

Depois de `std::move` em um objeto, não faça suposição nenhuma sobre o estado dele. Ele está em **estado válido mas não-especificado**, que não quer dizer vazio nem zerado. Destruir e reatribuir são as únicas operações seguras.

O caso que confirma a regra é o do próprio `std::unique_ptr` do Cap. 12: ali a origem **fica nula**, e fica porque a operação de movimento dele promete isso explicitamente. Promessa da classe, e não regra geral do `std::move`.

§14.5 A Regra dos Cinco

Com C++11, o compilador pode gerar operações de movimento (construtor de movimento e atribuição de movimento). Se você declara qualquer uma das três operações da regra dos três, o compilador não gera as de movimento. Para ter todas as cinco corretas, declare todas as cinco.

A regra fecha o trio que a disciplina usa, e a ordem em que se decide entre as três é o conteúdo, e não a lista.

A regra do zero é a primeira escolha, e o Cap. 9 §9.2 já a defendeu: se todo membro é RAII da biblioteca padrão, declare zero operações e o compilador gera as cinco, corretas e sempre atualizadas. É onde a grade está.

A regra dos três é o que se cumpre quando há recurso à mão, e no Deriva ela aparece na forma curta, com as duas operações de cópia = delete, em `terminal_bruto`.

A regra dos cinco é para o caso restante, e ele é mais estreito do que parece. Ela cabe quando a classe tem de declarar o destrutor por uma razão que não é gestão de recurso - e é exatamente o caso de mapa, que declara destrutor e cópia para mexer no contador de instâncias vivas. Declarar o destrutor apagou as operações de movimento; declarar as cinco as traz de volta.

Repare no que isso diz sobre a caça ao bug 1 do Cap. 9. Lá, o conserto “melhor” foi voltar à regra do zero trocando o ponteiro cru por `std::vector`, e o argumento foi que ela devolve as cinco operações de graça, inclusive as duas de movimento. Este é o capítulo em que aquela frase fecha: as duas de movimento que a v0.3 boa ganhou sem escrever uma linha são estas, e a versão escrita à mão da regra dos três não as teria.

As cinco de mapa estão extraídas no fim deste capítulo, declaradas no mesmo lugar, com a razão escrita ao lado - e o comentário de lá registra a medição que contrariou o que ele mesmo dizia antes, que é o §14.1 deste capítulo. Vale ler os dois lados do movimento lado a lado, porque a assimetria entre eles é o que mais confunde: o **construtor** de movimento incrementa o contador, porque um objeto nasceu, ainda que sem alocar nada; a **atribuição** de movimento não toca no contador, porque atribuir não cria nem destrói ninguém, e a guarda de autoatribuição é o único cuidado extra que ela pede.

Sobre a atribuição de cópia, um idioma que vale conhecer pelo nome e que a mapa não usa: **copy-and-swap**. O parâmetro entra por valor, então a cópia já aconteceu na passagem; o corpo só troca os membros, e o objeto antigo sai pelo destrutor do parâmetro. Ele resolve a autoatribuição sem guarda explícita e dá garantia forte de exceção de graça, porque tudo o que poderia lançar já lançou antes de o estado

deste objeto ser tocado. O preço é uma cópia onde a versão com guarda poderia reaproveitar a alocação existente, e é o exercício 2 que mede esse preço.

E a obrigação real do construtor de movimento, quando a classe gerencia memória à mão, não é a que o folclore diz. Numa classe com `celula*` cru, o ponteiro da origem tem de ficar `nullptr` **porque o destrutor dela vai rodar e vai chamar `delete[]`**; se o ponteiro continuasse lá, seria liberação dupla. A origem é deixada num estado que o destrutor dela suporta, e é essa a obrigação. Que o estado escolhido pareça “vazio” é consequência, e é por isso que quem escreve a classe pode contar com ele e quem apenas usa a classe não pode.

—| DICA |—

Antes de escrever as cinco, pergunte por que você está declarando a primeira. Se a resposta é “para liberar um recurso”, há um tipo `RAII` da biblioteca padrão que faz isso, e a regra do zero é melhor. Se a resposta é outra - contar instâncias, registrar-se num observador, tocar num estado global -, a regra dos cinco é a resposta certa, e as cinco vêm juntas.

§14.6 `noexcept`, e Por Que o `vector` Depende Dele

Operação de movimento se marca `noexcept`, e não é cerimônia: é o que decide o comportamento de `std::vector`.

Quando um `vector` cresce, ele aloca um bloco novo e transfere os elementos do antigo para o novo. Ele quer mover, porque mover é barato. Só que a transferência tem de dar garantia forte: se algo lançar no meio, o vetor precisa ficar como estava. Com cópia isso é possível - o bloco antigo continua intacto e basta descartar o novo. Com movimento não é, porque os elementos já transferidos foram esvaziados na origem e não há como desfazer.

A saída que a biblioteca adota é uma pergunta feita em tempo de compilação: se o construtor de movimento do elemento for `noexcept`, o `vector` **move**; se não for, ele **copia**, para preservar a garantia. O efeito é que uma classe com construtor de movimento não marcado `noexcept` tem movimento que nunca é usado pelo `vector`, e o programa paga cópia em toda realocação sem que nada avise. É o custo mais invisível deste capítulo, e o remédio são nove caracteres.

Duas notas. `noexcept` só é honesto se a operação de fato não puder lançar, e a de movimento em geral não pode, porque ela troca ponteiros e inteiros. Se um membro tem movimento que lança, a sua classe não pode prometer o contrário - `noexcept` mentiroso não vira exceção propagada, vira `std::terminate`. E `std::move_if_noexcept` é a ferramenta que a biblioteca usa por dentro para fazer essa escolha, e vale conhecer o nome para reconhecer o mecanismo quando ele aparecer no rastro de erro de um `template`.

§14.7 Encaminhamento Perfeito e `std::forward` - Panorama

Esta seção é panorama, e não conteúdo de prova. Ela existe porque o estudante vai ler `T&&` e `std::forward` em qualquer biblioteca moderna, inclusive na `std::make_unique` que ele já usou no Cap. 12, e não reconhecer o idioma é não conseguir ler a assinatura.

O problema é o seguinte. Uma função que recebe um argumento e o repassa a outra função quer preservar a categoria do valor: se recebeu `lvalue`, repassar `lvalue`; se recebeu `rvalue`, repassar `rvalue`. Com sobrecargas, isso custa duas funções para um parâmetro, quatro para dois, e 2^n para n - e é por isso que sobrecarga não é a resposta.

A resposta é uma construção com duas partes.

A primeira é a **referência universal**, que se escreve `T&&` num parâmetro cujo `T` é parâmetro de template dedutível daquela função. Ela parece uma referência a `rvalue` e não é: pelas regras de colapso de referências, quando o argumento é `lvalue` o `T` é deduzido como referência a `lvalue`, e `T&&` colapsa em referência a `lvalue`. Uma mesma assinatura passa a aceitar as duas categorias, guardando na dedução qual delas veio. Repare no que a distingue de uma referência a `rvalue` de verdade: `void f(mapa&& m)` tem `mapa` fixo e é referência a `rvalue`; `template <class T> void f(T&& m)` tem `T` dedutível e é universal.

A segunda é `std::forward<T>`, que é a conversão que restaura a categoria guardada. Onde `std::move` converte **sempre** para `rvalue`, `std::forward<T>` converte **conforme** o `T` deduzido: para `rvalue` se o argumento era `rvalue`, e para `lvalue` se era `lvalue`. É a razão de ele existir, e é a razão de ele ser sempre escrito com o parâmetro de template explícito, porque é justamente esse parâmetro que carrega a informação.

Juntas, as duas dão o **encaminhamento perfeito**: uma função de fábrica que recebe os argumentos do construtor e os repassa sem copiar o que não precisa ser copiado nem mover o que não pode ser movido. É exatamente o que `std::make_unique<drone>(pos, carga)` faz com `pos` e `carga`, e saber disso responde à pergunta que aparece na primeira vez que alguém usa `make_unique` com um argumento grande.

Panorama não quer dizer sem código. `include/deriva/encaminhamento.hpp` traz duas fábricas lado a lado - uma que copia sempre, escrita com parâmetro por valor, e a `criar_encaminhando<T>(Args&&...)`, que é a referência universal mais `std::forward` na forma mínima -, e as duas estão no portão. O que o par prova está no segundo trecho extraído: encaminhado como `lvalue`, o argumento chega **copiado** ao construtor, e a origem **continua utilizável** depois da chamada. Trocar `std::forward` por `std::move` ali compilaria, passaria a construir por movimento, e roubaria o objeto de quem chamou - o defeito clássico deste idioma, e um que nenhum aviso pega. É esse teste que a troca quebraria, e é por isso que ele existe.

A regra de uso, para esta disciplina, é de duas linhas: **`std::move` quando você sabe que é `rvalue` e quer transferir; `std::forward<T>` só dentro de função com referência universal, e só uma vez por argumento**. Encaminhar duas vezes o mesmo argumento é mover duas vezes o mesmo objeto, e o segundo movimento pega o que o primeiro deixou.

§14.8 Semântica de Valor e Semântica de Referência

Tipos com semântica de valor se comportam como `int`: copiar cria um objeto independente, e modificar a cópia não afeta o original. É o comportamento padrão de classes C++ com a regra do zero corretamente aplicada. Semântica de referência é quando vários handles apontam para o mesmo dado subjacente, como `shared_ptr`.

A escolha entre as duas é de projeto, e o Deriva usa as duas de propósito, em camadas separadas.

`vetor2`, `celula`, `grade` e `mapa` têm semântica de **valor**: copiar um mapa produz um mapa independente, e é isso que o teste do Cap. 9 verifica quando escreve numa célula da cópia e confirma que a original não mudou. Valor é o que permite o `despejar()` determinístico, porque um mapa não muda por baixo de quem o está lendo.

entidade e as derivadas têm semântica de **referência**, e não por escolha estética: objeto polimórfico não cabe num contêiner por valor. Guardar `std::vector<entidade>` seria fatiar o objeto - o `vector` guardaria só a parte entidade de cada derivada, descartando o resto, sem aviso e sem erro de execução, e o despacho virtual passaria a chamar a base. É o defeito que o glossário chama de fatiamento, e é a razão técnica de as entidades viverem atrás de `unique_ptr` no Cap. 12.

A regra que fecha: **valor para dado, referência para comportamento**. O que a hierarquia existe para variar é o comportamento, e comportamento não se copia, se aponta.

Exercícios Propostos

1. Dê a `mapa` as cinco operações: destrutor, cópia, atribuição de cópia, construtor de movimento `noexcept` e atribuição de movimento `noexcept`. Trate o contador de instâncias vivas em cada uma, e escreva **antes de compilar** qual delas incrementa vivos e qual não. Depois rode testes/`test_mapa.cpp` e confira o que o §14.1 mediu: a contagem de construções de `de_texto` **não** muda, e a única diferença observável é o endereço do buffer da `grade`. Explique por escrito por que a contagem não caiu.

2. Implemente a regra dos cinco completa para a `grade_crua` da caça ao bug 1 - a versão com `celula*`, usando o idioma **copy-and-swap** no operador de atribuição de cópia. Escreva testes que verifiquem: (a) a cópia produz objeto independente; (b) o movimento deixa a origem num estado que o destrutor dela suporta; (c) não há liberação dupla. Depois compare com a versão da regra do zero e conte as linhas de cada uma. (*Exercício 2 do Cap. 10 do v1, reescrito sobre o Deriva.*)

3. Escreva o construtor de movimento de `mapa` esquecendo o `std::move` no corpo: `nome_(o.nome_)` em vez de `nome_(std::move(o.nome_))`. Compile - não há erro, não há aviso - e prove, com um contador de cópias na `std::string` ou com o endereço de `data()`, que o seu construtor de movimento está copiando. Explique a armadilha de nome do §14.2 que produz isso.

4. Por que as operações de movimento devem ser marcadas `noexcept`? Meça: crie um `std::vector<mapa>` e faça-o realocar várias vezes, contando construções de cópia e de movimento, primeiro com o construtor de movimento `noexcept` e depois sem. Reporte os dois números e explique a decisão que o `vector` tomou em cada caso. (*Exercício 4 do Cap. 10 do v1, reescrito sobre o Deriva.*)

5. Meça o que a otimização de string curta faz com o movimento. Para uma `std::string` de 6 caracteres e outra de 44, imprima o endereço de `data()` antes do movimento e o do destino depois. Explique os dois resultados, e compare com o que `testes/test_move_string.cpp` afirma. Depois responda: no seu compilador, a origem esvaziou? E o que o padrão garante sobre isso?
6. Encontre, no seu próprio código do semestre, todo teste ou `assert` que leia o objeto de origem depois de um `std::move`, e apague-o - substituindo por uma verificação sobre o **destino**, que é o que interessava. Este é o portão do **LAB-08**, e o critério dele é o do título: cópia versus movimento em grade, e o objeto de origem depois, instrumentado e lido na saída em vez de afirmado.
7. Escreva uma função `mundo::criar` com referência universal e `std::forward`, que construa qualquer derivada de entidade e a insira. Depois escreva a mesma coisa com sobrecargas para `lvalue` e `rvalue` e conte quantas funções seriam necessárias com dois parâmetros e com três. Explique de onde vem o 2^n .
8. Demonstre o fatiamento de objeto do §14.8: guarde um drone num `std::vector<entidade>` - ou tente, e leia o erro se entidade for abstrata -, e depois num `std::vector<entidade*>`. Explique a diferença em termos de leitura do objeto, usando os números do Cap. 11 §11.4.
9. Devolva um mapa de uma função de três formas: `return m;` sobre variável local, `return mapa{...}` sobre um `prvalue`, e `return std::move(m);`. Instrumente as cinco operações e registre quais rodam em cada caso. A terceira forma é pior que a primeira: explique por que, usando o que o §14.1 disse sobre elisão obrigatória e NRVO.

O código, extraído do Deriva

Todo trecho abaixo vem de `exemplos/deriva/`, que compila com `-std=c++17 -Wall -Wextra -Wpedantic` sem um aviso e passa `make verifica`. Nenhum foi digitado neste texto.

```

┌─── testes/test_move_string.cpp:28 ──── a origem, nos quatro casos ────┐
28 TEST_CASE("nesta implementacao a origem esvazia nos quatro casos") {
29     SECTION("curta, por construcao") {
30         std::string a(kCurta);
31         const std::string b(std::move(a));
32         REQUIRE(b == kCurta);
33         REQUIRE(a.empty());
34     }
└───┘

```

Curta e longa, construção e atribuição: a origem esvazia nos quatro. É justamente porque `REQUIRE(origem.empty())` PASSA aqui que o folclore sobrevive e o erro embarca - o padrão promete estado válido, não vazio.

```

┌───| testes/test_move_string.cpp:55 |─── a diferença que se reproduz ──┐
55 // A diferença que REALMENTE se observa entre curta e longa não é o estado da
56 // origem: é se algum byte foi copiado. É este o fato que o interativo da Aula
57 // 14 mostra, porque é o único que se reproduz.
58 TEST_CASE("move de string curta COPIA bytes; de string longa transfere ponteiro") {
59     SECTION("curta: o buffer é interno, então não há ponteiro a roubar") {
60         std::string a(kCurta);
61         const void* antes = a.data();
62         const std::string b(std::move(a));
63         REQUIRE(a.data() == antes);           // a origem segue apontando para si mesma
64         REQUIRE(b.data() != antes);           // e o destino teve de copiar 8 bytes
65         REQUIRE(b.size() == 8);
66     }
}
└───┐

```

Curta: o endereço do destino é NOVO, porque oito bytes foram copiados - não havia ponteiro a roubar. Longa: o mesmo ponteiro de heap troca de dono, e nenhum byte de conteúdo se move.

```

┌───| src/mapa.cpp:41 |─── o construtor de movimento ──┐
41 // Mover um mapa não copia a grade: o `std::vector` de dentro dela transfere o
42 // ponteiro do heap. O que ainda custa é o contador - um objeto NASCEU, mesmo
43 // que sem alocar, e por isso `criados` sobe.
44 //
45 // `noexcept` não é decoração: `std::vector<mapa>` só usa o movimento ao
46 // realocar se ele for `noexcept`; sem a palavra, ele copia (Aula 14).
47 mapa::mapa(mapa&& o) noexcept
48     : nome_(std::move(o.nome_)),
49       grade_(std::move(o.grade_)),
50       entrada_(o.entrada_),
51       marca_(std::move(o.marca_)) {
52     ++contador_mapa::vivos;
53     ++contador_mapa::criados;
54 }
└───┐

```

`noexcept` não é decoração: `std::vector<mapa>` só usa o movimento ao realocar se ele for `noexcept`; sem a palavra, copia.

```

| testes/test_mapa.cpp:122 |----- o que o movimento muda -----|
122 // v1.4 · Aula 14 - o que o movimento muda, e o que ele NÃO muda.
123 //
124 // Surpresa medida: o contador de `criados` continua em 3 para um `carregar`,
125 // exatamente como antes do construtor de movimento existir. O contador conta
126 // OBJETOS, e três objetos nascem nos dois casos. O que mudou é o custo de
127 // cada nascimento, e isso o contador não vê.
128 TEST_CASE("mover um mapa transfere o buffer da grade em vez de copia-lo") {
129     auto a = mapa::de_texto(kSetor, "origem");
130     REQUIRE(a.has_value());
131     const void* buffer_antes = &a->g().em({0, 0});
132
133     const mapa movido(std::move(*a));
134     REQUIRE(&movido.g().em({0, 0}) == buffer_antes); // o MESMO bloco de heap
135
136     const mapa copiado(movido);
137     REQUIRE(&copiado.g().em({0, 0}) != buffer_antes); // buffer novo, 60 células
138 }

```

Mover devolve o MESMO endereço de buffer; copiar devolve um novo. E o contador não distingue os dois, porque conta objetos e não alocações.

```

| src/mapa.cpp:88 |----- variável nomeada devolvida -----|
88 mapa m(std::move(nome), static_cast<int>(largura),
89         static_cast<int>(fileiras.size()));
90
91 // Ligação estruturada sobre índice e conteúdo seria mais elegante com
92 // `enumerate`, que é C++23. Aqui, um laço com índice explícito.
93 for (int y = 0; y < static_cast<int>(fileiras.size()); ++y) {
94     for (int x = 0; x < static_cast<int>(largura); ++x) {
95         const char c = fileiras[static_cast<std::size_t>(y)][static_cast<std::size_t>(x)];
96         celula& cel = m.grade_.em({x, y});
97         cel.glifo = c;
98         switch (c) {
99             case '#': cel.massa = 1000; break; // parede: não se move
100             case '!': cel.massa = 3; cel.energia = 1; break;
101             case '@': m.entrada_ = {x, y}; cel.glifo = '.'; break;
102             default: break;
103         }
104     }
105 }
106 return m;

```

É o caso do NRVO: o compilador pode construir `m` diretamente no lugar do retorno e não copiar nem mover. Ele **pode**, e não é obrigado - a elisão obrigatória de C++17 vale para `prvalue`, que este não é.

```

┌─| src/mundo.cpp:10 |────────────────────────────────── por valor, e depois 'std::move' ───────────────────┐
10 entidade& mundo::acrescentar(std::unique_ptr<entidade> e) {
11     entidades_.push_back(std::move(e));
12     return *entidades_.back();
13 }
└────────────────────────────────────────────────────────────────────────────────────────────────────────────────┘

```

Parâmetro por valor num tipo só-movível é o idioma: quem chama decide se move ou constrói no lugar, e a função move para dentro do vetor sem cópia nenhuma.

```

┌─| include/deriva/mapa.hpp:38 |────────────────────────────────── as cinco, juntas ───────────────────┐
38 // A regra dos cinco, completa (v1.4 · Aula 14).
39 //
40 // MEDIDO, e o resultado contraria o que este comentário dizia antes. Com o
41 // construtor de movimento, 'de_texto' custa **duas** construções; sem ele,
42 // custa **duas** também. O movimento não mudou número nenhum que o contador
43 // relate, porque o contador conta OBJETOS - e dois objetos nascem nos dois
44 // casos: o local e o que vai para dentro do 'std::optional'.
45 //
46 // O que o movimento mudou é o CUSTO da segunda construção: com ele, o
47 // ponteiro de heap da grade troca de dono; sem ele, as células são copiadas
48 // uma a uma. 'testes/test_mapa.cpp' mede a diferença por identidade de
49 // endereço, que é o único jeito de vê-la - e é essa a lição da Aula 14: o
50 // instrumento da Aula 07 não distingue cópia de movimento, e saber o que ele
51 // não vê vale tanto quanto saber usá-lo.
52 //
53 // 'noexcept' não é decoração: 'std::vector<mapa>' só usa o movimento ao
54 // realocar se ele for 'noexcept'.
55 ~mapa();
56 mapa(const mapa& o);
57 mapa& operator=(const mapa& o);
58 mapa(mapa&& o) noexcept;
59 mapa& operator=(mapa&& o) noexcept;
└────────────────────────────────────────────────────────────────────────────────────────────────────────────────┘

```

Declaradas no mesmo lugar, com a razão escrita ao lado. O comentário registra uma medição que contrariou o que ele mesmo dizia antes.

```

src/mapa.cpp:56 |----- a atribuição de movimento-----
56 mapa& mapa::operator=(mapa&& o) noexcept {
57     if (this != &o) {
58         nome_ = std::move(o.nome_);
59         grade_ = std::move(o.grade_);
60         entrada_ = o.entrada_;
61         marca_ = std::move(o.marca_);
62     }

```

Ela não toca no contador, e o construtor toca: atribuir não cria nem destrói ninguém. É a assimetria que mais confunde na regra dos cinco.

```

include/deriva/encaminhamento.hpp:47 |----- encaminhamento perfeito-----
47 /// Fábrica **com** encaminhamento perfeito.
48 ///
49 /// `T&&` num parâmetro de template dedutível NÃO é referência a rvalue: é
50 /// referência universal, e ela deduz `T` como `X&` para lvalue e como `X` para
51 /// rvalue. O colapso de referências faz `X& &&` virar `X&`, e é por isso que a
52 /// mesma assinatura serve para os dois casos.
53 ///
54 /// `std::forward<T>(x)` devolve `x` na categoria de valor com que ele chegou.
55 /// Trocá-lo por `std::move(x)` moveria SEMPRE - inclusive de um lvalue que o
56 /// chamador ainda vai usar, e esse é o defeito clássico deste idioma.
57 template <class T, class... Args>
58 [[nodiscard]] std::unique_ptr<T> criar_encaminhando(Args&&... args) {
59     return std::make_unique<T>(std::forward<Args>(args)...);
60 }

```

T&& num parâmetro dedutível não é referência a rvalue: é universal, e o colapso de referências faz a mesma assinatura servir para lvalue e rvalue. Trocar forward por move moveria SEMPRE, inclusive de um lvalue que o chamador ainda vai usar.

```

testes/test_encaminhamento.cpp:31 |----- a origem segue utilizável-----
31 TEST_CASE("e o lvalue continua chegando como lvalue") {
32     como_chegou::limpar();
33     carga_marcada original("peca");
34     const auto p = criar_encaminhando<carga_marcada>(original);
35     REQUIRE(p->rotulo() == "peca");
36     REQUIRE(como_chegou::traco[1].find("COPIADA") == 0);
37     // A origem continua utilizavel, e é isso que `std::move` no lugar de
38     // `std::forward` teria estragado.
39     REQUIRE(original.rotulo() == "peca");
40 }

```

É este teste que `std::move` no lugar de `std::forward` quebraria, e a quebra seria silenciosa: o programa continuaria compilando.

Sobrecarga de operadores

O QUE ESTE CAPÍTULO ENTREGA

- ▶ Implementar sobrecarga de operadores aritméticos e de comparação.
- ▶ Sobrecarregar os operadores de stream << e >> para I/O idiomático.
- ▶ Seguir as convenções de membro vs. função livre para cada operador.
- ▶ Escrever o par const e não-const de operator[], e dizer o que cada um promete.
- ▶ Evitar armadilhas comuns na sobrecarga de operadores.

§15.1 Por que Sobrecarregar Operadores?

Sobrecarga de operadores permite que tipos definidos pelo programador se comportem como tipos built-in. Em vez de `a.somar(b)`, escreve-se `a + b`. O código fica mais natural e consistente com a biblioteca padrão.

O critério que autoriza sobrecarregar é mais estreito que “fica mais bonito”, e ele é este: **o operador só se sobrecarrega quando o significado dele no domínio já é o significado dele na aritmética**. Somar dois `vetor2` é somar deslocamentos, e isso é adição em qualquer sentido da palavra; quem lê `pos + delta` não precisa de documentação. Já `mapa + mapa` não significa nada, e nenhuma quantidade de conveniência sintática justificaria inventar um significado.

O Deriva tem três lugares em que o critério passa, e é a v1.5 inteira.

`vetor2` pede aritmética, porque o sistema não faz outra coisa: toda movimentação é uma posição mais um deslocamento. Sem os operadores, `mover` escreve `vetor2{p.x + d.x, p.y + d.y}` em cada ponto de uso, e a tabela de comandos de `src/main.cpp` - que associa cada tecla a um `vetor2` de deslocamento - fica com a aritmética espalhada por fora do tipo que a define.

`mapa` pede indexação, porque `m[{3, 4}]` é o que se quer escrever quando se quer a célula naquela posição. O caminho anterior era `m.g().em({3, 4})`, que atravessa duas chamadas e expõe a grade interna a quem só queria uma célula.

E `mapa` pede saída em stream, porque ele já sabe se despejar em texto, e `std::cout << m` é a forma que o resto do C++ usa para isso.

§15.2 Regras de Ouro

1. Operadores que modificam o objeto (`+=`, `-=`, `*=`) devem ser membros - recebem acesso ao estado interno.
2. Operadores binários simétricos (`+`, `-`, `*`) devem ser funções livres - permitem conversão implícita em ambos os operandos.

3. Operadores de stream `<<` e `>>` devem ser funções livres - o primeiro argumento é o stream, não o objeto.
4. Preservar a semântica intuitiva: `a + b` não deve modificar `a` nem `b`.
5. Se `operator==` é definido, defina também `operator!=` (ou use `= default` em C++20).

A terceira regra tem uma razão que vale explicitar, porque ela é a única que não é convenção: `operator<<` **não pode** ser membro. O primeiro operando de `os << m` é o stream, e membro de uma classe recebe como primeiro operando um objeto daquela classe; escrever `mapa::operator<<` produziria um operador que se usa como `m << os`, o que ninguém escreve. A regra sai da linguagem, e não do gosto.

A quarta é a que mais se viola sem perceber. `a + b` devolve um objeto novo e não toca nos operandos, e a forma de garantir isso é a construção que o §15.3 desmonta: implementar `+=` como membro, e implementar `+` como função livre que recebe o primeiro operando **por valor**, aplica `+=` sobre a própria cópia, e devolve a cópia. Uma implementação, duas operações, e a promessa de não mexer no original vem de o parâmetro ser cópia.

A quinta precisa de uma correção de versão, e ela importa nesta disciplina. `bool operator==(const T&) const = default;` é C++20, e o alvo do laboratório é C++17. Em C++17, quem define `==` define `!=` à mão, e é o que `include/deriva/vetor2.hpp` faz - com o `!=` escrito em termos do `==`, para que exista uma definição só do que é igualdade. A forma com `= default` está aqui para que você a reconheça quando a encontrar em código de terceiros; na sua entrega ela não compila com `-std=c++17`.

§15.3 A Construção Composto-para-Simétrico, no Deriva

Os operadores de `vetor2` são o idioma da segunda e da quarta regras de ouro escritos por completo, e os cinco trechos no fim deste capítulo os trazem inteiros. Dois deles se leem como par - o composto é membro e binário é função livre -, porque é o par que ensina.

`operator+=` é **membro**, e é `constexpr` e `noexcept`: ele modifica o objeto da esquerda, então precisa de acesso ao estado interno, e o `return *this` do Cap. 7 §7.3 aparece aqui com uso - é ele que permite encadear. `operator+` é **função livre**, e o corpo dela tem uma linha: `return a += b;`, com `a` recebido **por valor**. Uma implementação da regra de soma, duas operações expostas, e a garantia de que `a + b` não toca nos operandos vem de `a` ser cópia. Trocar a ordem - escrever `+` primeiro e `+=` em termos dele - duplicaria a regra e daria duas chances de ela divergir.

Três coisas a levar dessa construção.

O **par de sobrecargas de multiplicação por escalar** existe porque `operator*` como membro cobriria só `v * 3`, e não `3 * v`: o primeiro operando de um membro é o objeto, e `int` não tem membro nenhum. `vetor2.hpp` declara as duas funções livres, `operator*(vetor2, int)` e `operator*(int, vetor2)`, e o trecho `simetria`, medida as afirma em duas linhas vizinhas de teste - `v * 3` e `3 * v` dão o mesmo `vetor2{6, 9}`, e é o argumento da segunda regra de ouro na forma medida.

O **retorno por valor** no `operator+` é onde o Cap. 14 encosta neste. O parâmetro é variável local nomeada, então devolvê-lo é o caso do NRVO, permitido e não obrigatório; se o compilador não elidir, ele usa o construtor de movimento. Para `vetor2`,

que são dois int, isso é indiferente; para um tipo que gerencia recurso, é o que separa o operador barato do operador caro.

E o `operator<< devolve std::ostream&`, e não `void`, porque é isso que permite `std::cout << a << " " << b`. Encadeamento de operador binário à esquerda só funciona se cada aplicação devolver algo que sirva de operando à seguinte, e é a mesma razão pela qual `operator=` e `operator+=` devolvem `T&`. O último trecho do capítulo é o do Deriva, e ele tem duas linhas: recebe o fluxo e o mapa, escreve `despejar()` no fluxo, devolve o fluxo. O lado esquerdo é o fluxo, e o fluxo não é nosso - é a razão da terceira regra de ouro, e é por isso que a função tem de ser livre.

§15.4 O Par `const` e `Não-const` de `operator[]`

Este é o operador que o Deriva mais precisa, e o par de sobrecargas dele é o padrão que o Cap. 7 §7.4 já apresentou com outro nome. `grade::em(vetor2) const` devolve `const celula&`; `grade::em(vetor2)` devolve `celula&`. As duas existem, e o compilador escolhe pela constância do objeto: numa `grade const` só a primeira é viável, e ela não deixa ninguém escrever na célula.

`mapa::operator[](vetor2)` é a mesma coisa com a sintaxe que se quer escrever, e o segundo trecho no fim deste capítulo traz as duas declarações em duas linhas, cada uma delegando ao `em()` correspondente da `grade`. O comentário de cima registra a razão de o par existir, e ela é de contrato e não de conveniência: uma sobrecarga só, devolvendo referência não-const, permitiria escrever através de um mapa constante; devolvendo referência const, impediria escrever em qualquer um. Nenhuma das duas isoladas serve, e é por isso que são duas.

A passagem de `em()` para `[]` acrescenta uma pergunta e uma armadilha.

A pergunta é sobre **verificação de limite**. `grade::em()` não verifica, e o comentário no cabeçalho diz por quê: quem chama já perguntou dentro(). Isso é uma pré-condição, e pré-condição é contrato: ela vale se estiver escrita e se quem chama a cumprir. O `operator[]` da biblioteca padrão adota a mesma política em `std::vector`, e é por isso que `v[i]` fora do limite é comportamento indefinido enquanto `v.at(i)` lança. A decisão do Deriva é seguir a biblioteca padrão: `m[p]` não verifica e `m.em_verificado(p)` lança, de forma que quem quer a garantia a pede pelo nome e quem já verificou não paga por ela duas vezes.

A armadilha é sobre **encapsulamento**. `operator[]` não-const devolve referência não-const a um membro interno, e isso é abrir o estado da classe para escrita direta - exatamente o que o item 3 da rubrica do Cap. 4 pergunta na terceira linha. Aqui está autorizado, e vale dizer por que: `celula` é agregado sem invariante nenhuma, como o Cap. 7 §7.2 argumentou, então não há nada que a escrita direta possa violar. Se a `celula` ganhar uma invariante - energia que não pode ser negativa, por exemplo -, o `operator[]` não-const passa a ser um furo, e a saída deixa de ser referência e passa a ser função que valida.

Uma nota de sintaxe, para o exercício não travar: em C++17, `operator[]` recebe **um** argumento, e é por isso que a chave dupla de `m[{3, 4}]` é necessária - o argumento é um `vetor2` construído na chamada. Índice de dois argumentos, `m[3, 4]`, é C++23.

§15.5 operator<< e a Saída que o Replay Compara

mapa já sabe se despejar: `despejar()` devolve uma `std::string` com o nome, o ponto de entrada e a grade, linha por linha, e o comentário no cabeçalho registra o requisito - mesma entrada, mesma saída, byte a byte. O `operator<<` deste capítulo é uma função livre de duas linhas escrita sobre ela: recebe o stream e o mapa, escreve `despejar()` no stream, devolve o stream.

Duas decisões que essa forma toma, e as duas importam ao Cap. 16.

A primeira é que **a formatação vive num lugar só**. Se o `operator<<` formatasse por conta própria, passariam a existir duas descrições textuais do mapa - a de `despejar()` e a dele -, e elas divergiriam na primeira mudança de formato. O portão de replay compara byte a byte contra `esperado.txt`; duas formatações significam que a saída que o teste compara não é necessariamente a que o jogo imprime.

A segunda é que o `operator<<` **não** consulta nada além do objeto. Nenhuma configuração global, nenhuma localidade, nenhuma hora do sistema. Saída que depende de estado externo é saída não reproduzível, e o replay do Cap. 16 é precisamente a exigência de que não haja nada disso. É o mesmo motivo pelo qual o sorteio do Deriva é um gerador congruente com semente explícita, e não `std::random_device`.

Sobre `operator>>`, que a caixa de objetivos menciona: o Deriva **não** o define, e a decisão é deliberada. A leitura de um mapa a partir de texto é `mapa::de_texto`, que devolve `std::optional<mapa>` porque linha malformada é ausência de resultado e não exceção, como o Cap. 9 e o Cap. 20 tratam. `operator>>` teria de sinalizar a falha pelo estado do stream, que é um canal mais pobre e mais fácil de ignorar. Simetria de sintaxe não é razão suficiente para trocar um `optional` por um bit de erro.

▲ ATENÇÃO

Alguns operadores não se sobrecarregam, ainda que a linguagem permita. `&&` e `||` sobrecarregados perdem a avaliação em curto-circuito, porque passam a ser chamadas de função e função avalia todos os argumentos; o operador vírgula perde a ordem de avaliação garantida. E `operator&`, sobrecarregado, quebra todo código genérico que tome o endereço do objeto.

A regra que cobre os três: se sobrecarregar o operador muda a **regra de avaliação** e não apenas o que ele calcula, não sobrecarregue.

► DERIVA

A v1.5 é a versão em que o código do jogo encurta sem perder nada, e o teste é a diferença de leitura:

- antes: `deriva::celula& c = m.g().em({s.ate(g.largura()), s.ate(g.altura())});`
- depois: `deriva::celula& c = m[{s.ate(g.largura()), s.ate(g.altura())}];`

A segunda linha não expõe a grade interna a quem só queria uma célula, e é essa a razão de projeto - a economia de caracteres é o efeito colateral.

Exercícios Propostos

1. Acrescente a `vetor2`: `operator+=` e `operator-=` como membros; `operator+` e `operator-` como funções livres implementadas sobre os compostos; e `operator*` por escalar nos dois lados, $v * 2$ e $2 * v$. Marque `constexpr`, `noexcept` e `[[nodiscard]]` onde couber e confirme, com um `static_assert`, que a soma é avaliada em tempo de compilação.
2. Reescreva a tabela de comandos de `src/main.cpp` usando os seus operadores: a posição nova passa a ser `pos + delta`. Rode `make verifica` e confirme que o `replay` continua idêntico byte a byte. Se ele mudou, o seu operador não faz a mesma aritmética que a versão anterior fazia, e o `diff` diz onde.
3. Implemente `mapa::operator[] (vetor2)` nas duas versões, `const` e `não-const`, mais um `em_verificado` que lança `std::out_of_range`. Escreva testes que provem três coisas: que a versão `const` é a escolhida num `const mapa&`; que a `não-const` permite escrever; e que o código que tenta escrever através de um `const mapa&` **não compila**. O terceiro é um teste de compilação, e vale escrever como comentário com a mensagem do compilador.
4. Por que `operator+=` deve devolver `T&`, e não `void` nem `T`? Escreva um uso que quebre com cada uma das duas alternativas erradas, e ligue-o ao `return *this` do Cap. 7 §7.3.
5. Implemente `operator<<` para `mapa` como função livre sobre `despejar()`. Depois escreva uma segunda versão que formate por conta própria, de um jeito ligeiramente diferente, e rode `make verifica`. Explique qual dos quatro portões falhou, e por que duas descrições textuais do mesmo objeto são um defeito e não uma conveniência.
6. Escreva o `operator==` de `vetor2` na forma de C++20, `bool operator==(const vetor2&) const = default;`, e compile-o com `-std=c++17`. Leia o erro. Depois escreva a versão de C++17, com `==` e `!=` à mão, e explique por que o `!=` deve ser escrito em termos do `==` e não de forma independente. Compare com o que `include/deriva/vetor2.hpp` faz.
7. Dê à `celula` uma invariante - energia nunca negativa - e mostre que o `operator[]` `não-const` passa a ser um furo nela. Proponha duas saídas, uma que troca a referência por função que valida e outra que mantém a referência e move a invariante para outro lugar, e diga qual delas você escolheria no Deriva e por quê.
8. Pesquise o operador de comparação de três vias de C++20, `<=>`. Como ele reduz a implementação de todos os operadores de comparação? Implemente-o para `vetor2` num arquivo compilado com `-std=c++20`, à parte do Deriva, e diga por que ele não entra na entrega desta disciplina.

O código, extraído do Deriva

Todo trecho abaixo vem de `exemplos/deriva/`, que compila com `-std=c++17 -Wall -Wextra -Wpedantic` sem um aviso e passa `make verifica`. Nenhum foi digitado neste texto.

```

| include/deriva/vetor2.hpp:55 |----- binário é função livre-----
55 // Funções livres: simétricas por construção, e é isso que as torna a forma
56 // idiomática. Um `operator+` membro aceitaria `a + b` e recusaria conversões
57 // do lado esquerdo.
58 [[nodiscard]] constexpr vetor2 operator+(vetor2 a, const vetor2& b) noexcept {
59     return a += b;
60 }

```

O composto é membro porque modifica o objeto da esquerda; o binário é livre porque trata os dois lados igual. Escrever `+` em termos de `+=` é a forma que não duplica a regra.

```

| include/deriva/mapa.hpp:79 |----- o par que existe por uma razão-----
79 /// Render determinístico. Mesma entrada, mesma saída, byte a byte - é o que
80 /// torna o replay da Aula 16 possível.
81 /// v1.5 · Aula 15. O par const / não-const existe porque `mapa` const tem
82 /// de deixar LER a célula e não deixar escrever nela. Uma sobrecarga só,
83 /// devolvendo referência não-const, permitiria escrever através de um mapa
84 /// constante; devolvendo referência const, impediria escrever em qualquer um.
85 [[nodiscard]] const celula& operator[](vetor2 p) const { return grade_.em(p); }
86 [[nodiscard]] celula& operator[](vetor2 p) { return grade_.em(p); }

```

Uma sobrecarga só, devolvendo referência não-const, permitiria escrever através de um mapa constante; devolvendo referência const, impediria escrever em qualquer um.

```

| testes/test_operadores.cpp:12 |----- simetria, medida-----
12 TEST_CASE("os operadores de vetor2 sao simetricos e constexpr") {
13     REQUIRE(vetor2{1, 2} + vetor2{3, 4} == vetor2{4, 6});
14     REQUIRE(vetor2{5, 5} - vetor2{1, 2} == vetor2{4, 3});
15     REQUIRE(vetor2{2, 3} * 3 == vetor2{6, 9});
16     REQUIRE(3 * vetor2{2, 3} == vetor2{6, 9});    // funcao livre: aceita os dois lados
17
18     vetor2 v{1, 1};
19     v += {2, 3};
20     REQUIRE(v == vetor2{3, 4});
21     v -= {1, 1};
22     REQUIRE(v == vetor2{2, 3});
23 }

```

`3 * v` e `v * 3` funcionam os dois, e é isso que função livre compra. Membro aceitaria só um dos lados.

```

26 // v1.5 · Aula 15. Os compostos são MEMBROS porque modificam o objeto da
27 // esquerda; os binários são funções LIVRES, logo abaixo, porque tratam os
28 // dois lados igual. Escrever '+' em termos de '+=' é a forma que não
29 // duplica a regra.
30 constexpr vetor2& operator+=(const vetor2& o) noexcept {
31     x += o.x;
32     y += o.y;
33     return *this;
34 }

```

Ele modifica o objeto da esquerda, e por isso é membro. O binário livre é escrito em termos dele, e assim a regra existe num lugar só.

```
src/mapa.cpp:138 | _____ `operator<<` é função livre
138 std::ostream& operator<<(std::ostream& os, const mapa& m) {
139     return os << m.despejar();
140 }
```

O lado esquerdo é o fluxo, e o fluxo não é nosso. Duas linhas sobre despejar(), devolvendo a referência para encadear.

Testes com Catch2 e replay determinístico

O QUE ESTE CAPÍTULO ENTREGA

- ▶ Configurar Catch2 em um projeto CMake.
- ▶ Escrever casos de teste com TEST_CASE e REQUIRE/CHECK.
- ▶ Organizar testes com SECTION e tags.
- ▶ Testar comportamentos de exceção com REQUIRE_THROWS.
- ▶ Escrever o teste como especificação executável, e distingui-lo do teste como caçador de defeito.
- ▶ Montar um replay determinístico: semente fixa, roteiro gravado, despejo idêntico byte a byte.

§16.1 O Teste como Especificação Executável

O framework adotado nesta disciplina é o Catch2, integrado por FetchContent no CMake do Cap. 2.

O papel do teste, neste material, é mais estreito e mais exigente do que “achar bug”. Achar defeito é o que ele faz por acidente; o que ele existe para fazer é **ser a descrição executável do comportamento que se decidiu ter**. A diferença aparece na hora de escrever: teste que caça defeito se escreve depois, olhando o código, e acaba afirmando o que o código faz; teste como especificação se escreve antes ou junto, olhando o requisito, e afirma o que o código **deve** fazer. O segundo pega o defeito de amanhã; o primeiro documenta o defeito de hoje como se fosse a regra.

A Unidade I já produziu três exemplos disso, e vale relê-los com esse critério.

testes/test_celula.cpp e os static_assert de celula.hpp afirmam um leiaute de memória que **se decidiu** ter, por uma razão medida em bytes. Nenhum defeito motivou aquelas quatro linhas; elas existem para que a decisão não se desfaça sem que alguém perceba.

testes/test_mapa.cpp afirma que o contador de instâncias vivas fecha em zero, e afirma também quantas construções um mapa custa. A segunda afirmação virou a melhor lição de teste que este material tem, e não pela razão prevista: esperava-se que ela caísse quando o Cap. 14 desse a mapa as operações de movimento, e ela **não caiu**. de_texto custa duas construções com o movimento e duas sem, porque o contador conta objetos e dois objetos nascem nos dois casos. O teste antigo prometia no título o que a asserção dele não verificava, e o caso que está no repositório hoje substituiu aquele, com o comentário registrando o que foi medido. Teste que afirma um número tem de afirmar o número que interessa, e descobrir qual é ele é metade do trabalho.

testes/test_ciclo_de_vida.cpp afirma a ordem exata em que os destrutores rodam quando uma exceção desenrola a pilha. Não há como escrever aquele teste

duas construções

construções em de_texto

aferido por

./build/testes "mapa"

testes/test_mapa.cpp

depois: ele é a especificação de uma garantia da linguagem, escrita para que o estudante a leia como texto em vez de acreditar nela.

Hoje o portão do Deriva roda **188 testes** verdes, e é a segunda das quatro condições de `make verifica`.

§16.2 Configurando Catch2 com CMake

A declaração inteira está extraída no fim deste capítulo, do `CMakeLists.txt` do Deriva: `FetchContent_Declare(Catch2 ...)` atrás de `if(DERIVA_TESTES)`, a lista dos arquivos de teste do alvo, o `find_package(Threads)` que o teste de concorrência do Cap. 22 exige, e o `catch_discover_tests` no fim. Três coisas a fixar sobre ela, porque as três são decisões de reprodutibilidade e não de conveniência.

A dependência entra marcada **SYSTEM**. É a diferença que o Cap. 2 §2.3 argumentou: aviso emitido dentro do Catch2 não conta no portão de zero aviso, porque aquele código não é do estudante e ele não pode consertá-lo. Sem essa palavra, o portão mais duro do material passaria a depender da higiene de um repositório de terceiros.

A **tag é fixa**. `GIT_TAG v3.5.2` nomeia uma versão, e não um branch. Apontar para `devel` significa que o build de hoje e o de amanhã compilam código diferente, e um portão que muda sozinho não é portão. A mesma regra vale para a FTXUI do Cap. 2, fixada em `v5.0.0` pela razão que o comentário no `CMakeLists.txt` registra: as versões seguintes podem elevar o padrão exigido e derrubar o alvo C++17 da disciplina.

O `catch_discover_tests` **registra cada caso no CTest**, um por um, em vez de registrar o executável inteiro como um teste só. É o que faz `ctest --output-on-failure` dizer qual caso falhou sem que ninguém precise ler a saída completa, e é o que faz a contagem do portão ter significado: `188 testes verdes` são 188 casos, e não 188 execuções do mesmo binário.

Repare também na linha de `target_compile_options` do alvo de testes, que desliga `-Wnon-virtual-dtor` ali e só ali, com a razão escrita ao lado: o alvo de testes inclui o cabeçalho gerado por IA do Cap. 4, que carrega um destrutor não virtual de propósito. O arquivo com o defeito plantado tem alvo próprio pela mesma razão, e são essas duas as **únicas** supressões de aviso do Deriva: as duas são locais a um alvo, as duas trazem o motivo escrito ao lado, e nenhuma delas cobre a variante da caça ao bug 2, que continua sob o conjunto completo e é acusada por ele. É a forma como o §11.5 admite que uma supressão exista - com escopo de um arquivo e razão registrada -, e é o mesmo gesto que um projeto de verdade usa para esconder problema, com a diferença de que aqui o motivo está escrito.

§16.3 Primeiro Arquivo de Teste

O arquivo mais didático que o Deriva tem para esta seção é `testes/test_grade.cpp`, e ele está extraído no fim do capítulo. São dois `TEST_CASE` sobre um tipo que o estudante escreveu no Cap. 7, e neles cabem as quatro macros que o resto do semestre usa: `REQUIRE` para o que tem de valer, `REQUIRE_FALSE` para o que tem de não valer, e `REQUIRE_THROWS_AS` para a exceção que se espera, com o tipo dela nomeado. Lido de cima a baixo, o arquivo é a documentação de `grade` - e é executável.

Repare no que ele afirma: as **quatro bordas** de `dentro()`, e não uma amostra. `{0, 0}` e `{19, 9}` dentro, `{20, 9}` e `{-1, 0}` fora. Teste de fronteira que verifica só o meio do intervalo passa por construção e não prova nada, e o erro de um-a-mais mora exatamente nos dois extremos.

E repare no que ele **não** usa: `SECTION` não aparece em nenhum dos dois casos, e isso também é conteúdo. `SECTION` se paga quando há preâmbulo comum a repetir - cada seção roda numa execução própria do corpo do `TEST_CASE`, do começo, então a construção do objeto que vem antes das seções acontece uma vez por seção, limpa, e é o que substitui o `setUp` de outros frameworks e impede que uma seção contamine a seguinte. Aqui não há preâmbulo a repetir, então dois `TEST_CASE` dizem a mesma coisa com menos aparato. Quem espera que as seções rodem em sequência sobre o mesmo objeto escreve testes que passam na ordem em que foram escritos e falham quando alguém acrescenta uma seção no meio.

E `Catch::Approx` é obrigatório na comparação de ponto flutuante, por uma razão que o livro não trata em nenhum capítulo e que vale em uma linha: `0.1 + 0.2 == 0.3` é falso, porque nenhum dos três valores é representável em binário. O Deriva quase não usa ponto flutuante - a `grade` é de inteiros, de propósito, e o §16.5 explica que essa escolha é o que torna o replay possível.

— DICA —

Rode o portão a cada commit, e não a cada entrega. Um atalho no terminal resolve: `alias v="make -C exemplos/deriva verifica\"`, que roda as quatro condições e não só os testes. Rodar apenas `ctest` é meia verificação: a cópia rasa do Cap. 9 e o ciclo do Cap. 13 passam nos testes e falham na quarta condição.

— Sobre integração contínua, e por que ela não está neste livro

Vale dizer o que este livro decidiu não cobrir, porque a decisão é de projeto e o estudante vai perguntar. **Integração contínua fica fora**, e não porque não valha: porque o portão desta disciplina tem de rodar na máquina de quem escreve o código, com quatro condições e um comando, e é essa reprodutibilidade local que se cobra. Servidor que fica verde e que ninguém consegue reproduzir no próprio terminal é pior que servidor nenhum, porque desloca a autoridade sobre a correção para um lugar onde o estudante não pode olhar.

O que a integração contínua acrescenta, quando entra, é uma frase: o mesmo `make verifica`, na máquina de outro, a cada envio. Se você quiser montá-la no seu repositório, é um arquivo de fluxo de trabalho que instala compilador e CMake, configura o projeto e roda `make verifica` - e o critério de que ela está correta é o mesmo de sempre: o que ela roda tem de ser o portão inteiro, e não só o `ctest`. O Deriva não traz esse arquivo, e é por isso que este livro não o descreve linha por linha.

§16.4 Testando Exceção e Casos de Fronteira

Há duas maneiras de uma operação não entregar o que se pediu, e o Deriva as trata com tipos diferentes de propósito. O último trecho do capítulo é o lado do `optional: mapa::de_texto("")`, `mapa::de_texto` sobre um texto com fileiras de larguras diferentes, e `mapa::carregar` sobre um caminho que não existe. As três devolvem `std::optional` vazio, e **nenhuma** lança. O contraste está no trecho vizinho, de `test_grade.cpp`: `grade(0, 5)` e `grade(5, -1)` lançam `std::invalid_argument`.

O critério que separa os dois é o que o Cap. 20 vai fechar, e ele cabe numa linha: **ausência de resultado sai por optional; erro de programação sai por exceção**. Um arquivo de mapa mal formado é entrada do mundo, e o programa tem de continuar; uma grade de largura zero é chamada errada de quem escreveu o código, e continuar seria propagar o erro. Quem lança onde deveria devolver `optional` obriga todo chamador a escrever `try`; quem devolve `optional` onde deveria lançar deixa o defeito de programação passar como se fosse dado ruim.

Escrever o teste dessas fronteiras é mais curto do que descrevê-las, e é o que os dois trechos mostram: três linhas para as três ausências, duas para os dois erros. O que custa é decidir de que lado do critério cada caso fica, e essa decisão é da assinatura, não do teste.

Vale fechar a seção com o caso oposto, que é o do teste que passa e está errado. Ele tem uma linha, e ela aparece em todo material sobre movimento:

```
 REQUIRE(origem.size() == 0), escrito depois de um std::move.
```

A linha afirma uma coisa que o padrão não promete, pela razão que o Cap. 14 §14.4 mediu: depois do movimento, a origem está em estado válido mas **não-especificado**. Nesta `libstdc++` ela esvazia, então o `REQUIRE` passa, e é por isso que essa linha sobrevive em tanto lugar - ela confirma o folclore em vez de acusá-lo, e o dia em que ela falhar será numa outra implementação, num outro semestre, longe de quem a escreveu. `testes/test_move_string.cpp` é a forma correta de afirmar a mesma medida: ele diz “neste alvo a origem esvazia” no título do caso, e a distinção entre o que se mediu e o que se garante fica no texto do teste.

A instrução da entrega desta aula é apagar toda linha desse tipo do seu código, substituindo-a por uma verificação sobre o **destino**, que é o que interessava - e é o mesmo critério que o portão do LAB-08 cobra no Cap. 14. Guarde o caso, porque ele é o argumento do §16.5 na sua forma mais curta: teste que passa não é teste que está certo.

§16.5 O Replay Determinístico

Teste de unidade afirma o comportamento de uma peça. O que ele não afirma é o comportamento do **sistema inteiro**, e é justamente esse que uma refatoração ameaça: dividir uma classe grande em três, trocar um laço por um algoritmo, mover uma responsabilidade de lugar - nenhuma dessas mudanças quebra necessariamente um teste de unidade, e todas podem mudar o que o programa faz.

O replay é o instrumento que fecha esse buraco, e ele tem três peças e uma regra.

Semente fixa. Todo sorteio do Deriva passa por um gerador congruente linear com semente explícita, em `src/main.cpp`. Não é `std::mt19937`, e muito menos `std::random_device`: precisa ser reproduzível byte a byte em qualquer máquina,

e as duas alternativas não são - `random_device` por definição, e `mt19937` porque a sequência que ela produz para uma semente é a mesma, porém o que se faz com ela depende de distribuições cuja implementação o padrão não fixa. Aleatoriedade de verdade seria pior aqui, e essa é a frase que o comentário no código traz.

Roteiro gravado. `roteiro.txt` é a lista de comandos, um por linha, na ordem. O programa a lê com `--replay roteiro.txt` e a executa passo a passo, sem esperar teclado. É o que substitui a mão humana, e é o que faz a execução ser a mesma em toda máquina.

Despejo comparado. Ao fim de cada passo, o programa imprime o estado - `mapa:despejar()`, que é determinístico pela razão do Cap. 15 §15.5 -, e a saída inteira é comparada com `esperado.txt` por diff, byte a byte. É a terceira das quatro condições de `make verifica`.

E a regra, que é a metade do instrumento: **regravar `esperado.txt` é uma decisão**. O alvo que faz isso existe, e o comentário dele no Makefile diz o que ele significa - você está afirmando que a saída nova é a correta. Numa refatoração, isso é exatamente o que não se pode fazer, porque refatoração é, por definição, a mudança que não altera o comportamento observável. Quem regrava o `esperado` para “consertar” o portão apagou o único oráculo que tinha.

Os dois trechos que sustentam esta seção estão extraídos no fim do capítulo: o sorteio que não sorteia, e o alvo `replay` do Makefile.

— Por que este capítulo vem antes dos Caps. 24 e 25

Porque o `replay` é o oráculo deles, e não o contrário.

O Cap. 24 pede a refatoração do mundo como `god class` sob os princípios SOLID, e a lição de lá não é a lista de cinco letras: é que **refatoração correta é a que não muda a saída**. A afirmação só tem sentido se existir uma saída a comparar, e ela precisa existir antes da refatoração - gravada com a versão antiga, comparada contra a nova. O Cap. 25 faz o mesmo com os padrões de projeto.

A caça ao bug 3, na semana 13, é precisamente a refatoração que mudou a saída, e o instrumento que a acusa é este. Escrever o teste que trava a refatoração **antes** de refatorar é a ordem, e é a ordem que se cobra.

— DERIVA —

As quatro condições de `make verifica`, e o lugar de cada uma:

- **(1/4) warning** - zero aviso com `-Wall -Wextra -Wpedantic -Wconversion`, no núcleo e só nele (Cap. 2).
- **(2/4) ctest** - 188 testes verdes, um caso por registro (este capítulo).
- **(3/4) replay** - despejo idêntico byte a byte, semente 7 (este capítulo).
- **(4/4) contadores** - `vivos=0`, nenhum destrutor deixou de rodar (Cap. 7).

Nenhuma das quatro depende de sanitizer, e é deliberado: o laboratório não os tem. O lugar do ASan é o Cap. 2, como ferramenta, e não como critério de aceitação.

§16.6 O Que o Teste Não Faz

Três limites, porque instrumento sem limite declarado vira crença.

Teste não prova ausência de defeito. Ele prova que os casos escritos passam. A caça ao bug 1 do Cap. 9 é a demonstração: a grade com cópia rasa passa em todo teste que verifique valores, porque os valores estão certos - o que está errado é a posse, e nenhum REQUIRE sobre conteúdo de célula a alcança.

Teste que espelha a implementação não vale nada. Escrito depois, olhando o código, ele afirma o que o código faz, inclusive o que ele faz de errado. O caso do §16.4 é esse, e o pior é que ele dá confiança: o portão fica verde e a decisão errada fica travada.

Replay idêntico não quer dizer correto. Ele quer dizer *igual ao de antes*. Se a versão antiga já estava errada, o replay preserva o erro com fidelidade byte a byte. Ele é oráculo de invariância, e não de correção, e é por isso que ele complementa os testes de unidade em vez de substituí-los.

Exercícios Propostos

1. Configure o Catch2 por FetchContent no seu Deriva, com SYSTEM e a tag fixa, e escreva testes/test_vetor2.cpp do zero: construção padrão na origem, igualdade, desigualdade, e os operadores do Cap. 15. Confirme que ctest registra um teste por caso, e não um por executável.
2. Escreva, para a grade, os testes de fronteira: as quatro bordas em dentro(), dimensão zero e dimensão negativa lançando, e o par de sobrecargas de em(). Depois conte quantos casos você escreveu e compare com quantos testes/test_grade.cpp tem.
3. Aplique TDD a uma peça nova: mapa::caminho_livre(vetor2 de, vetor2 ate), que responde se há linha reta de piso entre duas posições. Escreva **primeiro** os testes - incluindo os casos degenerados de origem igual ao destino e de posição fora da grade -, e só depois a implementação. Registre quantos testes falharam antes de a implementação existir.
4. Monte o seu replay: acrescente --replay e --semente ao seu programa, grave um roteiro.txt de dez comandos, e grave o esperado.txt da versão atual. Depois faça uma mudança que **deve** preservar a saída - extrair uma função, renomear uma variável - e confirme que o diff fica vazio. Este exercício e o 5 são o portão do **LAB-09**, e o critério dele é a ordem: escrever o teste que trava a refatoração **antes** de refatorar.
5. Agora faça uma mudança que **muda** a saída, de propósito, e sem alterar teste de unidade nenhum: troque a ordem em que as entidades são desenhadas. Todos os testes continuam verdes, e o replay falha. Leia o diff e explique o que ele mostra que o ctest não mostrou.
6. Troque o gerador congruente do Deriva por std::mt19937 com a mesma semente e rode o replay em duas máquinas diferentes, ou em dois compiladores. Registre o que aconteceu. Depois explique por que o material escolheu um gerador mais fraco de propósito.
7. Encontre, no seu código do semestre, um teste que espelhe a implementação em vez do requisito, e reescreva-o a partir do requisito. Diga o que a versão nova passou a permitir que a antiga proibia.
8. Rode make esperado no Deriva depois de uma mudança de comportamento, e depois leia o diff no Git antes de aceitar. Descreva, em quatro linhas, o procedi-

mento que você adotaria numa equipe para que regravar o esperado nunca acontecesse por acidente.

O código, extraído do Deriva

Todo trecho abaixo vem de `exemplos/deriva/`, que compila com `-std=c++17 -Wall -Wextra -Wpedantic` sem um aviso e passa `make verifica`. Nenhum foi digitado neste texto.

```
src/fov.cpp:7 |----- Bresenham em inteiros-----
7 std::vector<vetor2> linha(vetor2 a, vetor2 b) {
8   // Bresenham em inteiros. Nenhum ponto flutuante, nenhum arredondamento
9   // dependente de plataforma: a mesma entrada dá a mesma linha em qualquer
10  // máquina, e é essa propriedade que o replay da Aula 16 compra.
11  std::vector<vetor2> pontos;
12  int x = a.x, y = a.y;
13  const int dx = std::abs(b.x - a.x), dy = std::abs(b.y - a.y);
14  const int sx = a.x < b.x ? 1 : -1, sy = a.y < b.y ? 1 : -1;
15  int erro = dx - dy;
16  for (;;) {
17    pontos.push_back({x, y});
18    if (x == b.x && y == b.y) break;
19    const int e2 = 2 * erro;
20    if (e2 > -dy) { erro -= dy; x += sx; }
21    if (e2 < dx) { erro += dx; y += sy; }
22  }
```

Nenhum ponto flutuante, nenhum arredondamento dependente de plataforma. É essa propriedade que o replay compra, e é por isso que o campo de visão é o PRIMEIRO alvo dos testes e não o último: função pura se testa sem montar o mundo.

```

src/main.cpp:31 |----- o sorteio que não sorteia -----
31 /// Gerador congruente linear, com constantes de Numerical Recipes.
32 ///
33 /// Não é `std::mt19937` nem, muito menos, `std::random_device`: precisa ser
34 /// reproduzível byte a byte em qualquer máquina, porque é sobre isso que o
35 /// replay da Aula 16 se apoia. Aleatoriedade de verdade seria pior aqui.
36 class sorteio {
37 public:
38     explicit sorteio(unsigned semente) noexcept : estado_(semente) {}
39
40     [[nodiscard]] unsigned proximo() noexcept {
41         estado_ = estado_ * 1664525u + 1013904223u;
42         return estado_;
43     }
44     [[nodiscard]] int ate(int limite) noexcept {
45         return limite <= 0 ? 0 : static_cast<int>(proximo() % static_-
cast<unsigned>(limite));
46     }
47
48 private:
49     unsigned estado_;
50 };

```

Não é `std::mt19937` nem `std::random_device`: precisa ser reproduzível byte a byte em qualquer máquina, porque é sobre isso que o replay se apoia. Aleatoriedade de verdade seria pior aqui.

```

Makefile:39 |----- o portão de replay -----
39 # (3) replay: despejo idêntico byte a byte. É o oráculo das Aulas 16, 24 e 25.
40 replay:
41     @./$(BUILD)/deriva --replay roteiro.txt --semente $(SEMENTE) > $(BUILD)/replay.out
42     @diff -u esperado.txt $(BUILD)/replay.out > $(BUILD)/replay.diff || \
43
44     { cat $(BUILD)/replay.diff; echo "verifica: FALHA (3/4 replay) - a saída mudou"; exit 1; }
45     @echo " (3/4) replay      OK idêntico byte a byte (semente $(SEMENTE))"

```

Semente fixa, roteiro gravado, diff contra o esperado. Regravar o esperado é uma DECISÃO - numa refatoração, é justamente o que não se pode fazer.

```

CMakeLists.txt:142 | Catch2 como SYSTEM, com tag fixa
142 FetchContent_Declare(Catch2
143   GIT_REPOSITORY https://github.com/catchorg/Catch2.git
144   GIT_TAG v3.5.2
145   GIT_SHALLOW TRUE
146   SYSTEM
147 )
148 FetchContent_MakeAvailable(Catch2)
149
150 enable_testing()
151 add_executable(testes
152   testes/test_vetor2.cpp
153   testes/test_celula.cpp
154   testes/test_grade.cpp
155   testes/test_mapa.cpp
156   testes/test_ciclo_de_vida.cpp
157   testes/test_leiaute.cpp
158   testes/test_posse.cpp
159   testes/test_string_view.cpp
160   testes/test_move_string.cpp
161   testes/test_corrida.cpp
162   testes/test_entidade.cpp
163   testes/test_mundo.cpp
164   testes/test_estacao.cpp
165   testes/test_operadores.cpp
166   testes/test_fov.cpp
167   testes/test_diamante.cpp
168   testes/test_inspetor.cpp
169   testes/test_generico.cpp
170   testes/test_erro.cpp
171   testes/test_inventario.cpp
172   testes/test_partida.cpp
173   testes/test_concorrencia.cpp
174   testes/test_padroes.cpp
175   testes/test_comparativo.cpp
176   testes/test_revisao_ia.cpp
177   testes/test_tipos.cpp
178   testes/test_uml.cpp
179   testes/test_encaminhamento.cpp
180   testes/test_padroes_extra.cpp
181   testes/test_solid.cpp
182 )
183 find_package(Threads REQUIRED)
184 target_link_libraries(testes PRIVATE deriva_nucleo deriva_
revisao Catch2::Catch2WithMain Threads::Threads)
185 # o teste do código gerado inclui o cabeçalho com o defeito plantado
186 target_compile_options(testes PRIVATE -Wall -Wextra -Wpedantic -Wno-non-virtual-dtor)
187
188 include(CTest)
189 include(Catch)
190 catch_discover_tests(testes)

```

SYSTEM para que o aviso da dependência não conte no portão, e GIT_TAG fixo para que a suíte não mude de comportamento sem que ninguém tenha mexido nela.

```
┌───| testes/test_grade.cpp:11 |────────────────────────── o teste como especificação ───────────────────┐
11 TEST_CASE("grade conhece seus limites") {
12     const grade g(20, 10);
13     REQUIRE(g.largura() == 20);
14     REQUIRE(g.altura() == 10);
15     REQUIRE(g.dentro({0, 0}));
16     REQUIRE(g.dentro({19, 9}));
17     REQUIRE_FALSE(g.dentro({20, 9}));
18     REQUIRE_FALSE(g.dentro({-1, 0}));
19 }
20
21 TEST_CASE("dimensao nao-positiva e erro de programacao") {
22     REQUIRE_THROWS_AS(grade(0, 5), std::invalid_argument);
23     REQUIRE_THROWS_AS(grade(5, -1), std::invalid_argument);

```

Ele não procura defeito: ele diz o que a classe promete. Lido de cima a baixo, é a documentação de grade - e é executável.

```
┌───| testes/test_mapa.cpp:44 |────────────────────────── as três fronteiras ───────────────────┐
44 TEST_CASE("ausencia de resultado nao e excecao") {
45     REQUIRE_FALSE(mapa::de_texto("", "vazio").has_value());
46     REQUIRE_FALSE(mapa::de_texto("###\n###\n", "torto").has_value());
47     REQUIRE_FALSE(mapa::carregar("nao/existe/em/lugar/algum.txt").has_value());
48 }

```

Vazio, torto e ausente: os três devolvem optional vazio. Ausência de resultado não é exceção, e o tipo diz isso.

```

┌───┤ testes/test_fov.cpp:45 ┤─────────────────── `SECTION` refaz o arranjo em cada ramo ───────────┐
45 TEST_CASE("a parede e vista e bloqueia o que esta atras") {
46     const auto m = mapa::de_texto(kSala, "sala");
47     const auto v = visiveis(*m, {1, 2}, 6);
48
49     REQUIRE(v.count(vetor2{4, 2}) == 1);    // a coluna: vista, porque bloqueia
50     REQUIRE(v.count(vetor2{5, 2}) == 0);    // logo atras dela: sombra
51
52     SECTION("mas se ve em volta da coluna, por cima e por baixo") {
53         REQUIRE(v.count(vetor2{4, 1}) == 1);
54         REQUIRE(v.count(vetor2{4, 3}) == 1);
55         REQUIRE(v.count(vetor2{6, 1}) == 1);    // na borda do raio, dist 6
56     }
57     SECTION("o raio e de Manhattan, e a borda e exata") {
58         REQUIRE(vetor2{1, 2}.manhattan({6, 1}) == 6);
59         REQUIRE(v.count(vetor2{6, 1}) == 1);
60         REQUIRE(vetor2{1, 2}.manhattan({7, 1}) == 7);
61         REQUIRE(v.count(vetor2{7, 1}) == 0);    // um passo alem, e ja nao se ve

```

O corpo do TEST_CASE roda uma vez POR seção, e não uma vez para as duas: cada ramo entra num mapa recém-montado. É o que torna as seções independentes, e é a propriedade que se perde ao guardar estado entre elas.

Herança múltipla e o diamante

O QUE ESTE CAPÍTULO ENTREGA

- ▶ Usar herança múltipla de interfaces sem ambiguidades.
- ▶ Compreender o problema do diamante e suas consequências.
- ▶ Resolver o diamante com herança virtual, e dizer o que ela custa.
- ▶ Determinar a ordem de construção de uma hierarquia com base virtual.
- ▶ Contrastar com a solução de Java/C# (interfaces únicas, herança simples de classes).

§17.1 Herança Múltipla: Capacidades e Riscos

C++ permite que uma classe herde de várias classes base ao mesmo tempo. O caso seguro é herdar de várias classes puramente abstratas, sem estado, que é o que Java e C# chamam de “implementar várias interfaces”.

A pergunta que separa o caso seguro do perigoso é sobre **estado**, e não sobre número de bases. Herdar de cinco interfaces sem dado nenhum não produz problema algum: não há campo a duplicar, não há construtor de base a chamar duas vezes, e a única ambiguidade possível é de nome de função, que se resolve qualificando. Herdar de duas classes que carregam dado, e que descendem de uma terceira que também carrega dado, é onde o diamante mora.

No Deriva, o caso seguro é o que a v1.7 precisa de fato. A sonda_reparadora é uma sonda que sabe consertar o que encontra, e “saber consertar” é uma capacidade sem estado próprio: duas funções virtuais puras, `reparar(celula&)` e `reparos_feitos()`, e nenhum campo. Declarada assim, `i_reparavel` é interface, `sonda_reparadora` : `public sonda`, `public i_reparavel` compila sem uma ambiguidade, e não há diamante.

O trecho no fim deste capítulo é essa interface, extraída do arquivo que compila, e ela tem três linhas de corpo: o destrutor virtual, `reparar(celula&)` pura, e `reparos_feitos()` pura. Nenhum membro de dado, nenhum construtor. Quem a implementa é a sonda_reparadora, declarada no mesmo cabeçalho como `class sonda_reparadora final : public sonda, public i_reparavel`, e o teste `testes/test_diamante.cpp` confirma as três identidades do mesmo objeto: ele é entidade, é sonda, e é `i_reparavel`, com os três ponteiros de base apontando para ele ao mesmo tempo.

Repare no nome. A interface se chama `i_reparavel`, e não `reparadora`, porque o que ela declara é a capacidade e não o papel: `sonda_reparadora` é uma sonda que também é reparável no sentido de saber reparar, e o prefixo distingue a interface da classe concreta na leitura de uma assinatura. O §17.2 e o §17.3 abaixo continuam falando de uma reparadora que deriva de entidade, e é de propósito: aquela é a versão que **não** foi escrita, e existe no texto para ser medida.

Duas notas sobre o caso seguro, e a segunda liga ao Cap. 11.

Cada interface pura precisa do **seu próprio destrutor virtual**, e a razão é a do Cap. 11 §11.5 aplicada duas vezes: um `sonda_reparadora` pode ser destruído através de `sonda*` ou através de `i_reparavel*`, e as duas bases têm de estar preparadas. Base polimórfica sem destrutor virtual é defeito ainda quando ela não tem dado nenhum a liberar, porque o que ela deixa de liberar é a parte derivada.

E um objeto com duas bases polimórficas tem **dois** ponteiros de `vtable`, e não um. Converter `sonda_reparadora*` para `i_reparavel*` deixa de ser gratuito: o endereço muda, porque o subobjeto `i_reparavel` não começa no mesmo byte que o objeto completo. É a primeira vez neste livro que uma conversão de ponteiro para base ajusta o endereço, e é a razão pela qual `reinterpret_cast` entre ponteiros de uma hierarquia com múltiplas bases é errado e `static_cast` é certo.

§17.2 O Problema do Diamante

O problema do diamante ocorre quando uma classe herda indiretamente de outra classe por duas rotas diferentes. Isso cria duas cópias dos membros da classe ancestral, causando ambiguidade e desperdício de memória.

Duas frases sobre o desenho que **não** foi escrito, porque é ele que o estudante vai tentar. Se `reparadora` precisasse da posição para saber o que consertar, a saída natural e errada seria fazê-la derivar de entidade; a partir daí, `sonda_reparadora` herdaria entidade duas vezes, uma por cada ramo. O objeto passaria a ter **duas** posições, duas `vtables` da base, e dois contadores de instâncias vivas incrementados na construção - vivos valeria 2 para um objeto, e seria a primeira vez que o instrumento do Cap. 7 mentiria por causa da forma da hierarquia e não por causa de um destrutor. E `sr.pos()` deixaria de compilar, com uma mensagem sobre acesso ambíguo a membro de base; a ambiguidade se resolve qualificando - `sr.sonda::pos()` -, e o fato de a solução ser essa já é o diagnóstico: se é preciso dizer *qual* posição, então há duas, e um objeto com duas posições não é um objeto, são dois colados.

O Deriva recusou esse desenho, e o §17.4 diz por quê. O que o material precisava, então, era de um diamante que existisse de verdade para ser medido, e ele está em `include/deriva/diamante.hpp`, fora do jogo, no mesmo molde de `include/deriva/leiaute.hpp` - cabeçalho que existe para que os números sejam os que o `g++` produz, e não para que a estação funcione. O trecho está extraído no fim deste capítulo, com as três formas lado a lado.

O quarteto é o mínimo que produz o problema, e a peça que o produz é o **estado na base do meio**: `nucleo` tem um `int leituras` e um destrutor virtual. `movel` e `sensor` herdam dele por herança comum, `patrulha_duplicada` herda dos dois, e a folha passa a ter **dois** campos leituras, em endereços diferentes.

E o defeito não é o desperdício, é a **ambiguidade de valor**. O trecho `dois leituras`, e nenhum é o certo é a prova em quatro linhas: escreve-se 7 pelo ramo `movel`, escreve-se 9 pelo ramo `sensor`, e as duas leituras devolvem o que cada uma escreveu. A pergunta “quantas leituras esta patrulha tem?” não tem resposta, e um objeto sobre o qual essa pergunta não tem resposta não é um objeto. Escrever por um ramo e ler pelo outro devolve lixo coerente, que é a pior categoria de lixo - ele passa em qualquer teste que não tenha sido escrito para pegá-lo.

DIAGRAMA UML

Diagrama UML do diamante medido: núcleo no topo, com o campo leituras; move1 e sensor herdam dele; patrulha_ duplicada herda dos dois. Desenhar duas vezes: à esquerda, sem herança virtual, mostrando os DOIS subobjetos núcleo dentro da folha, cada um com o seu leituras, e o total de 40 bytes; à direita, com virtual núcleo nos dois ramos do meio, mostrando UM só subobjeto, mais o ponteiro que cada ramo carrega para alcançá-lo, e o total de 48. É o quadro que o interativo Inspetor de objeto desta aula exibe, com os números de include/deriva/diamante.hpp.

§17.3 Resolvendo com Herança Virtual

Com herança virtual, a classe ancestral compartilhada tem exatamente uma instância em toda a hierarquia, não importa quantas rotas de herança existam.

A palavra virtual vai nas heranças **do meio**, e não na de baixo: struct move1_v : virtual nucleo e struct sensor_v : virtual nucleo, e a folha patrulha_unica : move1_v, sensor_v não escreve virtual em lugar nenhum. É onde ela tem de estar, e isso tem uma consequência que decide projeto: **quem decide se a base é virtual é a classe do meio, e não a classe que sofre o diamante**. Se move1 já existir sem virtual, o diamante não se resolve na folha; ele se resolve mudando move1, e recompilando tudo o que dependia dela. Herança virtual acrescentada depois é mudança que atravessa a hierarquia inteira, e é por isso que o par move1_v/sensor_v existe ao lado de move1/sensor em vez de substituí-los.

O trecho com base virtual, um campo é a resolução medida, e ela é medida por **identidade** e não por tamanho: escreve-se 7 pelo ramo move1_v e lê-se 7 pelo ramo sensor_v, e depois 9 na outra direção. Há um campo, e é o mesmo campo pelas duas rotas. O caso vizinho conta os subobjetos: dois na forma comum, um na virtual.

E aqui os números contrariam a intuição, e por isso estão em static_assert e não em prosa. patrulha_duplicada ocupa **40 bytes**; patrulha_unica, a de herança virtual, ocupa **48**; e patrulha_composta, que não tem diamante nenhum, ocupa **56**. A herança virtual é a **maior** das três, e não a menor: o ponteiro que cada ramo virtual carrega para alcançar a base compartilhada custa mais do que duplicar uma base de 16 bytes. Escrever que ela “economiza memória” seria mentir, e a primeira versão daquele cabeçalho tinha um static_assert invertido afirmando exatamente isso - o compilador o recusou, que é o motivo de o número morar ali e não aqui.

O que a herança virtual compra, então, não é tamanho: é **correção**. Um campo em vez de dois ambíguos, e a pergunta do §17.2 volta a ter resposta. E a forma composta, que é a maior de todas em bytes, continua sendo a recomendada pela mesma razão levada mais longe - ela compra a ausência de uma pergunta que não tem resposta boa, e paga em bytes por isso.

Três coisas mudam quando uma base é virtual.

A ordem de construção muda, e é o que aparece na prova. Bases virtuais são construídas **primeiro**, todas elas, antes de qualquer base não virtual, e na ordem em que aparecem no percurso em profundidade da hierarquia; só depois vêm as bases não virtuais e, por fim, os membros. A destruição é o espelho.

A responsabilidade de inicializar a base virtual passa para a classe mais derivada. A folha nomeia a base virtual na sua própria lista de inicialização, mesmo não herdando diretamente dela; e o argumento que a classe do meio passasse para a base é **ignorado** quando ela é subobjeto de uma folha. Isto é a regra que mais

40 bytes

sizeof(patrulha_duplicada)

aferido por

./build/testes "*heranca
virtual custa mais"

include/deriva/diamante.hpp

surpreende, porque o código do meio fica lá, escrito, e deixa de ter efeito conforme quem o herda. Se a folha esquecer de nomear a base virtual, o compilador exige um construtor padrão nela, e a mensagem não fala de herança virtual. As três formas de `diamante.hpp` são agregados, sem construtor escrito, justamente para que o `sizeof` meça o leiaute e nada mais; a ordem de construção é o exercício 5, e o comentário de `src/reparadora.cpp` a registra em prosa para quem quiser conferir antes de escrever.

O acesso à base virtual passa a custar uma indireção. O compilador não sabe, em tempo de compilação, a que distância do início do objeto o subobjeto virtual está, porque a distância depende da classe mais derivada; então ele guarda essa informação no objeto e a consulta. É por isso que `static_cast` de uma base virtual **para** a derivada é malformado, e o único caminho é `dynamic_cast`, que é o Cap. 18.

A resolução da **função** ambígua é outra coisa, e nada do que foi dito até aqui a resolve: herança virtual unifica o subobjeto, e duas funções de mesmo nome herdadas por rotas diferentes continuam ambíguas. A saída mais direta é a folha declarar a sua própria sobrescrita e, no corpo, chamar a versão que ela quer, com o nome qualificado - `move1_v::mover(...)`, por exemplo. A alternativa é `using move1_v::mover;` dentro da folha, que traz um dos nomes ao escopo e desfaz a ambiguidade sem escrever corpo nenhum; ela cabe quando a escolha é uma das duas versões inteiras, e não uma combinação delas.

▲ ATENÇÃO

Com herança virtual, a classe mais derivada é responsável por inicializar a base virtual, mesmo que não herde diretamente dela - e o argumento que a classe do meio passava para a base passa a ser ignorado.

Herança virtual acrescenta indireção no acesso à base e informação de leiaute no objeto. Use apenas quando o diamante realmente ocorrer, e prefira não fazê-lo ocorrer.

§17.4 Contraste com Java/C#, e o Que o Deriva Escolhe

Java e C# eliminam o problema proibindo herança múltipla de classes concretas. Em compensação, permitem implementar várias interfaces, equivalentes às classes abstratas puras de C++. O resultado é uma solução mais restrita e mais simples, e a maior parte dos usos práticos de herança múltipla em C++ envolve apenas interfaces puras, que não têm o problema do diamante.

A conclusão de projeto, para esta disciplina, é a que o §17.1 já anunciou e que vale escrever sem rodeio: **o diamante é objeto de estudo, e não recomendação.** A saída, quando ele aparece num desenho, quase nunca é herança virtual - é reconhecer que uma das duas heranças não era is-a.

No caso da sonda `reparadora`, o diamante nasceria de `reparadora` precisar da posição. E ela não precisa **herdar** a posição: ela precisa **receber** o que vai consertar. `virtual bool reparar(celula& c) = 0;` resolve, com a célula chegando por parâmetro, e a interface volta a não ter estado. Foi a pergunta do Cap. 10 §10.1 aplicada uma vez mais - is-a ou has-a -, e a resposta aqui é *nem uma nem outra*: é passagem de argumento. É por isso que a v1.7 do Deriva entrega `i_reparavel` como interface pura e **nenhum** diamante no jogo, e é por isso que o diamante deste capítulo mora num cabeçalho de medição, que é o lugar de um objeto de estudo que não se recomenda escrever.

Isto não torna a seção inútil. Herança virtual aparece em código de terceiros, aparece em hierarquias de exceção da própria biblioteca padrão, e aparece em `std::basic_iostream`, que herda de `basic_istream` e `basic_ostream`, ambas com base virtual `basic_ios` - o exemplo canônico, e um que o estudante usa em toda linha de `std::cout`. Saber ler é obrigatório; saber escrever é para o caso raro.

§17.5 Neste Encontro: a Caça ao Bug 2

A caça ao bug 2 do semestre acontece neste encontro, e o conteúdo dela é do Cap. 11: **o destrutor que não é virtual**. O protocolo e as cinco observações estão no §11.6, e o defeito e o mecanismo no §11.5.

A distância entre os dois é de três encontros, e ela muda o que o exercício mede: deixa de ser “você acabou de ver isto” e passa a ser “você reconhece isto duas semanas depois”, que é a pergunta mais honesta das duas. Revise o §11.5 antes, e chegue com o roteiro de observação na mão.

E há uma razão para a caça cair aqui e não lá, além do calendário: este capítulo acabou de acrescentar duas bases polimórficas ao mesmo objeto, e com elas a possibilidade de destruir um `sonda_reparadora` através de `sonda*` ou de `i_reparavel*`. O defeito do Cap. 11 duplica-se junto com a hierarquia, e a pergunta do item 5 da rubrica passa a ter duas respostas a conferir em vez de uma.

Exercícios Propostos

1. Implemente `i_reparavel` como interface pura, sem estado, e `sonda_reparadora` final: `public sonda, public i_reparavel`. Ponha `marca_de_vida` em cada classe e escreva a ordem do traço antes de rodar; compare com a ordem que o comentário de `src/reparadora.cpp` registra. Confirme que `vivos` fecha em zero e que destruir através de `i_reparavel*` funciona - e diga por que o destrutor virtual da interface é obrigatório mesmo ela não tendo dado nenhum.
2. Agora produza o diamante de propósito: faça a sua interface derivar de entidade, e assim deixar de ser interface. Tente chamar `sr.pos()` e leia a mensagem de ambiguidade. Depois imprima `vivos` depois de construir **um** `sonda_reparadora`, explique o valor, e diga o que isso acrescenta ao “o que o contador não acusa” do Cap. 7. Este é o desenho que a v1.7 recusou, e o exercício existe para que você veja o que ela recusou.
3. Resolva o diamante do exercício 2 com `public virtual entidade` nas duas classes do meio. Meça o `sizeof` do objeto nas três formas - diamante comum, diamante virtual e composição - e escreva o `static_assert` de cada uma **antes** de compilar. Depois compare com os três números de `include/deriva/diamante.hpp`, que são 40, 48 e 56, e responda: qual das três você previu como a maior, e por que errou?
4. Com a base virtual do exercício 3, faça `sonda` passar um argumento para o construtor de entidade e **não** nomeie entidade na lista de inicialização de `sonda_reparadora`. Leia o erro. Depois nomeie-a, passando outro valor, e mostre que o argumento de `sonda` foi ignorado. Escreva a regra em uma frase.
5. Ponha `marca_de_vida` nas quatro classes do diamante virtual e escreva a ordem completa de construção e destruição **antes** de rodar. Compare com a saída. Se errou, diga qual das três regras do §17.3 você não aplicou.

6. Tente `static_cast<sonda_reparadora*>` a partir de um ponteiro para a base virtual `entidade*`. Leia o erro, e explique-o em termos do que o compilador não sabe em tempo de compilação. Depois faça a mesma conversão com `dynamic_cast` e guarde o resultado para o Cap. 18.
7. Abra a declaração de `std::basic_istream` na sua biblioteca padrão e identifique o diamante: quem são as duas bases, qual é o ancestral comum, e onde está o virtual. Explique por que a biblioteca precisou disso, e por que o seu código provavelmente não precisa.
8. Pesquise o padrão *mixin* em C++, com herança múltipla de classes que têm implementação. Como um mixin difere do diamante? Escreva um mixin que acrescenta uma capacidade a uma entidade sem produzir ancestral comum, e diga o que ele tem em comum com o `contador_de_instancias<T>` que chega no Cap. 19.

O código, extraído do Deriva

Todo trecho abaixo vem de `exemplos/deriva/`, que compila com `-std=c++17 -Wall -Wextra -Wpedantic` sem um aviso e passa `make verifica`. Nenhum foi digitado neste texto.

```

└─ include/deriva/diamante.hpp:39 ─────────────────────────────────── os números do diamante ───────────────────────────────────
39 // Os números medidos em g++ 13.3 x86-64, e eles contrariam a intuição:
40 //
41 //     nucleo                16      vptr 8 + int 4 + padding 4
42 //     movel / sensor        16      a base cabe no mesmo alinhamento
43 //     patrulha_duplicada    40      DUAS bases de 16, mais rota e padding
44 //     patrulha_unica        48      UMA base, e mais um ponteiro por ramo virtual
45 //     patrulha_composta     56      nenhum diamante, e o maior de todos
46 //
47 // A herança virtual é a MAIOR das três, não a menor. O ponteiro para a base
48 // virtual que cada ramo carrega custa mais do que duplicar uma base de 16
49 // bytes. Escrever aqui que ela "economiza memória" seria mentir, e a primeira
50 // versão deste cabeçalho tinha um `static_assert` invertido afirmando
51 // exatamente isso - o compilador o recusou.
52 //
53 // O que a herança virtual compra não é tamanho: é **correção**. Na forma
54 // duplicada existem dois campos `leituras` com endereços diferentes, e nenhum
55 // dos dois é "o" valor; escrever por um ramo e ler pelo outro devolve lixo
56 // coerente. Na forma virtual existe um campo só. É por isso que se paga.
57 static_assert(sizeof(nucleo) == 16, "vptr 8 + int 4 + padding 4");
58 static_assert(sizeof(patrulha_duplicada) == 40, "duas bases de 16, mais rota");
59 static_assert(sizeof(patrulha_unica) == 48, "uma base, mais um ponteiro por ramo");
60 static_assert(sizeof(patrulha_unica) > sizeof(patrulha_duplicada),
61               "a indireção virtual custa MAIS bytes que a duplicação que evita");

```

Contraria a intuição, e por isso está medido: a herança virtual é a MAIOR das três formas, 48 bytes contra 40 da duplicada. O que ela compra não é tamanho, é correção - um campo em vez de dois ambíguos.

```

| include/deriva/reparadora.hpp:13 |----- o caso fácil -----|
13 /// Interface pura: nenhum dado, nenhum construtor, destrutor virtual.
14 ///
15 /// Interface pura é o **único** uso de herança múltipla que este material
16 /// recomenda sem ressalva. Ela não traz estado, então não há o que duplicar,
17 /// e o diamante que ela formaria é inofensivo.
18 class i_reparavel {
19 public:
20     virtual ~i_reparavel() = default;
21     [[nodiscard]] virtual bool reparar(celula& c) = 0;
22     [[nodiscard]] virtual int reparos_feitos() const = 0;
23 };

```

Interface pura é o único uso de herança múltipla que este material recomenda sem ressalva: nenhum dado, então não há o que duplicar, e o diamante que ela formaria é inofensivo.

```

| include/deriva/diamante.hpp:15 |----- as três formas, lado a lado -----|
15 /// A base com estado. Um `int` e uma função virtual.
16 struct nucleo {
17     virtual ~nucleo() = default;
18     int leituras = 0;
19 };
20
21 /// Herança comum nos dois ramos: o `nucleo` é DUPLICADO na folha.
22 struct movel : nucleo { int passos = 0; };
23 struct sensor : nucleo { int alcance = 0; };
24 struct patrulha_duplicada : movel, sensor { int rota = 0; };
25
26 /// Herança virtual: existe UM `nucleo` na folha, e o compilador paga por isso
27 /// com um ponteiro extra por ramo virtual.
28 struct movel_v : virtual nucleo { int passos = 0; };
29 struct sensor_v : virtual nucleo { int alcance = 0; };
30 struct patrulha_unica : movel_v, sensor_v { int rota = 0; };
31
32 /// A saída que o material recomenda: composição no lugar do segundo ramo.
33 /// Nenhum diamante, nenhuma ambiguidade, nenhuma pergunta sobre qual
34 /// `leituras` é o certo. É também a MAIOR das três em bytes (56), e continua
35 /// sendo a recomendada: o que se está comprando é a ausência de uma pergunta
36 /// que não tem resposta boa.
37 struct patrulha_composta : movel { sensor_v olho; int rota = 0; };

```

Duplicada, virtual e composta. Os tamanhos estão no fim do arquivo, afirmados, e contrariam a intuição.

```

| testes/test_diamante.cpp:29 |----- dois `leituras`, e nenhum é o certo-----
29 TEST_CASE("com heranca comum, os dois ramos escrevem em campos diferentes") {
30     patrulha_duplicada p;
31     static_cast<movel>&(p).leituras = 7;
32     static_cast<sensor>&(p).leituras = 9;
33     REQUIRE(static_cast<movel>&(p).leituras == 7);
34     REQUIRE(static_cast<sensor>&(p).leituras == 9);    // e este é o defeito
35 }

```

Escrever 7 por um ramo e 9 pelo outro, e ler os dois de volta. Não há resposta boa para “qual é o valor”, e é esse o defeito - não o tamanho.

```

| testes/test_diamante.cpp:37 |----- com base virtual, um campo-----
37 TEST_CASE("com heranca virtual, ha um campo so") {
38     patrulha_unica p;
39     static_cast<movel_v>&(p).leituras = 7;
40     REQUIRE(static_cast<sensor_v>&(p).leituras == 7);
41     static_cast<sensor_v>&(p).leituras = 9;
42     REQUIRE(static_cast<movel_v>&(p).leituras == 9);
43 }

```

Escrever por um ramo e ler pelo outro devolve o mesmo valor. É isto que se compra com a indireção, e não bytes - a forma virtual é a maior das três.

```

| src/reparadora.cpp:5 |----- a ordem das bases é a da declaração-----
5 sonda_reparadora::sonda_reparadora(vetor2 pos, int energia)
6     : sonda(pos, energia) {
7     // A ordem de construção é: entidade, sonda, i_reparavel, sonda_reparadora.
8     // Bases antes de membros, e bases na ordem de DECLARAÇÃO da lista de
9     // herança - não na ordem em que a lista de inicialização as menciona
10    // (Aula 17).
11    ++vivos;
12    ++criados;
13 }

```

Bases antes de membros, e bases na ordem em que a lista de HERANÇA as declara - não na ordem em que a lista de inicialização as menciona. Trocar a ordem na lista de inicialização não muda a ordem de execução, e o `-Wreorder`, que o `-Wall` do portão já liga, avisa quando as duas divergem.

```
┌───| testes/test_diamante.cpp:77 |────────────────── a destruição percorre o inverso exato ───────────┐
77 TEST_CASE("a ordem de construcao e destruicao da reparadora fecha em zero") {
78     zerar_entidades();
79     {
80         const sonda_reparadora r({1, 1});
81         REQUIRE(sonda_reparadora::vivos == 1);
82         REQUIRE(sonda::vivos == 1);    // ela TAMBEM e uma sonda, e conta nos dois
83     }
84     REQUIRE(sonda_reparadora::vivos == 0);
85     REQUIRE(sonda::vivos == 0);
└────────────────────────────────────────────────────────────────────────────────────────────────────────┘
```

Uma sonda_reparadora conta nos DOIS contadores, porque ela também é uma sonda, e os dois voltam a zero no fim do escopo. É a prova de que nenhum destrutor da cadeia deixou de rodar.

Polimorfismo dinâmico e RTTI

O QUE ESTE CAPÍTULO ENTREGA

- ▶ Compreender polimorfismo dinâmico via vtable.
- ▶ Usar `dynamic_cast` para downcasting seguro.
- ▶ Entender RTTI (`typeid`, `type_info`) e seus limites.
- ▶ Identificar quando `dynamic_cast` indica problema de design, e reconhecer o caso em que ele é a resposta certa.

§18.1 Polimorfismo Dinâmico em Ação

Polimorfismo dinâmico permite processar objetos de tipos diferentes através de uma interface comum. O tipo real é irrelevante para o código cliente: ele apenas chama o método, e o objeto correto responde.

Esta é a v1.8, e ela é o ponto em que o Deriva colhe o que a unidade construiu. O laço de turno percorre `std::vector<std::unique_ptr<entidade>>`, chama `agir()` em cada uma, e não sabe de subtipo nenhum. Vale reler a lista do que teve de estar no lugar para essa linha existir, porque é a arquitetura da unidade inteira em quatro itens: a hierarquia com o subobjeto base (Cap. 10), o método virtual e o destrutor virtual (Cap. 11), a posse exclusiva atrás de `unique_ptr` (Cap. 12), e a semântica de referência que impede o fatiamento (Cap. 14 §14.8). Falte um, e o laço deixa de funcionar por uma razão diferente em cada caso.

A propriedade que o laço tem, e que interessa ao resto do livro, é a de ser **fechado à modificação**. Uma derivada nova de entidade entra no sistema sem que uma linha do laço mude, porque o laço nunca nomeou tipo concreto. É o princípio aberto/fechado, que o Cap. 24 vai cobrar pelo nome; aqui ele aparece como consequência do despacho virtual, o que é a ordem certa - o princípio descreve o que o mecanismo já faz.

Os dois últimos trechos no fim deste capítulo são os dois laços, extraídos de `src/mundo.cpp`, e vale ler os dois pelo que **não** está escrito neles: nenhum `dynamic_cast`, nenhum `typeid`, nenhum `switch`, nenhum nome de tipo concreto. `mundo::turno()` tem uma linha de trabalho, `entidades_[i]->agir(*this)`, e `mundo::despejar()` pede `pos()` e `glifo()` a cada entidade e escreve o caractere na posição calculada.

Uma decisão de implementação do turno merece nota, porque ela é observável no replay. O laço percorre **por índice**, com o limite fixado antes de começar, e não por iterador nem por `for` de intervalo. A razão está no comentário de lá: `agir()` pode acrescentar entidade ao mundo, e acrescentar a um `std::vector` durante um percurso por iterador invalida o iterador. Com índice e limite fixo, quem entra durante o turno age no turno **seguinte**, e essa é uma regra do jogo, e não um

detalhe de contêiner: ela decide a ordem em que as coisas acontecem, e o despejo comparado byte a byte do Cap. 16 a trava.

§18.2 `dynamic_cast` - Downcasting Seguro

Downcast é a operação de converter um ponteiro ou referência para um tipo base em um tipo derivado. `dynamic_cast` verifica em tempo de execução se o objeto realmente é do tipo alvo: devolve `nullptr`, no caso de ponteiro, ou lança `std::bad_cast`, no caso de referência, se não for.

Três fatos que decidem quando usá-lo.

Ele exige classe polimórfica. `dynamic_cast` precisa do `vptr` para chegar à informação de tipo, e classe sem função virtual não tem `vptr`; a tentativa é erro de compilação, e não falha em execução. É a mesma razão pela qual `entidade_simples` do Cap. 11 §11.4 não tem os 8 bytes a mais: sem `vtable`, não há de onde tirar o tipo dinâmico.

Ele custa tempo de execução, e mais que uma chamada virtual. A verificação percorre a informação de tipo da hierarquia, e o custo depende da profundidade e da forma dela - num diamante com base virtual, como o do Cap. 17, ele é mais caro. Não é operação para pôr dentro do laço de turno.

Ele funciona onde `static_cast` não funciona, e recusa onde `static_cast` mente. `static_cast<drone*>(ent)` compila e não verifica nada: se o objeto não for um drone, você tem um ponteiro de tipo errado apontando para memória válida, e o programa continua a rodar até chamar algo que não está ali. `dynamic_cast` devolve `nullptr` no mesmo caso. Quando você não sabe o tipo, `static_cast` é a ferramenta errada; quando você sabe, `dynamic_cast` é gasto inútil.

O trecho no fim deste capítulo é esse inspetor, e as duas versões estão nele. A primeira linha, `std::string s = e.descrever();`, é a versão por despacho virtual, que não pergunta o tipo; tudo o que vem depois é a cadeia de `dynamic_cast`, que existe para acrescentar o campo que só aquela derivada tem - os reparos da reparadora, a energia da sonda, o rumo do drone, a massa do item. Ler o trecho de cima para baixo é ler o contraste do §18.4 na ordem em que ele foi escrito.

E a ordem da cadeia é obrigatória, pelo motivo que o comentário registra: a sonda_reparadora também É sonda, então perguntar por sonda primeiro a capturaria e o ramo específico dela nunca rodaria. `testes/test_inspetor.cpp` trava as duas metades disso, afirmando que a saída da reparadora traz [reparadora, e **não** traz [sonda, energia.

O idioma que o trecho usa em cada ramo - a conversão dentro da condição do `if`, com a variável declarada ali, `if (const auto* r = dynamic_cast<const sonda_reparadora*>(&e))` - é o que se escreve, e em C++17 ele fica melhor ainda com inicializador de `if` quando o teste não é o próprio ponteiro: `if (auto* d = dynamic_cast<drone*>(e); d != nullptr)`. O ganho é de escopo: `d` não existe fora do `if`, e não há como usá-lo por acidente depois, num ponto em que ele possa ser nulo.

▲ ATENÇÃO

Usar `dynamic_cast` frequentemente é um sintoma de design ruim - indica que o código cliente sabe demais sobre os tipos concretos.

Se você se pega fazendo `if (dynamic_cast<tipo_a>()) ... else if (dynamic_cast<tipo_b>()) ...`, considere redesenhar usando o padrão Visitor ou adicionando o método necessário na interface base.

§18.3 RTTI - typeid e type_info

RTTI (Run-Time Type Information) fornece informação sobre o tipo real de um objeto em tempo de execução. `typeid(expr)` devolve uma referência a um objeto `type_info`, que pode ser comparado com `==` e tem um método `name()`.

Quatro armadilhas, e as quatro custam tempo de laboratório.

name() devolve nome manglado, e o formato é escolha do compilador. Medido no g++ 13.3.0: um `deriva::drone` sai como `N6deriva5droneE`, e o padrão não diz nada sobre isso. Desmanglar depende de extensão - `c++filt` na linha de comando, ou `abi::__cxa_demangle` no código, que é interface do ABI do GNU e não existe em toda implementação. Portanto: `name()` serve para diagnóstico, e não para saída que alguém vá ler nem para comparação.

typeid sobre ponteiro nulo desreferenciado lança. `typeid(*p)` com `p` nulo lança `std::bad_typeid`, e não é comportamento indefinido - é a única operação deste capítulo que se defende sozinha. Já `typeid(p)`, sem o asterisco, devolve o tipo **estático** do ponteiro e não lança nunca, o que é a confusão mais comum: `typeid(p)` de um `entidade*` apontando para um `drone` diz `PN6deriva8entidadeE`, que é `entidade*` manglado, e não diz `drone`.

typeid compara igualdade exata de tipo, e não subtipo. `typeid(*e) == typeid(sonda)` é falso para uma `sonda_reparadora`, que é uma sonda. `dynamic_cast<sonda*>(e)` é verdadeiro. Confundir os dois produz um inspetor que não reconhece as derivadas das derivadas, e o defeito só aparece quando a hierarquia cresce - isto é, depois do Cap. 17.

RTTI pode estar desligado. `-fno-rtti` é uma opção real, usada em código embarcado e em partes de motores de jogo, e ela torna `typeid` e `dynamic_cast` indisponíveis. Não é o caso desta disciplina, e é a razão pela qual bibliotecas que querem ser universais evitam os dois.

As quatro armadilhas acima têm uma consequência de projeto para o inspetor, e ela foi decidida assim: **o nome legível que o inspetor imprime não vem de typeid**. Ele vem de `name()`, que é virtual pura na base, através do `descrever()` do Cap. 10 - a moldura não-virtual sobre virtuais. `typeid` continua no sistema, e num lugar só: `inspetor.hpp` declara `tipo_cru(const entidade&)`, que devolve `typeid(e).name()` sem desembaralhar, e o comentário de lá diz para que ela existe - mostrar que `typeid` responde sobre o tipo real e que o nome que ele devolve é definido pela implementação. Ela é material didático, e não caminho de saída.

A regra que fecha a seção: prefira `dynamic_cast` a comparação com `typeid`, porque o primeiro funciona com toda a hierarquia e o segundo é igualdade exata de tipo. E prefira função virtual aos dois, quando o que você quer é um nome ou um texto - foi assim que o inspetor do Deriva ficou com uma linha de despacho virtual e uma cadeia de conversões que só acrescenta o campo específico de cada derivada.

§18.4 Quando `dynamic_cast` é Sintoma, e Quando Ele é a Resposta

O aviso do §18.2 é verdadeiro e é insuficiente, porque ele não diz como distinguir os dois casos. O critério é este: **`dynamic_cast` é sintoma quando o cliente pergunta o tipo para decidir o comportamento, e é a resposta quando a finalidade do cliente é relatar o tipo.**

A cadeia de `if` (`dynamic_cast<...>`) que escolhe o que fazer é sintoma, sempre, e por uma razão estrutural: ela recria o `switch` sobre etiqueta de tipo que o Cap. 11 §11.4 disse que o despacho virtual substitui, e recria também o defeito dele - uma derivada nova entra no sistema e a cadeia não a trata, sem que nada avise. O laço do §18.1 é fechado à modificação; a cadeia de conversões não é. A saída é acrescentar o método que faltava na interface base, e a pergunta que a encontra é: *o que eu ia fazer depois do cast?* Aquilo é o método.

O inspetor de entidade da v1.8 é o outro caso. Ele é o comando do console que responde “o que é esta coisa na minha frente, e quais são os campos dela?”, e a resposta correta é o tipo concreto - relatá-lo não é decidir comportamento, é o serviço prestado. Um inspetor de depuração escrito com função virtual `descrever()` funciona, e paga um preço: cada derivada passa a carregar, na sua interface pública, um método que existe só para a ferramenta. Quando o número de derivadas cresce e as ferramentas se multiplicam, é a interface que fica poluída, e é aí que o Visitor do Cap. 25 se paga.

Duas decisões sobre o inspetor, e as duas são de projeto e não de estilo.

Ele é ferramenta, e não parte do jogo. O inspetor entra por `--inspecionar` na linha de comando, ao lado de `--leiaute` e de `--fov`, e a saída dele **não** entra no despejo que o portão de replay compara. Se ele fizesse parte do jogo, cada campo novo de cada derivada passaria a ser comparado byte a byte pelo Cap. 16, e uma ferramenta de depuração passaria a travar o formato do estado do mundo. Ferramenta que trava o que ela observa deixa de ser ferramenta.

Ele pergunta por capacidade, e não só por tipo. O segundo trecho extraído é `listar_reparadoras`, e a conversão dela é `dynamic_cast<const i_reparavel*>` - para a **interface** do Cap. 17, e não para a classe concreta. É a pergunta certa: o que interessa àquele comando é quem sabe reparar, e não quem é uma sonda-reparadora. Uma derivada nova que implemente a interface aparece na lista sem que aquela função mude, e é a diferença entre uma conversão que fecha o código à extensão e uma que não fecha.

O contraste entre as duas versões do inspetor é o exercício central deste capítulo, e ele não tem resposta única: a versão com `descrever()` é melhor para o jogo, a versão com `dynamic_cast` é melhor para a ferramenta, e o critério é quem paga a conta. Escrever as duas e argumentar a escolha é o que se cobra.

DERIVA

O que a Unidade II acrescentou ao sistema, versão por versão:

- **v1.0** hierarquia entidade → sonda/drone/item, com o subobjeto base
- **v1.1** desenhar() e agir() virtuais, classe abstrata, destrutor virtual
- **v1.2** mundo com vector<unique_ptr<entidade>>
- **v1.3** grafo de conexões da estação, com shared_ptr e weak_ptr
- **v1.4** movimento em grade e mapa, e as cinco operações de mapa
- **v1.5** operadores de vetor2 e mapa[pos]
- **v1.6** replay determinístico: semente fixa, roteiro gravado, despejo comparado
- **v1.7** sonda_reparadora, e o diamante
- **v1.8** inspetor de entidade no console

As nove versões acima são a trilha da unidade, e a numeração delas é didática: o repositório é um só, e o que ele traz hoje é o núcleo inteiro, com todas as peças de todas as nove no lugar. O Anexo C traz a lista completa das vinte versões, da v0.0 à v2.7, com o que cada uma acrescenta.

A Prova 2 é no segundo encontro desta semana, escrita, em papel, e fecha a Unidade II. O que ela cobra é o que os nove capítulos formaram, e a forma de estudar é a mesma do regime de predição das aulas: dado um trecho, dizer qual construtor roda, qual destrutor roda, qual função virtual é chamada, quanto o objeto ocupa, e o que sobra na origem depois do movimento - sem executar o trecho.

Exercícios Propostos

1. Escreva o laço de turno e o de render do mundo sem um único nome de tipo concreto. Depois acrescente uma derivada nova de entidade e conte quantas linhas dos dois laços você teve de mudar. Se a resposta não for zero, diga onde o tipo concreto vazou.
2. Implemente o inspetor de entidade **duas** vezes: com `dynamic_cast` para cada derivada, e com uma função virtual `descrever()`. Compare as duas em quatro pontos: o que acontece quando uma derivada nova aparece, o que cada uma pede da interface base, o custo em tempo de execução, e o que sobra de conhecimento de tipo no cliente. Depois escolha uma para o Deriva e justifique.
3. Faça `static_cast<drone*>` num ponteiro entidade* que aponta para um item, e chame um método específico do drone. Registre o que aconteceu. Repita com `dynamic_cast` e registre a diferença. Explique por que o primeiro compila sem um aviso.
4. Escreva `typeid(*e) == typeid(sonda)` para uma sonda_reparadora do Cap. 17, e depois `dynamic_cast<sonda*>(e)` para o mesmo objeto. Explique por que os dois resultados diferem, e diga qual dos dois responde à pergunta “isto é uma sonda?”.
5. Imprima `typeid(*e).name()` para cada uma das suas derivadas e passe a saída por `c++filt`. Compare com o que `deriva::tipo_cru` devolve. Depois responda: por que este material recusa usar `name()` na saída que o estudante lê, e o que ele usa no lugar?
6. Pesquise o padrão Visitor como alternativa ao `dynamic_cast` para operações específicas de tipo. Implemente um visitante que percorra as entidades do mundo

e recolha estatísticas por tipo. Depois diga o que ele resolve em relação ao exercício 2, e o que ele **piora** quando uma derivada nova aparece.

7. Por que `dynamic_cast` só funciona com classes polimórficas? Tente-o numa classe sem função virtual, leia o erro, e ligue a resposta aos 8 bytes do `vptr` do Cap. 11 §11.4.

8. Compile o seu Deriva com `-fno-rtti` e registre o que deixa de compilar. Depois diga o que você mudaria no desenho do inspetor para que ele sobrevivesse a essa opção, e se valeria a pena.

O código, extraído do Deriva

Todo trecho abaixo vem de `exemplos/deriva/`, que compila com `-std=c++17 -Wall -Wextra -Wpedantic` sem um aviso e passa `make verifica`. Nenhum foi digitado neste texto.

```
src/inspetor.cpp:10 | a ordem da cadeia é obrigatória
10 std::string inspecionar(const entidade& e) {
11     std::string s = e.descrever();
12
13     // A ordem importa: `sonda_reparadora` também É `sonda`, então testá-la
14     // depois nunca aconteceria. Perguntar pelo tipo mais derivado primeiro é a
15     // armadilha número um de cadeia de dynamic_cast - e é o argumento de que
16     // essa cadeia deveria ser uma função virtual.
17     if (const auto* r = dynamic_cast<const sonda_reparadora*>(&e)) {
18         s.append(" [reparadora, ").append(std::to_string(r->reparos_feitos()))
19         .append(" reparo(s), energia ").append(std::to_string(r->energia())).append("]");
20     } else if (const auto* sd = dynamic_cast<const sonda*>(&e)) {
21         s.append(" [sonda, energia ").append(std::to_string(sd->energia())).append("]");
22     } else if (const auto* dr = dynamic_cast<const drone*>(&e)) {
23         s.append(" [drone, rumo ").append(std::to_string(dr->rumo().x)).append(", ")
24         .append(std::to_string(dr->rumo().y)).append("]");
25     } else if (const auto* it = dynamic_cast<const item*>(&e)) {
26         s.append(" [item, massa ").append(std::to_string(it->massa())).append("]");
27     }
```

A `sonda_reparadora` também É `sonda`, então testá-la depois nunca aconteceria. Perguntar pelo tipo mais derivado primeiro é a armadilha número um - e é o argumento de que essa cadeia deveria ser uma função virtual.

```

src/inspetor.cpp:31 | perguntar por capacidade, não por tipo
31 std::string listar_reparadoras(const mundo& m) {
32     std::string s;
33     int achadas = 0;
34     for (std::size_t i = 0; i < m.quantas(); ++i) {
35         // `dynamic_cast` para a INTERFACE, e não para a classe concreta: é a
36         // pergunta certa, porque o que interessa é a capacidade e não o tipo.
37         if (dynamic_cast<const i_reparavel*>(&m.em(i)) != nullptr) {
38             s.append(" ").append(m.em(i).descrever()).append("\n");
39             ++achadas;
40         }
    }

```

`dynamic_cast` para a interface, e não para a classe concreta: é a pergunta certa, porque o que interessa é a capacidade. Este é o único lugar do Deriva onde `dynamic_cast` é a resposta e não o sintoma.

```

src/mundo.cpp:19 | o laço que não nomeia tipo concreto
19 void mundo::turno() {
20     // Índice em lugar de iterador: `agir` pode acrescentar entidade, e isso
21     // invalidaria o iterador. Quem entra durante o turno age no turno seguinte,
22     // e essa regra é observável no replay.
23     const std::size_t n = entidades_.size();
24     for (std::size_t i = 0; i < n; ++i) entidades_[i]->agir(*this);
25 }

```

Nenhum `dynamic_cast`, nenhum `typeid`, nenhum `switch`. É o polimorfismo funcionando - e o contraste com o inspetor da mesma aula, onde perguntar o tipo é a tarefa.

```

src/mundo.cpp:46 | o render, também sem nome de tipo
46 std::string mundo::despejar() const {
47     std::string s = setor_.despejar();
48     // As entidades entram por cima do glifo do terreno, na ordem de inserção.
49     const int largura = setor_.g().largura();
50     const std::size_t cabeca = s.find('\n', s.find('\n') + 1) + 1;
51     for (const std::unique_ptr<entidade>& e : entidades_) {
52         const vetor2 p = e->pos();
53         if (!setor_.g().dentro(p)) continue;
54         const std::size_t i = cabeca +
55             static_cast<std::size_t>(p.y) * static_cast<std::size_t>(largura + 1) +
56             static_cast<std::size_t>(p.x);
57         if (i < s.size()) s[i] = e->glifo();
58     }

```

Cada entidade desenha o próprio glifo por chamada virtual. Acrescentar uma entidade nova não toca nesta função.

Genericidade, robustez e projeto

Templates e CRTP com alvo concreto, erros, STL e lambdas, concorrência, serialização, SOLID verificado por replay, padrões e um segundo front-end sobre o mesmo núcleo.

Templates, polimorfismo estático e CRTP

O QUE ESTE CAPÍTULO ENTREGA

- ▶ Escrever e usar templates de função e de classe em C++17.
- ▶ Restringir um parâmetro de tipo com `static_assert`, e ler a mensagem que ele produz.
- ▶ Usar `if constexpr` para escolher ramo em tempo de compilação, no lugar de `SFINAE`.
- ▶ Distinguir polimorfismo estático de dinâmico pelo que cada um cobra e pelo que cada um permite.
- ▶ Generalizar por CRTP o contador de instâncias vivas escrito à mão desde o Cap. 7.
- ▶ Reconhecer `Concepts` e `Ranges` como API do C++20, fora do padrão-alvo da disciplina.

Este capítulo tem uma posição incomum na disciplina, e vale declará-la antes de qualquer sintaxe: **tudo o que ele generaliza já foi escrito à mão**. O contador de instâncias vivas apareceu no Cap. 7, foi copiado para `terminal_bruto` no Cap. 8 e copiado outra vez para a hierarquia de entidades no Cap. 10, sempre com as mesmas quatro linhas e o mesmo par de variáveis. A grade guarda `celula` desde o Cap. 8, e o cálculo de caminho da v2.3 vai querer uma grade de distâncias inteiras, enquanto o mapa do que já foi visitado, na v2.5, vai querer uma de caracteres. Assim, o template chega aqui como resposta a uma repetição que você sentiu, e não como recurso a ser exercitado por si.

A ordem importa. Template que chega antes de o estudante ter escrito o código repetido três vezes é solução para um problema que ele não teve, e o resultado previsível é o uso decorativo de genericidade: classe parametrizada por um tipo só, instanciada uma vez, com o parâmetro nunca variando. O comentário de cabeçalho de `include/deriva/contador.hpp` diz exatamente isso, e está lá desde a v0.1, escrito para ser lido agora.

§19.1 O que São Templates?

A palavra que carrega o mecanismo é **instanciar**, e convém não passar por ela rápido. O template não é código, e sim texto parametrizado por tipo: é a receita a partir da qual o compilador escreve código, uma vez para cada tipo que aparece de fato no programa. Nada é gerado para um tipo que ninguém usou, e o que é gerado para `grade<celula>` não tem relação de parentesco alguma com o que é gerado para `grade<int>`. São duas classes diferentes, que por acidente saíram do mesmo texto.

Disso decorre a consequência prática que mais atrapalha quem vem de Java: a definição de um template precisa estar visível no ponto de uso, e portanto vai no cabeçalho, e não num `.cpp`. `grade` tem hoje o par `grade.hpp` e `grade.cpp`, com a validação e o cálculo de índice no segundo; na versão genérica, que é `grade_generic.hpp`, a implementação inteira está no cabeçalho, e não há `.cpp` que lhe

corresponda. Não é preferência de estilo: o compilador precisa do corpo do método na unidade de tradução em que instancia.

§19.2 Templates de Função

O bloco desta seção é ilustração de sintaxe, e não sai de exemplos/deriva/: a `grade_de<T>` da §19.3 é template de classe, e o Deriva não tem template de função que sirva de exemplo mínimo. Leia-o com atenção ao comentário, porque ele diz `operator>` e não `operator<`: o requisito de um template é o que o **corpo** usa, e não o que a documentação afirma, e é exatamente essa distância entre a promessa escrita e a exigência real que a §19.4 vai fechar com `static_assert`.

```

1 #include <string>
2
3 // Template de função: serve a qualquer tipo que tenha operator>
4 template <typename T>
5 T max_of(T a, T b) {
6     return (a > b) ? a : b;
7 }
8
9 // O compilador instancia uma versão para cada tipo que aparece de fato:
10 auto x = max_of(3, 7);           // T = int
11 auto y = max_of(3.14, 2.71);    // T = double
12 auto z = max_of(std::string{"a"}, std::string{"b"}); // T = std::string
13
14 // Template com dois parâmetros de tipo:
15 template <typename From, typename To>
16 To convert(From value) {
17     return static_cast<To>(value);
18 }
19
20 auto f = convert<int, float>(42);

```

Repare no que **não** está escrito nas três primeiras chamadas: o tipo. A dedução de argumento de template olha o tipo dos argumentos passados e resolve `T` sozinha, e é por isso que `max_of(3, 7)` compila sem que ninguém escreva `max_of<int>`. A quarta chamada precisa da lista explícita, `convert<int, float>(42)`, porque `To` não aparece em nenhum parâmetro da função, e o que não aparece nos parâmetros não pode ser deduzido.

O que interessa aqui, mais que o caso que funciona, é o que acontece quando o tipo não serve. Chame `max_of` com dois `vetor2` e o compilador reclamará de dentro do corpo do template, dizendo que não existe `operator>` para `deriva::vetor2`, com um rastro de instanciação que aponta primeiro para a linha errada. A mensagem é correta e é péssima: ela relata o sintoma no lugar do requisito. As duas seções seguintes tratam disso, uma em C++17 e outra em C++20.

§19.3 grade<T>: o Template de Classe

A v2.0 acrescenta a versão genérica ao lado da grade da v0.2, e o desenho é o seguinte. A largura, a altura e o vetor de elementos permanecem; dentro() e o cálculo de índice permanecem, porque não dependem do que o elemento é; o par de sobrecargas de em() passa a devolver T& e const T&. Com isso, grade_de<celula> é o mapa da estação, grade_de<int> é o campo de distâncias de que o cálculo de caminho precisa, e grade_de<char> é o mapa do que já foi visitado, da v2.5. Um cálculo de índice, num lugar só, para os três. O tipo que **não** entra nessa lista é bool, e o motivo está num static_assert extraído no fim deste capítulo.

O código existe, e com um nome que vale explicar: ele está em include/deriva/grade_generica.hpp, chamado grade_de<T>, e **ao lado** da grade não genérica, que continua onde estava. A convivência é deliberada, e o comentário de cabeçalho do arquivo a registra: as classes das Aulas 7 a 18 seguem com a forma escrita à mão para que a comparação entre as duas continue possível. O alias using grade_de_celulas = grade_de<celula>; é o que mantém o nome curto para o caso comum.

Três trechos no fim deste capítulo mostram a classe por inteiro, e vale lê-los na ordem em que aparecem. O primeiro é a declaração com as duas restrições de T, e nele já se lê a herança pelo CRTP da §19.6. O segundo é o construtor, que valida antes de alocar, com o par de sobrecargas de em() devolvendo T& e const T&. O terceiro é o alias e o static_assert de que a parametrização e a herança não aumentaram o objeto - a afirmação da §19.5 sobre custo, medida no sizeof em vez de prometida.

Uma decisão de projeto merece ser explicitada, porque ela é a diferença entre generalizar e desfigurar. A grade não parametriza a largura nem a altura: elas continuam sendo argumentos de construtor, e não parâmetros de template. Parâmetro de template não-tipo existe, e grade<celula, 80, 24> compila; porém o mapa é lido de arquivo em tempo de execução, e a dimensão só é conhecida depois de ler a primeira fileira. Genericidade que exige saber em compilação o que só se sabe em execução não é generalização, é um acoplamento novo.

▲ ATENÇÃO

Cada instanciação é uma classe distinta, com código próprio no binário. grade<celula>, grade<char> e grade<int> não compartilham uma linha de código executável, e o que elas compartilham é o texto do template. Isso é o que dá a velocidade, e é também o que faz o tempo de compilação e o tamanho do binário crescerem com o número de tipos usados. Instanciar um template para vinte tipos porque é fácil é o defeito simétrico ao de escrever a classe vinte vezes à mão.

§19.4 Restringir o Tipo: `static_assert` e `if constexpr`

Uma `grade<T>` aceita qualquer `T` que se possa guardar num `std::vector` e copiar, o que é mais do que se quer. `grade<std::thread>` compila até a primeira tentativa de copiar a `grade`, e a mensagem que aparece então não tem relação visível com a decisão errada, tomada centenas de linhas antes. A defesa em C++17 é declarar a exigência no corpo da classe, através de `static_assert`, para que a violação seja relatada no ponto em que o tipo errado foi escolhido, com a frase que o autor da classe escreveu.

A restrição de que a `grade<T>` precisa é modesta, e o valor está em nomeá-la: `T` tem de ser copiável e ter construtor padrão, porque o construtor da `grade` cria largura × altura elementos antes de qualquer coisa ser desenhada. Uma linha na classe, com a mensagem "`grade<T>`: `T` precisa ter construtor padrão", troca um erro de dentro de `<vector>` por uma frase que diz o que fazer.

— `if constexpr`, e por que ele substitui SFINAE

O despejo determinístico é onde a genericidade cobra o preço. `mapa::despejar()` produz o texto do setor, e é ele que sustenta o replay do Cap. 16; uma `grade` genérica precisa saber transformar `T` em caractere, e a resposta é diferente para cada `T`: para `celula`, é o campo `glifo`; para `char`, é `+` ou `.`; para tipo inteiro, é o dígito, e `#` quando o valor é negativo. É o que o trecho extraído no fim deste capítulo mostra, com os três ramos lado a lado.

A saída ingênua é escrever funções livres sobrecarregadas, e ela funciona. A saída ruim é escrever um `if` comum dentro do template, e ela **não** funciona, por um motivo que vale entender bem: num `if` comum, os dois ramos precisam ser código válido para todo `T` instanciado, inclusive o ramo que nunca será executado. Escrever `if (é_celula) return valor.glifo; else return '#';` numa `grade_de<int>` faz o compilador tentar compilar `valor.glifo` para `int`, e falhar, apesar de aquela linha ser inalcançável.

`if constexpr`, que é C++17, resolve isso mudando o momento da decisão. A condição é avaliada em tempo de compilação, e o ramo descartado não é apenas pulado na execução: ele é **descartado antes da instanciação**, e portanto não precisa nem ser válido para aquele `T`. É esta propriedade, e não a economia de digitação, que faz de `if constexpr` a ferramenta certa aqui.

Antes do C++17, o mesmo efeito se obtinha por SFINAE - a sigla de *substitution failure is not an error* -, através de `std::enable_if` em parâmetro de template extra, ou por despacho por etiqueta, com uma sobrecarga por categoria de tipo. As duas técnicas funcionam, aparecem em código que você vai ler, e cobram o mesmo preço: a intenção fica espalhada por assinaturas, e a mensagem de erro de uma restrição violada é a de “nenhuma sobrecarga viável”, que não diz qual requisito falhou. O v1 deste livro tinha uma única ocorrência de SFINAE, e ela sai daqui: o padrão-alvo da disciplina tem `if constexpr`, e o que se ensina é o que se usa.

O que o Deriva escreve, portanto, é uma forma só, e ela é a extraída no fim deste capítulo: os três ramos sobre `std::is_same_v<T, celula>`, `std::is_same_v<T, char>` e `std::is_integral_v<T>`, dentro de `grade_de<T>::despejar()`, com o ramo final devolvendo `?`. A versão com `std::enable_if` **não** está no repositório, e a ausência é deliberada: manter no núcleo uma técnica que o material recusa é convidar a que ela seja copiada. A comparação que interessa, que é a das mensagens de erro para o mesmo `T` inválido, você produz no exercício 3 e no exercício 6, com o compilador do laboratório em mãos - as três formas, sem restrição, com

`static_assert` e com `concept`, e a leitura de qual delas diz o que fazer é o Anexo A.

§19.5 Polimorfismo Estático vs. Dinâmico

Característica	Templates (estático)	Virtual (dinâmico)
Quando é resolvido?	Compilação	Runtime
Overhead por chamada?	Nenhum (inlining possível)	Indireção via vtable
Código binário?	Instância separada por tipo (code bloat)	Uma implementação, despacho por <code>vptr</code>
Tipos em coleção heterogênea?	Não (todos do mesmo tipo)	Sim (ponteiros para base)
Extensão em plugins?	Não (tipos fixos em compilação)	Sim (tipos adicionados em runtime)
Erros detectados?	Compilação (mensagens podem ser crípticas)	Runtime (<code>dynamic_cast</code> , <code>bad_cast</code>)

O Deriva usa as duas, e a linha da tabela que decide é a quarta. O mundo guarda `vector<unique_ptr<entidade>>` desde o Cap. 12, com sonda, drone e item no mesmo contêiner, e cada um respondendo à sua maneira a `desenhar()` e a `agir()`: coleção heterogênea, conjunto de tipos que cresce ao longo do semestre, e portanto despacho dinâmico. A `grade<T>` é o oposto em todas as linhas, porque todos os elementos são do mesmo tipo, o conjunto de tipos é fechado em compilação, e não há nada a estender em execução.

O custo do despacho dinâmico está medido, e não é grande: `include/deriva/leiaute.hpp` afirma por `static_assert` que a entidade sem método virtual ocupa 8 bytes e a entidade com um método virtual ocupa 16, porque o `vptr` são 8 bytes **por objeto**, e não por classe. Numa estação com algumas centenas de entidades, isso é ruído. É por isso que trocar `virtual` por CRTP na hierarquia de entidades seria mau negócio: pagar-se-ia em contêiner heterogêneo, que é o que se usa, para economizar em indireção, que não é o gargalo.

A pergunta que separa os dois casos, e que serve de critério fora deste livro, é uma só: **o programa precisa guardar objetos de tipos diferentes no mesmo contêiner, ou decidir em execução qual tipo criar?** Se precisa, é `virtual`. Se não precisa, `template` resolve, sem `vptr` e com `inlining`.

8 bytes

custo do `vptr`

aferido por

`./build/deriva --leiaute`

`testes/test_leiaute.cpp`

§19.6 CRTP, e o Contador que Já Foi Escrito Três Vezes

O nome do padrão descreve a forma antes de dizer para que ela serve, e por isso ele parece truque na primeira leitura. O alvo concreto desfaz a impressão, e ele é o contador de instâncias vivas.

Abra `include/deriva/contador.hpp` e conte o que está lá: `contador_mapa` e `contador_terminal`, duas estruturas com o mesmo par de variáveis `inline static`, e um terceiro contador na hierarquia de entidades desde o Cap. 10. Três vezes as mesmas quatro linhas, e três lugares onde alguém pode esquecer o incremento no construtor de cópia, que é justamente o erro que o exercício 4 do Cap. 7 mandou cometer de propósito.

O que atrapalha generalizar isso é que o contador precisa de estado estático **por classe**, e não compartilhado. Uma base comum com um `inline static int` vivos, da qual `mapa`, `terminal` e entidade herdassem, daria um contador só, so-

mando os três, e a informação que interessa - qual classe deixou objeto vivo - se perderia. Herança comum não separa estado estático: a base é uma, e a variável dela é uma.

É aqui que o parâmetro de template entra, e não como enfeite. `contador_de_instancias<T>` é um template, logo `contador_de_instancias<peca>` e `contador_de_instancias<mochila>` são **classes diferentes**, cada uma com o seu próprio par de variáveis estáticas. A derivada se passa como argumento para a própria base - `class grade_de : public contador_de_instancias<grade_de<T>>` é a forma que o Deriva usa -, e é essa recorrência de aparência curiosa que dá nome ao padrão e que produz o efeito desejado: uma base por derivada, um contador por classe, um lugar só onde as quatro linhas estão escritas.

O construtor da base incrementa vivos e criados; o destrutor decrementa vivos; e o construtor de cópia da base, que agora existe e é escrito uma vez, incrementa também. **O defeito que o Cap. 7 mandou provocar deixa de ser possível**, porque não há mais três construtores de cópia a lembrar, e sim um. O trecho extraído no fim deste capítulo é o template inteiro, e vale conferir nele o que o parágrafo acima afirma: o construtor de cópia e o de movimento contam, e as duas atribuições não contam, porque atribuir não cria nem destrói objeto.

Quem herda dele hoje é `grade_de<T>`, `peca` e `mochila`. `mapa` e `terminal_bruto` continuam com o contador escrito à mão, e a convivência é deliberada: o material precisa das duas formas lado a lado para que a comparação seja possível, e trocar as duas é o que o **LAB-10** cobra do estudante.

O padrão tem duas formas, e vale separá-las, porque só uma delas está no Deriva. A que está é a forma **passiva**: a base guarda estado por tipo, faz a sua contabilidade no construtor e no destrutor, e nunca chama de volta a derivada. A outra é a forma de **interface estática**: a base declara `fazer()` e o implementa chamando `static_cast<Derived*>(this)->fazer_impl()`, de modo que a derivada forneça o corpo sem que nenhum método seja `virtual` e sem que nenhum objeto ganhe `vptr`. É a forma que aparece em biblioteca de terceiro, e é a que você vai reconhecer em código alheio.

O Deriva não a usa, e a decisão é de custo. A única hierarquia do sistema em que compor comportamento faz sentido é a de entidade, e ela está descartada pelo argumento da §19.5: o mundo guarda `vector<unique_ptr<entidade>>`, isto é um contêiner heterogêneo, e polimorfismo estático não guarda tipos diferentes no mesmo contêiner. Fora dela não há dívida de repetição a pagar, e interface estática sem dívida a pagar é um `static_cast` na base em troca de nada. O contador tinha a dívida - três cópias das mesmas quatro linhas -, e é por isso que ele foi generalizado e o resto não.

— DICA —

O destrutor de uma base de CRTP **não** deve ser `virtual`. Ninguém destrói um objeto através de `contador_de_instancias<grade_de<celula>>*`, porque essa base não é interface de nada; declarar o destrutor `virtual` acrescentaria um `vptr` de 8 bytes por objeto e desfaria o argumento inteiro do capítulo. O que protege o uso errado, no Deriva, são os **construtores e o destrutor protegidos**: a base não se instancia sozinha, e não se destrói por ponteiro para ela. A herança em si é pública, e de propósito - vivos, criados, `zerar()` e `fechou()` precisam ser lidos de fora, pelo portão e pelos testes, e herança privada os esconderia.

8 bytes

custo do `vptr`

aferido por

`./build/deriva --leiaute`

`testes/test_leiaute.cpp`

— O que o CRTP não conserta

Generalizar o contador não muda o que ele mede, e as duas limitações do Cap. 7 seguem valendo, palavra por palavra. Ele conta objetos, e não recursos: na variante de cópia rasa da caça ao bug 1, dois objetos apontam para o mesmo buffer, os dois são construídos e destruídos, vivos fecha em zero, e a memória é liberada duas vezes. E ele não é seguro entre threads: o Cap. 22 mostra exatamente esta variável perdendo um incremento, agora dentro do template, o que apenas concentra num lugar o defeito que antes estava em três.

Vale dizer também o que o CRTP custa em legibilidade, porque este capítulo não é propaganda de template. A declaração `class grade_de : public contador_de_instancias<grade_de<T>>` é estranha na primeira leitura, e leva um parágrafo de explicação para deixar de ser; o `static_cast<Derived*>(this)` da forma de interface estática é pior, porque só se justifica com o comentário ao lado dizendo por que ele é seguro, a saber, que a base só é instanciada com a derivada que herda dela. Numa classe que não paga a mesma dívida de repetição que o contador pagava, essa estranheza não se compra.

§19.7 Vinte Minutos de C++20: Concepts e Ranges

O padrão-alvo desta disciplina é C++17, e este é o único lugar do material obrigatório onde o C++20 aparece. São vinte minutos de reconhecimento de API, e não de uso: nenhum exercício, nenhum laboratório e nenhuma prova dependem do que está nesta seção, e o conteúdo integral está no **Anexo A**, que compila em alvo separado, com `-std=c++20`.

O gancho é a §19.4. O `static_assert` que restringe `T` dá uma mensagem escrita à mão, o que é melhora grande sobre o erro de dentro de `<vector>`, e continua sendo remendo: a restrição não faz parte da assinatura, não participa da resolução de sobrecarga, e não se pode consultar se um tipo a satisfaz sem tentar instanciar. Um **concept** é a mesma exigência promovida a entidade nomeada da linguagem, verificável, componível, e capaz de aparecer na assinatura no lugar de `typename`. **Ranges** é o assunto vizinho: composição de transformações sobre sequências, com avaliação preguiçosa, no lugar do par de iteradores que o Cap. 21 apresenta.

O que fica desta seção, para a prova, é a capacidade de reconhecer as duas coisas em código alheio e de dizer por que elas não estão nas suas entregas. A resposta é de infraestrutura, e não de gosto: o laboratório tem o compilador que tem, o portão exige `-std=c++17` sem extensões, e material que ensina uma construção que o portão recusa produz o estudante que não consegue entregar. O v1 deste livro fazia isso, ao declarar-se C++17 em doze lugares e ensinar Concepts e Ranges como capítulo, sem uma ocorrência de `string_view`, de ligações estruturadas ou de `std::filesystem`. O Anexo B é a lista do que ficou de fora, e o Anexo A é o que mudou de estatuto.

Exercícios Propostos

1. Transforme `grade` em `grade<T>`: mova a implementação de `src/grade.cpp` para o cabeçalho, parametrize o vetor e o par de sobrecargas de `em()`, e declare `using grade_de_celulas = grade<celula>;` para que o resto do `Deriva` continue compilando sem uma linha alterada. Rode `make verifica` e confirme as quatro condições, com atenção à primeira: zero aviso.
2. Acrescente a `grade<T>` o `static_assert` que exige de `T` construtor padrão e cópia, com mensagem própria. Depois tente, de propósito, instanciar `grade<std::thread>`, e compare a mensagem que você escreveu com a que o compilador dava antes do `static_assert`. Guarde as duas saídas: elas são a resposta ao exercício 6.
3. Escreva `glifo_de<T>()` com `if constexpr`, cobrindo `celula`, `char` e tipo inteiro, e faça `despejar()` usá-la. Em seguida troque o `if constexpr` por um `if` comum e explique, a partir da mensagem de erro, por que o ramo inalcançável precisou ser válido.
4. Generalize o contador em `contador_de_instancias<T>` e faça `mapa` e `terminal_bruto` herdarem dele, removendo as quatro linhas escritas à mão em cada um. O portão é o do **LAB-10**: `make verifica` continua passando, `vivos` fecha em zero, e o `static_assert` afirma que `sizeof(mapa)` não cresceu. Explique, em duas linhas, por que a herança não custou byte.
5. Faça o contador errado de propósito: escreva uma base **não** parametrizada, com um `inline static int vivos` só, e faça as duas classes herdarem dela. Rode o roteiro de `replay` e explique o que o número impresso passou a significar. Por que o parâmetro de `template` é o que separa os contadores?
6. Compile a `grade<T>` do exercício 2 com `-std=c++20` num alvo separado e troque o `static_assert` por um `concept`, como o Anexo A mostra. Compare as três mensagens de erro para o mesmo `T` inválido: sem restrição, com `static_assert`, e com `concept`. Qual delas diz o que fazer?
7. Compare o assembly gerado, via `Compiler Explorer`, para a mesma operação implementada com `virtual` e com CRTP, compilando com `-O2`. Identifique a indicação na primeira versão e a sua ausência na segunda, e diga quantas instruções a diferença representa por chamada.

O código, extraído do Deriva

Todo trecho abaixo vem de `exemplos/deriva/`, que compila com `-std=c++17 -Wall -Wextra -Wpedantic` sem um aviso e passa `make verifica`. Nenhum foi digitado neste texto.


```

┌─| include/deriva/contador_crtp.hpp:30 |────────────────── o contador generalizado ─────────┐
30 class contador_de_instancias {
31 public:
32     inline static int vivos = 0;
33     inline static int criados = 0;
34
35     static void zerar() noexcept { vivos = criados = 0; }
36
37     /// Verdadeiro quando todo objeto deste tipo já foi destruído. É o que o
38     /// portão `make verifica` consulta.
39     [[nodiscard]] static bool fechou() noexcept { return vivos == 0; }
40
41 protected:
42     // Protegido e não-virtual: esta base não é para ser usada por ponteiro, e
43     // por isso não paga destrutor virtual. Quem herdar dela e for base
44     // polimórfica declara o SEU destrutor virtual, como `entidade` faz.
45     contador_de_instancias() noexcept {
46         ++vivos;
47         ++criados;
48     }
49     contador_de_instancias(const contador_de_instancias&) noexcept {
50         ++vivos;
51         ++criados;    // cópia também é nascimento, e o manual esquecia disso
52     }
53     contador_de_instancias(contador_de_instancias&&) noexcept {
54         ++vivos;
55         ++criados;
56     }
57     contador_de_instancias& operator=(const contador_de_instancias&) noexcept {
58         return *this;    // atribuição não cria nem destrói
59     }
60     contador_de_instancias& operator=(contador_de_instancias&&) noexcept {
61         return *this;
62     }
63     ~contador_de_instancias() noexcept { --vivos; }
64 };

```

O parâmetro T é o truque: cada instanciação é um TIPO diferente, logo tem os seus próprios vivos. Herança comum compartilharia um contador só entre todas as derivadas, que é exatamente o erro que o contador na base cometeria.

```

┌─| include/deriva/grade_generica.hpp:65 |────────────────────────── if constexpr no despejo ───────────────────┐
65 /// O despejo depende do que `T` é, e a decisão acontece em tempo de
66 /// COMPILAÇÃO. Com `if` comum, os três ramos teriam de ser válidos para
67 /// todo `T` - e `c.glifo` não existe em `int`, então nem compilaria. É a
68 /// diferença que `if constexpr` faz, e o motivo pelo qual ele substitui
69 /// SFINAE (Aula 19).
70 [[nodiscard]] std::string despejar() const {
71     std::string s;
72     for (int y = 0; y < altura_; ++y) {
73         for (int x = 0; x < largura_; ++x) {
74             const T& c = em({x, y});
75             if constexpr (std::is_same_v<T, celula>) {
76                 s.push_back(c.glifo);
77             } else if constexpr (std::is_same_v<T, char>) {
78                 s.push_back(c ? '+' : '.');
79             } else if constexpr (std::is_integral_v<T>) {
80                 s.push_back(c < 0 ? '#' : static_cast<char>('0' + (c % 10)));
81             } else {
82                 s.push_back('?');
83             }

```

Com `if` comum os três ramos teriam de ser válidos para todo `T`, e `c.glifo` não existe em `int` - não compilaria. É a diferença que `if constexpr` faz, e o motivo pelo qual ele substitui SFINAE.

```

┌─| include/deriva/grade_generica.hpp:32 |────────────────────────── a mensagem é a nossa ───────────────────┐
32 // `std::vector<bool>` é a especialização mais famosa da biblioteca padrão, e
33 // a mais lamentada: ela empacota os bits, então `operator[]` devolve um
34 // PROXY e não um `bool&`. Uma grade que promete `T&` não pode ser
35 // instanciada com `bool`, e a mensagem abaixo é o que o estudante lê em vez
36 // de "cannot bind non-const lvalue reference to an rvalue".
37 //
38 // A saída é `char` ou `std::uint8_t`, e a v2.5 usa `char` no mapa do que já
39 // foi visitado. É por isso que a Aula 21 diz que `vector<bool>` não é um
40 // vetor de bool (Aula 19 e Aula 21).
41 static_assert(!std::is_same_v<T, bool>,
42               "grade_de<bool> nao existe: std::vector<bool> empacota bits e "
43               "devolve proxy, nao bool&. Use char ou std::uint8_t");

```

`std::vector<bool>` empacota bits e devolve proxy, não `bool&`. Sem este `static_assert`, o estudante leria “cannot bind non-const lvalue reference to an rvalue” vindo de dentro da biblioteca.

```
include/deriva/grade_generica.hpp:27 | a classe e as restrições
27 class grade_de : public contador_de_instancias<grade_de<T>> {
28     static_assert(std::is_default_constructible_v<T>,
29                 "grade_de<T> precisa saber criar celula vazia: T sem construtor "
30                 "padrao nao serve");
31     static_assert(!std::is_reference_v<T>, "grade de referencias nao existe");
```

`grade_de<T>` CONVIVE com a `grade` não genérica em vez de substituí-la: generalizar não é obrigação retroativa. E o `static_assert` na definição, não no uso, é o que faz a mensagem de erro ser a nossa.

```

46 grade_de(int largura, int altura)
47     : largura_(exigir_positivo(largura, "largura")),
48       altura_(exigir_positivo(altura, "altura")),
49       celulas_(static_cast<std::size_t>(largura_) *
50               static_cast<std::size_t>(altura_)) {}
51
52 [[nodiscard]] int largura() const noexcept { return largura_; }
53 [[nodiscard]] int altura() const noexcept { return altura_; }
54 [[nodiscard]] bool dentro(vetor2 p) const noexcept {
55     return p.x >= 0 && p.y >= 0 && p.x < largura_ && p.y < altura_;
56 }
57
58 [[nodiscard]] const T& em(vetor2 p) const { return celulas_[indice(p)]; }
59 [[nodiscard]] T& em(vetor2 p) { return celulas_[indice(p)]; }

```

A validação continua na lista de inicialização, como na versão não genérica - e o par `const/não-const` de `em()` é o mesmo padrão do Cap. 7, agora sobre `τ`.

```
113 /// A grade de células continua sendo o caso comum, e ganha um nome curto.
114 using grade_de_celulas = grade_de<celula>;
115
116 /// Herdar do CRTP não custa byte nenhum: base vazia, e o compilador a
117 /// otimiza. É a diferença entre polimorfismo estático e dinâmico, medida.
118 static_assert(sizeof(grade_de<celula>) == sizeof(int) * 2 + sizeof(std::vector<celula>),
119               "o contador por CRTP nao aumenta o objeto");
```

Base vazia, e o compilador a otimiza. É a diferença entre polimorfismo estático e dinâmico, afirmada no sizeof.

Tratamento de erros

O QUE ESTE CAPÍTULO ENTREGA

- ▶ Definir uma hierarquia de exceções própria a partir de `std::exception`.
- ▶ Nomear a garantia de exceção que uma operação oferece: nenhuma, básica, forte ou `noexcept`.
- ▶ Explicar o desenrolar da pilha e por que ele é o que faz RAII funcionar.
- ▶ Usar `std::optional` para ausência de resultado e `std::variant` para resultado ou erro.
- ▶ Usar `std::filesystem` no carregamento de mapa, e reconhecer a corrida entre verificar e abrir.
- ▶ Escolher, para uma operação dada, entre exceção, `optional` e `variant`.

O Deriva chega a este capítulo com uma política de erro só, e ela foi deliberada: **arquivo ausente ou mapa torto devolve `std::optional` vazio, e nada é lançado**. A decisão está escrita no comentário de `include/deriva/mapa.hpp` desde a v0.3, com a promessa de que o erro de verdade chegaria na v2.2, que é agora.

O que muda aqui é a distinção entre duas coisas que a v0.3 tratava igual. Arquivo que não existe é **ausência de resultado**: o programa pediu um mapa, não havia mapa, e a resposta “não há” é uma resposta legítima que o chamador sabe tratar. Permissão negada no meio da leitura, disco cheio na hora de gravar a partida, fileira com caractere que nenhum glifo do jogo define, tudo isso é **falha**: o programa pediu algo que devia ter funcionado, alguma coisa deu errado no caminho, e quem chamou não tem como consertar localmente. A primeira categoria é do `optional`; a segunda é da exceção.

§20.1 Exceções em C++

Uma exceção sai do ponto em que foi lançada e sobe a pilha de chamadas até o primeiro `catch` que aceite o tipo dela, destruindo no caminho tudo o que já havia sido construído, e sem que os níveis intermediários escrevam uma linha a respeito - é a §20.3 que mostra por que essa destruição é o que faz o RAII do Deriva funcionar no caminho de erro. Onde não há `catch` que a aceite, o programa termina.

A hierarquia que o Deriva tem é uma raiz e duas folhas, e a razão de ela ser uma hierarquia, e não uma classe só com uma mensagem dentro, é a cláusula `catch`. Quem captura escolhe a granularidade: `catch (const erro_de_deriva&)` trata qualquer problema do jogo; `catch (const mapa_invalido&)` trata só o conteúdo que não é mapa e deixa o resto passar. Sem hierarquia, essa escolha viraria uma cadeia de comparações de string dentro de um `catch` único, que é o padrão que se vê em código ruim e que não sobrevive à primeira mensagem reescrita.

A raiz é `deriva::erro_de_deriva`, derivada de `std::runtime_error`, e a escolha da base importa: `std::runtime_error` já guarda a mensagem e já implementa `what()`, portanto derivar dela custa um construtor de uma linha. Derivar diretamente de `std::exception` obrigaria a guardar a `std::string` à mão e a lembrar de que `what()` é `noexcept`, o que é trabalho sem retorno - e o comentário no cabeçalho acrescenta a armadilha que fecha o argumento: se o construtor de cópia da exceção lançar durante o desenrolar da pilha, o programa termina. As duas folhas são `falha_de_leitura`, para o que o sistema de arquivos recusou, que guarda o `std::error_code`; e `mapa_invalido`, para o conteúdo que não é mapa, que guarda o caminho e o motivo. Repare no que cada folha carrega além da mensagem: dado estruturado, e não texto a ser analisado outra vez por quem capturou.

Três trechos no fim deste capítulo mostram a hierarquia inteira, e a ordem de leitura importa. O primeiro é a raiz, com o comentário que diz por que ela não deriva de `std::exception` direto. O segundo são as duas folhas, e nele o que se olha é a lista de inicialização: cada uma monta a mensagem na base e guarda o dado estruturado ao lado. O terceiro é `carregar_ou_lancar`, e é o único lugar do Deriva em que a mesma ausência de arquivo produz exceção em vez de `optional` - o comentário interno diz por quê, e a razão é a promessa da função, e não a natureza do erro.

§20.2 As Garantias de Exceção

Dizer que uma função “trata erros” não diz nada verificável. O que se pode afirmar e conferir é a **garantia** que ela oferece quando uma exceção passa por ela, e são quatro níveis, do mais forte ao mais fraco.

A garantia **noexcept** é a promessa de que a função não lança, em nenhuma circunstância. Não é otimismo: é verificada em execução, e se uma exceção tentar sair de uma função `noexcept`, o programa chama `std::terminate`. As funções de acesso do Deriva a carregam desde a v0.1, e `grade::dentro()` é o exemplo mínimo, porque comparar dois inteiros com dois limites não tem como falhar.

A garantia **forte** é a de tudo ou nada: se a operação lança, o objeto fica exatamente como estava antes da chamada, e nenhum efeito parcial é observável. É a garantia que `std::vector::push_back` oferece, e é ela que explica a regra do Cap. 14 sobre `noexcept` no construtor de movimento.

A garantia **básica** é a de que nenhum recurso vazou e nenhuma invariante da classe foi violada, porém o objeto pode ter mudado. Ele continua utilizável e destrutível; só não se sabe qual dos dois estados ele tem.

E há a ausência de garantia, que é o objeto em estado inconsistente, com invariante quebrada e destrutor que pode piorar as coisas. Código nessa condição não é aceitável em entrega desta disciplina.

— Onde o Deriva está, hoje, e o que falta

`mapa::operator=` oferece a garantia **básica**, e não a forte, e vale ler o código para ver por quê. Ele atribui membro por membro - nome, grade, entrada, marca - depois da guarda de autoatribuição; a atribuição da grade aloca, porque copia um `std::vector`, e portanto pode lançar `std::bad_alloc`. Se ela lançar, o nome já foi trocado e a grade não, e o mapa passa a ter o nome de um setor com o desenho de outro. Nenhuma invariante de classe foi violada, nada vazou, e o objeto é destrutível: garantia básica, cumprida.

A promoção para garantia forte tem nome e é curta: **copiar e trocar**. Constrói-se uma cópia completa do objeto de origem, o que é a única parte que pode lançar, e só depois se trocam os membros com `swap`, que é `noexcept`. Se a construção da cópia falhar, o objeto de destino não foi tocado. O custo é uma alocação a mais no caminho de sucesso, e é por isso que a técnica é escolha, e não regra.

— DICA —

`noexcept` não é decoração otimista: é promessa verificada em execução, e quebrá-la chama `std::terminate`. Marque `noexcept` o que de fato não pode falhar - acesso a membro, comparação de inteiros, troca de ponteiros - e deixe sem marca o que aloca ou chama código de terceiro. Há um lugar em que a marca muda o comportamento, e não só a documentação: o construtor de movimento. `std::vector` só usa movimento ao realocar se ele for `noexcept`; sem a marca, ele copia, para poder oferecer a garantia forte. É o assunto do Cap. 14, e é a razão pela qual as funções de acesso do Deriva carregam `noexcept` desde a v0.1.

§20.3 O Desenrolar da Pilha, e por que RAII Depende Dele

O Cap. 8 estabeleceu a garantia da linguagem em uma frase, e ela é a que sustenta este capítulo inteiro: **quando uma exceção é lançada, os destrutores dos objetos já construídos no caminho são chamados, de dentro para fora, enquanto a pilha é desenrolada**. A exceção não pula os destrutores; ela os chama. `testes/test_ciclo_de_vida.cpp` afirma essa ordem linha por linha, e o trecho está extraído no fim do Cap. 8.

O que este capítulo acrescenta é a consequência de projeto. Se a liberação de recurso está no destrutor, ela acontece no caminho de erro sem que ninguém escreva uma linha para isso. Se está numa chamada explícita depois do uso, a exceção passa por cima dela e o recurso vaza. Não há terceira opção em C++: não existe `finally`, e a proposta de escrever `catch (...) { liberar(); throw; }` em cada nível é a versão manual, verbosa e esquecível daquilo que o destrutor faz de graça.

`terminal_bruto` é o caso em que essa diferença é física. Ele põe o terminal em modo bruto no construtor e o restaura no destrutor; se uma exceção sobe de dentro do laço de jogo, o desenrolar da pilha destrói o `terminal_bruto` no caminho, o modo é restaurado, e a mensagem de erro aparece num terminal que funciona. Na variante v0.2-quebrada, que omite o destrutor, a mesma exceção deixa o terminal do estudante sem eco e sem Enter **depois** que o programa sai, e ele descobre RAII digitando `reset` às cegas. É a melhor demonstração de RAII que a disciplina tem, e não é metáfora.

▲ ATENÇÃO

Destrutor não lança. A regra parece arbitrária até que se veja o mecanismo: se uma exceção está em curso e o destrutor de um objeto que o desenrolar está destruindo lança uma segunda, o programa chama `std::terminate` e morre sem passar por handler nenhum. Desde o C++11 os destrutores são implicitamente `noexcept`, o que transforma o vazamento de exceção de um destrutor em terminação imediata. Se o seu destrutor faz algo que pode falhar - fechar arquivo, gravar buffer, restaurar o modo do terminal -, ele trata a falha ali, registra e segue; não a propaga.

§20.4 `std::optional` - Resultado Ausente sem Exceção

O `optional` do Deriva não é ilustração: `mapa::carregar` e `mapa::de_texto` devolvem `std::optional<mapa>` desde a v0.3, e o trecho está extraído no fim deste capítulo. O que ele compra é uma coisa só, e é grande: **a ausência de resultado aparece no tipo**. Quem chama `carregar` não pode esquecer de verificar, porque não existe `mapa` para usar antes de desembulhar o `optional`; a alternativa antiga, devolver um `mapa` vazio e um `bool` de sucesso ao lado, permite ignorar o `bool` e seguir com o objeto vazio, que é o defeito que este tipo elimina por construção.

Duas armadilhas valem nomear. A primeira é `value_or`, que é cômodo e avalia o argumento sempre, mesmo quando o `optional` tem valor; se o padrão custa uma alocação, `value_or` a paga em todas as chamadas. A segunda é o operador `*`, que desreferencia sem verificar: usá-lo sem ter perguntado antes é comportamento indefinido, e não exceção. A versão que lança é `value()`, e a diferença entre as duas é a única coisa que este parágrafo pede que você memorize.

§20.5 `std::variant` - Resultado ou Erro

O `optional` diz que não houve resultado, e não diz por quê. Quando o motivo importa para quem chamou, e quando lançar seria pesado demais porque a falha é frequente e esperada, a resposta é uma **soma de tipos**: o valor, ou o erro, num tipo só, com a garantia de que exatamente um dos dois está lá.

`std::variant<mapa, razao>` é a forma dessa resposta no Deriva, e ela tem nome: `resultado_de_mapa`, declarada em `include/deriva/erro.hpp` e extraída no fim deste capítulo, com o comentário que registra a escolha inteira da aula. A alternativa de erro não é uma string nem uma classe de exceção: é o enumerado `razao`, e o motivo está no código - quem trata o erro decide o que fazer a partir dele, e comparar string é frágil.

A consulta se faz de duas formas, e as duas valem conhecer. `std::visit` exige tratamento de todas as alternativas e falha em compilação quando uma fica de fora, e é a diferença prática em relação ao par de tipos com um `bool`: aqui o compilador cobra a exaustividade. `std::get_if` pergunta por uma alternativa e devolve ponteiro nulo quando não é ela, e é a forma que `carregar_ou_lancar` usa, porque ali há uma pergunta só a fazer - se veio `razao`, converte em `mapa_invalido` e lança; se veio `mapa`, devolve. Repare que a exaustividade que `visit` cobra é justamente o que `get_if` não cobra, e por isso ele serve onde as alternativas são duas e atrapalha onde são muitas.

O carregamento de `mapa` é o lugar do Deriva onde o `variant` se justifica, e não por elegância. A v2.2 passa a distinguir três desfechos, e o `optional` só distingue

dois: o arquivo não existe, e é ausência; o arquivo existe e o conteúdo não é mapa, e o motivo é um dos que `razao` nomeia - vazio, fileira torta, sem entrada, entrada duplicada, glifo desconhecido -, cada um com a frase que descrever devolve; o arquivo existe e o sistema recusou a leitura, e aí é exceção, porque o programa não tem o que fazer com isso. O `variant` cobre o segundo caso sem transformar erro de conteúdo em exceção, o que importa porque conteúdo malformado, num arquivo que o próprio estudante escreve, é situação corriqueira e não excepcional.

O que a v2.2 **não** faz, e vale dizer para que ninguém procure: reportar a fileira e a coluna do defeito. `razao` nomeia a categoria, e não a posição; pôr a posição na resposta é acrescentar um campo ao enumerado ou trocá-lo por uma `struct`, e é decisão de projeto, não detalhe de escrita.

§20.6 `std::filesystem` no Carregamento de Mapa

O v1 deste livro se declarava C++17 em doze lugares e não tinha uma ocorrência de `std::filesystem`. O caminho de arquivo aparecia como `std::string`, a existência era verificada abrindo um `std::ifstream` e testando-o, e a construção de caminho era concatenação com barra literal, o que já é errado fora de POSIX.

`std::filesystem::path` é o tipo do caminho, e ele resolve três coisas de uma vez. A primeira é a construção correta, com `operator/` no lugar da concatenação de `string`, respeitando o separador da plataforma. A segunda é a decomposição sem análise de `string` à mão: `stem()` devolve o nome sem extensão, e é ele que o `Deriva` usa para nomear o setor a partir do arquivo, de forma que mapas/estacao-01.txt produza um mapa chamado `estacao-01`. A terceira é a consulta ao sistema de arquivos com `exists` e `is_regular_file`, o que permite distinguir arquivo ausente de diretório passado por engano, distinção que o `ifstream` não oferece.

O trecho extraído no fim deste capítulo é o de `mapa::carregar`, e há três coisas nele que valem leitura atenta.

A primeira é o `std::error_code`. Toda função de `std::filesystem` tem duas formas, uma que lança `filesystem_error` e outra que recebe um `error_code&` e não lança; `carregar` usa a segunda e testa o código, porque nesta camada a ausência é resposta e não falha. A escolha é consciente e é o exemplo mais limpo do critério do capítulo: a mesma biblioteca oferece as duas políticas, e quem decide é o chamador, pelo significado que aquele erro tem no seu domínio.

A segunda é a corrida. Verificar que o arquivo existe e depois abri-lo são duas operações distintas, e entre uma e outra o arquivo pode desaparecer, mudar de permissão ou virar diretório. Nenhuma checagem prévia elimina isso; é uma condição de corrida inerente a sistemas de arquivos, conhecida como janela entre verificar e usar. Por isso o código testa `arq` depois de abrir, apesar de já ter perguntado por `exists`: a verificação prévia serve ao diagnóstico, e a única prova de que a leitura vai acontecer é a leitura.

A terceira é que a política do capítulo se lê no texto do código. `carregar` fala com o sistema de arquivos e devolve `optional`; `de_texto` recebe o conteúdo já em memória e é a que os testes usam, porque teste que depende do sistema de arquivos é teste frágil. A separação foi feita para o teste e paga também aqui, ao deixar claro que a decisão sobre erro pertence à camada que conhece o sistema operacional.

§20.7 Qual dos Três, e por Quê

Situação	Mecanismo	No Deriva
Ausência é resposta legítima, e o motivo não importa	<code>std::optional<T></code>	<code>mapa::carregar</code> , arquivo inexistente
Falha esperada, frequente, e o motivo importa	<code>std::variant<T, E></code>	fileira de largura errada
Falha rara, o chamador local não tem o que fazer	exceção	permissão negada, disco cheio
Não pode falhar	<code>noexcept</code>	<code>grade::dentro</code> , funções de acesso
Violação de contrato do programador	<code>assert</code> , <code>static_assert</code>	índice fora da grade sem ter perguntado <code>dentro()</code>

A última linha é a que se esquece. Índice fora da grade, chamada de método em objeto movido, argumento negativo onde a documentação exige positivo: nada disso é erro de execução a ser tratado, é defeito de programa. Tratar defeito de programa com exceção transforma um erro reproduzível, que o teste pega, num caminho de recuperação que ninguém exercita e que esconde o defeito. `grade::em()` não verifica limite de propósito, e o comentário no código diz por quê: quem chama já perguntou `dentro()`.

Exercícios Propostos

1. Escreva `include/deriva/erro.hpp` com `deriva::erro : std::runtime_error` e as derivadas `erro_de_arquivo` e `erro_de_formato`, herdando o construtor com `using`. Escreva três testes Catch2 com `REQUIRE_THROWS_AS`, um por tipo, e um quarto que captura por `const deriva::erro&` e confirma que as três chegam lá. Este é o portão do **LAB-11**.
2. Faça `mapa::carregar` distinguir os três desfechos da §20.6: devolver `nullopt` quando o arquivo não existe, lançar `erro_de_arquivo` quando o `error_code` indica outra coisa, e lançar `erro_de_formato` quando uma fileira tem largura diferente das outras, com a fileira e a largura na mensagem. Rode `make_verifica` e confirme que o replay continua idêntico byte a byte.
3. Promova `mapa::operator=` à garantia forte por copiar e trocar. Demonstre a diferença com um teste: um alocador que falha na enésima alocação, uma atribuição que lança no meio, e a asserção de que o mapa de destino continua com o nome e o desenho que tinha. `testes/test_posse.cpp` já traz um alocador que conta, e serve de ponto de partida.
4. Reescreva o carregamento com `std::variant<mapa, erro_de_formato>` em vez de exceção para o caso de formato, e consulte com `std::visit`. Compare as duas versões em duas linhas: qual delas o chamador pode ignorar, e qual delas o compilador cobra?
5. Provoque a terminação: escreva um destrutor que lança, construa o objeto dentro de um escopo que também lança, e rode. Explique, a partir da mensagem, por que não houve catch nenhum. Depois conserte o destrutor tratando a falha no lugar de propagá-la.
6. Passe um diretório para `mapa::carregar` no lugar de um arquivo, e depois um arquivo sem permissão de leitura. Preveja por escrito o que cada chamada devolve

ou lança antes de rodar, e explique qual das três funções de `std::filesystem` distinguiu os dois casos.

7. Pesquise `std::expected` (C++23). O que ele melhora sobre `std::variant<T, E>` para tratamento de erro, e o que ele **não** resolve em relação à exceção? Diga também por que ele não entra nas suas entregas desta disciplina.

O código, extraído do Deriva

Todo trecho abaixo vem de `exemplos/deriva/`, que compila com `-std=c++17 -Wall -Wextra -Wpedantic` sem um aviso e passa `make verifica`. Nenhum foi digitado neste texto.

```

└─| include/deriva/erro.hpp:69 |────────────────── optional, variant ou exceção ───────────┐
69 /// O resultado de tentar interpretar um texto como mapa.
70 ///
71 /// **A escolha de projeto da Aula 20**, e ela é declarada no tipo: três formas
72 /// de dizer que algo não deu certo, cada uma para um caso diferente.
73 ///
74 /// - `std::optional` para **ausência**: o arquivo não existe. Não é erro, é
75 ///   resposta. É o que `mapa::carregar` devolve desde a v0.3.
76 /// - `std::variant` para **erro esperado com informação**: o texto existe e
77 ///   não é mapa, e quem chamou precisa saber por quê para decidir. Não é
78 ///   exceção porque acontece no fluxo normal - o estudante vai errar o mapa
79 ///   dele muitas vezes.
80 /// - **exceção** para o que rompe a operação: não deu para LER o arquivo.
81 ///   Quem chamou não tem o que decidir, e o desenrolar da pilha é a resposta
82 ///   certa.
83 using resultado_de_mapa = std::variant<mapa, razao>;

```

Três formas de dizer que algo não deu certo, e a escolha está no tipo: `optional` para ausência, `variant` para erro esperado com informação, exceção para o que rompe a operação.

```

src/erro.cpp:30 |----- validar antes de construir -----
30 resultado_de_mapa interpretar(std::string_view texto, std::string nome) {
31     // Repare que a validação acontece ANTES de qualquer construção: nada é
32     // alocado até se saber que o texto serve. É o que torna a garantia forte
33     // barata - não há o que desfazer.
34     if (texto.empty()) return razao::vazio;
35
36     std::size_t largura = 0, entradas = 0, fileiras = 0, inicio = 0;
37     while (inicio <= texto.size()) {
38         const std::size_t fim = texto.find('\n', inicio);
39         const std::string_view linha = texto.substr(
40             inicio, fim == std::string_view::npos ? std::string_view::npos : fim - inicio);
41         if (!linha.empty()) {
42             if (fileiras == 0) largura = linha.size();
43             else if (linha.size() != largura) return razao::fileira_torta;
44             ++fileiras;
45             for (const char c : linha) {
46                 if (c == '@') ++entradas;
47                 else if (c != '#' && c != '.' && c != '!') return razao::glifo_desconhecido;
48             }

```

A garantia forte é barata aqui porque nada é alocado até se saber que o texto serve - não há o que desfazer. É por isso que a leitura é separada da aplicação.

```

include/deriva/erro.hpp:14 |----- a raiz, e por que não std::exception -----
14 /// A raiz da hierarquia de erros do Deriva.
15 ///
16 /// Deriva de `std::runtime_error` e não de `std::exception` direto, porque
17 /// `runtime_error` já guarda a mensagem e resolve o `what()`. Herdar de
18 /// `std::exception` e reimplementar `what()` é trabalho sem retorno, e a
19 /// tentação de guardar um `std::string` membro esconde uma armadilha: se o
20 /// construtor de cópia da exceção lançar durante o desenrolar da pilha, o
21 /// programa termina.
22 class erro_de_deriva : public std::runtime_error {
23 public:
24     explicit erro_de_deriva(const std::string& msg) : std::runtime_error(msg) {}
25 };

```

`runtime_error` já guarda a mensagem e resolve o `what()`. Herdar de `std::exception` e guardar uma `std::string` membro esconde uma armadilha: se a cópia da exceção lançar durante o desenrolar, o programa termina.

```

src/erro.cpp:9 |----- as duas folhas-----
9 mapa_invalido::mapa_invalido(std::filesystem::path caminho, std::string motivo)
10 : erro_de_deriva("mapa invalido em '" + caminho.string() + "': " + motivo),
11   caminho_(std::move(caminho)),
12   motivo_(std::move(motivo)) {}
13
14 falha_de_leitura::falha_de_leitura(std::filesystem::path caminho, std::error_code ec)
15 : erro_de_-
16   deriva("nao foi possivel ler '" + caminho.string() + "': " + ec.message()),
17   caminho_(std::move(caminho)),
18   ec_(ec) {}

```

Cada uma monta a mensagem na base e guarda o dado estruturado ao lado: quem trata precisa do caminho e do código, e não de uma string para reanalisar.

```

src/erro.cpp:63 |----- o ponto exato em que ausência deixa de ser optional-----
63 mapa_carregar_ou_lancar(const std::filesystem::path& caminho) {
64   std::error_code ec;
65   if (!std::filesystem::exists(caminho, ec)) {
66     // Ausência tratada como erro AQUI, e como `optional` em `mapa::carregar`.
67     // A diferença não é capricho: esta função promete devolver um mapa, e
68     // aquela promete responder se há um.
69     throw falha_de_leitura(caminho, std::make_error_code(std::errc::no_such_file_or_
70 directory));
71   }
72   if (ec) throw falha_de_leitura(caminho, ec);

```

`mapa::carregar` devolve `optional` para o mesmo arquivo ausente. A diferença não é capricho: esta função PROMETE devolver um mapa, e aquela promete responder se há um.

```
src/mape.cpp:109 optional e filesystem
109 std::optional<mapa> mapa::carregar(const std::filesystem::path& caminho) {
110     std::error_code ec;
111     if (!std::filesystem::exists(caminho, ec) || ec) return std::nullopt;
112     if (!std::filesystem::is_regular_file(caminho, ec) || ec) return std::nullopt;
113
114     std::ifstream arq(caminho);
115     if (!arq) return std::nullopt;
116     std::ostringstream buf;
117     buf << arq.rdbuf();
118     const std::string texto = buf.str();
119     return de_texto(texto, caminho.stem().string());
120 }
```

Arquivo ausente é *ausência de resultado*, não exceção - optional diz isso no tipo. Erro de verdade, permissão negada, é exceção, e chega na v2.2. Repare que exists e abrir são operações distintas: a corrida entre elas é real.

STL panorâmica e lambdas

O QUE ESTE CAPÍTULO ENTREGA

- ▶ Escolher o contêiner adequado a partir do padrão de acesso, e não do hábito.
- ▶ Compreender o modelo de iteradores e o intervalo semiaberto.
- ▶ Escrever lambdas, e reconhecer nelas a classe anônima que o compilador gera.
- ▶ Distinguir captura por valor de captura por referência, e o risco de cada uma.
- ▶ Substituir laço manual por algoritmo da STL, e saber quando o laço é mais claro.
- ▶ Usar `std::clamp` e `std::size`, com as armadilhas de cada um.

Este capítulo tem duas metades desiguais em história e iguais em peso. A primeira, a dos contêineres e dos algoritmos, existia no v1 e vem de lá com poucas mudanças. A segunda, a das lambdas, é conteúdo novo: o livro inteiro tinha **uma** menção a lambda, de passagem, num bloco de código. É pouco para uma construção da qual dependem o Cap. 25, quando Strategy deixa de ser hierarquia e passa a ser função, e a metade dos algoritmos desta aula.

O Deriva chega aqui na v2.3, com duas entregas que são as candidatas naturais: o campo de visão da sonda, que percorre a grade decidindo o que é visível, e o inventário, que ordena, soma e filtra o que a sonda carrega. As duas eram laço com índice na versão anterior. O que este capítulo pede não é que virem algoritmo por moda, e sim que a escolha entre laço e algoritmo passe a ser deliberada.

§21.1 Contêineres Essenciais

Contêiner	Caso de uso	Acesso	Inserção/remoção
<code>vector<T></code>	Array dinâmico - padrão para sequências	$O(1)$ random access	$O(1)$ amortizado no final
<code>deque<T></code>	Fila dupla - inserção/remoção eficiente nos dois extremos	$O(1)$ random access	$O(1)$ nos extremos
<code>list<T></code>	Lista ligada - inserção/remoção $O(1)$ em qualquer posição com iterador	$O(n)$ sequencial	$O(1)$ com iterador
<code>map<K,V></code>	Mapa ordenado - busca $O(\log n)$	$O(\log n)$	$O(\log n)$
<code>unordered_map<K,V></code>	Mapa hash - busca $O(1)$ amortizado	$O(1)$ amortizado	$O(1)$ amortizado
<code>set<T></code>	Conjunto ordenado sem duplicatas	$O(\log n)$	$O(\log n)$
<code>priority_queue<T></code>	Heap - acesso ao maior/menor em $O(1)$	$O(1)$ no topo	$O(\log n)$

A tabela ordena por complexidade assintótica, e a decisão real quase nunca se toma por ela. `std::vector` é o padrão, e continua sendo o padrão para umas centenas de elementos mesmo quando a tabela sugere outra coisa, porque os elementos ficam contíguos na memória e o processador lê linha de cache inteira de uma vez. Uma `std::list` de mil inteiros tem inserção $O(1)$ e perde de um `std::vector` em quase todo percurso, porque cada nó está num lugar diferente e cada salto é uma falha de cache. A regra prática é: comece com `vector`, meça, e só troque com o número na mão.

O Deriva ilustra isso três vezes, e as três valem como argumento.

A grade guarda `std::vector<celula>` e calcula o índice a partir da posição, em vez de guardar um vetor de vetores. São duas razões: as células ficam contíguas, o que faz o percurso do desenho ler memória em sequência; e há um só objeto a copiar, mover e destruir, o que é o que permite a regra do zero do Cap. 9. `bytes_das_celulas()` mede o resultado, e o Cap. 7 mostrou que 4 bytes por célula de diferença viram 7,5 KB numa grade de 80 por 24.

A tabela de comandos do roteiro, em `src/main.cpp`, é um array de cinco `std::pair<std::string_view, vetor2>`, percorrido linearmente. Um `std::unordered_map<std::string, vetor2>` daria busca $O(1)$ e seria pior em tudo o que importa aqui: cinco comparações de `string_view` são mais rápidas que uma função de espalhamento, o array é `static const` e não aloca nada, e a tabela cabe numa tela. Estrutura de dados escolhida pela complexidade assintótica sobre cinco elementos é o caso mais comum de otimização que atrapalha.

O inventário da sonda, na v2.3, é `std::vector` de itens, e não `std::map` por nome. A operação frequente é percorrer tudo para desenhar a lista; a busca por nome acontece uma vez por comando do jogador. Quando o padrão de acesso é o percurso, o contêiner é o sequencial.

A exceção legítima chega com o cálculo de caminho: uma busca por melhor-primeiro precisa retirar repetidamente o nó de menor custo, e `std::priority_queue` faz isso em $O(\log n)$ enquanto o `vector` faria em $O(n)$. Aí a troca se paga, e o motivo se escreve em uma frase.

§21.2 Iteradores e o Intervalo Semiaberto

Um iterador é a generalização do ponteiro: um objeto que sabe apontar para um elemento, avançar, e ser comparado com outro. É ele que desacopla algoritmo de contêiner, e é por isso que `std::sort` funciona sobre `vector`, sobre array cru e sobre `std::deque` sem uma linha diferente.

Todo intervalo da STL é **semiaberto**: `[primeiro, ultimo)` inclui o primeiro e exclui o último, e `end()` aponta para uma posição depois do fim, que não se pode desreferenciar. A convenção parece arbitrária e resolve duas coisas: o tamanho do intervalo é a subtração dos dois iteradores, sem ajuste; e o intervalo vazio se escreve com os dois iguais, sem caso especial. Quase todo erro de um-a-mais em código C++ vem de esquecer que `end()` não é um elemento.

A consequência prática que mais morde é a **invalidação**. Inserir num `std::vector` pode realocar, e realocar invalida todos os iteradores, ponteiros e referências para os elementos dele. Apagar invalida a partir do ponto apagado. O laço que percorre um vetor apagando elementos por dentro é a forma mais rápida de produzir comportamento indefinido em C++, e a solução idiomática tem nome: `remove_if` seguido de `erase`, que é o que a §21.4 mostra.

§21.3 Lambdas: a Classe Anônima que o Compilador Escreve

Uma expressão lambda é, na leitura mais útil para esta disciplina, **açúcar sintático para uma classe**. Quando você escreve

```
auto e_visivel = [limite](const celula& c) { return c.energia > li-  
mite; };
```

o compilador gera um tipo de classe sem nome, com um membro para cada coisa capturada, e um `operator()` `const` que recebe os parâmetros declarados e executa o corpo. `e_visivel` é um objeto desse tipo. Não é ponteiro para função, não é `std::function`, e o tipo dele não tem nome que você possa escrever, o que é a razão de auto ser obrigatório ali.

Essa leitura paga em três lugares. Primeiro, explica por que a lambda é rápida: o `operator()` é conhecido em compilação, portanto pode ser inlinado, e o algoritmo que a recebe por parâmetro de template compila com a chamada aberta no lugar. Segundo, explica a captura: `[limite]` é um membro, e a lista de captura é a lista de inicialização daquele membro. Terceiro, explica `mutable`: o `operator()` é `const` por padrão, e é por isso que modificar um valor capturado por cópia não compila sem essa palavra.

— Captura por valor e por referência

`[x]` copia; `[&x]` guarda referência; `[=]` e `[&]` fazem o mesmo com tudo o que o corpo usar; `[this]` captura o ponteiro do objeto, o que dá acesso a todos os membros. A escolha não é de estilo, é de tempo de vida, e o critério é o mesmo do `string_view` do Cap. 3: **captura por referência é segura enquanto vive o escopo capturado**.

Uma lambda que só é usada dentro da função onde foi escrita, como argumento de um algoritmo que roda ali, pode capturar por referência sem risco, e é o caso da maioria. Uma lambda que é guardada para depois - numa fila de comandos, num observador, num `std::function` membro de classe - captura por referência algo que talvez já não exista quando ela for chamada, e o defeito não aparece na compilação nem no primeiro teste. A regra que o Deriva segue: guardou, copiou.

`[=]` merece uma advertência específica. Ele parece seguro porque copia, e captura `this` por **ponteiro** quando o corpo usa um membro, o que é captura por referência disfarçada: a lambda continua dependendo do objeto estar vivo. Capturar `[*this]`, que é C++17, copia o objeto inteiro e resolve o problema quando o objeto é pequeno.

— O que a lambda substitui

O lugar onde isso muda o projeto, e não só a digitação, é o Cap. 25. *Strategy*, no livro dos padrões e no Cap. 26 do v1, é uma classe base abstrata com um método virtual, uma derivada por algoritmo, e um `unique_ptr` no contexto: quatro declarações, um `vptr` por objeto, e uma indireção por chamada, para trocar uma função. Com lambda, a estratégia é o objeto invocável, o contexto a guarda por parâmetro de template ou por `std::function`, e cada algoritmo novo é uma expressão de uma linha no ponto de uso. O padrão continua sendo *Strategy*, com a mesma intenção; só a mecânica é outra, e é a que o C++ moderno usa.

§21.4 Algoritmos: Trocar o Laço pelo Nome do que Ele Faz

Seis trechos no fim deste capítulo cobrem o que o Deriva de fato usa, e todos os seis saem de `src/inventario.cpp`: `std::clamp` no construtor, o par `remove_if/erase` no descarte, `std::sort` com o desempate que o replay exige, `std::accumulate` com o zero que define o tipo da soma, `std::count_if` com o predicado vindo de fora, e `std::max_element` com a comparação contra `end()`. Vale saber o que **não** está lá: `std::any_of`, `std::all_of`, `std::none_of`, `std::transform` e `std::for_each` não aparecem em uma linha do sistema, e é o exercício 1 que traz o primeiro deles.

O argumento a favor do algoritmo não é a brevidade, é que **o nome diz a intenção**. `std::count_if` num laço de campo de visão informa, na primeira linha, que nada será modificado e que o resultado é um número; o laço equivalente com `for` e um acumulador exige ler o corpo inteiro para descobrir o mesmo. Sobre `std::sort`, há a garantia adicional de que a ordenação é a da biblioteca, testada por décadas, e não a que alguém escreveria às pressas.

Quatro algoritmos cobrem quase tudo o que o Deriva precisa, e vale saber o que cada nome promete. `for_each` e o laço `for` de intervalo fazem a mesma coisa, e o segundo é mais claro em C++ moderno. `count_if`, `any_of`, `all_of` e `none_of` respondem perguntas sem modificar nada, e o Deriva usa o primeiro, em `inventario::contar_se`. `transform` produz uma sequência a partir de outra. `accumulate`, que vive em `<numeric>` e não em `<algorithm>`, reduz a sequência a um valor.

O par `remove_if` e `erase` é o único que exige explicação, porque o nome mente um pouco. `remove_if` não remove nada: ele reorganiza os elementos, empurrando para o fim os que casam com o predicado, e devolve o iterador para o começo da parte a descartar. É `erase` que encurta o contêiner. Chamar `remove_if` sozinho compila, não emite aviso, e deixa o vetor com o mesmo tamanho e lixo no fim, o que é um dos defeitos mais comuns em código de quem está aprendendo a biblioteca.

E vale o contrário também, porque este capítulo não é campanha. Quando o laço tem duas condições de parada, ou modifica dois contêineres de uma vez, ou precisa do índice além do elemento, o `for` explícito é mais legível que a composição de três algoritmos com lambdas encadeadas. `mapa::despejar()` é assim: dois laços aninhados com índice, escrevendo caractere por caractere, e transformá-lo em `transform` não o tornaria mais claro.

§21.5 `std::clamp` e `std::size`

`std::clamp(v, lo, hi)` devolve `lo` se `v` é menor, `hi` se é maior, e `v` no resto. É C++17, vive em `<algorithm>`, e existe porque a alternativa manuscrita, `std::max(lo, std::min(v, hi))`, é fácil de escrever ao contrário e difícil de ler.

O alvo no Deriva é a v2.3, e são dois. O primeiro é manter a sonda dentro da grade quando o movimento é calculado antes de a colisão ser verificada, o que substitui um par de `if` por uma linha em que o limite aparece explícito. O segundo é a saturação de valores: energia e massa de célula têm faixa, e `clamp` é o nome dessa faixa no código.

▲ ATENÇÃO

`std::clamp` devolve **referência** ao menor, ao maior ou ao próprio valor, e não uma cópia. Escrever `const int& x = std::clamp(pos.x, 0, g.largura() - 1);` liga a referência a um temporário quando o resultado é um dos limites, e o temporário morre no fim da expressão. A forma segura é receber por valor - `const int x = std::clamp(...)` -, e é a única forma que aparece no Deriva. Há uma segunda condição, menos conhecida: `clamp` exige que o limite inferior não seja maior que o superior. Numa grade de largura zero, `std::clamp(x, 0, -1)` é comportamento indefinido, e não zero.

`std::size(c)`, também C++17, devolve o tamanho de qualquer coisa que tenha `size()` e de array cru, o que é a parte útil: a tabela de comandos de `src/main.cpp` é um array, e `std::size(tabela)` é a forma correta de contá-la. A forma antiga, `sizeof(tabela) / sizeof(tabela[0])`, funciona no array e devolve silenciosamente lixo quando o argumento decaiu para ponteiro, que é o defeito clássico de passar array para função e contá-lo lá dentro.

✓ DICA

Prefira `auto` para guardar uma lambda, e parâmetro de template para recebê-la. `std::function` apaga o tipo, o que é útil quando o contêiner precisa guardar objetos invocáveis de tipos diferentes; em troca, ele custa uma indireção por chamada, e possivelmente uma alocação, e impede o inlining que é a razão de a lambda ser rápida. A regra prática: `template <typename F> void para_cada_visivel(F f)` quando a função é chamada logo; `std::vector<std::function<void()>>` quando os invocáveis são guardados para depois, como na fila de comandos do Cap. 25.

Exercícios Propostos

1. Substitua o laço de contagem do campo de visão por `std::count_if` com lambda, e o teste de item adjacente por `std::any_of`. Confirme, com `make verifica`, que o replay continua idêntico byte a byte: é o oráculo do Cap. 16 aplicado a uma mudança que não deve alterar comportamento.
2. Escreva a lambda que ordena o inventário por massa decrescente e, em caso de empate, por nome. Guarde-a em `auto`, passe-a a `std::sort`, e depois tente declarar o tipo dela explicitamente. O que o compilador diz, e por quê?
3. Retire do inventário os itens de massa zero usando `remove_if` sozinho, sem `erase`. Imprima o tamanho do vetor antes e depois, e explique o que sobrou no

fim. Depois conserte com `erase` e explique por que a biblioteca separou as duas operações.

4. Escreva uma lambda que captura por referência uma variável local, guarde-a num `std::function` membro de uma classe, e chame-a depois de a função original ter retornado. Preveja o que acontece antes de rodar. Rode com `-Wall -Wextra` e verifique se o compilador avisou; explique por que ele não tem como saber.

5. Aplique `std::clamp` ao movimento da sonda e, em seguida, escreva a versão errada de propósito: ligue o resultado a um `const int&`. Rode. O que muda, e por que o defeito é intermitente?

6. Meça: percorra um `std::vector<celula>` de 100 mil elementos somando a energia, e faça o mesmo com um `std::list<celula>` de igual tamanho. Use `std::chrono`. Explique a diferença em termos de localidade de memória, e não de complexidade assintótica.

7. Reescreva a tabela de comandos de `src/main.cpp` como `std::unordered_map<std::string, vetor2>` e meça o tempo de leitura do roteiro nas duas versões. Diga qual é mais rápida, por quanto, e o que isso ensina sobre escolher contêiner pela tabela da §21.1.

O código, extraído do Deriva

Todo trecho abaixo vem de `exemplos/deriva/`, que compila com `-std=c++17 -Wall -Wextra -Wpedantic` sem um aviso e passa `make verifica`. Nenhum foi digitado neste texto.

```

src/inventario.cpp:10 |----- std::clamp em vez de min/max aninhado -----
10 inventario::inventario(int capacidade) noexcept
11     // `std::clamp` no lugar do min/max aninhado: a capacidade fica entre 0 e
12     // 999 sem que ninguém precise ler duas chamadas encaixadas para saber
13     // disso. Cuidado com o retorno por referência - aqui o resultado é
14     // copiado para um `int`, e é isso que o torna seguro.
15     : capacidade_(std::clamp(capacidade, 0, 999)) {}

```

`clamp` devolve **referência**, e é a armadilha dele: não o alimente com temporário guardando o resultado por referência. Aqui o valor é copiado para um `int`, e é isso que o torna seguro.

```

src/inventario.cpp:51 |----- erase-remove -----
51 std::size_t inventario::descartar_se(
52     const std::function<bool(const componente&>& criterio) {
53         // Erase-remove. Em C++20: std::erase_if(pecas_, ...), uma chamada.
54         const auto novo_fim = std::remove_if(
55             pecas_.begin(), pecas_.end(),
56             [&criterio](const std::unique_ptr<componente>& c) { return criterio(*c); });
57         const std::size_t saiu = static_cast<std::size_t>(pecas_.end() - novo_fim);
58         pecas_.erase(novo_fim, pecas_.end());
59         return saiu;
60     }

```

`remove_if` empurra para o fim e devolve o novo fim; `erase` corta. Em C++20 seria `std::erase_if`, uma chamada só - e é por isso que o material nomeia o idioma antigo em vez de fingir que ele é natural.

```

src/inventario.cpp:62 |----- o desempate que o replay exige -----
62 void inventario::ordenar_por_massa() {
63     std::sort(pecas_.begin(), pecas_.end(),
64         [](const std::unique_ptr<componente>& a,
65            const std::unique_ptr<componente>& b) {
66             // Desempate pelo rótulo: sem ele, a ordem entre massas iguais
67             // seria a que o `sort` quisesse, e o despejo deixaria de ser
68             // determinístico - o replay quebraria.
69             if (a->massa() != b->massa()) return a->massa() > b->massa();
70             return a->rotulo() < b->rotulo();
71         });
72 }

```

`std::sort` é instável. Sem o desempate pelo rótulo, a ordem entre massas iguais seria a que o algoritmo quisesse, e o despejo deixaria de ser determinístico.

```

src/inventario.cpp:24 |----- o zero que define o tipo da soma -----
24 int inventario::massa_total() const {
25     // `accumulate` com lambda: o zero inicial é `int`, e é ele que define o tipo
26     // da soma. Passar `0.0` daria soma em double sem ninguém pedir.
27     return std::accumulate(pecas_.begin(), pecas_.end(), 0,
28         [](int soma, const std::unique_ptr<componente>& c) {
29             return soma + c->massa();
30         });
31 }

```

Passar `0.0` daria soma em double sem ninguém pedir, e o truncamento apareceria três capítulos depois.

```
src/inventario.cpp:44 |----- o predicado vem de fora-----
44 std::size_t inventario::contar_se(
45     const std::function<bool(const componente&)>& criterio) const {
46     return static_cast<std::size_t>(std::count_if(
47         pecas_.begin(), pecas_.end(),
48         [&criterio](const std::unique_ptr<componente>& c) { return criterio(*c); }));
49 }
```

A função serve sem saber o que se vai perguntar, e é isso que a torna útil - acrescentar um critério não a edita.

```
src/inventario.cpp:35 |----- o iterador de fim não é elemento-----
35 const componente* inventario::mais_pesada() const {
36     const auto it = std::max_element(
37         pecas_.begin(), pecas_.end(),
38         [](const std::unique_ptr<componente>& a, const std::unique_ptr<componente>& b) {
39             return a->massa() < b->massa();
40         });
41     return it == pecas_.end() ? nullptr : it->get();
42 }
```

`max_element` devolve `end()` quando a faixa é vazia, e desreferenciar isso é comportamento indefinido. A comparação com `end()` não é cerimônia.

Concorrência em C++ - panorâmica

O QUE ESTE CAPÍTULO ENTREGA

- ▶ Criar e aguardar threads com `std::thread`, e reconhecer o que acontece sem `join`.
- ▶ Dizer o que as threads compartilham e o que cada uma tem só para si.
- ▶ Explicar a corrida de dados sobre o contador vivos, incremento por incremento.
- ▶ Proteger dado compartilhado com `std::mutex`, `std::lock_guard` e `std::scoped_lock`.
- ▶ Reconhecer o que a concorrência cobra do projeto orientado a objetos.
- ▶ Situar o que fica para a disciplina de Programação Concorrente.

Este capítulo é panorâmica, e a palavra é literal: são duas horas, não há laboratório, e o plano de ensino registra que, havendo perda de encontro no semestre, esta é a primeira aula a virar leitura dirigida. O motivo não é que o assunto seja menor, é que ele tem disciplina própria. **Programação Concorrente** cobre o modelo de memória, atômicos, variáveis de condição, produtor-consumidor e conjuntos de threads, com o rigor que nada disso cabe aqui.

O que esta aula deve, e é o que justifica ela existir dentro de POO, é uma constatação sobre objetos: **encapsulamento não protege contra concorrência**. Uma classe com todos os atributos privados, todas as invariantes estabelecidas no construtor e todos os métodos corretos pode estar errada no instante em que duas threads a usam, e nada no texto da classe muda. O detector da disciplina serve de prova, e é por isso que este capítulo o usa como exemplo.

§22.1 `std::thread` - Criação e Join

Uma `std::thread` é um objeto que possui uma linha de execução. O construtor recebe o invocável e os argumentos, e a thread começa a rodar imediatamente, em paralelo com quem a criou; `join()` bloqueia até ela terminar; `detach()` desfaz a relação e deixa a thread solta.

Duas coisas da passagem de argumento merecem atenção, porque as duas produzem defeito silencioso. A primeira é que os argumentos são **copiados** por padrão, mesmo que o parâmetro da função seja referência; passar por referência de verdade exige `std::ref`, e passar por referência constante exige `std::cref`. A segunda é a consequência disso: se você usa `std::ref` para uma variável local e a função que criou a thread retorna antes do `join`, a thread passa a escrever num objeto destruído. É o mesmo risco da captura de lambda por referência do Cap. 21, com a agravante de que aqui o momento da falha depende do escalonador.

O par de threads que o Deriva tem está extraído no fim deste capítulo, e são dois trechos que se leem juntos. O primeiro é a declaração de `exercitar_fila`, com o comentário que responde à pergunta que a maioria não faz: por que duas threads

não implicam saída indeterminada. A resposta é que a fila é FIFO e o consumidor é um só, e com dois consumidores a ordem deixaria de ser garantida - e o replay do Cap. 16 pararia de servir. O segundo é o corpo da função, onde a thread produtora é construída com lambda e a captura por referência é segura, porque o join acontece antes de o escopo fechar.

Vale registrar o que o sistema não tem, para que ninguém procure: `src/main.cpp` não cria thread nenhuma. A thread de entrada da v2.4 é a fronteira desenhada na §22.5, e o que existe em código é a fila que a atravessa, exercitada por um produtor sintético em lugar do teclado. A escolha é do portão: teste que depende de tecla digitada não roda no ctest, e a fila é a parte da qual o material precisa afirmar algo.

▲ ATENÇÃO

`std::thread` que sai de escopo sem join nem detach chama `std::terminate`. Não é vazamento silencioso nem thread órfã: o programa morre, e a escolha da linguagem foi deliberada, porque a alternativa - destruir um objeto de thread enquanto a thread roda - não tem semântica defensável. É o mesmo raciocínio do destrutor do Cap. 8, aplicado a um recurso que é uma linha de execução: o destrutor exige que a posse tenha sido resolvida antes.

§22.2 O que as Threads Compartilham

A resposta curta é: quase tudo. Threads do mesmo processo veem o mesmo espaço de endereçamento, e portanto compartilham o heap, as variáveis globais, os membros `static` de qualquer classe, e todo objeto alcançável por ponteiro ou referência que uma delas passe à outra. O que cada thread tem só para si é a pilha, e com ela as variáveis locais e os parâmetros.

Essa lista, lida com o Cap. 7 em mente, já entrega o problema deste capítulo. `inline static int vivos` é membro estático, o que significa que **existe uma única variável para todos os objetos da classe** - foi essa propriedade que a tornou útil como contador. A mesma propriedade a torna compartilhada por todas as threads, sem que ninguém tenha escrito uma linha sobre concorrência.

§22.3 A Corrida de Dados sobre o Contador vivos

O comentário de cabeçalho de `include/deriva/contador.hpp` traz, desde a v0.1, a frase que esta seção cobra: “Não é seguro entre threads. A Aula 22 mostra exatamente esta variável perdendo um incremento.” O interativo desta aula é essa demonstração, e ele vem da disciplina de Laboratório de Programação II com a legenda trocada, que é o primeiro reaproveitamento de peça entre as duas ementas.

O mecanismo é o que interessa, e ele é mais simples do que a intuição sugere. `++vivos` **não** é uma operação; são três: ler o valor da memória para um registrador, somar um, escrever de volta. Duas threads que executam as três etapas de forma entrelaçada podem ambas ler 7, ambas somar um, e ambas escrever 8. Dois objetos foram construídos, o contador subiu um, e nada no programa acusa. Quando os dois destrutores rodarem, `vivos` fechará em menos um.

O que faz deste o exemplo certo para a disciplina é a inversão que ele produz. Por dezenove capítulos, vivos foi o oráculo: fechou em zero, ninguém vazou; não fechou, um destrutor não rodou. A quarta condição de `make verifica` é exatamente essa leitura. Aqui o oráculo mente, e mente nas duas direções - pode acusar vazamento onde não houve, ou fechar em zero com objeto vivo -, porque o instrumento passou a ter o mesmo defeito que ele existe para detectar. Instrumento é código, e código compartilhado sem proteção quebra igual.

Há três respostas possíveis, e vale saber por que a terceira é a do Deriva.

A primeira é trocar `int` por `std::atomic<int>`, o que torna o incremento indivisível e resolve o caso. Custa uma instrução mais cara em cada construção e destruição de objeto, o que é irrelevante aqui, e resolve só o contador: criados e vivos continuariam sendo dois atômicos separados, e ler os dois em sequência ainda daria um par inconsistente.

A segunda é proteger com mutex, o que funciona e é desproporcional para somar um.

A terceira, que é a que o desenho da v2.4 adota, é **não compartilhar**. A thread de entrada não constrói nem destrói objeto de domínio: ela lê tecla e enfileira um comando. Quem constrói entidade, avança turno e destrói é a thread principal, sozinha. Assim o contador continua sendo `int` simples, continua correto, e o programa não paga nada - e a lição registrada é que a melhor correção de uma corrida de dados costuma ser eliminar o compartilhamento, e não sincronizá-lo.

Repare que os parágrafos acima descrevem o mecanismo e não dizem **quantos** incrementos se perdem, e a omissão é a lição, e não uma lacuna. A medição existe: `include/deriva/medida_corrida.hpp` roda o mesmo par de threads sem proteção várias vezes e devolve o mínimo, o máximo e quantas execuções não perderam nada, e o trecho está extraído no fim deste capítulo. O que ela produz é uma **faixa**, que muda de uma execução para a outra e de uma máquina para a outra, porque corrida de dados é comportamento indefinido, e comportamento indefinido não tem valor esperado a publicar. Um número único no texto seria falsa precisão; a faixa, se for citada, vem com a máquina e a data ao lado, medidas no laboratório.

A consequência disso chega ao teste, e é o segundo trecho a ler: sobre a versão sem proteção não há o que afirmar, e por isso `testes/test_concorrenca.cpp` apenas mede e relata. O que ele afirma é o outro lado, a versão com `scoped_lock`, cuja conta fecha em toda execução.

§22.4 `std::mutex`, `std::lock_guard` e `std::scoped_lock`

Um `std::mutex` é o objeto que garante que apenas uma thread execute uma região por vez. Trancar e destrancar à mão, com `lock()` e `unlock()`, funciona e é errado pelo mesmo motivo que liberar memória à mão é errado: qualquer retorno antecipado, e qualquer exceção, pula o `unlock` e deixa o mutex trancado para sempre.

A resposta é RAIL, outra vez, e é a terceira aparição do mesmo idioma neste livro. `std::lock_guard` tranca no construtor e destranca no destrutor, e portanto destranca também quando uma exceção desenrola a pilha, o que é a garantia do Cap. 20. Escrever um bloco `try/catch` para destrancar é a versão manual daquilo que o destrutor faz de graça.

O par de comparação está extraído no fim deste capítulo, e são as mesmas duas funções com uma linha de diferença: `contar_sem_mutex`, com o comentário que decompõe ++vivos em ler, somar e escrever, e `contar_com_mutex`, com a seção crítica serializada. Ao lado delas ficam as duas pontas da fila da v2.4, `empurrar` e `puxar`, que é onde o idioma aparece em código de produção em lugar de em código de demonstração - com o `notify_one` fora da região trancada, e o predicado no `wait` que não é conveniência.

O que essas funções usam é `std::scoped_lock`, que é **C++17**, e não o `std::lock_guard` de C++11 que aparece na maior parte do material que você vai encontrar. É a escolha padrão a partir daqui, por duas razões: ele aceita mais de um mutex de uma vez, trancando-os em ordem que evita o abraço mortal, e dispensa a lista de tipos, porque deduz. Onde há um mutex só, os dois são equivalentes; onde há dois, `scoped_lock` é a diferença entre o programa que funciona e o que trava na terça-feira.

O abraço mortal, ou *deadlock*, é o defeito complementar da corrida. Duas threads, dois mutexes, e cada uma tranca um e espera pelo outro: nenhuma avança, e o programa não morre nem prossegue. A regra que o evita é dar uma **ordem total** aos mutexes e sempre trancar nessa ordem; `std::scoped_lock` com os dois mutexes de uma vez faz isso por você.

✓ DICA

Antes de tornar uma classe segura entre threads, pergunte se ela precisa ser compartilhada. Classe com mutex dentro paga o travamento em toda chamada, inclusive nas de uma thread só, e o mutex ainda não garante nada sobre sequências de duas chamadas: `if (!inv.cheio()) inv.acrescentar(x);` é uma corrida mesmo que os dois métodos sejam individualmente protegidos, porque a decisão foi tomada com informação que expirou. A alternativa quase sempre melhor é confinar: um dono por dado, comunicação por fila, e nenhum objeto de domínio compartilhado. É o desenho da v2.4 do Deriva.

§22.5 O que a Concorrência Cobra do Projeto Orientado a Objetos

Três consequências valem registro, porque as três aparecem em decisão de projeto e nenhuma é sobre sintaxe.

A primeira é que **const deixa de significar só “não modifica”**. Na biblioteca padrão, e por convenção que vale seguir, um método `const` é o que se pode chamar de várias threads ao mesmo tempo sem sincronização. Isso muda o que se pode fazer dentro dele: um método `const` que guarda resultado calculado em um membro precisa marcar esse membro como `mutable` e protegê-lo com mutex, e o mutex por sua vez tem de ser `mutable`, porque trancá-lo modifica o objeto. Cache preguiçoso, que é inofensivo numa thread, é dado compartilhado em duas.

A segunda é que **a granularidade da invariante muda**. Encapsulamento garante que cada método deixa o objeto consistente; nada garante que dois métodos chamados em sequência vejam o mesmo estado. Verificar e agir são duas operações, e entre elas o mundo mudou, o que é a mesma estrutura da corrida entre `exists` e `abrir` do Cap. 20, agora com outra thread no lugar do sistema de arquivos. A correção não está em proteger cada método: está em oferecer a operação composta, `acrescentar_se_couber`, como um método só.

A terceira é que **a fronteira do compartilhamento é decisão de arquitetura**. O Deriva escolhe uma: o núcleo de domínio não conhece thread nenhuma, e a única coisa que atravessa a fronteira é uma fila de comandos. É a mesma separação que o Cap. 26 usa para pôr Qt sobre o mesmo núcleo, e ela paga duas vezes pelo mesmo desenho.

§22.6 A Ponte com Programação Concorrente

O que fica desta aula é reconhecimento de API e um vocabulário: thread, junção, corrida de dados, região crítica, mutex, abraço mortal, e a distinção entre corrida de dados e condição de corrida. Não fica competência em escrever código concorrente, e o material não finge que fica.

O que a disciplina de Programação Concorrente desenvolve, e que aqui só se nomeia: o modelo de memória do C++ e as ordens de consistência, que é o que explica **por que** ler e escrever sem sincronização é comportamento indefinido, e não apenas resultado imprevisível; `std::atomic` e as operações de leitura-modificação-escrita; `std::condition_variable` e o problema do produtor-consumidor, que é o que a fila da v2.4 resolve na versão simples; conjuntos de threads e escalonamento de tarefas; e as ferramentas de detecção, entre elas o `-fsanitize=thread` que o laboratório não tem e que aparece em aula, na máquina de projeção.

O interativo de corrida de dados é compartilhado entre as duas disciplinas, com a legenda trocada: aqui ele mostra o contador vivos perdendo incremento, lá ele mostra o mesmo mecanismo sem o contexto do detector de vazamento. Uma peça, dois cursos.

Exercícios Propostos

1. Escreva um programa que cria duas threads, cada uma construindo e destruindo mil objetos de uma classe com o contador vivos escrito à mão, e imprime vivos no fim. Rode dez vezes e registre os dez valores. Explique por que eles diferem, e por que alguns podem ser exatamente zero.
2. Corrija o exercício 1 de três formas: com `std::atomic<int>`, com `std::mutex`, e eliminando o compartilhamento. Compare as três em uma frase cada, e diga qual delas o Deriva adota e por quê.
3. Crie uma `std::thread` e deixe-a sair de escopo sem `join` nem `detach`. Preveja o que acontece antes de rodar. Depois leia a mensagem de terminação e explique por que a linguagem escolheu morrer em vez de desanexar sozinha.
4. Passe uma variável local a uma thread com `std::ref`, retorne da função que a criou sem `join`, e chame `join` depois, do chamador. Explique por que isso é comportamento indefinido, e por que o programa pode passar em todos os testes.
5. Escreva o abraço mortal mínimo: dois mutexes, duas threads, cada uma trancando na ordem oposta. Confirme que o programa trava. Conserte de duas maneiras, primeiro impondo ordem total aos mutexes e depois com um `std::scoped_lock` que recebe os dois, e diga o que a segunda forma dispensa você de lembrar.
6. Dado um inventário com `cheio()` e `acrescentar()`, ambos protegidos por mutex interno, escreva a sequência de duas chamadas que ainda produz erro com

duas threads. Conserte oferecendo a operação composta, e explique por que proteger cada método não bastava.

O código, extraído do Deriva

Todo trecho abaixo vem de `exemplos/deriva/`, que compila com `-std=c++17 -Wall -Wextra -Wpedantic` sem um aviso e passa `make verifica`. Nenhum foi digitado neste texto.

```

| include/deriva/fila_de_comandos.hpp:13 | a fila é a única fronteira |
13 /// A fila entre a thread que lê o teclado e a que desenha.
14 ///
15 /// **0 que se compartilha é esta fila, e nada mais.** É a decisão de projeto
16 /// da Aula 22, e ela é o oposto da tentação: seria mais fácil deixar as duas
17 /// threads mexerem no `mundo`, e é exatamente isso que produz a corrida que o
18 /// interativo mostra. Aqui o `mundo` continua sendo de uma thread só; o que
19 /// atravessa a fronteira são comandos, um por vez, protegidos.
20 ///
21 /// `std::scoped_lock` (C++17) no lugar de `std::lock_guard`: aceita mais de um
22 /// mutex e resolve a ordem de travamento sozinho, o que elimina uma classe
23 /// inteira de impasse. Para um mutex só, os dois são equivalentes, e usar o
24 /// novo é hábito.
25 ///
26 /// A condição de parada não é uma variável booleana lida sem proteção - esse é
27 /// o erro que a Aula 22 mostra medido. Ela é parte do estado guardado pelo
28 /// mesmo mutex, e a espera usa `condition_variable`, que acorda quem espera em
29 /// vez de queimar processador.
30 class fila_de_comandos {
31 public:
32     /// Põe um comando na fila e acorda quem estiver esperando.
33     void empurrar(std::string comando);
34
35     /// Bloqueia até haver comando ou a fila ser fechada. `std::nullopt` quando
36     /// fechada e vazia - é o sinal de fim, e não uma exceção.
37     [[nodiscard]] std::optional<std::string> puxar();
38
39     /// Tira sem bloquear. Serve ao laço de render, que não pode parar.
40     [[nodiscard]] std::optional<std::string> tentar_puxar();
41
42     /// Fecha para sempre e acorda todos. Idempotente.
43     void fechar();
44
45     [[nodiscard]] bool fechada() const;
46     [[nodiscard]] std::size_t tamanho() const;
47
48 private:
49     mutable std::mutex mutex_;
50     std::condition_variable tem_coisa_;
51     std::deque<std::string> fila_;
52     bool fechada_ = false;
53 };

```

O mundo continua sendo de uma thread só. O que atravessa a fronteira são comandos, um por vez, protegidos - e é o oposto da tentação de deixar as duas threads mexerem no mundo.

```

src/fila_de_comandos.cpp:19 |----- o predicado não é conveniência-----
19 std::optional<std::string> fila_de_comandos::puxar() {
20     std::unique_lock trava(mutex_);
21     // O predicado no `wait` não é conveniência: sem ele, um despertar espúrio
22     // faria a thread seguir com a fila vazia. Ele é reavaliado a cada despertar,
23     // e é o que torna a espera correta.
24     tem_coisa_.wait(trava, [this] { return !fila_.empty() || fechada_; });
25     if (fila_.empty()) return std::nullopt;    // fechada e vazia: fim
26     std::string c = std::move(fila_.front());
27     fila_.pop_front();
28     return c;
29 }

```

Sem o predicado no wait, um despertar espúrio faria a thread seguir com a fila vazia. E notificar fora da região travada economiza uma ida e volta no escalonador.

```

testes/test_concorrenzia.cpp:51 |----- o que não se pode afirmar-----
51 // ATENCAO ao que este teste NAO afirma, e por que.
52 //
53 // A primeira versao dele exigia `r.perdidos_max > 0`: que a corrida se
54 // manifestasse. Ela falhou no portao, e com razao - a medida anterior mostrou
55 // oito execucoes de dez sem perda alguma. Um teste que depende de
56 // comportamento indefinido se manifestar e um teste INSTAVEL, e teste instavel
57 // e pior que teste ausente: ele treina a equipe a reexecutar o portao ate
58 // passar.
59 //
60 // A licao e essa mesma. Sobre a versao sem mutex nao ha o que afirmar, e por
61 // isso o teste apenas MEDE e relata. O que se afirma e o outro lado: com
62 // `scoped_lock` a conta fecha em toda execucao, sempre, e e disso que um teste
63 // pode falar.
64 TEST_CASE("com scoped_lock a conta fecha sempre; sem ele, nao ha o que afirmar") {
65     SECTION("a versao protegida e exata em toda execucao") {
66         for (int k = 0; k < 8; ++k) {
67             REQUIRE(medida::contar_com_mutex(25000) == 50000);
68         }

```

A primeira versão deste teste exigia que a corrida se manifestasse, e falhou no portão - oito execuções de dez não perdem nada. Teste que depende de comportamento indefinido é teste instável, e instável é pior que ausente: treina a equipe a reexecutar até passar.

```

┌ include/deriva/fila_de_comandos.hpp:55 ─────────── duas threads, saída determinística ───────────
55 /// Roda `quantos` comandos por uma fila, de uma thread produtora e uma
56 /// consumidora, e devolve a ordem em que foram consumidos.
57 ///
58 /// Determinístico apesar de haver duas threads, e é isso que a Aula 22 quer
59 /// mostrar: concorrência não implica indeterminismo. A ordem é preservada
60 /// porque a fila é FIFO e há um consumidor só. Com dois consumidores, a ordem
61 /// deixaria de ser garantida - e aí o replay não serviria mais.
62 [[nodiscard]] std::string exercitar_fila(int quantos);
└──────────────────────────────────────────────────────────────────────────────────────────────────────────┘

```

Concorrência não implica indeterminismo: a fila é FIFO e há um consumidor só. Com dois consumidores a ordem deixaria de ser garantida, e o replay não serviria mais.

```

┌ src/fila_de_comandos.cpp:57 ─────────── produtora e consumidora ───────────
57 std::string exercitar_fila(int quantos) {
58     fila_de_comandos fila;
59     std::string consumido;
60
61     std::thread produtora(&fila, quantos) {
62         for (int i = 0; i < quantos; ++i) fila.empurrar("cmd" + std::to_string(i));
63         fila.fechar();
64     };
65
66     // O consumidor é um só, e é isso que preserva a ordem. Com dois, a saída
67     // deixaria de ser determinística e o replay não serviria.
68     while (const std::optional<std::string> c = fila.puxar()) {
69         consumido.append(*c).push_back(' ');
70     }
71     produtora.join();
72     return consumido;
└──────────────────────────────────────────────────────────────────────────────────────────────────────────┘

```

A captura por referência é segura porque o join acontece antes de o escopo fechar. Sem ele, seriam referências a objetos destruídos - e o join esquecido é o defeito mais comum de quem começa.

```

src/fila_de_comandos.cpp:8 |----- notificar fora da região travada
8 void fila_de_comandos::empurrar(std::string comando) {
9     {
10         const std::scoped_lock trava(mutex_);
11         if (fechada_) return;
12         fila_.push_back(std::move(comando));
13     }
14     // Notificar FORA da região travada: quem acorda tentaria travar de imediato,
15     // e acordar antes de destravar custa uma ida e volta a mais no escalonador.
16     tem_coisa_.notify_one();
17 }

```

Quem acorda tentaria travar de imediato, e acordar antes de destravar custa uma ida e volta a mais no escalonador. O escopo interno do `scoped_lock` existe para isso.

```

include/deriva/medida_corrida.hpp:32 |----- `vivos++` em duas threads
32 /// `vivos` compartilhado, sem proteção, incrementado por duas threads.
33 [[nodiscard]] inline int contar_sem_mutex(int por_thread) {
34     int vivos = 0;
35     auto somar = [&vivos, por_thread] {
36         for (int i = 0; i < por_thread; ++i) {
37             ++vivos;           // lê, soma, escreve: três passos, nenhum atômico
38         }
39     };
40     std::thread a(somar), b(somar);
41     a.join();
42     b.join();
43     return vivos;
44 }

```

Três passos - lê, soma, escreve -, e nenhum deles atômico. É o mesmo contador da Aula 07, e é ele que perde o incremento.

```

46 /// 0 mesmo, com a seção crítica serializada. `scoped_lock` é C++17 e
47 /// substitui `lock_guard` por aceitar mais de um mutex.
48 [[nodiscard]] inline int contar_com_mutex(int por_thread) {
49     int vivos = 0;
50     std::mutex m;
51     auto somar = [&vivos, &m, por_thread] {
52         for (int i = 0; i < por_thread; ++i) {
53             const std::scoped_lock trava(m);
54             ++vivos;
55         }
56     };
57     std::thread a(somar), b(somar);
58     a.join();
59     b.join();
60     return vivos;
61 }

```

`std::scoped_lock` é C++17 e aceita mais de um mutex, resolvendo a ordem de travamento sozinho - o que elimina uma classe inteira de impasse. Para um mutex só é equivalente ao `lock_guard`, e usar o novo é hábito.

```

63 [[nodiscard]] inline corrida medir(int execucoes = 20, int por_thread = 100000) {
64     corrida r;
65     r.execucoes = execucoes;
66     r.esperado = 2 * por_thread;
67     r.perdidos_min = r.esperado;
68     for (int k = 0; k < execucoes; ++k) {
69         const int perdidos = r.esperado - contar_sem_mutex(por_thread);
70         if (perdidos < r.perdidos_min) r.perdidos_min = perdidos;
71         if (perdidos > r.perdidos_max) r.perdidos_max = perdidos;
72         if (perdidos == 0) ++r.execucoes_sem_perda;
73     }
74     return r;
75 }

```

Corrida é comportamento indefinido, e o resultado varia entre execuções. O que se mede é a distribuição, e é ela a lição: oito execuções de dez não perderam nada.

Serialização

O QUE ESTE CAPÍTULO ENTREGA

- ▶ Reconhecer o despejo determinístico do Cap. 16 como o serializador que o Deriva já tem.
- ▶ Escrever e ler um formato de texto com cabeçalho e número de versão.
- ▶ Manter compatibilidade regressiva ao acrescentar campo, e saber o que a quebra.
- ▶ Restabelecer a invariante da classe na desserialização, através do construtor.
- ▶ Escrever o teste de ida e volta como especificação do formato.
- ▶ Situar JSON e formato binário, e dizer o que cada um custa.

Este capítulo começa com uma constatação que economiza metade do trabalho: **o Deriva já serializa, desde a v0.3, e o fez por outro motivo.** `mapa::despejar()` produz o texto do setor - cabeçalho com nome e dimensão, linha de entrada, e uma fileira de glifos por linha da grade -, e ele existe porque o replay do Cap. 16 precisa comparar duas execuções byte a byte. Um formato determinístico de leitura humana já estava lá, construído para servir de oráculo de teste.

O que a v2.5 acrescenta é o resto do estado. `despejar()` grava o mapa e não grava a sonda, o inventário, o turno nem a semente do sorteio; quem carrega o despejo obtém o setor e perde a partida. Serializar, aqui, é responder a uma pergunta de projeto antes de qualquer sintaxe: **qual é o estado mínimo a partir do qual a partida continua idêntica?**

A resposta tem uma propriedade que vale notar. Se a partida guarda a semente e o número do turno, e se todo sorteio deriva daquela semente, então o arquivo salvo é equivalente ao prefixo de um replay: carregar é o mesmo que reexecutar o roteiro até ali. É o determinismo do Cap. 16 pago pela segunda vez, e é o que permite que o teste de ida e volta seja tão curto.

§23.1 Estratégias de Serialização

Estratégia	Vantagens	Desvantagens
Texto puro	Simples, legível, depurável	Parsing manual, sem schema
JSON (nlohmann/json)	Legível, amplamente suportado, bibliotecas maduras	Verboso, sem tipos binários
Binário (Boost.Serialization)	Compacto, eficiente, versionamento	Não legível por humanos, dependência

O Deriva escolhe a primeira linha, e as razões são de disciplina antes de serem de engenharia.

A primeira é que o formato de texto se compara com diff. Quando o replay da v2.6 acusar que a refatoração mudou a saída, o que o estudante vê é a linha que mudou, e não que dois arquivos binários diferem em algum lugar. A caça ao bug 3 depende disso para ser conduzível em duas horas.

A segunda é que o texto se escreve à mão. Construir o caso de teste da partida corrompida, da versão futura, da fileira torta, é editar um arquivo num editor; em binário, seria escrever um gerador de casos, que é código a mais para manter e para depurar.

A terceira é o portão de dependências. O `CMakeLists.txt` do Deriva fixa FTXUI e Catch2 por `FetchContent`, e cada dependência nova é uma versão a fixar, uma inclusão `SYSTEM` a declarar para que o aviso dela não conte, e um risco de o laboratório não compilar na segunda-feira. Formato de texto próprio custa umas dezenas de linhas de leitura e escrita, e zero dependência.

O que a escolha custa é honesto dizer: sem esquema declarado, a validação é escrita à mão; e o arquivo é maior que o binário equivalente, o que não importa para uma estação de 80 por 24.

§23.2 O Formato da Partida Salva

O formato da v2.5 é este, e ele existe: `include/deriva/partida.hpp` declara a `struct partida`, e `src/partida.cpp` implementa as duas pontas. O trecho de escrita está extraído no fim deste capítulo.

A primeira linha é o **cabeçalho**, e traz duas coisas: a marca `deriva-partida`, que identifica o arquivo, e o número de versão do formato. A marca existe para que abrir o arquivo errado dê um erro claro em vez de um estado absurdo; a versão existe para tudo o que a §23.3 trata, e ela é a **primeira** coisa gravada, porque quem lê precisa saber com que regras ler antes de ler qualquer outra coisa.

Seguem os campos, um por linha, no formato `chave valor`: o setor, a posição da sonda, a carga, o turno e a energia. Aqui há uma assimetria deliberada entre as duas pontas. A escrita tem ordem fixa, e não é detalhe: ordem estável é o que permite comparar dois saves com diff, que é a mesma razão do replay. A leitura, porém, casa por **chave** e não por posição, num laço que aceita os campos em qualquer ordem - e é essa escolha que a §23.4 vai cobrar, porque ela muda quais mudanças de formato quebram a compatibilidade.

Duas decisões merecem estar escritas no arquivo, e não na cabeça de quem o escreveu. A primeira é que os números vão em decimal, sem separador de milhar e sem localidade, porque `std::ostream` respeita a localidade do sistema e uma vírgula decimal em vez de ponto quebra o carregamento na máquina do colega. A segunda é que o desenho do setor **não** vai no arquivo: o que vai é o nome dele, num campo só. É escolha, e o preço está declarado - um setor editado depois do save carrega diferente do que foi salvo -, e a alternativa, gravar o mapa inteiro, é o caso de acrescentar campo no fim que a §23.4 trata.

Duas coisas sobre o alcance da v2.5 valem estar escritas aqui, porque as duas são escolha e não omissão.

A primeira é que as funções que existem operam sobre **texto em memória**, e não sobre arquivo: `serializar()` devolve uma `std::string`, e `desserializar(std::string_view)` recebe uma. É a mesma separação de `mapa::de_texto` e `mapa::carregar` do Cap. 20, e pela mesma razão, que é o teste - teste que depende do

sistema de arquivos é teste frágil, e os três trechos extraídos no fim deste capítulo rodam sem tocar em disco. As duas pontas de arquivo são camada por cima dessas, com o `std::error_code` e a corrida entre verificar e abrir que a §20.6 já discutiu, e é o exercício 1 que as pede.

A segunda é o sentido em que a partida se converte. `partida::de(const mundo&, int)` extrai o estado de um mundo em execução, e essa é a direção que a v2.5 entrega; a volta, isto é reconstruir o mundo a partir de uma partida, é o que o aviso abaixo tem por alvo e o que o exercício 4 cobra, justamente porque é ali que a invariante da grade pode ser violada por um arquivo escrito à mão. Fazer as duas direções ao mesmo tempo esconderia a assimetria: salvar não constrói nada, e carregar constrói tudo.

▲ ATENÇÃO

Desserializar escrevendo direto nos atributos é o caminho mais curto para um objeto que compila, roda e viola a própria invariante. Uma partida salva à mão, ou corrompida pela metade, pode trazer largura 40 e trinta e nove fileiras; se o carregamento preencher os campos um a um, a grade passa a ter menos células do que largura × altura promete, e o primeiro acesso à última fileira lê memória que não é dela. A regra é a do Cap. 8: o construtor é o único lugar onde a invariante se estabelece, e o desserializador reúne os dados, valida, e **chama o construtor** - ou devolve erro sem construir nada.

§23.3 Versionamento de Formato

Todo formato que sobrevive a dois semestres muda, e a decisão a tomar não é se vai mudar, é o que acontece com os arquivos gravados antes da mudança. Sem número de versão, a resposta é que eles param de carregar, ou pior, carregam errado.

O número de versão no cabeçalho estabelece três regras, e o leitor precisa implementar as três.

O leitor aceita a sua versão e as anteriores que ainda conhece. É isso que compatibilidade regressiva significa: uma partida salva na v2.5 continua carregando na v2.7. O custo é um ramo de leitura por versão suportada, e é por isso que a lista não cresce para sempre - em algum momento se declara que versões abaixo de N não são mais lidas, e essa é uma decisão documentada, e não um defeito.

O leitor recusa versão maior que a sua. Arquivo gravado por uma versão futura pode ter campos que este código não sabe interpretar, e adivinhar é pior que falhar. O desserializar do Deriva recusa devolvendo `std::nullopt`, e a escolha é a da §20.7 aplicada a este caso: save que não serve é **ausência de resultado**, e não falha, porque o arquivo que o estudante edita à mão vai estar errado muitas vezes ao longo do semestre, e o que acontece com frequência esperada não é excepcional. O comentário de `include/deriva/partida.hpp` declara isso em uma linha, e a hierarquia de erro. `hpp` continua com as duas folhas que o Cap. 20 lhe deu, sem uma terceira para estado inválido.

O preço da escolha é real e vale nomeá-lo: `nullopt` não diz **qual** versão o arquivo trazia, e uma mensagem útil precisaria dizer as duas, a do arquivo e a do programa. O lugar dessa mensagem é a camada de cima, a que abre o arquivo e por isso conhece o caminho, e não a que interpreta texto - exatamente a divisão que `mapa::carregar` e `mapa::de_texto` já estabeleceram. O exercício 5 é onde você a escreve.

O escritor grava apenas a versão corrente. Escrever formato antigo “para compatibilidade” dobra o código de escrita e produz, invariavelmente, um dos dois caminhos sem teste.

§23.4 O que Quebra a Compatibilidade Regressiva

Quais mudanças são seguras depende de como o leitor casa os campos, e é aqui que a escolha da §23.2 se paga. Num formato lido **por posição**, acrescentar campo no fim é seguro, acrescentar no meio quebra tudo, e reordenar quebra em silêncio, com os valores do tipo certo e do significado errado. O Deriva lê **por chave**, e a lista fica outra.

Acrescentar campo, em qualquer lugar, é seguro nas duas direções, e testes/`test_partida.cpp` prova as duas. O leitor novo abre o arquivo antigo, e o valor padrão do membro responde pelo campo ausente: é o caso de energia, que a versão 2 acrescentou, e o trecho está extraído no fim deste capítulo. O leitor antigo abre o arquivo novo, porque chave que ele não conhece é **pulada** em vez de recusada, e é essa decisão que separa formato extensível de formato frágil. Remover campo é seguro pela mesma razão, e reordenar não quebra nada, porque na leitura a posição não significa nada.

O que quebra, e quebra em silêncio, é reinterpretar. Reaproveitar o nome de um campo para outro significado produz um arquivo que carrega e está errado, e ninguém suspeita do nome, justamente porque o nome continua o mesmo. Mudar a unidade de um campo - massa em unidades para massa em décimos - sem incrementar a versão é o mesmo defeito, e ele aparece meses depois, num arquivo salvo antes da mudança.

A regra é curta, e o casamento por chave a torna mais curta ainda: **campo é acrescentado, nunca reinterpretado; e toda reinterpretação incrementa a versão.**

§23.5 A Invariante, o Contador, e o Teste de Ida e Volta

Desserializar é construir objeto, e portanto todo o Cap. 8 se aplica. O carregamento reúne os dados lidos, valida o que a classe exige, e chama o construtor; se a validação falha, nada é construído e o erro sobe. Preencher atributo a atributo, atravessando o construtor, é o que o aviso acima trata.

Há uma consequência que só esta disciplina cobra, e é boa notícia: **o contador de instâncias vivas verifica o carregamento de graça**. Reconstruir o mundo a partir de uma partida - a direção que a §23.2 deixou para o exercício 4 - vai construir um mapa e entidades; salvar não constrói nada. Se o carregamento cria objeto por caminho de erro e o abandona, vivos acusa, e a quarta condição de `make verifica` falha antes de qualquer teste de conteúdo. Um serializador com vazamento não passa no portão.

DICA

O teste de ida e volta é a especificação do formato, e vale escrevê-lo antes do leitor. Serialize um estado, desserialize o resultado, serialize outra vez, e exija que os dois textos sejam idênticos byte a byte. Ele pega, de uma vez, campo esquecido na escrita, campo esquecido na leitura, ordem trocada e valor truncado - e é o mesmo oráculo do replay do Cap. 16, aplicado a um arquivo em lugar de a uma execução. Um segundo teste, que carrega um arquivo da versão anterior gravado no repositório, é o que impede a compatibilidade regressiva de se perder sem ninguém notar.

§23.6 JSON, Formato Binário e Serialização Polimórfica

Esta seção não traz código, e a ausência é a decisão do capítulo levada a sério. O Deriva **não** usa JSON, e o motivo está no comentário de cabeçalho de `include/deriva/partida.hpp`: o formato é texto de linhas `chave valor`, porque o despejo do replay já é texto e o diff do portão já compara texto, e JSON entraria se houvesse estrutura aninhada de verdade, o que não há. Escrever um exemplo em JSON sobre o Deriva só para ter o que mostrar aqui contrariaria a §23.1, e acrescentar `nlohmann/json` ao `CMakeLists.txt` por causa de uma seção de reconhecimento contrariaria o portão de dependências. O que segue, portanto, é comparação, e ela se lê com a documentação da biblioteca ao lado.

JSON entra por reconhecimento, e por um motivo prático: é o formato que você vai encontrar em serviço, em configuração e em intercâmbio de dados, e `nlohmann/json` é a biblioteca de C++ que o faz com menos cerimônia, através de conversão implícita para os tipos da linguagem. O que ele traz e o formato próprio não tem é estrutura aninhada sem trabalho e ferramenta de terceiros para inspecionar. O que ele cobra é a dependência e a verbosidade, além de não distinguir inteiro de ponto flutuante da forma que C++ distingue.

Formato binário aparece pela outra ponta do compromisso: compacto e rápido, ilegível e frágil. Duas armadilhas valem nomear, porque são as que aparecem em trabalho de estudante. A primeira é gravar a estrutura com `write(reinterpret_cast<const char*>(&obj), sizeof(obj))`, o que grava o padding do Cap. 7 e, se a classe tiver método virtual, grava o `vptr` - um ponteiro para a `vtable` daquela execução, que é lixo na próxima. A segunda é a ordem dos bytes e o tamanho dos tipos, que mudam entre plataformas.

A serialização de hierarquia polimórfica é o caso que junta este capítulo com o próximo. Um `vector<unique_ptr<entidade>>` guardado em arquivo precisa gravar, para cada elemento, **qual** tipo ele é, porque a leitura tem de decidir qual classe construir. A etiqueta de tipo no arquivo e a função que a converte em objeto são o padrão Factory, e é assim que ele chega no Cap. 25, com um problema concreto atrás dele.

Exercícios Propostos

1. Escreva `partida::salvar` e `partida::carregar` no formato da §23.2, com cabeçalho de marca e versão. Escreva o teste de ida e volta antes do leitor: serialize, carregue, serialize outra vez, e exija igualdade byte a byte.
2. Grave no repositório uma partida da versão 1 do formato. Acrescente um campo no fim do bloco da sonda, incremente a versão para 2, e faça o leitor aceitar as duas. Confirme, com o arquivo gravado, que a partida antiga continua carregando.
3. Quebre a compatibilidade de propósito de duas formas: primeiro acrescentando um campo no **meio** de um bloco, depois trocando a ordem de dois campos existentes. Para cada uma, diga se o defeito aparece como erro de leitura ou como partida silenciosamente trocada, e por quê.
4. Faça o carregamento preencher os atributos diretamente, sem passar pelo construtor, e alimente-o com um arquivo em que a largura declarada não corresponde ao número de fileiras. Descreva o que acontece no primeiro acesso à última fileira. Depois conserte passando pelo construtor e devolvendo erro.
5. Provoque a versão futura: edite o cabeçalho de uma partida para uma versão maior que a do programa e carregue. Confirme que o resultado é `std::nullopt` e explique por que adivinhar seria pior que falhar. Depois acrescente, na camada que abre o arquivo, a mensagem que nomeia as duas versões, a do arquivo e a do programa, e diga por que ela não pertence ao desserializar.
6. Salve e carregue dez vezes em sequência, dentro de um escopo, e imprima vivos ao final. Explique o que um valor diferente de zero diria sobre o seu serializador, e por que o contador pega isso sem que exista teste de conteúdo.

O código, extraído do Deriva

Todo trecho abaixo vem de `exemplos/deriva/`, que compila com `-std=c++17 -Wall -Wextra -Wpedantic` sem um aviso e passa `make verifica`. Nenhum foi digitado neste texto.

```

src/partida.cpp:10 |----- a versão é a primeira linha
10 std::string partida::serializar() const {
11     // A ordem das linhas é fixa e a versão vem primeiro. Ordem estável é o que
12     // permite comparar dois saves com `diff`, e é a mesma razão do replay.
13     std::ostringstream os;
14     os << "deriva-partida " << versao << '\n'
15         << "setor " << setor << '\n'
16         << "sonda " << sonda.x << ' ' << sonda.y << '\n'
17         << "carga " << carga << '\n'
18         << "turno " << turno << '\n'
19         << "energia " << energia << '\n';
20     return os.str();
21 }

```

Quem lê precisa saber com que regras ler antes de ler qualquer outra coisa. Ordem de linha fixa é o que permite comparar dois saves com diff, e é a mesma razão do replay.

```

testes/test_partida.cpp:35 |----- o padrão do membro responde pelo campo ausente
35 // Compatibilidade regressiva: um arquivo da versao 1 nao tem `energia`, e o
36 // leitor v2 tem de aceitar isso. O valor padrao do membro e o que responde
37 // por ele - sem esse padrao, a sonda carregaria com zero de energia e
38 // apareceria morta, que e o defeito classico de migracao de formato.
39 TEST_CASE("o leitor v2 abre uma partida v1") {
40     const std::string v1 =
41         "deriva-partida 1\n"
42         "setor estacao-01\n"
43         "sonda 4 5\n"
44         "carga 3\n"
45         "turno 10\n";
46
47     const auto lida = partida::desserializar(v1);
48     REQUIRE(lida.has_value());
49     REQUIRE(lida->versao == 1);
50     REQUIRE(lida->sonda == vetor2{4, 5});
51     REQUIRE(lida->energia == 100); // o padrao, e nao zero
52 }

```

Sem esse padrão, a partida antiga carregaria com zero de energia e a sonda apareceria morta - o defeito clássico de migração de formato.

```
src/partida.cpp:24 |----- chaves, e não parênteses-----
24 // Chaves, e não parênteses. `std::istringstream is(std::string(texto));`
25 // é a *most vexing parse*: o compilador lê isso como a DECLARAÇÃO de uma
26 // função chamada `is` que devolve `istringstream` e recebe um `std::string`,
27 // e depois reclama que não há `operator>>` para função. A inicialização
28 // uniforme com `{}` da Aula 03 não tem essa ambiguidade, e é por isso que o
29 // material a recomenda desde o começo.
30 std::istringstream is{std::string(texto)};
31 std::string chave;
32 partida p;
33 bool tem_cabeca = false;
```

`std::istringstream is(std::string(texto));` é a *most vexing parse*: o compilador lê a DECLARAÇÃO de uma função. A inicialização uniforme da Aula 03 não tem essa ambiguidade, e é por isso que o material a recomenda desde o começo.

SOLID e invariância de comportamento

O QUE ESTE CAPÍTULO ENTREGA

- ▶ Identificar e aplicar cada um dos cinco princípios SOLID sobre o Deriva.
- ▶ Reconhecer violações de SOLID em código existente, e dizer o que cada uma torna difícil.
- ▶ Refatorar o mundo sem alterar um byte da saída, verificando por replay.
- ▶ Explicar por que replay é o oráculo, e o que ele não cobre.
- ▶ Conduzir a caça ao bug 3 na ordem: reproduzir, explicar em uma frase, corrigir, provar.
- ▶ Avaliar código gerado por LLM sob a ótica SOLID, com a rubrica do Cap. 4.

Os cinco princípios deste capítulo têm quarenta anos de literatura e cabem em cinco frases. O que não cabe em cinco frases, e é o que esta aula acrescenta ao que o v1 tinha, é a pergunta que vem depois de cada refatoração: **como você sabe que não quebrou nada?**

A resposta desta disciplina é o replay do Cap. 16. Dada uma semente fixa e um roteiro gravado de comandos, o Deriva refatorado tem de produzir um despejo de estado idêntico, byte a byte, ao da versão anterior. É essa comparação, e não a leitura do código nem a impressão de que ficou mais limpo, que torna a correção objetiva. A lição a levar do capítulo, e ela é uma frase: **refatoração correta é a que não muda a saída.**

§24.1 O mundo como God Class

O alvo da refatoração é o mundo, e ele chegou até aqui de propósito. Desde a v1.2 ele é a classe que guarda `vector<unique_ptr<entidade>>`, e a cada aula lhe foi acrescentada uma responsabilidade, porque acrescentar ali era sempre o caminho mais curto. A variante `variantes/v2.6-antes/` é essa classe, e ela faz sete coisas: guarda o estado do domínio, desenha direto em `std::cout`, lê a entrada num `switch`, decide o comportamento de cada tipo com `dynamic_cast`, abre o arquivo de log ali dentro, grava o save com o formato incluído, e cria entidade a partir de glifo numa terceira tabela.

Sete responsabilidades numa classe não é estimativa, é a contagem dos motivos independentes para editar aquele arquivo, e cada um dos sete está nomeado duas vezes: em comentário numerado na própria classe, e na tabela do LEIA-ME da variante, ao lado do que ele impede. Vale registrar aqui uma coisa que o material anterior fazia errado: o interativo do site exibia a contagem de linhas de mundo. `cpp` antes e depois da refatoração, com dois números que eram estimativa de documento de projeto e não medida de arquivo nenhum, e eles foram **removidos** do painel em vez de mantidos, porque número inventado é pior que número ausente. A contagem de responsabilidades é a métrica que ficou, e ela se confere lendo o

código. O que a peça ensina, que são as dependências de saída do nó central, é estrutural, e continua lá.

O sintoma prático de uma classe assim não é o tamanho, é o que ela impede. Testar a resolução de turno exige um terminal, porque desenhar está na mesma classe. Trocar o front-end exige mexer na regra do jogo. Duas pessoas mexendo em dois assuntos diferentes mexem no mesmo arquivo. É por isso que a refatoração acontece **antes** do Cap. 26 e não depois: sem ela, o argumento do segundo front-end não fecha.

§24.2 S - Responsabilidade Única (SRP)

O enunciado do princípio é uma frase: uma classe deve ter um motivo, e um só, para mudar. O mundo da variante v2.6-antes tem sete, e a §24.1 os contou um por um, na numeração que está em comentário dentro da própria classe.

A frase “um motivo para mudar” é mais útil que “faz uma coisa só”, porque ela é verificável: pergunte quem pede a mudança. Uma alteração no formato da partida salva vem de uma decisão de compatibilidade; uma alteração no cálculo de campo de visão vem de uma decisão de regra de jogo; uma alteração no desenho vem de uma decisão de apresentação. Três pedintes diferentes, três classes.

A decomposição da v2.6 sai da tabela da §24.1 quase mecanicamente, e ela está escrita: `include/deriva/apresentacao.hpp` é onde as costuras foram feitas. O mundo fica com o estado do domínio e a resolução de turno, e nada mais. O render sai para `i_apresentacao`, que é interface pura. A entrada sai para `i_comando`, com `historico` guardando o que foi feito - que é justamente o que o `switch` não tinha onde guardar. O log sai para `i_observador`. A criação de entidade sai para `criar_por_glifo`, com a tabela de glifos num lugar só. A IA sai para `estrategia`, que é `std::function` e não hierarquia. A persistência sai para a `struct partida` do Cap. 23. E o carregamento de mapa já estava fora desde a v0.3, em `mapa::carregar`.

O campo de visão é o caso duvidoso, e vale dizer por quê: ele é regra de jogo, porque decide o que a sonda percebe, e é usado só pelo desenho. Ficou fora das duas, em `src/fov.cpp`, como função livre e pura - e isso resolve a dúvida de um jeito que nenhuma das duas classes resolveria, porque função pura não precisa pertencer a ninguém.

O contraste que esta seção pede se lê no primeiro trecho extraído no fim do capítulo. Ele é a classe inteira da variante v2.6-antes, com as sete responsabilidades numeradas em comentário, uma por bloco de métodos: o estado, o render em `std::cout`, a entrada num `switch`, a IA com `dynamic_cast`, o log abrindo arquivo, o formato do save, e a tabela de glifos. Leia os sete cabeçalhos em sequência e a lição está dada, sem que seja preciso ler um corpo de método.

§24.3 O - Aberto/Fechado (OCP)

Classes devem estar abertas para extensão e fechadas para modificação, e o par que o Deriva oferece a essa frase é mundo e entidade: o mundo guarda `std::vector<std::unique_ptr<entidade>>` e aceita qualquer derivada da base, de forma que acrescentar uma entidade nova não toca uma linha de `src/mundo.cpp`.

No Deriva a violação tem endereço. A resolução de turno da variante v2.6-antes decide o que cada coisa faz olhando o glifo da célula, numa cadeia de comparações: se é `!`, acrescenta ao inventário; se é `#`, bloqueia; e assim por diante. Acrescentar um tipo de entidade exige achar essa cadeia e emendá-la, e nada no compilador avisa quem esqueceu de emendar as outras três cadeias espalhadas pelo arquivo.

A forma fechada para modificação já existe desde o Cap. 11: `entidade::agir()` é virtual, cada derivada responde à sua maneira, e o laço de turno chama `agir()` sem saber de quem. Acrescentar `drone_reparador` passa a ser escrever uma classe nova, e o laço de turno não é tocado.

Uma ressalva, porque este princípio é o mais fácil de aplicar demais. Fechar para modificação custa uma hierarquia, um `vptr` por objeto e uma indireção por chamada, e só se paga quando o conjunto de casos **cresce**. Onde o conjunto é fechado e conhecido - os cinco comandos do roteiro, os quatro tipos de terreno -, a cadeia de comparações ou o `switch` é mais legível que a hierarquia, e o Cap. 19 mostrou uma terceira via, que é `if constexpr` quando a decisão é de tipo em compilação. O critério é o do Cap. 19, e é o mesmo: pergunte se o conjunto de tipos precisa crescer sem recompilar o cliente.

As duas formas estão em código, e a leitura lado a lado é o que esta seção pede. A que viola está no trecho da variante v2.6-antes, no fim deste capítulo, nos blocos numerados (3), (4) e (7): três cadeias de comparação, uma por glifo e duas por tipo com `dynamic_cast`, no mesmo arquivo. A que está fechada para modificação é `mundo::turno()`, extraída no fim do **Cap. 18**, e o que se olha nela é o que ela **não** tem - nenhum `dynamic_cast`, nenhum `typeid`, nenhum `switch`. Acrescentar uma classe de entidade não toca naquela função, e é essa a diferença que o princípio nomeia.

§24.4 L - Substituição de Liskov (LSP)

Objetos de uma derivada devem poder substituir objetos da base sem alterar o comportamento que o programa espera. Se uma função que recebe `obstaculo*` deixa de funcionar quando lhe chega um `parede_que_lanca*`, há violação de LSP, e é essa a forma exata em que o Deriva a exhibe, de propósito.

LSP é o princípio que mais se confunde com sintaxe, e é útil dizer o que ele **não** é: não é escrever `override` corretamente, e não é a hierarquia compilar. O compilador verifica assinatura; LSP é sobre contrato, que o compilador não vê.

A regra prática tem duas metades. A derivada **não pode exigir mais** que a base - não pode fortalecer a pré-condição -, e **não pode prometer menos** - não pode enfraquecer a pós-condição. É por isso que o caso clássico do retângulo e do quadrado é violação: `quadrado::set_largura` também muda a altura, o que enfraquece a pós-condição de `retangulo::set_largura`, e o código que confiava na base passa a errar.

No Deriva a violação está escrita, e ela mora numa hierarquia própria, `include/deriva/solid.hpp`, que existe para isso. A base é `obstaculo`, e a promessa dela

está no comentário acima da declaração, em uma frase: `mover(delta)` devolve onde o obstáculo ficou depois de tentar mover, **nunca lança**, e para quem não pode mover a resposta é a posição atual. `caixa` cumpre a promessa. `parede_que_lanca` a quebra, e o corpo dela tem uma linha: `lanca std::logic_error`, porque `parede` não se move.

A assinatura está correta, o override está correto, o `-Wall -Wextra -Wpedantic` não diz nada, e o programa está errado. `parede_que_lanca` fortaleceu a pré-condição de `mover()`: a base aceita qualquer chamada, a derivada aceita nenhuma. Repare em onde o sintoma aparece, porque é isso que separa LSP de sintaxe: ele não aparece na parede, aparece em `empurrar_todos`, que recebe `std::vector<obstaculo*>` e foi escrita **antes** de a parede existir, quando estava certa. O teste extraído no fim deste capítulo é essa demonstração, e a última asserção dele é a parte que dói - a caixa **já** foi movida quando a exceção subiu, e a função deixou o sistema no meio do caminho sem ter feito nada errado. Violar LSP não produz erro local, produz garantia de exceção rebaixada em quem chama, no vocabulário da §20.2.

A correção não é `try/catch` no chamador, e é aqui que a maioria erra: envolver a chamada em `catch` é tratar o sintoma e deixar a promessa quebrada para o próximo chamador. A correção é honrar a promessa, e ela está ao lado, em `parede_honesta`, cujo `mover` devolve a posição em que está - quem chama não precisa saber que aquilo é parede, que é exatamente o que substituição significa. Uma segunda correção, quando a base não pode prometer isso uniformemente, é declarar `pode_mover()` na base e mudar o contrato de `mover()` para “move, se puder”, também uniformemente: a pré-condição desce para a base, e a substituição volta a valer.

E vale registrar por que essa hierarquia é separada da de entidade. `include/deriva/entidade.hpp` tem sonda, drone e item, e a parede do jogo **não** é entidade: é propriedade da célula desde a v0.3, e `munho::livre` pergunta pelo glifo `#` da célula em lugar de perguntar por um objeto. Isto é, o Deriva já aplica a primeira das duas correções, por modelagem, desde antes de o princípio ter nome no material. `solid.hpp` existe para que a violação possa compilar e rodar sem contaminar o núcleo, do mesmo modo que as variantes quebradas existem.

§24.5 I - Segregação de Interfaces (ISP)

Clientes não devem ser forçados a depender de interfaces que não usam. Uma interface grande que inclui funcionalidades irrelevantes para alguns clientes viola ISP.

O sintoma deste princípio tem forma reconhecível, e é o **método vazio**: a derivada que não faz o que a interface pede escreve um corpo com nada dentro e segue. O Deriva tem o caso de verdade, e ele aparece assim que o `item` entra na hierarquia - a base entidade oferece `agir(mundo&)`, e o `item` fica no chão até alguém o pegar -, e tem também o par escrito para ser comparado, que é onde a seção começa.

O par que mostra o princípio está em `include/deriva/solid.hpp`, ao lado da hierarquia de LSP e pela mesma razão. `i_tudo` é a interface gorda: três métodos puros, desenhar, salvar e reparar. `so_desenha_gordo` é quem só desenha, e o corpo dela é a prova - dois métodos vazios, com o comentário ao lado nomeando cada um deles como confissão. Método vazio numa interface não é economia, é a declaração de que a interface pede mais do que o implementador tem a dar; e o método que **lança** no lugar do vazio é a mesma coisa piorada, porque transforma a violação de ISP na violação de LSP da seção anterior.

Ao lado ficam as interfaces segregadas, `i_desenhavel` e `i_salvavel`, com `so_desenha` implementando uma e `desenha_e_salva` implementando as duas. E a métrica de ISP é **contável**, o que é raro num princípio de projeto: `metodos_obrigados_gordo()` devolve três e `metodos_obrigados_segregado()` devolve um, as duas em `constexpr`, e são esses dois números que o teste compara. Três obrigados contra um, para o mesmo implementador, é o argumento inteiro.

Dois avisos acompanham. O primeiro é que segregar cria herança múltipla, e o Cap. 17 mostrou o que ela custa: com interfaces puras e nenhum dado na base, não há diamante e não há herança virtual a declarar, e é por isso que interface pura é o caso seguro da herança múltipla em C++ - `desenha_e_salva` herda de duas e não paga nada por isso. O segundo é o do aviso ao fim desta seção: quatro interfaces com um implementador cada são pior que uma interface com um método vazio, e a hora de segregar é quando o segundo implementador de assinatura diferente aparece.

E é esse segundo aviso que explica o que a v2.6 fez com a hierarquia de entidade, porque a decisão foi a contrária. A base pede `glifo()` e `nome()`, que toda entidade tem, e oferece `agir(mundo&)`, que o item não faz. Segregar entidade em `desenhavel` e `agente` produziria dois laços sobre duas interfaces, e o mundo guarda um `vector<unique_ptr<entidade>>` desde a v1.2: os dois laços querem ou dois vetores, com uma entidade em dois lugares e a pergunta de quem a possui, ou um `dynamic_cast` por elemento, que é o que o Cap. 18 apresentou como sintoma. Com três classes concretas e um único implementador que não age, o custo é maior que o defeito, e a segregação **espera** - pelo próprio critério desta seção. entidade continua base única, `agir` continua virtual com corpo vazio na base, e o preço está declarado: a interface promete `agir` a quem não age, e nada no tipo distingue os dois. A única interface pura que o núcleo tem é `i_reparavel`, do Cap. 17, e ela existe porque ali o segundo implementador chegou.

§24.6 D - Inversão de Dependência (DIP)

Módulo de alto nível não deve depender de módulo de baixo nível: os dois dependem de uma abstração. A segunda metade do princípio fixa o sentido da seta, e é a metade que se cita errado com mais frequência - o detalhe depende da abstração, e a abstração não depende do detalhe.

Este é o princípio cuja aplicação a disciplina inteira cobra, e é o único dos cinco cujo retorno está agendado: o Cap. 26 põe um front-end Qt sobre o mesmo núcleo, sem alterar uma linha de regra de jogo, e isso só é possível se o mundo não conhecer o terminal.

Antes da refatoração ele conhecia, e a variante v2.6-antes preserva a forma: `std::cout` escrito de dentro da própria classe, no método que `desenha`, e o arquivo de log aberto no método que registra. Duas consequências decorriam, e a segunda é a que dói. A primeira é que verificar a regra do jogo obrigava a capturar a saída do processo, que é o que a tabela do LEIA-ME da variante anota ao lado da responsabilidade de render. A segunda é que a camada de apresentação não era substituível, e portanto a afirmação “o núcleo não conhece UI nenhuma” seria propaganda.

Vale registrar de passagem que essa pressão apareceu antes, e foi contornada de outra maneira: o `terminal_bruto` da v0.2 pergunta por `isatty` no construtor, e é essa guarda que impede o `ctest` de deixar o terminal de quem roda os testes em estado imprevisível. Contornar não é resolver, e é a diferença entre a v0.2 e esta aula.

A inversão, no Deriva, é `i_apresentacao`: interface pura com dois verbos, `dese-nhar(const mundo&)` e `mensagem(std::string_view)`, e ela é o segundo trecho extraído no fim deste capítulo. Repare no sentido da seta, porque ele é mais forte do que a formulação usual do princípio. Em lugar de o mundo receber um apresentador por referência no construtor, é o apresentador que recebe o mundo em cada chamada - e `include/deriva/mundo.hpp` não inclui `apresentacao.hpp`, de forma que o núcleo não depende nem da abstração. O que ele oferece à camada de cima é `despejar()`, o render determinístico em texto que o replay do Cap. 16 já usava desde a v1.6.

Injeção no construtor continua sendo a forma que o princípio descreve, e ela é a certa quando o objeto de alto nível precisa **chamar** o de baixo. Aqui ele não precisa, porque quem decide o momento de desenhar é o laço, que está fora dos dois - e essa é a diferença entre inverter a dependência e apenas movê-la para um parâmetro.

Quem implementa a interface hoje são `apresentacao_em_texto`, que acumula o que foi pedido numa `std::string` e é o que os testes usam, e a `tela_qt` do Cap. 26. O `main` de terminal não passa por ela: ele escreve `despejar()` em `std::cout` direto, e isso não é violação, porque `main` é a raiz da composição e não camada de domínio - o que a v2.6 tirou de dentro do mundo foi o `std::cout`, e é a variante v2.6-antes que preserva a forma anterior para comparação.

O que se ganha é verificável e vale listar, porque DIP é o princípio que mais atrai aplicação decorativa. O teste de regra de jogo roda com uma apresentação que só registra o que foi pedido, sem terminal e sem `std::cout`, e `testes/test_padroes.cpp` o faz na primeira asserção. O `CMakeLists.txt` do Deriva já separa `deriva_nucleo` como biblioteca, e a separação é real: o alvo do núcleo compila sem `FTXUI` e sem `Qt`, porque as duas dependências estão atrás de opção desligada por padrão. E o Cap. 26 é demonstração em vez de afirmação.

§24.7 Invariância de Comportamento, Verificada por Replay

Aqui está o que este capítulo tem que o v1 não tinha. Os cinco princípios dizem para onde mover o código; nenhum deles diz como saber que o movimento foi correto. O replay diz.

O procedimento é o do Cap. 16, e tem quatro passos. Antes de tocar no código, rode `./build/deriva --replay roteiro.txt --semente 7` e guarde a saída, que é a **referência**. Refatore. Rode o mesmo comando. Compare com `diff`. Saída idêntica byte a byte significa que nenhum comportamento observável mudou; uma linha de diferença significa que algo mudou, e a tarefa passa a ser descobrir o quê, e não decidir se o novo comportamento é aceitável.

A razão de isso funcionar está no determinismo, e ele foi construído para este momento. A semente é fixa, portanto o sorteio de itens é o mesmo; o roteiro é gravado, portanto a sequência de comandos é a mesma; `mapa::despejar()` produz texto na mesma ordem, sempre. Nada na execução depende de relógio, de endereço de memória ou de ordem de contêiner associativo, e o comentário em `mapa::despejar()` registra isso desde a v0.3.

Vale dizer o que é diferente de teste unitário, porque as duas coisas são complementares e não substitutas. O teste unitário afirma que uma unidade faz o que se pretende, e portanto **muda** quando se decide mudar o comportamento. O replay afirma que o sistema faz o que fazia, sem opinião sobre estar certo: ele preserva

inclusive o defeito. É exatamente essa indiferença que o torna oráculo de refatoração, e é também por isso que ele não substitui os testes.

▲ ATENÇÃO

O replay prova que a saída não mudou **nos caminhos que o roteiro percorre**, e não prova nada sobre os outros. Se o roteiro nunca esbarra numa parede, a refatoração pode ter invertido o teste de colisão e passar no portão. Daí a ordem que o LAB-12 cobra: **estender o roteiro antes de refatorar**, até que ele exercite todo comportamento que se pretende preservar, e só então mexer no código. Roteiro escrito depois da refatoração descreve o que o código passou a fazer, o que é o oposto de um oráculo.

§24.8 Caça ao Bug 3: a Refatoração que Mudou a Saída

A terceira e última caça ao bug do semestre é na semana 13, em dupla, sobre a variante v2.6-antes. Ela é a mais difícil das três, e a razão é que aqui não há travamento, não há vazamento, e o contador vivos fecha em zero. O programa roda, parece certo, e o replay acusa uma linha diferente.

O defeito plantado é de ordem. A extração da resolução de turno para fora do mundo troca o contêiner que o laço percorre, e a ordem em que as entidades agem deixa de ser a de inserção. O resultado é que a sonda recolhe um item um turno depois, e a linha do despejo que traz a carga difere de três para zero em um único passo do roteiro. Todo o resto é idêntico.

A ordem de trabalho é cobrada, e é a mesma das outras duas caças: **reproduzir a diferença; explicar em uma frase antes de tocar no código; corrigir; provar** com o replay voltando a ser idêntico. A rubrica de revisão do Cap. 4 é o instrumento, e o item dela que mais importa aqui é o que pergunta se a explicação identifica a causa ou descreve o sintoma. “A carga está errada” é sintoma; “a ordem de iteração deixou de ser a de inserção porque o contêiner mudou” é causa.

◇ LLM

Prompt para análise SOLID:

“Analise o código abaixo e identifique violações dos cinco princípios SOLID. Para cada violação: nomeie o princípio, cite a linha problemática, explique o impacto (o que fica difícil de fazer) e mostre o código refatorado.”

[colar o mundo da variante v2.6-antes]

Avalie a resposta com a rubrica do Cap. 4, e conte quantos dos cinco itens dela a análise cumpre. Dois erros são recorrentes e valem checar primeiro: LLMs acertam SRP e OCP, que são os princípios mais documentados, e erram LSP, que confundem com override correto, e DIP, que confundem com qualquer uso de abstração. O terceiro erro é mais custoso: a refatoração sugerida costuma mudar a saída, e o replay é quem descobre.

Exercícios Propostos

1. Abra a variante v2.6-antes e liste as violações de SOLID, uma por princípio, com três colunas: o princípio, a linha, e o que aquela violação torna difícil de fazer. A terceira coluna é a que vale nota, e nenhuma resposta ali pode ser “fica menos limpo”.
2. Refatore o mundo aplicando a decomposição da §24.2, com o portão do **LAB-12**: o despejo do replay com semente 7 continua idêntico byte a byte, e `make verifica` passa nas quatro condições. Antes de começar, estenda `roteiro.txt` até que ele esbarre em parede, recolha item e chegue a uma borda, e explique por que essa extensão vem primeiro.
3. Aplique DIP: declare `apresentacao` como interface pura, injete-a no construtor do mundo, e escreva `apresentacao_de_teste` que registra as chamadas. Demonstre que um turno completo pode ser testado sem terminal e sem `std::cout`, e confirme que o alvo do núcleo compila sem FTXUI.
4. Abra `include/deriva/solid.hpp` e escreva a terceira derivada de `obstaculo`, uma que devolva a posição errada em vez de lançar. Rode `empurrar_todos` sobre as três e diga em qual das duas violações o sintoma é mais difícil de achar, e por quê. Depois conserte a sua, e explique por que `try/catch` no chamador não é conserto.
5. Segregue entidade em `desenhavel` e `agente`, e remova o `agir()` vazio da base. Confirme, com `make verifica`, que a saída não mudou, e conte quantos `dynamic_cast` a sua versão precisou para percorrer as duas interfaces a partir de um `vector<unique_ptr<entidade>>`. Esse número é a resposta ao porquê de a v2.6 **não** ter segregado, e diga se você concorda com a decisão. Explique também por que esta segregação não produz diamante.
6. Faça a refatoração errada de propósito: extraia a resolução de turno mudando o contêiner de percurso, como na caça ao bug 3. Rode o replay, guarde o `diff`, e escreva a frase de causa antes de consertar. Guarde as duas coisas, porque a frase escrita antes é o que a rubrica avalia.
7. Peça a um LLM a refatoração do exercício 2 e rode o replay sobre o resultado dele. Se a saída mudou, identifique onde e por quê; se não mudou, aplique a rubrica do Cap. 4 e diga quais dos cinco princípios a resposta de fato endereçou.

O código, extraído do Deriva

Todo trecho abaixo vem de `exemplos/deriva/`, que compila com `-std=c++17 -Wall -Wextra -Wpedantic` sem um aviso e passa `make verifica`. Nenhum foi digitado neste texto.


```

18 // =====
19 // DIP - a interface de apresentação
20 //
21 // O núcleo depende DESTA abstração, e não de terminal nem de Qt. É o que
22 // torna a separação demonstrável em vez de afirmada: a v2.7 acrescenta uma
23 // segunda implementação e o núcleo não muda uma linha.
24 //
25 // Antes da refatoração, o `mundo` escrevia direto em `std::cout` - e trocar a
26 // saída significava editar o `mundo`. A variante `v2.6-antes` preserva essa
27 // forma para comparação.
28 // =====
29 class i_apresentacao {
30 public:
31     virtual ~i_apresentacao() = default;
32     virtual void desenhar(const mundo& m) = 0;
33     virtual void mensagem(std::string_view texto) = 0;
34 };

```

O núcleo depende desta abstração, e não de terminal nem de Qt. Antes da refatoração o mundo escrevia direto em `std::cout`, e trocar a saída significava editá-lo.

```

└─▲ variantes/v2.6-antes/mundo_god_class.cpp:44 ── quebrado de propósito · sete responsabilidades · CAÇA AO BUG 3─┐
44 /// AS SETE RESPONSABILIDADES. Cada uma é um motivo independente para editar
45 /// este arquivo, e é essa contagem - não o número de linhas - que a Aula 24
46 /// pede para medir.
47 class mundo {
48 public:
49     mundo(int largura, int altura) : largura_(largura), altura_(altura) {
50         terreno_.assign(static_cast<std::size_t>(largura * altura), '.');
51         for (int x = 0; x < largura; ++x) {
52             terreno_[idx({x, 0})] = '#';
53             terreno_[idx({x, altura - 1})] = '#';
54         }
55         for (int y = 0; y < altura; ++y) {
56             terreno_[idx({0, y})] = '#';
57             terreno_[idx({largura - 1, y})] = '#';
58         }
59     }
60
61     // (1) estado do dominio
62     void acrescentar(std::unique_ptr<entidade> e) { entidades_.push_back(std::move(e)); }
63     bool livre(vetor2 p) const {
64         return p.x >= 0 && p.y >= 0 && p.x < largura_ && p.y < altura_ &&
65             terreno_[idx(p)] != '#';
66     }
67
68     // (2) RENDER. Escreve direto em `std::cout`, e é isso que torna a troca por
69     // Qt impossível sem editar esta classe. Também impede testar sem capturar a
70     // saída do processo.
71     void desenhar() const {
72         for (int y = 0; y < altura_; ++y) {
73             for (int x = 0; x < largura_; ++x) {
74                 char c = terreno_[idx({x, y})];
75                 for (const auto& e : entidades_) {
76                     if (e->pos == vetor2{x, y}) c = e->glifo();
77                 }
78                 std::cout << c;
79             }
80             std::cout << '\n';
81         }
82     }
83
84     // (3) ENTRADA. O `switch` de comandos mora aqui, e por isso não há onde
85     // guardar o desfazer.
86     bool comando(const std::string& c) {
87         sonda* s = nullptr;
88         for (const auto& e : entidades_) {
89             if (auto* p = dynamic_cast<sonda*>(e.get())) s = p;
90         }
91         if (!s) return false;
92         vetor2 alvo = s->pos;
93         if (c == "norte") alvo.y -= 1;
94         else if (c == "sul") alvo.y += 1;
95         else if (c == "leste") alvo.x += 1;
96         else if (c == "oeste") alvo.x -= 1;
97         else if (c == "esperar") { /* nada */ }
98         else return false;
99         if (!livre(alvo)) { registrar("bloqueado"); return false; }
100         s->pos = alvo;
101         registrar("moveu");
102         return true;
103     }
104 }

```

```

105 // (4) IA. O comportamento de cada tipo está codificado aqui, com
106 // `dynamic_cast`, em vez de na entidade ou numa estratégia.
107 void turno() {
108     for (const auto& e : entidades_) {
109         if (auto* d = dynamic_cast<drone*>(e.get())) {
110             vetor2 alvo{d->pos.x + d->rumo.x, d->pos.y + d->rumo.y};
111             if (livre(alvo)) d->pos = alvo;
112             else d->rumo = {-d->rumo.x, -d->rumo.y};
113         }
114     }
115 }
116
117 // (5) LOG. Abre o arquivo aqui dentro, então o teste que quiser verificar o
118 // log precisa mexer no sistema de arquivos.
119 void registrar(const std::string& evento) {
120     std::ofstream log("deriva.log", std::ios::app);
121     log << evento << '\n';
122 }
123
124 // (6) PERSISTÊNCIA. O formato do save também mora aqui.
125 std::string salvar() const {
126     std::string s = "antes 1\n";
127     for (const auto& e : entidades_) {
128         s += e->glifo();
129         s += " " + std::to_string(e->pos.x) + " " + std::to_string(e->pos.y) + "\n";
130     }
131     return s;
132 }
133
134 // (7) CRIAÇÃO. A tabela de glifos, uma terceira vez com `dynamic_cast`.
135 void povoar(const std::string& glifos) {
136     int x = 1, y = 1;
137     for (const char g : glifos) {
138         if (g == '@') { auto s = std::make_-
unique<sonda>(); s->pos = {x, y}; acrescentar(std::move(s)); }
139         else if (g == 'd') { auto d = std::make_-
unique<drone>(); d->pos = {x, y}; acrescentar(std::move(d)); }
140         if (++x >= largura_ - 1) { x = 1; ++y; }
141     }
142 }
143
144 private:
145     std::size_t idx(vetor2 p) const {
146         return static_cast<std::size_t>(p.y) * static_cast<std::size_t>(largura_) +
147             static_cast<std::size_t>(p.x);
148     }
149     int largura_, altura_;
150     std::vector<char> terreno_;
151     std::vector<std::unique_ptr<entidade>> entidades_;
152 };

```

Compila sem aviso, roda, e faz tudo o que a refatorada faz. O defeito não é funcional: são sete motivos independentes para editar o mesmo arquivo. A caça não é achar o erro, é refatorar e provar pelo replay que a saída não mudou.

```

┌──| src/mundo.cpp:19 |────────────────────────────────── o turno não conhece as derivadas ───────────────────┐
19 void mundo::turno() {
20     // Índice em lugar de iterador: `agir` pode acrescentar entidade, e isso
21     // invalidaria o iterador. Quem entra durante o turno age no turno seguinte,
22     // e essa regra é observável no replay.
23     const std::size_t n = entidades_.size();
24     for (std::size_t i = 0; i < n; ++i) entidades_[i]->agir(*this);
25 }

```

Aberto para extensão e fechado para modificação, e a prova é negativa: acrescentar uma entidade nova não muda uma linha deste arquivo, porque o mundo guarda `vector<unique_ptr<entidade>>` e chama `agir` pela base. O índice em lugar do iterador não é estilo: `agir` pode acrescentar entidade, e isso invalidaria o iterador.

```

┌──| testes/test_padroes.cpp:63 |────────────────────────────────── o despejo byte a byte como oráculo ───────────────────┐
63 TEST_CASE("desfazer em cadeia volta ao estado inicial") {
64     zerar_entidades();
65     mundo w = montar();
66     w.acrescentar(std::make_unique<sonda>(vetor2{1, 1}));
67     historico h;
68     const std::string antes = w.despejar();
69
70     // A coluna em (3,2) e parede: a sequencia desce ANTES de chegar nela.
71     REQUIRE(h.aplicar(std::make_unique<mover_sonda>(vetor2{1, 0}), w)); // (2,1)
72     REQUIRE(h.aplicar(std::make_unique<mover_sonda>(vetor2{0, 1}), w)); // (2,2)
73     REQUIRE(h.aplicar(std::make_unique<mover_sonda>(vetor2{0, 1}), w)); // (2,3)
74     REQUIRE(h.profundidade() == 3);
75     REQUIRE(w.despejar() != antes);
76
77     while (h.desfazer_ultimo(w)) {
78     }

```

O mesmo critério que a caça ao bug 3 cobra: `diff` vazio é a única evidência aceita, e teste verde não basta - os testes passam nas duas versões.

```

┌─▲ include/deriva/solid.hpp:51 ────────────────── quebrado de propósito · a violação de LSP ──────────────────┐
51 /// A VIOLAÇÃO. Compila, e quebra o contrato da base.
52 class parede_que_lanca final : public obstaculo {
53 public:
54     using obstaculo::obstaculo;
55     [[nodiscard]] vetor2 mover(vetor2) override {
56         throw std::logic_error("parede nao se move");
57     }
58 };
└────────────────────────────────────────────────────────────────────────────────────────────────────────────────┘

```

Compila, e quebra a promessa da base: mover promete não lançar. O sintoma não aparece na parede - aparece em toda função que recebe obstaculo&, escrita antes de a parede existir e correta quando foi escrita.

```

┌─└─ testes/test_solid.cpp:13 ─────────────────────────────────────────────────────────────────────────────────── LSP se mede no chamador ───────────────────────────────────────────────────────────────────────────────────┐
13 // ---- LSP -----
14 TEST_CASE("LSP: a violacao nao aparece na derivada, aparece no chamador") {
15     caixa c({1, 1});
16     parede_que_lanca p({5, 5});
17     std::vector<obstaculo*> quais{&c, &p};
18
19     // `empurrar_todos` foi escrita antes de a parede existir, e estava certa.
20     REQUIRE_THROWS_AS(empurrar_todos(quais, {1, 0}), std::logic_error);
21
22     // E o pior: a caixa JA foi movida quando a execucao subiu. A funcao deixou o
23     // sistema no meio do caminho, sem ter feito nada errado.
24     REQUIRE(c.pos() == vetor2{2, 1});
25 }
└────────────────────────────────────────────────────────────────────────────────────────────────────────────────┘

```

E o pior está na última linha: a caixa JÁ foi movida quando a exceção subiu. A função deixou o sistema no meio do caminho sem ter feito nada errado.

```

┌─└─▲ include/deriva/solid.hpp:89 ─────────────────────────────────────────────────────────────────────────────────── quebrado de propósito · método vazio é a confissão ───────────────────────────────────────────────────────────────────────────────────┐
89 /// Quem só desenha, obrigado a mentir em dois métodos.
90 class so_desenha_gordo final : public i_tudo {
91 public:
92     void desenhar() override { desenhou = true; }
93     void salvar() override {} // vazio: a confissão
94     void reparar() override {} // vazio: a segunda
95     bool desenhou = false;
96 };
└────────────────────────────────────────────────────────────────────────────────────────────────────────────────┘

```

Quem só desenha é obrigado a implementar salvar e reparar, e mente em dois métodos. Método vazio numa interface é a confissão de que ela pede demais - e a métrica de ISP é contável: três obrigados contra um.

Padrões de projeto canônicos em C++ moderno

O QUE ESTE CAPÍTULO ENTREGA

- ▶ Implementar Strategy com lambdas, e dizer por que não com herança.
- ▶ Implementar Command com desfazer, e reconhecer onde a lambda deixa de bastar.
- ▶ Implementar State, Observer, Factory, Composite e Decorator sobre o Deriva.
- ▶ Aplicar Singleton corretamente, e nomear os quatro problemas que ele traz.
- ▶ Decidir quando um padrão se paga, e quando ele é indireção adiantada.

Um padrão de projeto é o nome de uma solução que se repete, e não uma peça a encaixar. A distinção tem consequência prática imediata: quem estuda o catálogo antes de ter o problema tende a procurar onde aplicar cada padrão, o que é a inversão exata do movimento correto, que é reconhecer, num problema que já se tem, a forma de algo já resolvido antes.

Há um segundo motivo para tratar o catálogo com cuidado em C++ moderno, e ele é histórico. Boa parte dos padrões da formulação original, de 1994, existe para contornar o que as linguagens da época não tinham. Strategy é uma classe com um método porque não havia função como valor; Command é uma classe com `executar()` pela mesma razão; Iterator é um padrão porque não havia biblioteca de iteradores; Prototype resolve a cópia polimórfica, que em C++ é um construtor de cópia. Com lambdas, `std::function`, templates e `std::variant`, vários deles encolhem de uma hierarquia para poucas linhas, e a intenção permanece idêntica. É por isso que este capítulo trata o padrão como intenção, e discute a mecânica separadamente.

O Deriva chega na v2.6, depois da refatoração do Cap. 24, e cada padrão daqui tem endereço nela: Command para a entrada, State para as telas, Observer para o registro de eventos, Factory para a geração de entidades, Strategy por lambda, Composite para o inventário, e Decorator sobre a apresentação. **Os sete existem em código**, e vale saber onde: Command, Observer, Factory e Strategy em `include/deriva/apresentacao.hpp` e `src/apresentacao.cpp`; Composite em `include/deriva/inventario.hpp`; State, Decorator e o Singleton da §25.8 em `include/deriva/padroes.hpp` e `src/padroes.cpp`. Os trechos estão extraídos no fim deste capítulo, e as provas em `testes/test_padroes.cpp` e `testes/test_padroes_extra.cpp`.

§25.1 Strategy com Lambdas, e não com Herança

Strategy encapsula uma família de algoritmos intercambiáveis, de forma que o contexto que os usa não conheça qual está em uso. A intenção é essa, e ela é boa.

A mecânica da formulação original é uma classe base abstrata com um método virtual, uma derivada por algoritmo, e um `unique_ptr` no contexto. São quatro declarações e um destrutor virtual para trocar uma função, mais um `vp_ptr` por objeto de estratégia e uma indireção por chamada. Em C++17 isso se escreve com uma `lambda`, e a razão de a versão com `lambda` ser preferível não é a brevidade: é que a estratégia deixa de precisar de nome, de arquivo e de tipo, o que reduz a distância entre onde ela é escrita e onde é usada.

No Deriva o alvo é o comportamento das entidades, e o tipo cabe numa linha: `using estrategia = std::function<vetor2(const entidade&, const mundo&)>;`. Uma estratégia recebe a entidade e o mundo e devolve para onde ela quer ir, e há três, construídas por função de fábrica - patrulha, perseguição e parada. Cada comportamento novo é uma `lambda`, e nenhuma classe nova aparece; trocar de estratégia é atribuir a uma variável, e é isso que uma das seções de testes/`test_padroes.cpp` afirma, com a frase que vale decorar: trocar de estratégia é trocar de valor, e não de tipo.

A escolha entre parâmetro de template e `std::function` é a do Cap. 21, e vale repeti-la aqui porque é onde ela decide projeto: **parâmetro de template quando a estratégia é chamada logo**, o que permite inlining e não custa nada; **`std::function` quando ela é guardada** como membro para uso posterior, ao preço de uma indireção e de uma possível alocação.

O Deriva escolhe `std::function` nos dois casos, e o segundo motivo não é o do parágrafo acima. `inventario::contar_se` e `inventario::descartar_se` recebem o predicado e o usam ali mesmo, o que pediria parâmetro de template; recebem `const std::function<...>&` porque estão **declaradas no cabeçalho e implementadas no .cpp**, e template obrigaria a implementação a subir para o cabeçalho, pelo motivo da §19.1. É um preço pago por tempo de compilação e por fronteira de módulo, e não por elegância - e é o tipo de decisão que só aparece quando o projeto tem mais de um arquivo.

Duas ressalvas honestas. Herança continua sendo a resposta quando a estratégia tem **estado** com ciclo de vida próprio, ou mais de uma operação, e não uma função só - aí ela é um objeto, e a classe é a forma certa de escrevê-la. E `std::function` não é grátis: guardar cem estratégias num contêiner de `std::function` é pagar cem indireções, o que a versão com herança também pagaria, e nenhuma das duas é o que se quer num laço quente.

Conte as declarações das duas formas, porque é aí que a comparação fica objetiva: quatro mais um destrutor virtual na clássica, contra a linha do `using` e uma função de fábrica por comportamento na que o Deriva usa. O trecho extraído no fim deste capítulo é a estratégia de patrulha, e o comentário dentro dela é a armadilha que sobra depois de a hierarquia desaparecer: a captura é por valor, e tem de ser, porque a `lambda` sobrevive à chamada que a criou. É a armadilha do `string_view` do Cap. 3, noutra roupa.

§25.2 Command: a Entrada, e o Desfazer

Command transforma uma operação em objeto, de forma que ela possa ser guardada, enfileirada, registrada e desfeita. No Deriva o alvo é a entrada: cada tecla do jogador se torna um comando, e o historico guarda os que foram executados, na ordem. Vale precisar a relação com o Cap. 22, porque as duas coisas têm o mesmo nome e não são a mesma: a `fila_de_comandos` da v2.4 atravessa a fronteira entre threads carregando o **nome** do comando, em texto, e é do lado de cá da fronteira que o texto se torna objeto de comando. O padrão já estava no desenho antes de ter nome.

Aqui a lambda **não** basta, e a razão vale mais que o padrão. Um comando que se desfaz precisa de duas operações, `executar()` e `desfazer()`, e de estado suficiente para reverter - qual era a posição antes de mover, qual era a carga antes de recolher. Uma lambda oferece uma operação. `std::pair` de duas lambdas com captura compartilhada resolve na aparência e piora na leitura, porque a relação entre as duas fica implícita na captura. Duas operações relacionadas e estado comum são a definição de objeto, e portanto Command é classe.

Assim, a discriminação deste capítulo fica clara: **Strategy virou função porque tem uma operação e nenhum estado; Command continua classe porque tem duas operações e estado**. O critério não é o nome do padrão, é a forma do problema.

O desfazer no Deriva tem uma sutileza que vale registrar, porque ela é a razão de o padrão se pagar. Desfazer um movimento é devolver a sonda à posição anterior, o que é trivial; desfazer o recolhimento de um item é devolver o item à célula com a massa e a energia que ele tinha, o que exige que o comando os guarde. E há operação que não se desfaz: o avanço de turno consome energia de todas as entidades, e reverter isso exigiria guardar o estado inteiro. É onde a pilha de desfazer se declara limitada, por decisão documentada, e não por esquecimento.

O que está escrito é a interface `i_comando`, com `executar(mundo&)`, `desfazer(mundo&)` e `nome()`; `mover_sonda`, que guarda de onde saiu e é por isso que sabe voltar; e `historico`, a pilha, com a regra de que comando que falhou não entra nela. O trecho de `mover_sonda` está no fim deste capítulo, e o que se olha nele são os três membros: `delta_`, que é o pedido, `de_onde_`, que é o que permite desfazer, e `executou_`, que é o que impede desfazer duas vezes o que se fez uma. `testes/test_padroes.cpp` prova o desfazer em cadeia pelo despejo byte a byte, que é o oráculo do Cap. 24 aplicado a uma pilha de comandos.

Há uma coisa a admitir aqui, e ela é o critério da §25.9 aplicado contra o próprio capítulo: **há um comando concreto, e não dois**. `mover_sonda` é o único que existe, e por esse critério `i_comando` é indireção antes de ser padrão. O que justifica a interface, apesar disso, é o segundo **cliente** em lugar do segundo comando: o `historico` guarda `unique_ptr<i_comando>` porque tem de guardar e desfazer sem conhecer a classe concreta, e a janela em Qt do Cap. 26 aplica comandos pelo mesmo `historico` que o terminal aplica. Dois clientes de uma interface pedem interface, do mesmo modo que dois implementadores pedem.

O segundo comando é o de recolhimento, e é o exercício 2 que o pede: ele guardaria a célula limpa e a massa do item, que é o estado sem o qual o desfazer não fecha. O mesmo exercício traz o limite declarado da pilha, que hoje cresce sem teto - e o limite é decisão documentada, e não descuido, pela razão do parágrafo anterior sobre o avanço de turno.

§25.3 State: as Telas

State substitui uma variável de modo e a cadeia de comparações em torno dela por um objeto que representa o modo corrente, com transições explícitas. No Deriva os modos são as telas do console, e são três: `tela_mapa`, `tela_inventario` e `tela_inspecao`, cada uma respondendo de forma diferente ao mesmo comando.

A versão sem padrão é um `enum` e um `switch` em cada lugar que trata entrada, e ela merece defesa antes de ser trocada: com três estados e um lugar de despacho, o `enum` é mais curto, mais legível, e mais fácil de percorrer no depurador. O que a torna insustentável é a multiplicação, e o comentário de cabeçalho de `include/deriva/padroes.hpp` faz a conta: quatro telas e cinco comandos são vinte casos espalhados por tantas funções quantas reajam a comando, e acrescentar uma tela obriga a visitar todas. O compilador só avisa das que estiverem em `switch` sem `default`.

State resolve isso movendo o comportamento para dentro do objeto de estado. A interface é `i_tela`, com `nome()` e `comando(std::string_view)`, e a decisão de projeto que vale a seção inteira está no tipo de retorno: **comando devolve a próxima tela**, como `std::unique_ptr<i_tela>`, ou `nullptr` para ficar onde está. O trecho de `tela_mapa::comando` está extraído no fim deste capítulo, e ele tem três linhas.

Devolver a transição em vez de mutar um campo compra duas coisas. A primeira é que não há como duas telas discordarem sobre qual está ativa: quem troca é uma só, a máquina console, que guarda a tela corrente por `unique_ptr` e mais nada. A segunda é o teste - a transição é o valor de retorno de uma função pura o bastante para ser exercitada sem console, sem terminal e sem mundo, e é isso que permite afirmar a máquina de estados numa suíte que roda no `ctest`. Acrescentar uma tela passa a ser escrever uma classe, e é OCP do Cap. 24 aplicado a modo em vez de a tipo de entidade.

Uma armadilha específica, e ela é a razão de as telas do Deriva não conhecerem o mundo. Estado que guarda referência ao contexto e contexto que guarda o estado corrente é um ciclo, e o Cap. 13 mostrou o que um ciclo de `shared_ptr` faz - prender 160 bytes, medidos, que nunca são liberados. A resposta é a posse em sentido único: a máquina possui a tela por `unique_ptr`, e a tela, quando precisar do mundo, o recebe por referência na chamada, sem guardá-lo.

160 bytes

presos pelo ciclo

aferido por

`./build/testes "*copiar"`

`testes/test_posse.cpp`

§25.4 Observer: o Registro de Eventos

Observer notifica interessados sobre mudança de estado, sem que o notificante os conheça. No Deriva o alvo é o registro de eventos: quando a sonda recolhe um item, quando um drone se move, quando a energia acaba, o mundo anuncia, e quem se interessou reage. Os dois interessados que o desenho prevê são a apresentação, que escreve a linha de mensagem na tela, e o registro em arquivo, que serve à depuração e ao replay.

O padrão é o que o Cap. 26 vai encontrar pronto no Qt: signals e slots são Observer implementado pelo framework, com o registro feito por `connect()` em vez de por `adicionar_observador()`. Ver os dois lado a lado é útil, porque mostra o que o framework acrescenta - a desconexão automática quando o objeto morre - e o que ele custa, que é o `QObject` e o pré-processador de metaobjetos.

O defeito clássico do Observer escrito à mão é o observador pendurado: um observador é destruído sem se desinscrever, o notificante guarda um ponteiro cru

para ele, e a próxima notificação escreve em objeto morto. Três respostas, em ordem crescente de custo. A mais simples é o observador se desinscrever no próprio destrutor, o que é RAII do Cap. 8 aplicado a inscrição. A segunda é o notificante guardar `weak_ptr`, verificar antes de notificar, e descartar o que expirou, que é o uso canônico do Cap. 13. A terceira, quando os observadores têm vida garantidamente maior que o notificante, é documentar essa garantia e usar ponteiro cru - o que só é aceitável com a garantia escrita, porque ponteiro cru sem posse é exatamente o que o Cap. 12 apresentou como contraexemplo.

O lado do observador está escrito, e são três trechos no fim deste capítulo. A interface `i_observador` tem um método, `aconteceu(std::string_view)`, e o comentário acima dela registra o que a refatoração do Cap. 24 tirou do caminho: antes, o mundo chamava o log direto, abrindo arquivo, e por isso não dava para testar o log sem mexer no sistema de arquivos. `registro_em_memoria` é o observador que acumula os eventos num `std::vector<std::string>`, tem vinte linhas, e é o que substitui o arquivo. E o teste é a prova do ganho inteiro: nenhum caminho, nenhuma permissão, nenhuma limpeza depois.

O lado do notificante é onde a v2.6 parou, e a parada é deliberada. `include/deriva/mundo.hpp` não conhece `i_observador`: ninguém guarda lista de inscritos, e o mundo não anuncia nada. A razão é a do próprio §25.9 - com um observador, uma lista de inscritos é uma chamada de método com cerimônia -, e a consequência é que o defeito do observador pendurado, com os três consertos que o parágrafo acima descreve, é o exercício 5, e não trecho a extrair. Escrever o notificante sem ter dois observadores seria justamente o erro que este capítulo passa dez páginas ensinando a não cometer.

§25.5 Factory: a Geração de Entidades

Factory concentra num lugar a decisão de qual classe construir, de forma que o cliente receba a base e não conheça as derivadas. No Deriva ele tem duas entradas, e as duas são reais.

A primeira é o carregamento de mapa, e é a que está escrita: `criar_por_glifo(char glifo, vetor2 pos)` devolve `std::unique_ptr<entidade>`, com @ produzindo a sonda, d o drone e ! o item. O trecho está extraído no fim deste capítulo, e repare no caso default: glifo desconhecido devolve `nullptr`, e não lança. A escolha está comentada ao lado e é de domínio, não de estilo - a maioria dos glifos de um mapa é terreno, e terreno não é entidade; transformar # em erro obrigaria quem lê o mapa a filtrar antes de perguntar, o que é a decisão da §20.7 outra vez, e aqui ausência de entidade é resposta e não falha.

A segunda entrada é o carregamento de partida, e ela é o gancho deixado no fim do Cap. 23: um arquivo salvo que trouxesse, para cada entidade, uma etiqueta de tipo, precisaria de alguém que convertesse a etiqueta em objeto da classe certa - e esse alguém é esta mesma função, com a etiqueta no lugar do glifo. Fica registrado que a `struct partida` de hoje não guarda entidade nenhuma além da sonda, e portanto essa segunda entrada é desenho.

Aqui vale uma observação sobre a cadeia de comparações, porque ela é o que a §24.3 condenou e o que este padrão contém. A diferença é que na Factory a cadeia está em **um** lugar, e é o lugar cuja razão de existir é justamente decidir tipo - o switch de `criar_por_glifo` é o mesmo switch que na variante v2.6 - antes aparecia três vezes, e uma delas com `dynamic_cast`. Acrescentar uma entidade continua exigindo uma linha ali, e essa linha é o preço declarado do padrão; o que ele evita

é a mesma decisão espalhada por quatro laços. Quando nem essa linha se quer, a saída é um registro - um mapa de etiqueta para função construtora, preenchido por cada classe ao ser registrada -, e ele custa inicialização estática de ordem não especificada, que é problema real e é o assunto do §25.8. Para o número de entidades que o Deriva tem, a cadeia num lugar é a escolha certa.

§25.6 Composite: o Inventário

Composite deixa que um agrupamento seja tratado como um elemento único, através da mesma interface. No Deriva o alvo é o inventário, e ele está escrito: a base é componente, com `massa()`, `rotulo()` e `pecas()` virtuais; a folha é peca; e o nó é mochila, cuja `massa()` soma a tara com o que está dentro, recursivamente. O trecho está extraído no fim deste capítulo. O `inventario` percorre componente, e não precisa saber se algum deles é mochila - `massa_total()` chama `massa()` e a recursão acontece sozinha, do outro lado da interface.

O padrão é curto de descrever e traz duas armadilhas que o Deriva já tem ferramenta para nomear.

A primeira é o ciclo. Nada na estrutura impede pôr a mochila A dentro da mochila B e a B dentro da A, e o resultado é recursão infinita no cálculo de massa, ou, com `shared_ptr`, o ciclo que nunca libera - a mesma forma dos 160 bytes medidos no Cap. 13. A resposta é a mais simples possível, e é a que o código usa: a mochila possui o conteúdo por `std::vector<std::unique_ptr<componente>>`, e posse exclusiva torna o ciclo impossível de construir, porque uma peça não pode estar em dois lugares.

A segunda é a interface pela metade. A formulação original põe `acrescentar()` e `remover()` na interface comum, para que cliente nenhum precise distinguir folha de nó, e a consequência é que a peça simples herda dois métodos que ela não pode cumprir, o que é a violação de ISP do §24.5 e a de LSP do §24.4 ao mesmo tempo. A alternativa é declarar os dois métodos apenas no nó, e aceitar que o cliente que quer acrescentar precise saber que está falando com uma mochila. É um compromisso, e é a escolha que está no código: `por_dentro` existe em mochila e não em componente, e componente declara `pecas()` com corpo padrão devolvendo um, que é o que permite ao despejo contar sem perguntar o tipo de ninguém.

160 bytes

presos pelo ciclo

aferido por

./build/testes "*copiar"

testes/test_posse.cpp

§25.7 Decorator: a Apresentação

Decorator acrescenta comportamento a um objeto envolvendo-o em outro da mesma interface, sem alterar nem a classe original nem quem a usa. A forma é a que define o padrão, e são duas condições ao mesmo tempo: **o decorador implementa a mesma interface que decora, e guarda um ponteiro para ela**. Sem a primeira, ele não é substituível por quem consome; sem a segunda, ele não é decorador, é outra implementação.

No Deriva as duas condições estão em `include/deriva/padroes.hpp`, e os decoradores são dois. `com_numero_de_linha` numera as linhas do que passa por ele. `com_moldura` desenha um quadro com título em volta. Os dois implementam `i_apresentacao` e os dois guardam um `std::unique_ptr<i_apresentacao>` dentro, e a razão de ser `unique_ptr` e não referência é a posse: quem constrói a pilha entrega o decorado ao decorador e sai de cena, e a destruição desce em cascata sem que ninguém precise lembrar da ordem.

O que a pilha compra é o que a herança não daria, e o teste extraído no fim deste capítulo é essa demonstração. Com herança, “numerado e com moldura” exigiria uma classe para cada combinação, e três comportamentos combináveis dariam sete classes. Com decorador, é a ordem em que se empilha - e a ordem é **observável na saída**, que é o que torna a afirmação verificável: o teste monta `com_numero_de_linha` por fora de `com_moldura`, desenha, e confere na string acumulada que a moldura, o número de linha e o fecho estão lá. Trocar as duas de lugar muda a saída, e é o mesmo critério de replay do Cap. 24 aplicado a uma composição.

A condição para que isso funcione é a inversão de dependência do §24.6, e vale explicitar a corrente: porque quem desenha é `i_apresentacao`, e não uma classe concreta, é possível pôr qualquer coisa que implemente aquela interface entre o núcleo e a tela. Decorator não é um padrão independente de DIP, é o que DIP permite fazer depois. E a terceira ponta da corrente é o Cap. 26: o mesmo par de decoradores se empilha sobre a `tela_qt` sem uma linha diferente, porque ela implementa a mesma interface que `apresentacao_em_texto`.

O que os decoradores escritos **não** fazem é registrar traço em arquivo, e o exercício 7 é onde isso entra. Vale dizer o que ele acrescenta ao que já está aqui, porque não é sintaxe: o ponto de composição. `com_numero_de_linha` e `com_moldura` são interpostos dentro do teste, e o `main` de terminal escreve `despejar()` em `std::cout` sem passar pela interface - o único lugar do repositório em que alguém monta uma apresentação para uso de verdade é `qt/janela.cpp`. Decidir onde a pilha se monta é decidir quem é a raiz da composição, e é a mesma pergunta que o §24.6 respondeu sobre o `main`.

§25.8 Singleton: a Forma Correta e os Quatro Problemas

Esta é a única seção do livro cujo código foi escrito **para ser recusado**, e vale dizer isso na abertura, porque muda como se lê o trecho. `registro_global`, em `include/deriva/padroes.hpp`, é o Singleton de Meyers, e ele é a forma correta de escrever o padrão em C++: uma variável `static local` na função de acesso, que a linguagem garante ser inicializada uma única vez, de forma segura entre threads desde o C++11, e destruída ao fim do programa, sem `new` e sem `delete`. Os construtores de cópia e a atribuição são `delete`, e o construtor padrão é privado, que é o que impede a segunda instância.

Vale saber por que essa forma é a certa entre as formas erradas, e o motivo é a alternativa: uma variável global em escopo de arquivo tem ordem de inicialização não especificada entre unidades de tradução, e um Singleton que depende de outro na inicialização estática produz o defeito mais difícil de depurar desta lista.

Os quatro custos estão no comentário acima da classe, e o aviso ao fim desta seção os desenvolve. Um deles, porém, deixou de ser argumento e passou a ser resultado, e é esse que vale ler no código: dois casos de teste compartilham o estado do registro **de propósito**, e o que eles mostram é que a ordem em que rodam altera o que o segundo vê. Teste que depende da ordem dos outros testes não é teste, e o Singleton é como se chega lá sem querer.

E a alternativa está escrita ao lado, na mesma dúzia de linhas, para que a comparação não dependa de imaginação: `anotar_em(i_observador& onde, std::string_view evento)`. A dependência aparece no parâmetro, o teste passa um

`registro_em_memoria` e verifica, e o custo é dois caracteres a mais na chamada. É a injeção de dependência do Cap. 24, e é o que este material recomenda.

▲ ATENÇÃO

Singleton é o padrão mais abusado, e o motivo é que ele resolve um problema de acesso, e não de projeto: ficar mais fácil chamar de qualquer lugar. Quatro custos vêm no pacote. O estado é global, e portanto compartilhado entre testes, o que faz um teste passar ou falhar conforme a ordem em que roda. A dependência é implícita: a assinatura do método não diz que ele fala com a configuração, e ninguém a descobre sem ler o corpo. Não há como injetar substituto no teste, porque não há parâmetro por onde. E, sob concorrência, a instância única é dado compartilhado por todas as threads, com tudo o que o Cap. 22 mostrou. A alternativa quase sempre melhor é a injeção de dependência do Cap. 24.

Há uma admissão a fazer aqui, e ela é do interesse desta disciplina em particular. O contador de instâncias vivas é estado global mutável: `inline static int vivos` é uma variável única por classe, alcançável de qualquer lugar, exatamente como a instância de um Singleton. A disciplina o usa por dezenove capítulos sabendo disso, porque ele é **instrumento**, e não projeto de domínio - ele existe para observar o programa, não para o programa funcionar. É a mesma licença que se dá a um contador de teste ou a um registro de depuração, e ela vem com as duas consequências que o Cap. 7 e o Cap. 22 já cobraram: `zerar_contadores()` existe justamente porque estado global entre testes é problema, e a corrida de dados da §22.3 é o que o estado global faz sob duas threads. Reconhecer que o instrumento tem o defeito que ele ensina a evitar é mais honesto que esconder o instrumento.

§25.9 Quando o Padrão Não se Paga

▼ DICA

O padrão se introduz quando o **segundo caso** aparece, e não quando alguém prevê que ele vai aparecer. Uma apresentação com um implementador não é DIP, é indireção; um Factory que constrói um tipo é uma chamada de construtor com cerimônia; um Observer com um observador é uma chamada de método. O v1 deste material tinha exemplos assim, e o Deriva tem uma vantagem para evitá-los: as versões da trilha dizem quando o segundo caso chega. O segundo apresentador é o Qt do Cap. 26; o segundo tipo de entidade é a v1.0; o segundo observador é o registro de eventos ao lado do desenho. Onde o segundo caso não está no plano, o padrão espera.

Três perguntas resolvem quase todos os casos, e valem mais que o catálogo.

Quantos casos existem hoje? Um implementador, um observador, um tipo a construir: não é padrão, é indireção. A abstração se justifica no segundo caso, e o Deriva tem a vantagem de que a trilha das versões diz quando o segundo caso chega.

O que a indireção esconde do depurador? Cada camada é um salto a mais para atravessar quando algo dá errado, e o gdb do Cap. 2 percorre a pilha de uma vez, o que é fácil, e a cadeia de decoradores um a um, o que não é. É custo real, e ele entra na conta.

A saída continua a mesma? Padrão aplicado é refatoração, e portanto vale o Cap. 24 inteiro: o replay com semente fixa tem de produzir despejo idêntico byte a byte. Se aplicar Composite ao inventário mudou a ordem em que os itens aparecem no despejo, o padrão foi aplicado errado, ou a mudança de comportamento precisa ser decidida em vez de acontecer.

Exercícios Propostos

1. Implemente `inventario::ordenar_por` com parâmetro de template e chame-o com três lambdas: por massa decrescente, por nome, e por energia com empate resolvido pela massa. Depois escreva a mesma coisa com hierarquia virtual e conte as declarações de cada versão. Diga em duas linhas o que a segunda versão oferece que a primeira não.
2. Implemente Command com desfazer para o movimento e para o recolhimento de item, e uma pilha de desfazer com limite documentado. Demonstre, com o replay, que executar dez comandos e desfazer os dez devolve o despejo ao estado inicial, byte a byte.
3. Tente escrever o comando de movimento com desfazer usando duas lambdas com captura compartilhada, em vez de uma classe. Explique, a partir do que você escreveu, por que Strategy virou função e Command não.
4. Escreva a máquina de telas duas vezes: primeiro com um enum e um switch em cada função que reage a comando, depois usando `i_tela` e a console que já estão em `include/deriva/padroes.hpp`. Acrescente uma quarta tela às duas versões e conte quantos arquivos você tocou em cada uma. Diga a partir de que número de telas a segunda versão passa a valer o preço.
5. Implemente Observer para o registro de eventos, com dois observadores. Provoque o observador pendurado: destrua um deles sem desinscrever e notifique. Depois conserte com desinscrição no destrutor, e outra vez com `weak_ptr`, e compare os dois consertos.
6. Implemente Composite no inventário com posse por `unique_ptr`. Tente construir um ciclo, colocando uma caixa dentro de si mesma. Explique por que o compilador ou o tipo impedem, e o que aconteceria se a posse fosse por `shared_ptr`.
7. Escreva um terceiro decorador, `com_traco_em_arquivo`, no molde de `com_numero_de_linha`, e empilhe-o com os dois que já existem sem alterar uma linha de mundo nem de `apresentacao_em_texto`. Depois troque a ordem das três camadas e mostre a diferença na saída. Explique por que nada disso seria possível antes da inversão de dependência do Cap. 24, e diga onde, no Deriva, a pilha deveria ser montada para valer no programa e não só no teste.
8. Identifique três lugares deste material, do site ou de código que você já escreveu, em que um Singleton foi usado onde injeção de dependência serviria. Para cada um, mostre a refatoração e diga qual dos quatro problemas do aviso do §25.8 ela resolve.

O código, extraído do Deriva

Todo trecho abaixo vem de exemplos/deriva/, que compila com `-std=c++17 -Wall -Wextra -Wpedantic` sem um aviso e passa make verifica. Nenhum foi digitado neste texto.

```
src/apresentacao.cpp:69 | Strategy é função, não hierarquia
69 estrategia estrategia_de_patrolha(vetor2 rumo) {
70     // A captura é por VALOR, e é obrigatório que seja: a lambda sobrevive à
71     // chamada que a criou, e capturar `rumo` por referência deixaria uma
72     // referência pendurada. É a mesma armadilha do `string_view` da Aula 03,
73     // noutra roupa.
74     return [rumo](const entidade& e, const mundo& m) {
75         const vetor2 alvo = e.pos() + rumo;
76         return m.livre(alvo) ? alvo : e.pos();
77     };
78 }
```

Uma operação e nenhum estado: é função, e `std::function` a guarda. A captura é por valor, e é obrigatório - a lambda sobrevive à chamada que a criou, e capturar por referência deixaria referência pendurada. É a armadilha do `string_view` da Aula 03 noutra roupa.

```
include/deriva/apresentacao.hpp:64 | Command guarda de onde saiu
64 /// Mover a sonda. Guarda de onde saiu, e é por isso que sabe voltar.
65 class mover_sonda final : public i_comando {
66 public:
67     explicit mover_sonda(vetor2 delta) noexcept : delta_(delta) {}
68
69     [[nodiscard]] bool executar(mundo& m) override;
70     void desfazer(mundo& m) override;
71     [[nodiscard]] std::string_view nome() const override { return "mover"; }
72
73 private:
74     vetor2 delta_;
75     vetor2 de_onde_{};
76     bool executou_ = false;
77 };
```

O que se ganha não é elegância, é o desfazer - e um switch não tem onde guardar de onde a sonda veio.

```

src/apresentacao.cpp:58 |----- Factory, e a tabela num lugar só
58 std::unique_ptr<entidade> criar_por_glifo(char glifo, vetor2 pos) {
59     // A tabela é aqui, e só aqui. Acrescentar uma entidade nova é acrescentar um
60     // caso - e nada no carregador de mapa muda.
61     switch (glifo) {
62         case '@': return std::make_unique<sonda>(pos);
63         case 'd': return std::make_unique<drone>(pos);
64         case '!': return std::make_unique<item>(pos, "sucata", 3);
65         default: return nullptr;    // glifo que não é entidade: o terreno cuida
66     }

```

Acrescentar entidade nova é acrescentar um caso aqui, e nada no carregador de mapa muda. Na variante v2.6-antes a mesma tabela aparece três vezes, com `dynamic_cast`.

```

src/inventario.cpp:90 |----- Composite, e o inventário não sabe a diferença
90 int mochila::massa() const {
91     return std::accumulate(dentro.begin(), dentro.end(), tara_,
92                             [](int soma, const std::unique_ptr<componente>& c) {
93                                 return soma + c->massa();
94                             });
95 }

```

A mochila responde massa somando o que tem dentro, e o inventário a trata como qualquer peça. Cuidado com o ciclo: mochila dentro de si mesma é o vazamento da Aula 13 noutra forma.

```

include/deriva/apresentacao.hpp:91 |----- Observer, a interface
91 // =====
92 // Observer · o log de eventos
93 //
94 // O 'mundo' avisa que algo aconteceu e não sabe quem escuta. Antes da
95 // refatoração ele chamava o log direto, e por isso não dava para testar sem
96 // arquivo.
97 // =====
98 class i_observador {
99 public:
100     virtual ~i_observador() = default;
101     virtual void aconteceu(std::string_view evento) = 0;
102 };

```

Antes da refatoração o mundo chamava o log direto, abrindo arquivo, e por isso não dava para testar o log sem mexer no sistema de arquivos.


```

| include/deriva/apresentacao.hpp:104 | o observador que substitui o arquivo
104 class registro_em_memoria final : public i_observador {
105     public:
106         void aconteceu(std::string_view evento) override;
107         [[nodiscard]] const std::vector<std::string>& eventos() const noexcept {
108             return eventos_;
109         }
110
111     private:
112         std::vector<std::string> eventos_;
113 };

```

Vinte linhas, e é o que torna o log verificável. O mundo não sabe quem escuta, e é isso que Observer compra.

```

| testes/test_padroes.cpp:83 | o log, verificado sem arquivo
83 TEST_CASE("Observer permite testar o log sem arquivo") {
84     registro_em_memoria log;
85     i_observador& como_interface = log;
86     como_interface.aconteceu("sonda entrou no setor");
87     como_interface.aconteceu("item recolhido");
88     REQUIRE(log.eventos().size() == 2);
89     REQUIRE(log.eventos().front() == "sonda entrou no setor");
90 }

```

Nenhum caminho, nenhuma permissão, nenhuma limpeza depois. É o teste que a versão anterior não conseguia ter.

```

| src/padroes.cpp:9 | State, e a transição é o retorno
9 std::unique_ptr<i_tela> tela_mapa::comando(std::string_view c) {
10     if (c == "i") return std::make_unique<tela_inventario>();
11     if (c == "x") return std::make_unique<tela_inspecao>();
12     return nullptr; // fica onde está
13 }

```

Devolver a próxima tela em vez de mutar um campo é o que impede duas telas de discordarem sobre qual está ativa. A alternativa é um switch com vinte casos espalhados por cinco funções.

```

┌───┐ testes/test_padroes_extra.cpp:50 ─────────────────── Decorator, empilhado ───────────────────┐
50 // ---- Decorator -----
51 TEST_CASE("Decorator: empilhar dois nao exige classe para cada combinacao") {
52     zerar_entidades();
53     mundo w = montar();
54     w.acrescentar(std::make_unique<sonda>(vetor2{2, 1}));
55
56     auto base = std::make_unique<apresentacao_em_texto>();
57     apresentacao_em_texto* espiar = base.get();
58
59     // numerado POR FORA da moldura: a ordem da pilha e observavel
60     std::unique_ptr<i_apresentacao> tela = std::make_unique<com_numero_de_linha>(
61         std::make_unique<com_moldura>(std::move(base), "SETOR"));
62
63     tela->desenhar(w);
64     const std::string saida = espiar->acumulado();
65
66     REQUIRE(saida.find("┌ SETOR └") != std::string::npos);
67     REQUIRE(saida.find("[1]") != std::string::npos);
68     REQUIRE(saida.find("└┘") != std::string::npos);
69 }

```

Com herança, “numerado e com moldura” exigiria uma classe para cada combinação. Com decorador, é a ordem da pilha - e a ordem é observável na saída.

```

┌─| include/deriva/padroes.hpp:104 |──────────────────────────────── Singleton, escrito para ser criticado ─────────────────┐
104 // =====
105 // Singleton · escrito para ser criticado
106 //
107 // O material recomenda NÃO usá-lo, e escrevê-lo é a forma de mostrar por quê.
108 // Este é o Singleton de Meyers, que é a versão correta da forma errada: a
109 // inicialização de estático local é garantidamente única e segura entre
110 // threads desde C++11.
111 //
112 // Os quatro custos, e nenhum deles é opinião:
113 // 1. o teste não consegue substituí-lo, porque quem o usa o pede por nome;
114 // 2. dois testes na mesma execução compartilham o estado dele;
115 // 3. a ordem de destruição entre estáticos não é controlável;
116 // 4. ele esconde uma dependência que a assinatura deveria declarar.
117 //
118 // O contador `vivos` da Aula 07 é estado global mutável, e este material o usa
119 // assim - como INSTRUMENTO, e não como projeto. A diferença vale ser dita: o
120 // contador não participa da lógica do jogo, e nenhuma decisão do domínio
121 // depende dele.
122 // =====
123 class registro_global {
124 public:
125     static registro_global& instancia();
126
127     void anotar(std::string_view evento);
128     [[nodiscard]] std::size_t quantos() const noexcept { return eventos_.size(); }
129     void limpar() { eventos_.clear(); }
130
131     registro_global(const registro_global&) = delete;
132     registro_global& operator=(const registro_global&) = delete;
133
134 private:
135     registro_global() = default;
136     std::vector<std::string> eventos_;
137 };
└────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────┘

```

É o Singleton de Meyers, a versão correta da forma errada. Os quatro custos estão no comentário, e nenhum é opinião - o segundo deles é medido por dois casos de teste que compartilham estado de propósito.

Qt e a separação domínio/apresentação

O QUE ESTE CAPÍTULO ENTREGA

- Compreender a arquitetura do Qt: QObject, signals e slots, laço de eventos.
- Reconhecer signals e slots como Observer implementado pelo framework.
- Distinguir a posse por árvore de objetos do Qt da posse por ponteiro inteligente do núcleo.
- Conectar domínio e apresentação sem que o domínio inclua um header do Qt.
- Dizer o que o segundo front-end prova, e o que ele não prova.

Uma nota de estatuto antes de qualquer coisa, porque ela muda como se lê o capítulo. O plano v2 rebaixou esta aula a **demonstração do docente com esqueleto publicado, sem entrega obrigatória**. Não há laboratório, não há item de prova que dependa dela, e o código do front-end é distribuído pronto, atrás de uma opção de compilação. O plano registra também que, se o semestre perder mais de um encontro, esta aula é reduzida a quarenta minutos no início do encontro de reserva, depois de a Aula 22 já ter virado leitura dirigida.

A razão do rebaixamento é de infraestrutura, e é a mesma que decidiu o padrão-alvo da linguagem: exigir Qt6 instalado em máquina de laboratório é exigir uma dependência grande, com versões que divergem entre distribuições, na última semana do semestre, para um conteúdo que não é o objeto da disciplina. O que é objeto da disciplina, e é o que sobrevive ao rebaixamento, é o argumento que este capítulo permite fazer.

Uma remissão, porque este capítulo é metade de um que se partiu. O último capítulo do v1 tratava de Qt e de uso de LLM no ciclo orientado a objetos completo, e as duas metades foram para pontas opostas do livro: o uso de LLM está no **Cap. 4**, onde ganhou peso por virar instrumento de trabalho desde o começo do semestre, com a rubrica de revisão que os capítulos seguintes usam. O post-mortem do projeto, que era o último exercício de lá, também está no Cap. 4. Aqui não há repetição de nenhum dos dois, e o exercício 3 é o único ponto em que os dois assuntos se cruzam.

§26.1 O que o Segundo Front-end Prova

Por vinte e cinco capítulos este material afirmou que o núcleo do Deriva não conhece a camada de apresentação. A afirmação apareceu no comentário do `CMakeLists.txt` da v0.0, quando `deriva_nucleo` foi declarado como biblioteca separada; apareceu no `terminal_bruto` da v0.2, que não sabe nada sobre o mapa; e apareceu como princípio no Cap. 24, quando a inversão de dependência substituiu a chamada direta a `std::cout` de dentro do mundo pela interface `i_apresentacao`.

Afirmção de arquitetura, porém, não se verifica lendo o código: verifica-se tentando substituir a camada. É isso que este capítulo faz, e é por isso que ele fica no fim. Pôr uma interface gráfica em Qt sobre o mesmo núcleo, sem alterar uma linha de regra de jogo, transforma “o núcleo é independente da apresentação” de promessa em resultado observável.

O critério de sucesso é objetivo e tem três partes. Nenhum arquivo de `include/deriva/` passa a incluir header do Qt. O alvo `deriva_nucleo` continua compilando com o Qt ausente da máquina. E a suíte de testes do núcleo passa sem alteração, com o mesmo número de casos: se um teste precisou mudar para o Qt entrar, o núcleo mudou, e o argumento não fecha. É essa igualdade, e não a captura de tela da janela, que é o resultado a exibir.

Vale dizer também o que ele **não** prova. Não prova que a arquitetura é boa em geral, nem que Qt é a escolha certa para um roguelike, que ele não é. Prova uma coisa só, e é suficiente: a fronteira que o curso desenhou existe e aguenta peso.

§26.2 Arquitetura do Qt

Qt é um framework C++ de interface gráfica, e o que este capítulo usa dele é pouco e específico: `QObject`, a classe raiz de toda a hierarquia; o laço de eventos de `QApplication::exec()`, que retira evento de interface de uma fila e despacha o slot correspondente; e signals e slots, o mecanismo de notificação frouxa entre objetos que é o Observer do Cap. 25 implementado pelo framework. A tabela abaixo põe cada peça ao lado do conceito de POO que ela realiza, e vale lê-la com o Deriva em mente: tudo o que está na coluna da esquerda mora em `qt/`, e nada disso entra em `include/deriva/`.

Conceito Qt	Equivalente OO clássico
Signal	Notificação de evento - análogo ao método “notify” do Observer
Slot	Handler de evento - análogo ao “update” do Observer
<code>connect()</code>	Registrar observer - Liga um signal a um slot, sem acoplamento direto
<code>QObject</code>	Classe base com metadados (MOC) para signals/slots e propriedades
<code>QApplication::exec()</code>	Event loop - processa eventos de UI e dispara slots

Três coisas da tabela merecem desenvolvimento, porque as três têm consequência de projeto.

A primeira é o **MOC**, o compilador de metaobjetos. A macro `Q_OBJECT` não é código C++ que o compilador entenda sozinho: um pré-processador do Qt lê o cabeçalho, gera um `.cpp` adicional com a tabela de metadados da classe, e é esse arquivo gerado que implementa a conexão entre signal e slot. Duas consequências práticas

decorrem. O sistema de build precisa saber disso, o que no CMake é a opção de automação de MOC, e é a razão pela qual copiar uma classe Qt para um projeto sem essa configuração produz erro de ligação em vez de erro de compilação. E `QObject` só funciona em classe que derive de `QObject`, o que faz de “emitir signal” uma capacidade que se herda, e não uma que se acrescenta.

A segunda é o **laço de eventos**. `QApplication::exec()` não retorna até a aplicação fechar: ele fica retirando eventos de uma fila e despachando-os para slots. Isso inverte o controle em relação ao Deriva de terminal, cujo `main` tem um laço próprio que lê comando, avança turno e desenha. As duas coisas não podem coexistir ingenuamente, e é o assunto da §26.4.

A terceira é a posse. Um `QObject` construído com pai é destruído pelo pai, e essa é a disciplina de posse do Qt: uma árvore de objetos, com destruição em cascata a partir da raiz.

▲ ATENÇÃO

O Qt tem um modelo de posse próprio: um `QObject` construído com pai é destruído pelo pai, o que é por que `new QWidget{this}` aparece sem `delete` em todo código Qt e não é vazamento. Isso convive mal com `std::unique_ptr` no mesmo objeto, e a regra a seguir é não misturar as duas disciplinas dentro de uma classe: widget é possuído pela árvore do Qt, objeto de domínio é possuído por ponteiro inteligente, e a fronteira entre os dois é a classe adaptadora. Envolver um widget com pai em `unique_ptr` produz destruição dupla, e o contador de instâncias vivas não a acusa, porque nenhum objeto de domínio nasceu ou morreu ali.

Quatro trechos no fim deste capítulo mostram tudo o que o Deriva tem de Qt, e são poucos de propósito. A janela, com a macro `Q_OBJECT`, a seção `private slots:` e o comentário sobre a árvore de posse. O `main`, que constrói a `QApplication` e o mundo e não faz mais nada. `tela Qt`, o adaptador que implementa `i_apresentacao` sem derivar de `QObject`. E o construtor da janela, que é o único lugar do repositório onde as duas disciplinas de posse convivem, com `new QPlainTextEdit(this)` a três linhas de `std::make_unique<tela Qt>`.

Nesse último trecho há um detalhe de forma que vale ler com atenção: os três `connect` usam **ponteiro de função**, `&QAction::triggered` e `&janela::ao_mover_norte`, e não as macros `SIGNAL` e `SLOT` que aparecem em todo material de Qt anterior ao Qt5. A diferença é de verificação: a forma antiga casava nomes em execução, comparando strings, e um erro de digitação virava uma conexão que nunca acontece, sem aviso de compilação e sem erro de execução. A forma nova é conferida pelo compilador, que é onde erro de nome deve aparecer.

E vale registrar o que o Deriva **não** tem, porque é a parte do Qt que todo tutorial mostra primeiro: `signals:` e `emit` não aparecem em uma linha do repositório. A janela liga os seus slots a `QAction::triggered`, que é signal do próprio Qt, e não declara signal nenhum. A razão é de projeto, e é o assunto da §26.4.

§26.3 Signals e Slots como Observer Implementado pelo Framework

O Cap. 25 escreveu Observer à mão, com uma interface de ouvinte, uma lista de inscritos, e o problema do observador pendurado. Signals e slots são o mesmo padrão com a mesma intenção, implementado pelo framework, e a comparação lado a lado é o que este capítulo tem de mais aproveitável para quem nunca vai escrever Qt.

O que o framework acrescenta é a **desconexão automática**. Quando um dos dois objetos de uma conexão é destruído, o Qt desfaz a conexão, e o defeito que o §25.4 descreveu deixa de ser possível. É a mesma garantia que o `weak_ptr` do Cap. 13 dava, obtida por outro caminho: em lugar de o notificante verificar se o observador ainda existe, o objeto que morre avisa quem estava conectado a ele. O preço é o registro central de conexões que o `QObject` mantém.

Acrescenta também a conexão entre threads. Um signal emitido em uma thread pode acionar um slot que executa na thread do objeto receptor, com o Qt enfileirando a chamada no laço de eventos daquela thread. É a solução do Cap. 22 para o mesmo problema, isto é confinar o dado a um dono e comunicar por fila, oferecida pelo framework em vez de escrita à mão.

E cobra o que já foi dito: `QObject` como base, MOC no build, e a dependência inteira. É aqui que a decisão de projeto deste capítulo se torna inevitável: se as classes de domínio precisassem emitir signals, elas precisariam derivar de `QObject`, e o núcleo passaria a depender do Qt. O que resolve isso é a §26.4.

§26.4 A Separação Domínio/Apresentação com Qt

A regra é uma só, e vale para qualquer framework de interface: a camada de domínio não depende dele, e é a camada de apresentação que se adapta ao domínio. O que a torna aplicável aqui é `i_apresentacao`, a interface do Cap. 24 através da qual o núcleo desenha sem saber quem desenha, e é ela, e não o Qt, que faz o segundo front-end existir sem que o núcleo mude uma linha.

No Deriva a regra se escreve de forma verificável: **nenhum arquivo de `include/deriva/` inclui header do Qt, e nenhuma classe de domínio deriva de `QObject`**. O que atravessa a fronteira mora inteiro em qt/, e são duas classes.

A primeira é `tela_qt`, e ela é o adaptador: implementa `i_apresentacao`, a mesma interface que `apresentacao_em_texto` implementa, e põe o `despejar()` do núcleo dentro de um `QPlainTextEdit`. Repare no que ela **não** é. Não deriva de `QObject`, e portanto não emite signal nenhum; guarda o widget por ponteiro cru, que é observação e não posse, com o comentário dizendo que a árvore do Qt é a dona.

A segunda é a `janela`, e ela é o único lugar onde as duas disciplinas de posse se encontram: possui o mundo e o `historico` por valor, o `tela_qt` por `unique_ptr` - porque não é widget, e ninguém mais o destruiria -, e o `QPlainTextEdit` por ponteiro cru, porque passou `this` como pai e com isso entregou a posse. Os slots dela são os comandos do Cap. 25: `ao_mover_norte` aplica um `mover_sonda` ao histórico e redesenha, e `ao_desfazer` chama o `desfazer` da pilha.

O sentido das dependências, que é o que o Cap. 24 chamou de inversão, fica então este: a `janela` conhece o adaptador, o núcleo e o Qt; o adaptador conhece a interface do núcleo e um widget; o núcleo não conhece nenhum dos dois, e não sabe nem

que `i_apresentacao` existe, porque `include/deriva/mundo.hpp` não a inclui. A seta atravessa a fronteira sempre no mesmo sentido.

Um desenho alternativo vale ser nomeado, porque é o que a literatura de Qt costuma mostrar e o Deriva escolheu não fazer: um controlador que derive de `QObject`, implemente a interface de apresentação e **reemita** cada chamada do núcleo como signal, com a janela ligada a ele por `connect` em vez de chamá-lo. Isso compra a desconexão automática da §26.3 também para a ligação entre núcleo e janela, e custa `QObject`, MOC e um signal declarado por evento. Com uma janela e três comandos, a chamada direta é mais curta e mais fácil de depurar; com vários widgets interessados na mesma mudança, o signal se paga, e o critério é o da §25.9 - o padrão entra quando o segundo interessado aparece.

DIAGRAMA UML

Diagrama de classes do Deriva v2.7: à esquerda, `mundo`, `mapa` e a hierarquia de entidades, que não conhecem o Qt e compõem o alvo `deriva_nucleo`; ao centro, a interface `i_apresentacao`, que é do núcleo e não conhece nem o terminal nem o Qt; à direita, `janela : QMainWindow`, com `QObject`, e `tela_qt`, que implementa `i_apresentacao` e não deriva de `QObject`. A janela possui o `mundo` por valor, o `tela_qt` por `unique_ptr` e o `QPlainTextEdit` pela árvore do Qt, e as suas três `QAction` chegam aos slots por `connect`. A seta de dependência atravessa a fronteira sempre da direita para a esquerda.

O laço de eventos exige uma decisão a mais, e vale registrá-la porque é onde a maioria das tentativas ingênuas trava. O Deriva de terminal tem laço próprio: lê comando, resolve turno, desenha. Sob Qt, o laço é do framework, e portanto o turno não pode ser resolvido num laço paralelo. A adaptação é que o comando passa a chegar por slot - o clique ou a tecla aciona um slot do controlador, que resolve um turno e devolve o controle -, e o Deriva por turnos se acomoda a isso sem esforço, porque nada nele precisa de tempo contínuo. Um jogo de tempo real exigiria um temporizador e a decisão sobre passo fixo, e é por isso que a escolha do artefato por turnos paga também aqui.

Uma consequência agradável fecha a seção: como o comando entra pelo mesmo caminho que o `Command` do Cap. 25 definiu, e como o núcleo continua determinístico, o **replay do Cap. 16 continua valendo sob Qt**. O mesmo roteiro e a mesma semente produzem o mesmo despejo, e o front-end é indiferente. Se produzirem coisas diferentes, alguma regra de jogo migrou para a camada de apresentação.

§26.5 O Esqueleto Publicado

O código deste capítulo é distribuído pronto, e o que o estudante faz com ele é ler, compilar se puder, e responder aos exercícios. O que o esqueleto contém, no desenho:

O alvo de Qt fica atrás de uma opção de CMake, desligada por padrão, do mesmo jeito que a integração de FTXUI está atrás de `-DDERIVA_COM_FTXUI=ON`, e isso já está escrito: `option(DERIVA_COM_QT ... OFF)` no `CMakeLists.txt`, com o comentário registrando que ligar a opção não altera uma linha de `deriva_nucleo`. Com a opção desligada, e é assim que o portão roda, o Qt não é procurado e a ausência dele não é erro. Com ela ligada, o CMake liga `CMAKE_AUTOMOC`, procura o Qt6 com `find_package` e monta um segundo executável, `deriva_qt`, ligado ao mesmo `deriva_nucleo`; a falta do Qt é mensagem clara e não falha de ligação. E o alvo novo carrega

os mesmos `-Wall` `-Wextra` `-Wpedantic` que o núcleo carrega, porque o portão de zero aviso vale para o código deste repositório e não para o de terceiros.

E há uma condição que vale escrita: o alvo do núcleo e a suíte de testes **não** dependem da opção. Isso é o que torna o critério da §26.1 verificável por qualquer pessoa, em qualquer máquina, sem instalar nada.

— Onde o critério da §26.1 é provado, e onde ele é conferido à mão

O critério tem três partes, e elas não estão todas no mesmo lugar. Vale saber qual é qual, porque a diferença entre prova automatizada e conferência manual é a diferença entre um critério que resiste ao semestre e um que resiste até alguém esquecer.

A parte que está **provada pela suíte** é a mais forte das três, e ela não depende de o Qt estar instalado. `testes/test_padroes.cpp` traz dois casos que rodam em toda execução de `make verifica`. O primeiro põe duas implementações independentes de `i_apresentacao` sobre o **mesmo** mundo, no mesmo processo, e confere que as duas veem o mesmo sistema - é o que a variante `v2.6`-antes torna impossível, e é a forma testável de dizer que o núcleo não sabe quem está do outro lado. O segundo é o critério mais duro: desenha cinco vezes e exige que o despejo do domínio continue igual byte a byte, porque, se desenhar mudasse estado, a segunda interface veria um sistema diferente da primeira e a separação seria só de arquivo. Os dois trechos estão no fim deste capítulo.

As outras duas partes são **conferidas à mão**, e não estão no portão. A primeira é o `grep` por inclusão de header do Qt dentro de `include/deriva/` e de `src/`, cujo resultado vazio é o que se quer. A segunda é compilar `deriva_nucleo` e a suíte numa máquina sem Qt e confirmar que a contagem de casos não mudou. Nenhuma das duas é difícil, e as duas dependem de alguém se lembrar. Pô-las no portão é acrescentar um alvo de verificação ao `Makefile`, no molde do alvo de `replay`, e é exatamente o que o exercício 2 pede - o que o transforma, se você o fizer, na quinta condição de `make verifica`.

§26.6 O que Este Capítulo Fecha

A janela em Qt é a última entrega da trilha, e ela não acrescenta conceito novo: ela cobra o que foi decidido antes. Vale enumerar, porque é a leitura retrospectiva que o fim do semestre permite e nenhum outro momento permite.

A composição do Cap. 9, que fez mapa **ter** uma grade em vez de **ser** uma grade, é o que permite ao núcleo ser uma biblioteca sem entrada de programa. O `virtual` do Cap. 11 e a posse exclusiva do Cap. 12 são o que fazem o laço de turno percorrer entidades sem saber quais, o que é o que permite ao controlador não saber tampouco. O `replay` do Cap. 16 é o que prova que a regra de jogo não migrou para a interface. A inversão de dependência do Cap. 24 é a fronteira em si. E a interface do Cap. 24 é o que permite empilhar os decoradores da §25.7 sobre qualquer das duas apresentações, a de texto e a de Qt, sem tocar em nenhuma das duas.

Nada disso foi construído para o Qt. O Qt é o teste.

0 avisos

avisos do compilador

aferido por

`make verifica`

`Makefile`

Exercícios Propostos

1. Compile o esqueleto publicado com a opção de Qt ligada e acrescente à janela um controle que não existe nela: um seletor de semente e um botão que reinicia a partida. Conecte-o ao controlador, e confirme que nenhum arquivo de `include/deriva/` foi tocado.
2. Prove a separação de forma verificável, e não por leitura. Compile o alvo `deriva_nucleo` e a suíte de testes numa máquina, ou num contêiner, **sem** Qt instalado, e confirme que a suíte passa com o mesmo número de casos que passava antes da v2.7. Depois procure, com `grep`, por qualquer inclusão de header do Qt dentro de `include/deriva/` e de `src/`, e explique por que um resultado vazio é o que se quer.
3. Peça a um LLM o front-end completo em Qt para o Deriva, com o prompt do §4.8, declarando o portão. Aplique a rubrica do Cap. 4 ao resultado e responda, com a linha do código como evidência: (a) alguma classe de domínio passou a derivar de `QObject`? (b) a dependência aponta da apresentação para o núcleo, ou o contrário? (c) a posse dos widgets segue a árvore do Qt ou está envolvida em ponteiro inteligente? (d) o `main` gerado ainda permite compilar o núcleo sem Qt?
4. Escreva a comparação entre o Observer do Cap. 25, escrito à mão, e `signals/slots`, em uma página. Cubra: como cada um trata o observador destruído sem `desinscrever`; o que cada um exige da classe que notifica; e o que cada um custa em dependência e em tempo de build. Termine dizendo qual dos dois você usaria num projeto sem Qt, e por quê.

O código, extraído do Deriva

Todo trecho abaixo vem de `exemplos/deriva/`, que compila com `-std=c++17 -Wall -Wextra -Wpedantic` sem um aviso e passa `make verifica`. Nenhum foi digitado neste texto.

```

qt/janela.hpp:44 |----- posse no Qt é a exceção declarada-----
44 /// A janela.
45 ///
46 /// **Posse no Qt é diferente.** `QObject` tem árvore de pais, e o pai destrói
47 /// os filhos. Passar um `QWidget*` cru para o construtor do filho ENTREGA a
48 /// posse ao pai - e por isso `unique_ptr` sobre `QWidget` com pai é erro de
49 /// dupla liberação. É a exceção à regra da Aula 12, e ela é declarada e não
50 /// escondida.
51 ///
52 /// O `Q_OBJECT` exige o MOC, um pré-processador que gera código a partir do
53 /// cabeçalho. É por isso que `CMAKE_AUTOMOC` existe, e é o que faz esta classe
54 /// precisar de um sistema de build que o Qt entenda.
55 class janela : public QMainWindow {
56     Q_OBJECT
57
58 public:
59     explicit janela(mundo w, QWidget* pai = nullptr);
60     ~janela() override;
61
62 private slots:
63     /// Um slot é um método comum que o MOC registra para poder ser chamado por
64     /// nome, sem que quem chama conheça o tipo. É a versão do Qt para Observer,
65     /// e o compilador não verifica a assinatura - o erro aparece em execução, na
66     /// forma de uma conexão que não acontece.
67     void ao_mover_norte();
68     void ao_mover_sul();
69     void ao_desfazer();
70
71 private:
72     void redesenhar();
73
74     mundo mundo_;                // o núcleo, intacto
75     historico historico_;        // Command, da v2.6
76     std::unique_ptr<tela_qt> tela_;
77     QPlainTextEdit* visor_ = nullptr;    // filho: o Qt é o dono
78 };

```

`QObject` tem árvore de pais, e o pai destrói os filhos: passar um `QWidget*` cru ao construtor do filho ENTREGA a posse. `unique_ptr` sobre `QWidget` com pai é dupla liberação. É a exceção à regra da Aula 12, e ela é declarada, não escondida.

```

qt/main_qt.cpp:10 |----- o argumento inteiro da aula -----
10 int main(int argc, char** argv) {
11     QApplication app(argc, argv);
12
13     auto m = deriva::mapa::carregar("mapas/estacao-01.txt");
14     if (!m) return 2;
15     deriva::mundo w(std::move(*m));
16     w.acrescentar(std::make_unique<deriva::sonda>(w.setor().entrada()));
17
18     // Nenhuma linha do núcleo mudou para isto existir. É o argumento inteiro da
19     // Aula 26, e ele só é verdade porque a v2.6 extraiu `i_apresentacao`.
20     deriva::qt::janela janela(std::move(w));
21     janela.resize(900, 600);
22     janela.setWindowTitle("Deriva - segundo front-end");
23     janela.show();
24     return app.exec();
25 }

```

Nenhuma linha do núcleo mudou para esta janela existir, e isso só é verdade porque a v2.6 extraiu `i_apresentacao`. É a diferença entre separação demonstrada e separação afirmada.

```

qt/janela.hpp:27 |----- o adaptador, que não é QObject -----
27 /// A implementação Qt de `i_apresentacao`.
28 ///
29 /// Repare no que ela NÃO faz: não conhece regra de jogo, não decide o que
30 /// acontece num turno, não sabe o que é uma parede. Ela recebe um `mundo` e
31 /// desenha. É a mesma interface que `apresentacao_em_texto` implementa, e o
32 /// núcleo não sabe qual das duas está do outro lado.
33 class tela_qt final : public i_apresentacao {
34 public:
35     explicit tela_qt(QPlainTextEdit* alvo) : alvo_(alvo) {}
36
37     void desenhar(const mundo& m) override;
38     void mensagem(std::string_view texto) override;
39
40 private:
41     QPlainTextEdit* alvo_; // observação, não posse: a árvore do Qt é a dona
42 };

```

Ele implementa a mesma interface que `apresentacao_em_texto`, e o núcleo não sabe qual das duas está do outro lado. O widget é guardado como ponteiro cru porque é observação: a árvore do Qt é a dona.

```

qt/janela.cpp:20 |-----`new` com pai ao lado de unique_ptr-----
20 janela::janela(mundo w, QWidget* pai)
21     : QMainWindow(pai), mundo_(std::move(w)) {
22     visor_ = new QPlainTextEdit(this);    // `this` é o pai: o Qt destrói
23     visor_->setReadOnly(true);
24     visor_->setFont(QFont("IBM Plex Mono", 12));
25     setCentralWidget(visor_);
26
27     tela_ = std::make_unique<tela_qt>(visor_);
28
29     QMenu* menu = menuBar()->addMenu("Sonda");
30     QAction* norte = menu->addAction("Norte");
31     QAction* sul = menu->addAction("Sul");
32     QAction* desfazer = menu->addAction("Desfazer");
33
34     // A conexão por ponteiro de função é a forma verificada em compilação. A
35     // forma antiga, com as macros SIGNAL e SLOT, casava strings em execução - e
36     // um erro de digitação virava conexão que nunca acontece, sem aviso.
37     connect(norte, &QAction::triggered, this, &janela::ao_mover_norte);
38     connect(sul, &QAction::triggered, this, &janela::ao_mover_sul);
39     connect(desfazer, &QAction::triggered, this, &janela::ao_desfazer);
40
41     redesenhar();

```

`new QPlainTextEdit(this)` entrega a posse ao pai, e `unique_ptr` sobre o adaptador guarda o que é nosso. As duas formas convivem na mesma função, e a diferença está em quem destrói.

```

┌─── testes/test_padroes.cpp:170 ──── a prova do critério da §26.1 ────
170 TEST_CASE("duas apresentacoes sobre o MESMO mundo, e ele nao sabe qual") {
171     zerar_entidades();
172     mundo w = montar();
173     w.acrescentar(std::make_unique<sonda>(vetor2{1, 1}));
174
175     // (2) Duas implementações independentes da mesma interface, no mesmo
176     //      processo, sobre o mesmo mundo. É o que a v2.7 faz com Qt, e o que a
177     //      variante `v2.6-antes` torna impossível.
178     apresentacao_em_texto a;
179     apresentacao_em_texto b;
180     i_apresentacao& como_a = a;
181     i_apresentacao& como_b = b;
182
183     como_a.desenhar(w);
184     como_b.mensagem("segunda interface");
185     como_b.desenhar(w);
186
187     REQUIRE(a.acumulado().find("entidades 1") != std::string::npos);
188     REQUIRE(b.acumulado().find("> segunda interface") == 0);
189     REQUIRE(b.acumulado().find("entidades 1") != std::string::npos);
190 }

```

Duas implementações da mesma interface, no mesmo processo, sobre o mesmo mundo. É o que a variante v2.6-antes torna impossível, e é a diferença entre separação demonstrada e separação afirmada.

```

┌─── testes/test_padroes.cpp:192 ──── desenhar não altera o mundo ────
192 TEST_CASE("trocar a apresentacao nao muda o estado do dominio") {
193     // (3) O critério mais forte: desenhar não pode alterar o mundo. Se
194     //      desenhar mudasse estado, a segunda interface veria um sistema
195     //      diferente da primeira.
196     zerar_entidades();
197     mundo w = montar();
198     w.acrescentar(std::make_unique<sonda>(vetor2{1, 1}));
199     const std::string antes = w.despejar();
200
201     apresentacao_em_texto tela;
202     for (int k = 0; k < 5; ++k) tela.desenhar(w);
203
204     REQUIRE(w.despejar() == antes);    // byte a byte, depois de cinco renders
205 }

```

Cinco renders, e o despejo byte a byte igual. Se desenhar mudasse estado, a segunda interface veria um sistema diferente da primeira.

Anexo A - Concepts e Ranges (C++20)

Vem do Cap. 20 do v1, íntegro · rotulado C++20 · Deriva v2.1 (opcional)

Este anexo é o Cap. 20 do v1, íntegro. **O que mudou não foi o conteúdo, foi o estatuto.**

O v1 tratava Concepts e Ranges como aula, com duas horas, exercícios e lugar na sequência. O plano v2 desfez isso por um motivo aferido: o material se declarava C++17 em doze lugares e ensinava, como capítulo próprio, duas construções que são C++20, ao mesmo tempo que não tinha uma única ocorrência de `std::string_view`, de ligações estruturadas, de `[[nodiscard]]`, de `std::forward`, de `std::filesystem` ou de `std::tuple`. O tempo de aula gasto no padrão seguinte era tempo que faltava no padrão-alvo, e a correção foi trocar o capítulo por vinte minutos dentro do **Cap. 19**, como reconhecimento de API, com o conteúdo integral preservado aqui.

Três consequências valem estar explícitas, porque elas definem como este anexo se lê.

Nada do material obrigatório depende deste anexo. Nenhum exemplo, nenhum exercício, nenhum laboratório e nenhum item de prova o referenciam como pré-requisito. Ele pode ser saltado inteiro sem que uma linha do resto do livro deixe de fazer sentido.

O código daqui compila em alvo separado, e só nele. A v2.1 do Deriva é `exemplos/deriva/c20/`, atrás da opção `-DDERIVA_COM_CPP20=ON`, e o alvo `deriva_c20` é o único do repositório com `CXX_STANDARD 20`: o núcleo continua em C++17, e é essa separação que faz o rebaixamento ser real em vez de declarado. A trilha do Anexo C registra a v2.1 como opcional, e as versões seguintes partem da v2.0, e não dela. Leia com essa ressalva a linha de abertura da seção de código no fim deste anexo: ela é a mesma em todos os capítulos e fala de `-std=c++17`, porque é gerada uma vez para o livro inteiro; aqui, e só aqui, os trechos vêm do alvo de C++20 e ficam fora de `make verifica`.

O padrão-alvo da disciplina segue sendo C++17. O que está aqui serve para você reconhecer a construção quando a encontrar em código alheio, e para saber o que a linguagem fez depois com os problemas que o Cap. 19 resolveu com `static_assert` e `if constexpr`.

duas constru

construções em `de_texto`

aferido por

`./build/testes "mapa"`

`testes/test_mapa.cpp`

O QUE ESTE CAPÍTULO ENTREGA

- ▶ Usar concepts para restringir parâmetros de template com mensagens de erro legíveis. **(C++20)**
- ▶ Definir concepts próprios com `requires`. **(C++20)**
- ▶ Usar ranges e views para expressão funcional sobre sequências. **(C++20)**
- ▶ Reconhecer, em código alheio, que a construção é de C++20 e não do padrão-alvo.

▲ ATENÇÃO

Nada deste anexo entra em prova, em laboratório ou em entrega, e o motivo não é de gosto. O portão da disciplina compila com `-std=c++17` e `CXX_EXTENSIONS OFF`, e código que só compila com `-std=c++20` **falha o portão**. Se você usar `concept` ou `range` numa entrega, ela não passa, mesmo estando correta. O lugar de exercitar o que está aqui é o alvo opcional, e ele é opcional de verdade: a v2.1 do Deriva não é pré-requisito de nenhuma versão posterior, e a trilha salta da v2.0 para a v2.2 sem ela.

§A.1 O Problema dos Templates sem Restrições (C++20)

Templates de C++17 aceitam qualquer tipo - erros de tipo só são detectados ao tentar usar um tipo incompatível, gerando mensagens de erro dificilmente legíveis. Concepts resolvem isso declarando explicitamente o que um tipo deve suportar.

O Cap. 19 chegou até a metade desse caminho com o que o padrão-alvo oferece, e vale ler a §19.4 ao lado desta seção. Um `static_assert` no corpo da classe troca o erro de dentro de `<vector>` por uma frase escrita à mão, no ponto em que o tipo errado foi escolhido, e resolve o caso prático. O que ele **não** faz é o que separa os dois recursos: a restrição não faz parte da assinatura, não participa da resolução de sobrecarga, e não há como perguntar se um tipo a satisfaz sem tentar instanciar.

Um `concept` é a mesma exigência promovida a entidade nomeada da linguagem. Ela tem nome, é reutilizável, compõe com outras através de conjunção e disjunção, aparece na assinatura no lugar de `typename`, e pode ser consultada numa expressão booleana. É por isso que ele permite duas coisas que o `static_assert` não permite: escrever duas sobrecargas do mesmo template, uma para tipos que satisfazem o `concept` e outra para os que não, deixando o compilador escolher; e receber, na violação, a mensagem que nomeia **qual** requisito falhou, em vez da que diz que nenhuma sobrecarga era viável.

O `concept` que o alvo opcional escreve chama-se `guardavel`, e ele é a tradução direta dos três `static_assert` de `grade_generica.hpp`: construtor padrão, não ser referência, e não ser `bool`. A forma dele está extraída no fim deste anexo, e são duas linhas com três conjunções. Duas coisas nelas valem atenção. A primeira é que os requisitos vêm de `<concepts>` já nomeados - `std::default_initializable`, `std::same_as` -, o que significa que a biblioteca padrão passou a ter vocabulário de restrição, e não só de tipo. A segunda é a composição: `&&` entre `concepts` produz outro `concept`, e é isso que o `static_assert` nunca soube fazer.

O uso aparece na declaração da classe, `template <guardavel T> class grade_restrita`, e é aqui que a diferença deixa de ser cosmética. A restrição passou a fazer parte da assinatura, o que tem três consequências: ela participa da resolução de sobrecarga, ela pode ser consultada numa expressão booleana - o arquivo termina em quatro `static_assert` que perguntam `guardavel<celula>`, `guardavel<int>`, `!guardavel<bool>` e `!guardavel<int&>`, e as quatro respostas são conferidas em compilação -, e a mensagem de erro muda de lugar. Com `static_assert`, o compilador aponta a linha do `assert`, dentro da classe. Com `concept`, ele aponta a **chamada** e diz qual restrição falhou. É essa mudança de endereço, e não a economia de digitação, que é a razão de os `concepts` existirem, e é o que o exercício 1 manda você transcrever.

§A.2 Ranges - Programação Funcional sobre Sequências (C++20)

Ranges é o assunto vizinho de Concepts no mesmo padrão, e resolve outro incômodo. O Cap. 21 apresenta os algoritmos da STL na forma clássica, com um par de iteradores por chamada, e essa forma tem dois custos de escrita. O primeiro é a repetição de `c.begin()`, `c.end()` em toda chamada, quando o que se quer quase sempre é o contêiner inteiro. O segundo é a composição: filtrar e depois transformar exige um contêiner intermediário para guardar o resultado do primeiro passo, com a alocação que isso implica.

Ranges resolve o primeiro passando o contêiner, e o segundo com **views**, que são composições preguiçosas: nada é calculado até que alguém percorra o resultado, e nenhum contêiner intermediário é criado. O operador `|` encadeia as views, e a leitura fica na ordem em que as coisas acontecem, da esquerda para a direita, que é o oposto da leitura de chamadas aninhadas.

No alvo opcional o exemplo é `glifos_de_parede`, extraída no fim deste anexo: ela percorre as células da grade, filtra as que têm o glifo de parede e para nas dez primeiras, tudo numa expressão só, com `views::filter` e `views::take` encadeados por `|`. Em C++17 o equivalente seria um `copy_if` para um vetor temporário e um segundo laço depois - dois percursos e uma alocação -, e é essa alocação que a composição preguiçosa não faz.

Vale registrar que o ganho aqui é de expressão, e não de desempenho: a versão com um laço explícito sobre a grade contígua é rápida, e a composição de views não a supera. O que ela dá é a leitura na ordem em que as coisas acontecem, e a ausência do contêiner que existia só para ser descartado.

E vale registrar a armadilha, que é a mesma do `std::string_view` do Cap. 3, pela mesma razão. **Uma view não possui os elementos que enxerga.** Guardar uma view cujo contêiner de origem já foi destruído, ou modificado de forma a invalidar iteradores, é referência pendurada, e o compilador não avisa. View se compõe e se percorre no mesmo escopo; o que se guarda é o resultado materializado.

Exercícios Propostos

Os itens abaixo pressupõem o alvo opcional de C++20, e nenhum deles é exigido. Eles só fazem sentido depois dos exercícios 1 a 5 do Cap. 19, porque comparam com o que foi feito lá.

1. Compile `grade_de<bool>` no alvo de C++17 e `grade_restrita<bool>` no alvo de C++20, e transcreva as duas mensagens de erro inteiras, sem resumir. Aponte, em cada uma, qual linha o compilador culpou e qual requisito ele nomeou. Depois escreva o seu próprio concept, com nome diferente de `guardavel`, exigindo também que `T` seja copiável, e diga o que a terceira mensagem acrescenta.
2. Escreva um concept `desenhavel_como_glifo<T>` que exija de `T` a conversão para `char` através de uma função livre `glifo_de`. Use-o para restringir `grade<T>::despejar()`, e explique o que ele permite que o `if constexpr` do Cap. 19 não permitia.
3. Reescreva o cálculo do campo de visão com `std::views::filter` e `std::views::transform`, sem contêiner intermediário. Meça as duas versões com `std::chrono`, num setor grande, e diga qual é mais rápida e por quê.

4. Provoque a view pendurada: componha uma view sobre um `std::vector` local, devolva-a da função, e percorra-a no chamador. Explique por que o compilador aceitou, e compare com o caso do `string_view` do Cap. 3.

5. Compile o mesmo arquivo com `-std=c++17` e com `-std=c++20` e liste, a partir dos erros do primeiro, tudo o que o seu código passou a depender do padrão novo. É este o exercício que responde à pergunta de por que este material é um anexo.

O código, extraído do Deriva

Todo trecho abaixo vem de `exemplos/deriva/c20/`, que compila em alvo **separado** com `-std=c++20 -Wall -Wextra -Wpedantic` sem um aviso - e que fica **fora** do portão `make` verifica, porque o padrão-alvo da disciplina é C++17. Nenhum foi digitado neste texto, e nada do material obrigatório depende deles.

```

c20/restricoes.hpp:36 |----- o concept no lugar do static_assert -----
36 concept guardavel = std::default_initializable<T> && !std::is_reference_v<T> &&
37                      !std::same_as<T, bool>;

```

A diferença que se vê é a mensagem de erro: com `static_assert` o compilador aponta a linha do `assert`; com `concept`, aponta a CHAMADA e diz qual restrição falhou. É a razão de os `concepts` existirem.

```

c20/restricoes.hpp:61 |----- ranges, sem contêiner intermediário -----
61 /// Ranges no lugar do laço, sobre as células da grade.
62 ///
63 /// O que se ganha é composição sem contêiner intermediário: 'filter' e
64 /// 'transform' são vistas preguiçosas, e nada é copiado até alguém iterar. Em
65 /// C++17 o equivalente seria um 'copy_if' para um vetor temporário e um
66 /// 'transform' depois - dois laços e uma alocação.
67 [[nodiscard]] inline std::string glifos_de_parede(const grade_restrita<celula>& g) {
68     std::string s;
69     for (const celula& c : g.todas())
70         | std::views::filter([](const celula& x) {
71             return x.glifo == '#';
72         })
73         | std::views::take(10)) {
74         s.push_back(c.glifo);
75     }

```

`filter` e `transform` são vistas preguiçosas: nada é copiado até alguém iterar. Em C++17 o equivalente seria um `copy_if` para um vetor temporário e um `transform` depois - dois laços e uma alocação.

Anexo B - Referência rápida de C++17

Novo. Tabela de consulta para prova e laboratório. Mesmo conteúdo da página do site - uma fonte, dois meios.

Este anexo existe por uma razão aferida no material anterior, e vale registrá-la, porque ela é o argumento de por que a tabela merece páginas. O livro v1 **se declarava C++17 em doze lugares** e não tinha uma única ocorrência de `std::string_view`, de ligações estruturadas, de `[[nodiscard]]`, de `std::forward`, de `std::filesystem` ou de `std::tuple`, ao mesmo tempo que ensinava Concepts e Ranges, que são C++20, como capítulo próprio.

O padrão declarado, portanto, não era o padrão ensinado, e a distorção tinha as duas pontas: faltava o que o portão exige e sobrava o que o portão recusa. A correção foi distribuir as construções que faltavam por cerca de dez capítulos, rebaixar Concepts e Ranges ao **Anexo A**, e escrever esta tabela, que é a lista do que passou a ser exigido.

A tabela é de consulta, e as quatro colunas respondem a quatro perguntas. A primeira diz **o que** é a construção. A segunda diz **onde** ela entra, para que a consulta devolva também o lugar do material em que o mecanismo é desenvolvido, e não apenas a sintaxe. A terceira dá a **forma**, isto é a notação mínima que faz reconhecer a construção ao vê-la. A quarta é a que mais paga em prova: **a armadilha** que costuma acompanhar cada uma, escrita como a frase que você gostaria de ter lido antes de errar.

Uma observação de escopo, para que a terceira coluna não seja mal lida. Ela traz notação de referência rápida, e não trecho de programa: `auto [a, b] = par;` é a forma da ligação estruturada, sem contexto, sem inclusão e sem tipo declarado. Todo trecho de código deste livro que se pretende compilável vem extraído de exemplos/deriva/, e é sempre acompanhado do arquivo e da linha de onde saiu; nada aqui tem essa pretensão.

§B.1 As quinze construções

construção	cap.	forma	o que costuma dar errado
<code>std::string_view</code>	3	<code>void f(std::string_views)</code>	não possui os bytes que enxerga, e um <code>string_view</code> guardado além da vida da <code>string</code> de origem fica pendurado
ligações estruturadas	3	<code>for (auto& [pos, cel] : mapa)</code>	sem o <code>&</code> a ligação copia o elemento; use <code>const auto&</code> quando o laço apenas lê
<code>[[nodiscard]]</code>	3, 7	<code>[[nodiscard]] bool carregar(...)</code>	vale no que retorna status ou recurso, e o warning que ele provoca é o objetivo

construção	cap.	forma	o que costuma dar errado
<code>[[maybe_unused]]</code>	3	<code>[[maybe_unused]] int n = f();</code>	para o parâmetro que só é usado dentro de <code>assert</code> ou em um dos ramos compilados
<code>if constexpr</code>	19	<code>if constexpr (std::is_integral_v<T>)</code>	poda o ramo em tempo de compilação, e o ramo podado nem precisa ser válido para o T em questão
<code>std::optional</code>	20	<code>std::optional<mapa> carregar(...)</code>	modela ausência, e não falha; para reportar erro, use exceção ou <code>variant</code>
<code>std::variant</code>	20	<code>std::variant<mapa, erro></code>	soma de tipos fechada, consultada com <code>std::visit</code> ; o acesso por <code>get</code> com o tipo errado lança
<code>std::filesystem</code>	20	<code>fs::exists(caminho)</code>	verificar a existência e abrir são duas operações, e a corrida entre elas é real: abra e trate a falha
<code>std::clamp</code>	21	<code>std::clamp(x, 0, largura - 1)</code>	substitui o par <code>min/max</code> aninhado, porém devolve referência - não o alimente com temporário
lambdas	21, 25	<code>[&](const auto& e) { return e.vivo(); }</code>	Strategy sem herança; a captura por referência só é segura enquanto vive o escopo capturado
<code>std::forward</code>	14	<code>template<class T> void add(T&& x)</code>	encaminhamento perfeito; T&& em parâmetro de <code>template</code> dedutível é referência universal, e não referência a rvalue
CTAD	15	<code>std::pair p{1, 2.0};</code>	dedução a partir do construtor, o que dispensa <code>make_pair</code> ; agregado próprio pode exigir guia de dedução
fold expressions	19	<code>return (... + args);</code>	variádico sem recursão; o pacote vazio exige valor inicial ou operador com identidade definida
<code>inline variables</code>	7	<code>inline static int vivos = 0;</code>	membro estático definido no próprio cabeçalho, sem a definição avulsa no <code>.cpp</code> que o C++14 exigia
<code>std::byte</code>	7	<code>std::byte b{0xFF};</code>	não é aritmético nem caractere, e a conversão para inteiro passa por <code>std::to_integer</code>

§B.2 Como usar isto em prova

A prova desta disciplina é escrita, em papel, e não cobra sintaxe de memória. O que ela cobra, e onde esta tabela ajuda, é a quarta coluna: dado um trecho, dizer o que vai acontecer e por quê.

Três padrões de questão se repetem, e os três se preparam relendo a coluna da armadilha. O primeiro é o de **tempo de vida**: um `string_view`, uma view de ranges ou uma lambda com captura por referência que sobrevive ao que enxergava. Os três casos são o mesmo mecanismo, e reconhecer isso vale mais que memorizar os três.

O segundo é o de **momento da decisão**: o que é resolvido em compilação e o que é resolvido em execução. `if constexpr`, `static_assert` e CTAD estão do lado da compilação; `virtual`, `dynamic_cast` e `std::visit` estão do lado da execução. A pergunta “quando isto é decidido?” resolve a maioria das questões de despacho.

O terceiro é o de **quem tem a posse**: `optional` não possui o que não tem, `string_view` não possui os bytes, `unique_ptr` possui com exclusividade, e ponteiro cru não diz nada. É a pergunta central da disciplina, e ela reaparece em cada linha desta tabela que trate de referência ou de ponteiro.

§B.3 O que está fora, de propósito

Não está aqui o que é C++20 ou posterior, e a lista do que ficou de fora é curta e vale saber: `Concepts` e `Ranges` estão no Anexo A, rotulados; `std::expected` é C++23 e aparece como pergunta de pesquisa no fim do Cap. 20; módulos, corrotinas e `std::format` não aparecem em lugar nenhum. Nenhuma dessas construções passa pelo portão da disciplina, que compila com `-std=c++17` e `CXX_EXTENSIONS OFF`.

Também não estão aqui as construções de C++11 e C++14 que a disciplina usa sem cerimônia por serem anteriores ao padrão-alvo, tais como `auto`, `nullptr`, `override`, `final`, `noexcept`, os ponteiros inteligentes, `std::move` e a inicialização uniforme. Elas são pressuposto, e estão nos Caps. 3, 12, 13 e 14; esta tabela é o **delta** do C++17 sobre o que já se usava.

Uma ponta solta, que vale amarrar. A prosa de abertura cita `std::tuple` entre as construções de que o v1 não tinha uma ocorrência, e ele não tem linha própria na tabela. É deliberado: no Deriva o que se usa é `std::pair`, e ele entra por duas portas que **estão** na tabela, que são CTAD, no Cap. 15, e as ligações estruturadas, no Cap. 3 - a tabela de comandos de `src/main.cpp` é um array de pares percorrido com `for (const auto& [nome, delta] : tabela)`. `std::tuple` de três ou mais elementos não aparece em entrega nenhuma, e dar-lhe linha própria seria exhibir como exigência o que é apenas reconhecível.

Anexo C - O Deriva: as 20 versões

Novo. Cada versão com o capítulo que a introduz e as variantes deliberadamente quebradas. Mesmo conteúdo da trilha do site - uma fonte, dois meios.

§C.1 Por que a trilha existe

O Deriva é um roguelike de terminal: uma sonda de inspeção percorre uma estação orbital abandonada, através do console. Ele é o artefato que atravessa a disciplina inteira, e a tabela adiante é a forma dessa travessia: **cada aula entrega uma versão que compila**, e a versão seguinte parte dela.

Três decisões estão embutidas nesse formato, e vale nomeá-las, porque nenhuma é óbvia.

A primeira é que **não há projeto final**. Não existe o momento em que o estudante recebe um enunciado grande e monta tudo de uma vez; existe uma sequência de vinte incrementos, cada um pequeno o bastante para caber num encontro e grande o bastante para acrescentar um conceito. O que isso resolve é o problema de quem não consegue começar: o começo é sempre a versão anterior, que compila.

A segunda é que **o conceito chega depois da necessidade**. A ordem da tabela não é a ordem do índice da linguagem, é a ordem em que o sistema pede as coisas. Ponteiro inteligente entra na v1.2 porque a v1.0 já tem uma hierarquia e alguém precisa possuir as entidades; template entra na v2.0 porque o contador já foi escrito à mão três vezes; serialização entra na v2.5 porque já existe partida a salvar. Conceito que chega antes de a necessidade aparecer é solução para um problema que o estudante não teve, e a §19.6 trata disso longamente.

A terceira é que **a versão anterior continua rodando**. Nenhum incremento quebra o anterior, e é o portão `make verifica` que garante isso, com as suas quatro condições: zero aviso, testes verdes, replay idêntico byte a byte, e o contador de instâncias vivas fechando em zero.

Uma nota de contagem, para que a tabela não pareça errada. Ela tem vinte e uma linhas: as **vinte** versões da trilha obrigatória, da v0.0 à v2.7, mais a **v2.1**, que é o alvo opcional de C++20 do Anexo A. A v2.1 não é pré-requisito de nada, e a trilha salta da v2.0 para a v2.2 sem ela.

E uma nota de estado, porque a tabela é promessa até que alguém a confira: em código, hoje, o repositório de exemplos está na **v2.7**, com o alvo opcional de C++20 ao lado, e as **quatro** variantes deliberadamente quebradas da coluna da direita existem, cada uma com o seu LEIA-ME.md. O que a tabela descreve é o que compila, e não o que se pretende compilar. Onde uma versão entrega menos do que a linha promete, o capítulo correspondente diz o que ficou de fora e por quê, em lugar de calar.

0 avisos

avisos do compilador

aferido por

make verifica

Makefile

§C.2 A tabela

versão	capítulo	entrega	conceitos	variante quebrada
v0.0	2	esqueleto que compila: CMake, FetchContent, main vazio	CMake, FetchContent, FTXUI v5.0.0 e Catch2 como SYSTEM, make verifica	-
v0.1	7	vetor2, célula e o contador vivos	encapsulamento, const-correctness, static, this, [[nodiscard]]	-
v0.2	8	grade e terminal_bruto	construtores, destrutor, lista de inicialização, RAI, instrumentação de ciclo de vida	v0.2-quebrada - terminal_bruto sem destrutor
v0.3	9	mapa, carregamento de arquivo e o PRIMEIRO render	composição, regra do zero e do três, std::optional, std::filesystem, string_view	v0.3-quebrada - cópia rasa em grade
v1.0	10	hierarquia entidade → sonda / drone / item	herança pública, override, final, subobjeto base	-
v1.1	11	desenhar() e agir() virtuais; destrutor virtual	funções virtuais, classe abstrata, vptr e vtable, destrutor virtual	v1.1-quebrada - destrutor não virtual na base
v1.2	12	mundo com vector<unique_ptr>	posse exclusiva, unique_ptr, make_unique, ponteiro cru como contraexemplo	-
v1.3	13	grafo de conexões da estação com shared_ptr	posse compartilhada, shared_ptr, weak_ptr, contagem de referências, o ciclo que vaza	-
v1.4	14	movimento em grade e mapa	rvalue refs, std::move, regra dos cinco, noexcept, std::forward	-
v1.5	15	operadores de vetor2 e mapa[pos]	sobrecarga, funções livres, operator<<, operator[] const e não-const	-
v1.6	16	testes de FOV e caminho + replay determinístico	Catch2, semente fixa, roteiro gravado, despejo idêntico byte a byte	-
v1.7	17	sonda_reparadora: o diamante	herança múltipla, interface pura, herança virtual, ordem de construção	-
v1.8	18	inspetor de entidade no console	RTTI, dynamic_cast, typeid, quando NÃO usar	-
v2.0	19	grade genérica e contador_de_-instancias	templates, if constexpr, static_assert, CRTP	-
v2.1 (opcional, C++20)	Anexo A	restrições em grade por concept	Concepts e Ranges - fora do padrão-alvo, alvo de compilação separado	-

versão	capítulo	entrega	conceitos	variante quebrada
v2.2	20	erros de carregamento de mapa	exceções, garantias, std::optional, std::variant, std::filesystem	-
v2.3	21	FOV e inventário com algoritmos	STL, lambdas, std::clamp, std::size	-
v2.4	22	thread de entrada separada do render	std::thread, std::mutex, corrida de dados - panorâmica	-
v2.5	23	salvar e carregar partida	serialização, versionamento de formato, compatibilidade regressiva	-
v2.6	24, 25	refatoração do mundo + os seis padrões	SOLID; Command, State, Observer, Factory, Strategy (lambda), Composite	v2.6-antes - mundo como god class
v2.7	26	front-end Qt sobre o mesmo núcleo	QObject, signals/slots, separação domínio/apresentação	-

§C.3 A variante deliberadamente quebrada

A coluna da direita traz o recurso pedagógico mais valioso que o sistema-base anterior tinha, e que foi preservado por inteiro nesta migração: a **variante deliberadamente quebrada**.

Ela não é erro do material. É uma cópia da versão, com um defeito plantado, plausível, do tipo que se comete de verdade, e com uma propriedade que decide tudo: **o compilador não avisa**. As quatro variantes passam por -Wall -Wextra -Wpedantic sem uma palavra, porque em nenhuma delas a linguagem está fazendo algo diferente do que foi pedida. É esse silêncio que as torna úteis: elas ensinam que o portão de compilação é necessário e não é suficiente, e que a categoria de defeito que esta disciplina trata é justamente a que atravessa o portão.

▲ ATENÇÃO

A variante quebrada é para ser **rodada**, e não lida. Ler o código de variantes/v0.2-quebrada/ e concluir “falta o destrutor” ensina a reconhecer a ausência num arquivo de trinta linhas, que não é a habilidade cobrada. Rodar, ver o terminal parar de ecoar depois que o programa sai, e só então procurar a causa, é o que ensina. Cada variante tem LEIA-ME.md com o roteiro de observação na ordem certa: sem ferramenta, com o contador, com o alocador, com o ASan, com o compilador. E rode a v0.2 com `< /dev/null` na primeira vez, para não precisar digitar reset às cegas.

A variante da v0.2 tem estatuto especial, e não é caça ao bug: ela é demonstração de aula. `terminal_bruto` sem destrutor põe o terminal em modo bruto e não o restaura, e a consequência acontece **depois** que o programa sai, no terminal do estudante, que fica sem eco e sem Enter. É a melhor demonstração de RAII que a disciplina tem, e não é metáfora.

§C.4 As três caças ao bug

Uma vez por unidade, em dupla, cada equipe recebe uma versão do Deriva plausível e errada. A ordem de trabalho é cobrada, e é sempre a mesma: **reproduzir** a falha; **explicar em uma frase antes de tocar no código**; **corrigir**; **provar** a correção. O instrumento é a rubrica de revisão do Cap. 4, e o item dela que mais pesa é o que distingue causa de sintoma.

Caça ao bug 1, semana 5, v0.3-quebrada: a cópia rasa em grade. A grade guarda `celula*` cru, declara destrutor e esquece a cópia, e portanto duas grades passam a apontar para o mesmo buffer. Os dois objetos são construídos e destruídos corretamente, e o segundo destrutor libera memória já liberada. O que a torna instrutiva é o que o contador faz: vivos **fecha em zero**, porque nada de errado aconteceu com objetos, e o defeito é de recurso. É o limite do instrumento, ensinado pelo próprio instrumento, e é assunto do Cap. 9. Ela acontece no mesmo encontro da Prova 1, e o Cap. 9 registra a consequência disso.

Caça ao bug 2, semana 9, v1.1-quebrada: o destrutor não virtual na base. Deletar uma sonda através de `entidade*` chama o destrutor da base e não o da derivada. Aqui o contador funciona, e é o oposto do caso anterior: vivos nunca volta a zero, e o gdb com ponto de parada em destrutor mostra que `~sonda()` nunca roda. As duas ferramentas do Cap. 2 se combinam, e o Cap. 11 conduz.

Caça ao bug 3, semana 13, v2.6-antes: a refatoração que mudou a saída. A mais difícil das três, porque não há travamento, não há vazamento, e o contador fecha em zero. O programa roda e parece certo; o replay acusa uma linha diferente no despejo. O oráculo é o do Cap. 16, a lição é a do Cap. 24, e ela é uma frase: refatoração correta é a que não muda a saída.

Repare na progressão que as três desenham, porque ela é o argumento de haver três e não uma. Na primeira, o contador **mente**: fecha em zero e a memória foi liberada duas vezes, porque ele conta objetos e não recursos. Na segunda, o contador **acusa**: não fecha em zero, e o gdb com ponto de parada em destrutor mostra qual destrutor não rodou. Na terceira, o contador **cala**: fecha em zero, os testes passam, e o único instrumento que sobra é a comparação byte a byte com a execução anterior. Cada caça retira uma ferramenta, e a última retira todas menos a que o Cap. 16 construiu com dezessete semanas de antecedência.

Glossário

Termos fundamentais da disciplina, em ordem alfabética. Cada verbete registra o capítulo em que o conceito entra, de forma que a consulta devolva também o lugar do material onde o mecanismo é desenvolvido. Os verbetes coincidem com os do glossário do site: uma fonte, dois meios.

Abstração (Cap. 1): Mecanismo pelo qual detalhes de implementação são ocultados, expondo apenas o essencial para o usuário de uma classe ou módulo.

AddressSanitizer (ASan) (Cap. 2): Instrumentação do compilador que detecta em execução erros de memória: uso após liberação, acesso fora dos limites, vazamentos. Nesta disciplina ele é **ferramenta**, e não **portão**: as máquinas do laboratório não o têm, e o portão de aceitação usa as três técnicas sem dependência externa - o contador de instâncias vivas, a instrumentação de ciclo de vida, e o gdb com ponto de parada em destrutor.

auto (Cap. 3): Palavra-chave de C++11 que instrui o compilador a deduzir o tipo de uma variável a partir da expressão de inicialização. Obrigatório para guardar uma lambda, cujo tipo não tem nome que se possa escrever.

Caça ao bug (Caps. 9, 11 e 24): Atividade em dupla, uma por unidade, em que cada equipe recebe uma versão do Deriva plausível e errada. A ordem de trabalho é cobrada: reproduzir a falha, explicá-la em uma frase **antes** de tocar no código, corrigir, e provar a correção. O instrumento de avaliação é a rubrica de revisão do Cap. 4.

Classe abstrata (Cap. 11): Classe com pelo menos uma função virtual pura (= 0); não pode ser instanciada diretamente.

CMake (Cap. 2): Sistema de build multiplataforma para projetos C++; gera Makefiles ou projetos de IDE a partir de CMakeLists.txt. No Deriva ele também fixa as dependências por FetchContent, com versão travada, e as declara como SYSTEM, para que o portão de zero aviso incida apenas sobre o código do estudante.

Composição (Cap. 9): Relação “tem um” entre classes, alternativa à herança quando não há relação “é um”. mapa **compõe** uma grade: ele tem nome e ponto de entrada, que grade nenhuma tem, e não faz sentido passar um mapa onde se espera uma grade.

Concept (C++20) (Anexo A): Restrição sobre parâmetro de template promovida a entidade nomeada da linguagem, verificável, componível e capaz de aparecer na assinatura. Fora do padrão-alvo da disciplina; o equivalente em C++17 é static_assert no corpo da classe.

const-correctness (Cap. 7): Prática de marcar como const todos os métodos, parâmetros e variáveis que não devem modificar dados. No Deriva ela anda junto com [[nodiscard]]: um método const que devolve valor e não altera nada tem exatamente uma finalidade, e descartar o valor não pode ser intencional.

Contador de instâncias vivas (Cap. 7): `inline static int vivos`, incrementado no construtor e decrementado no destrutor. É o detector de vazamento que a disciplina usa do Cap. 7 ao fim do semestre, sem depender de sanitizer, e é a quarta condição do portão `make verifica`. Conta objetos, e não recursos: a cópia rasa da caça ao bug 1 fecha em zero. Não é seguro entre threads, e o Cap. 22 mostra exatamente esta variável perdendo um incremento.

C RTP (Cap. 19): *Curiously Recurring Template Pattern*: a classe derivada se passa como argumento de template para a própria base, o que resolve o polimorfismo em tempo de compilação, sem `vptr`. Aqui o alvo concreto é generalizar o contador vivos em `contador_de_instancias<T>`, e o que motiva o template é a repetição de tê-lo escrito à mão em três classes.

Deriva (Anexo C): O sistema-base da disciplina: um roguelike de terminal em que uma sonda de inspeção percorre uma estação orbital abandonada, através do console. Cada aula entrega uma versão que compila, da v0.0 à v2.7, e cada versão parte da anterior.

Desenrolar da pilha (Caps. 8 e 20): Processo pelo qual, ao ser lançada uma exceção, os destrutores dos objetos já construídos no caminho são chamados, de dentro para fora, até que um handler a capture. A exceção não pula os destrutores: ela os chama. É esta garantia, e nada mais, que faz RAII funcionar.

Despacho dinâmico (Cap. 11): Resolução em tempo de execução de qual implementação de um método virtual chamar, através da `vtable`.

Destrutor virtual (Cap. 11): Destrutor declarado `virtual` em classe base; garante que o destrutor da classe derivada seja chamado ao destruir através de ponteiro para a base. A sua ausência é a caça ao bug 2, e o contador vivos é o que a acusa.

DIP (Cap. 24): *Dependency Inversion Principle*: módulos de alto nível dependem de abstrações, e não de implementações concretas. No Deriva é o princípio cujo retorno está agendado, porque é ele que permite ao Cap. 26 pôr um front-end Qt sobre o mesmo núcleo.

dynamic_cast (Cap. 18): Operador de conversão de tipo verificado em execução; devolve `nullptr` quando aplicado a ponteiro incompatível, e lança `std::bad_cast` quando aplicado a referência.

Encapsulamento (Cap. 7): Agrupamento de dados e operações em uma classe, com controle de acesso. O critério para tornar um atributo privado é a existência de invariante a proteger, e não a regra automática: `vetor2` é um agregado de campos públicos porque qualquer par de inteiros é um `vetor2` válido.

event loop (Cap. 26): Laço que retira eventos de uma fila e os despacha para handlers ou slots; núcleo de aplicações Qt. Ele inverte o controle em relação ao laço próprio do Deriva de terminal, e é por isso que, sob Qt, o turno passa a ser resolvido dentro de um slot.

Factory Method (Cap. 25): Padrão que concentra num lugar a decisão de qual classe construir, devolvendo a base. No Deriva ele converte o glifo do mapa, ou a etiqueta de tipo do arquivo de partida, no objeto da classe certa.

Fatiamento de objeto (Cap. 10): Guardar um objeto de classe derivada por valor em contêiner da classe base descarta a parte específica da derivada. O sintoma é comportamento da base onde se esperava o da derivada, sem aviso e sem erro de execução.

Garantia de exceção (Cap. 20): O que uma função promete quando uma exceção passa por ela. São quatro níveis: `noexcept`, que é a promessa de não lançar, verifi-

cada em execução; **forte**, que é tudo ou nada, sem efeito parcial observável; **básica**, em que nada vazou e nenhuma invariante foi violada, porém o objeto pode ter mudado; e nenhuma, que é o objeto em estado inconsistente.

GDB (Cap. 2): Depurador do projeto GNU. A terceira das três técnicas que substituem o sanitizer ausente é o gdb com ponto de parada em destrutor, que mostra qual destrutor roda e de onde é chamado.

God class (Cap. 24): Classe que acumulou responsabilidades até ter vários motivos independentes para mudar. O sintoma prático não é o tamanho, é o que ela impede: no Deriva, testar a resolução de turno exigia um terminal, porque desenhar estava na mesma classe.

Herança múltipla (Cap. 17): Capacidade de uma classe derivar de mais de uma classe base; C++ a suporta, Java e C# a restringem a interfaces. O caso seguro em C++ é o de interfaces puras, sem dado na base, que não produz diamante.

Herança virtual (Cap. 17): Mecanismo de C++ para resolver o problema do diamante: garante que a classe base compartilhada tenha apenas um subobjeto na hierarquia.

if constexpr (Cap. 19): Construção de C++17 que decide o ramo em tempo de compilação. O ramo descartado é removido antes da instanciação, e portanto **não precisa nem ser válido** para o tipo em questão, o que é a propriedade que a distingue do if comum e a que a faz substituir SFINAE.

Instrumentação de ciclo de vida (Cap. 8): Segunda das três técnicas que substituem o sanitizer ausente: construtores, destrutores, cópia e movimento imprimindo a própria execução, de forma que a ordem de construção e de destruição fique observável. No Deriva é `src/instrumento.cpp`, e a opção `--traco` imprime o traço no fim.

ISP (Cap. 24): *Interface Segregation Principle*: interfaces pequenas e específicas, e clientes que não dependem de métodos que não usam. O sintoma da violação é o método virtual puro implementado com corpo vazio.

Lambda (Cap. 21): Expressão que produz um objeto invocável. Na leitura mais útil aqui, é açúcar sintático para uma classe anônima: a lista de captura são os membros, e o corpo é o `operator()`, que é `const` por padrão, o que é por que modificar uma captura por cópia exige `mutable`. Captura por referência é segura enquanto vive o escopo capturado.

Ligações estruturadas (Cap. 3): Construção de C++17 que decompõe um agregado, um par ou uma tupla em nomes: `for (const auto& [nome, delta] : tabela)`. Sem o `&` a ligação copia o elemento.

LLM (Cap. 4): *Large Language Model*: modelo de linguagem de grande escala treinado em texto e código, usado como assistente de programação. Nesta disciplina ele vem acompanhado de uma rubrica de revisão, que é o instrumento das três caças ao bug.

LSP (Cap. 24): *Liskov Substitution Principle*: objetos de uma subclasse devem poder substituir a superclasse sem alterar o comportamento correto do programa. A regra prática tem duas metades: a derivada não pode exigir mais que a base, nem prometer menos. Não é sobre assinatura, e portanto o compilador não a verifica.

lvalue (Cap. 14): Expressão com identidade e endereço acessível, que pode aparecer à esquerda de uma atribuição.

[[nodiscard]] (Cap. 3): Atributo de C++17 que faz o compilador avisar quando o valor devolvido é descartado. Vale no que devolve status ou recurso, e o aviso que ele provoca é o objetivo.

noexcept (Caps. 14 e 20): Especificador que promete que a função não lança. É verificado em execução, e quebrar a promessa chama `std::terminate`. Num construtor de movimento a marca muda o comportamento, e não só a documentação: `std::vector` só usa movimento ao realocar se ele for `noexcept`.

nullptr (Cap. 3): Literal de tipo `std::nullptr_t`, introduzido em C++11 para representar ponteiro nulo, em lugar de `NULL`, que é apenas o inteiro zero.

Observer (Cap. 25): Padrão em que interessados são notificados quando o estado de outro objeto muda. O defeito clássico da versão escrita à mão é o observador pendurado, e as três respostas são a desinscrição no destrutor, o `weak_ptr`, e a garantia de tempo de vida documentada. Signals e slots do Qt são este padrão implementado pelo framework.

OCP (Cap. 24): *Open/Closed Principle*: classes abertas para extensão e fechadas para modificação. Fechar para modificação custa uma hierarquia e uma indireção, e só se paga quando o conjunto de casos precisa crescer.

Otimização de string curta (Cap. 14): A `std::string` guarda cadeias curtas dentro do próprio objeto, sem alocar no heap - até 15 caracteres, neste alvo. A consequência prática, medida em testes/`test_move_string.cpp`: mover uma string curta **copia os bytes**, porque não há ponteiro a roubar, enquanto mover uma longa transfere o ponteiro de heap sem copiar conteúdo algum. Em nenhum dos dois casos a origem fica intacta nesta `libstdc++` - ela esvazia nos quatro -, e é justamente por isso que o teste errado passa: a origem de um movimento fica em estado **válido mas não-especificado**, que não é obrigatoriamente vazio, e depender do vazio é depender de detalhe de implementação.

override (Cap. 11): Especificador de C++11 que faz o compilador confirmar que a função realmente sobrescreve um virtual da base. Escrivê-lo corretamente não é o mesmo que respeitar a substituição de Liskov.

Padding (Cap. 7): Bytes que o compilador insere entre membros, e no fim do objeto, para respeitar o alinhamento exigido por cada tipo. A ordem de declaração é sua, o alinhamento não é, e da combinação sai o `sizeof`: a célula do `Deriva` ocupa 12 bytes agrupada por tamanho e 16 na ordem em que se pensa nela.

Polimorfismo (Cap. 11): Capacidade de objetos de tipos diferentes responderem à mesma mensagem de maneiras distintas. Na forma dinâmica, a chamada resolve pelo tipo dinâmico do objeto, e não pelo tipo do ponteiro através do qual se chega até ele; custa um `vptr` por objeto e uma indireção por chamada. Sem `virtual`, a decisão passa ao tipo estático, sem que o compilador emita aviso.

Portão make verifica (Cap. 2): As quatro condições que toda entrega do `Deriva` satisfaz: zero aviso com `-Wall -Wextra -Wpedantic`; todos os testes verdes no `ctest`; despejo de replay idêntico byte a byte com semente fixa; e o contador de instâncias vivas fechando em zero. Nenhuma das quatro depende de sanitizer, e isso é deliberado.

RAII (Cap. 8): *Resource Acquisition Is Initialization*: o construtor adquire, o destrutor libera, e a garantia vem da ordem de destruição, inversa à de construção e válida inclusive quando é uma exceção que desenrola a pilha. No `Deriva` ele tem consequência física: `terminal_bruto` sem destrutor deixa o terminal do estudante inutilizável **depois** que o programa sai.

Ranges (C++20) (Anexo A): Biblioteca que generaliza os algoritmos da STL para trabalhar com views preguiçosas e composição funcional. Fora do padrão-alvo. Uma view não possui os elementos que enxerga, o que é a mesma armadilha do `std::string_view`.

Regra do Zero (Cap. 9): Se a classe não gerencia recurso, nenhuma das operações especiais deve ser declarada: as que o compilador gera são corretas e não envelhecem quando um membro é acrescentado. A grade do Deriva é a regra do zero em forma pura, e o que vale ler nela é o que **não** está escrito.

Regra dos Cinco (Cap. 14): Ao declarar uma das cinco operações especiais, decida sobre as outras quatro. Declarar o destrutor e esquecer a cópia é o caminho pelo qual se produz cópia rasa silenciosa, que é a caça ao bug 1 do semestre.

Regra dos Três (Cap. 9): Classe que gerencia um recurso manualmente precisa das três operações juntas: destrutor, construtor de cópia e operador de atribuição por cópia. A forma curta de cumpri-la é declarar as duas operações de cópia = delete, e ela cabe quando o recurso é único por natureza, como o terminal.

Replay determinístico (Cap. 16): Semente fixa e roteiro gravado produzem despejo idêntico byte a byte, o que dá o oráculo com que se afirma que uma refatoração não alterou o comportamento observável. Ele preserva inclusive o defeito, e é essa indiferença que o torna oráculo de refatoração, e o que o distingue do teste unitário.

Semântica de movimento (Cap. 14): Transferência de recurso em lugar de cópia. O `std::move` não move nada por si: ele muda a categoria de valor do argumento, de forma que a sobrecarga escolhida seja a que rouba o recurso. A origem permanece em estado **válido mas não-especificado**, que não é necessariamente vazio, e a otimização de string curta é o caso em que isso se percebe.

shared_ptr (Cap. 13): Ponteiro inteligente de posse compartilhada, com contagem de referências; libera o objeto quando a última cópia é destruída. Dois objetos que se apontam formam um ciclo que nunca é liberado, e no Deriva ele prende 160 bytes, medidos.

Singleton (Cap. 25): Padrão que garante uma única instância com ponto de acesso global. A forma correta em C++ é a variável `static` local na função de acesso. Traz quatro custos: estado global entre testes, dependência implícita na assinatura, impossibilidade de injetar substituto, e dado compartilhado entre threads.

SOLID (Cap. 24): Cinco princípios de projeto orientado a objetos: SRP, OCP, LSP, ISP e DIP. Nenhum deles diz como saber se a refatoração foi correta, e é o replay que responde a isso.

SRP (Cap. 24): *Single Responsibility Principle*: uma classe deve ter apenas um motivo para mudar. A formulação por motivo é mais útil que a por “faz uma coisa só”, porque é verificável: pergunte quem pede a mudança.

std::clamp (Cap. 21): Função de C++17 que devolve o valor limitado a um intervalo. Substitui o par `min/max` aninhado, e devolve **referência**: ligar o resultado a uma referência quando um dos limites é temporário produz referência pendurada.

std::filesystem (Cap. 20): Biblioteca de C++17 para caminhos e consulta ao sistema de arquivos. Cada função tem duas formas, uma que lança e outra que recebe um `std::error_code`, e quem escolhe é o chamador. Verificar a existência e abrir são duas operações, e a corrida entre elas é real.

std::optional (Caps. 9 e 20): Tipo de C++17 que modela **ausência de resultado**, e não falha. `mapa::carregar` o devolve: arquivo inexistente é ausência; permissão negada é exceção. `value()` lança quando vazio, e `operator*` não verifica.

std::string_view (Cap. 3): Tipo de C++17 que enxerga uma sequência de caracteres sem possuí-la. Um `string_view` guardado além da vida da `string` de origem fica pendurado, e o compilador não avisa.

std::variant (Cap. 20): Tipo de C++17 que armazena exatamente um de vários tipos possíveis, consultado com `std::visit`, que cobra o tratamento de todas as alternativas em compilação. Serve a resultado **ou** erro, quando o motivo da falha importa e lançar seria pesado demais.

Strategy (Cap. 25): Padrão que encapsula algoritmos intercambiáveis, de forma que o contexto não saiba qual está em uso. Em C++ moderno a mecânica é a `lambda`, e não a hierarquia: uma operação e nenhum estado é função. `Command`, que tem duas operações e estado, continua sendo classe.

Template (Cap. 19): Mecanismo de C++ para código genérico parametrizado por tipo, instanciado pelo compilador uma vez para cada tipo usado de fato. Cada instanciação é uma classe distinta, e é por isso que `contador_de_instancias<mapa>` e `contador_de_instancias<terminal_bruto>` têm variáveis estáticas separadas.

this (Cap. 7): Ponteiro implícito dentro de todo método não-estático, que aponta para o objeto sobre o qual o método foi chamado. Os dois usos que o Deriva tem estão no operador de atribuição: a guarda `if (this != &o)` e o `return *this`.

typeid (Cap. 18): Operador de RTTI que devolve informação de tipo em execução; para que o tipo dinâmico seja consultado, a classe precisa ter pelo menos uma função virtual.

UML (Cap. 6): *Unified Modeling Language*: notação gráfica para modelagem de software, usada nesta disciplina em diagramas de classes e de sequência do Deriva.

UndefinedBehaviorSanitizer (UBSan) (Cap. 2): Instrumentação que detecta comportamento indefinido em execução: estouro de inteiro, desreferência de nulo, desalinhamento. Como o `ASan`, é ferramenta de aula e não portão de aceitação nesta disciplina.

unique_ptr (Cap. 12): Ponteiro inteligente de posse exclusiva; não é copiável, apenas movível, e não custa espaço em relação ao ponteiro cru. Posse exclusiva torna certos defeitos impossíveis de construir, e é por isso que o `Composite` do Cap. 25 a usa: um item não pode estar em dois lugares, logo não há ciclo.

Variante deliberadamente quebrada (Anexo C): Cópia de uma versão do Deriva com um defeito plantado, plausível, e com a propriedade que decide tudo: o compilador não avisa. Elas ensinam que o portão de compilação é necessário e não suficiente.

virtual (Cap. 11): Palavra-chave que indica que um método pode ser sobrescrito em classes derivadas e é resolvido em tempo de execução através da `vtable`.

vtable e vptr (Cap. 11): A `vtable` é a tabela de ponteiros para funções virtuais que o compilador gera para cada classe polimórfica; o `vptr` é o ponteiro para ela, que passa a ser o primeiro campo de cada objeto. São 8 bytes por **objeto**, e não por classe, aferidos por `static_assert` em `include/deriva/leiaute.hpp`.

weak_ptr (Cap. 13): Ponteiro inteligente observador, que não incrementa a contagem de referências; usado para quebrar ciclos e para verificar, antes de notificar, se o observado ainda existe.

Referências Bibliográficas

A regra que organiza esta lista está no plano do livro: **a bibliografia do livro se alinha à do plano de ensino, e não o contrário**. A do plano de ensino passou por auditoria; a do livro v1 não tinha passado, e o que segue é o resultado de a ter alinhado. As decisões que a auditoria produziu estão registradas aqui, e não em nota de rodapé, porque uma bibliografia é lista de decisões e não lista de títulos.

Três correções de autoria e de edição entraram. A do Catch2 é a maior: no v1 a referência aparecia **sem autoria nenhuma**, e a biblioteca é de **Phil Nash e Martin Hořeňovský**. O *Programming: Principles and Practice Using C++* está na **3ª edição, de 2024**, e não constava da lista do livro, apesar de constar da do plano de ensino. E o **FTXUI é de Arthur Sonzogni**, o que vale registrar porque uma versão anterior deste material atribuía a biblioteca a outra pessoa - invenção, e não erro de digitação, que é a categoria de defeito que uma auditoria de bibliografia existe para pegar.

Uma referência entrou por decisão de escopo: **Josuttis, C++17 - The Complete Guide**, que cobre o padrão-alvo construção por construção e é a fonte do Anexo B. Ele é publicado pelo próprio autor, pela Leanpub, e o registro dessa natureza fica aqui para que a escolha seja informada: título autopublicado costuma ser motivo de recusa, e neste caso não é, porque Josuttis é membro do comitê de padronização e o livro é a cobertura mais completa do C++17 que existe.

Duas referências do v1 **saíram**, e as duas por não estarem na bibliografia do plano de ensino. A primeira era uma *Introdução à programação orientada a objetos com C++* entrada por MENDES, e a auditoria encontrou nela um defeito de autoria antes de encontrar o de escopo: a obra é de Antônio Mendes da Silva Filho, sobrenome composto tratado como nome do meio, e a entrada correta começaria por SILVA FILHO. Corrigir a entrada e mantê-la seria alinhar o livro a si mesmo em vez de ao plano, e por isso ela foi retirada. A segunda era um *C++: como programar* de Deitel e Deitel, sem edição e sem ano, com a nota “edição mais recente disponível no acervo” no lugar do dado - o que funciona num plano de ensino, que é documento anual, e deixa uma referência de livro sem identificação. Nenhuma das duas foi substituída por invenção, e a decisão de retirar é reversível pelo caminho normal: se qualquer das duas voltar à bibliografia do plano, ela volta a esta lista com os dados conferidos no acervo.

Sobre duplicatas, a conferência foi feita e não achou nenhuma. Os pares que parecem duplicata não são: Stroustrup aparece três vezes, Meyers duas, Martin duas, Fowler duas e Josuttis duas, sempre com obras distintas. O par que mais se confunde é o de Josuttis, e vale desfazer a confusão de uma vez - *The C++ Standard Library* (2012) é referência de biblioteca padrão, e *C++17 - The Complete Guide* (2019) é o do padrão-alvo desta disciplina.

Uma última decisão, e ela é a que o v1 mais precisava: onde a obra original e a tradução brasileira coexistem, **o original é a referência**, como no plano de ensino, e as traduções ficam num bloco separado ao fim, identificadas como tais e regis-

tradas por serem os exemplares que o estudante encontra no acervo da BC/UFPB. O v1 citava as traduções nas listas principais e o site citava os originais, o que é exatamente a divergência entre livro e site que a arquitetura de fonte única existe para impedir.

Referências Básicas

1. STROUSTRUP, Bjarne. *A Tour of C++*. 3. ed. Boston: Addison-Wesley, 2022.
2. STROUSTRUP, Bjarne. *Programming: Principles and Practice Using C++*. 3. ed. Boston: Addison-Wesley, 2024.
3. MEYERS, Scott. *Effective Modern C++: 42 Specific Ways to Improve Your Use of C++11 and C++14*. Sebastopol: O'Reilly, 2014.
4. JOSUTTIS, Nicolai M. *C++17 - The Complete Guide*. Leanpub, 2019.
5. GAMMA, Erich; HELM, Richard; JOHNSON, Ralph; VLISSIDES, John. *Design Patterns: Elements of Reusable Object-Oriented Software*. Boston: Addison-Wesley, 1994.

Referências Complementares

1. ISO/IEC 14882:2017. *Programming languages - C++ (C++17)*.
2. STROUSTRUP, Bjarne. *The C++ Programming Language*. 4. ed. Boston: Addison-Wesley, 2013.
3. STROUSTRUP, Bjarne; SUTTER, Herb (eds.). *C++ Core Guidelines*. Disponível em: <https://isocpp.github.io/CppCoreGuidelines>
4. SUTTER, Herb; ALEXANDRESCU, Andrei. *C++ Coding Standards: 101 Rules, Guidelines, and Best Practices*. Boston: Addison-Wesley, 2004.
5. VANDEVOORDE, David; JOSUTTIS, Nicolai M.; GREGOR, Douglas. *C++ Templates: The Complete Guide*. 2. ed. Boston: Addison-Wesley, 2017.
6. MARTIN, Robert C. *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Boston: Prentice Hall, 2017.
7. OUSTERHOUT, John. *A Philosophy of Software Design*. 2. ed. Palo Alto: Yaknyam Press, 2021.
8. FREEMAN, Steve; PRYCE, Nat. *Growing Object-Oriented Software, Guided by Tests*. Boston: Addison-Wesley, 2009.
9. FOWLER, Martin. *Refactoring: Improving the Design of Existing Code*. 2. ed. Boston: Addison-Wesley, 2018.
10. FOWLER, Martin; SCOTT, Kendall. *UML Distilled*. 3. ed. Boston: Addison-Wesley, 2003.
11. CHACON, Scott; STRAUB, Ben. *Pro Git*. 2. ed. New York: Apress, 2014. Disponível em: <https://git-scm.com/book>
12. BERRYMAN, John; ZIEGLER, Albert. *Prompt Engineering for LLMs*. Sebastopol: O'Reilly, 2024.

Documentação de Ferramenta e de Biblioteca

As versões são fixadas e publicadas no início do semestre, e o Cap. 2 explica por que a tag não flutua.

1. cppreference.com. *Referência do C++*. Disponível em: <https://en.cppreference.com>
2. SONZOGNI, Arthur. *FTXUI - Functional Terminal (X) User interface*. Disponível em: <https://github.com/ArthurSonzogni/FTXUI>. Versão fixada para o semestre: v5.0.0.
3. NASH, Phil; HOŘEŇOVSKÝ, Martin *et al.* *Catch2 - A modern, C++-native test framework*. Disponível em: <https://github.com/catchorg/Catch2>. Versão fixada para o semestre.
4. THE QT COMPANY. *Qt Documentation*. Disponível em: <https://doc.qt.io>
5. LOHMANN, Niels. *JSON for Modern C++*. Disponível em: <https://github.com/nlohmann/json>

Referências em Português, no Acervo da BC/UFPB

Estas são as edições em português das obras acima, ou obras próximas a elas, e estão registradas por serem as que o estudante encontra no acervo. Onde a tradução e o original coexistem na lista, **o original é a referência**, e a tradução é o exemplar consultável.

1. MEYERS, Scott. *C++ Moderno e Eficaz: 42 formas específicas de aprimorar seu uso de C++11 e C++14*. Rio de Janeiro: Alta Books, 2015. [Tradução de *Effective Modern C++*.]
2. MEYERS, Scott. *C++ eficaz: 55 maneiras de aprimorar seus programas e projetos*. 3. ed. Porto Alegre: Bookman, 2011. [Tradução de *Effective C++*, 3ª ed.]
3. GAMMA, Erich; HELM, Richard; JOHNSON, Ralph; VLISSIDES, John. *Padrões de projeto: soluções reutilizáveis de software orientados a objetos*. Porto Alegre: Bookman, 2000. [Tradução de *Design Patterns*.]
4. MARTIN, Robert C. *Arquitetura limpa: o guia do artesão para estrutura e design de software*. Rio de Janeiro: Alta Books, 2019. [Tradução de *Clean Architecture*.]
5. MARTIN, Robert C. *Código limpo: habilidades práticas do Agile Software*. Rio de Janeiro: Alta Books, 2009.
6. FOWLER, Martin. *UML essencial: um breve guia para a linguagem-padrão de modelagem de objetos*. 3. ed. Porto Alegre: Bookman, 2005. [Tradução de *UML Distilled*.]
7. JOSUTTIS, Nicolai M. *The C++ Standard Library: a tutorial and reference*. 2. ed. Boston: Addison-Wesley, 2012.

Material da Disciplina

BATISTA, Carlos Eduardo Coelho Freire. *Programação Orientada a Objetos em C++ - POO/UFPB*. Livro, 26 aulas com exemplos interativos, e o repositório do **Deriva** com as versões que compilam. Distribuído via SIGAA e pelo site da disciplina.

Nota sobre a seção de documentação

Duas entradas da seção de ferramenta e de biblioteca não constam do plano de ensino, e as duas ficam por razão declarada. A documentação do **Qt** fica porque o Cap. 26 depende dela para ser conduzido, mesmo sendo aquela aula demonstração sem entrega. E o **nlohmann/json** fica com a autoria creditada, como reconhecimento de API: o Cap. 23 decide **não** usar JSON no Deriva, e a §23.6 explica por quê, de forma que a entrada serve ao estudante que vai encontrar a biblioteca fora daqui, e não a uma dependência deste repositório - o `CMakeLists.txt` fixa FTXUI e Catch2, e mais nada.

A conformidade com a ABNT, incluindo a forma de entrada de sobrenome composto e o uso de itálico, é trabalho de normalização e não entra aqui. As entradas seguem o formato que o plano de ensino já usa.